

RTL Design Sherpa

RTL AMBA Shared Infrastructure Module Reference — Rev 0.92

October 2026

Table of Contents

1 GAXI (Generic AXI) Modules....41
1.1 Overview....42
1.1.1 Key Features....42
1.2 Modules....43
1.2.1 Skid Buffers (Elastic Buffering)....43
1.2.2 FIFOs (Deep Buffering)....43
1.2.3 Register Slices (Timing Isolation)....43
1.2.4 Test-Code Multi-Instance Helpers....43
1.3 Protocol....45
1.3.1 Interface Signals....45
1.3.2 Handshake Rules....45
1.4 Usage Examples....46
1.4.1 Basic Skid Buffer....46
1.4.2 Synchronous FIFO (Deep Buffering)....46
1.4.3 Asynchronous FIFO (Clock Domain Crossing)....47
1.4.4 Type-Safe Struct Buffer....47
1.4.5 Drop FIFO (Packet Processing)....48
1.5 Design Patterns....50
1.5.1 Pattern 1: Timing Closure (Pipeline Stage)....50
1.5.2 Pattern 2: Rate Adaptation....50
1.5.3 Pattern 3: Clock Domain Crossing....50
1.6 Performance Characteristics....52

1.6.1 Latency....52
1.6.2 Throughput....52
1.6.3 Resource Utilization (Typical)....52
1.7 Parameter Selection Guidelines....53
1.7.1 Buffer Depth (DEPTH)....53
1.7.2 Output Mode (REGISTERED)....53
1.7.3 CDC Synchronizer Stages (N_FLOP_CROSS)....53
1.8 Common Use Cases....54
1.8.1 Video Processing (Skid Buffer)....54
1.8.2 Network Packet Processing (Drop FIFO)....54
1.8.3 Clock Domain Crossing (Async FIFO)....54
1.9 Design Notes....55
1.9.1 When to Use GAXI vs AXI4-Stream....55
1.9.2 Buffer vs FIFO Selection....55
1.10 Testing....56
1.11 Common Issues and Debugging....57
1.11.1 Issue 1: Data Loss....57
1.11.2 Issue 2: Timing Violation....57
1.11.3 Issue 3: CDC Metastability....57
1.12 Related Documentation....58
1.12.1 Comprehensive Guide....58
1.12.2 Protocol Specifications....58
1.12.3 RTL Design Sherpa Documentation....58
1.12.4 Source Code....58

1.13 References....59
1.14 Navigation....60
2 gaxi_drop_fifo_sync....61
2.1 Overview....62
2.1.1 Key Features....62
2.2 Parameters....63
2.2.1 Derived Parameters (do not override)....63
2.3 Ports....64
2.3.1 Clock and Reset....64
2.3.2 Write Interface....64
2.3.3 Read Interface....64
2.3.4 Drop Interface....64
2.3.5 Status....65
2.4 Functional Description....66
2.4.1 Block diagram....66
2.4.2 Drop by Count....66
2.4.3 Drop All....67
2.4.4 Simultaneous Operations....67
2.5 Timing Characteristics....68
2.5.1 Fill FIFO....68
2.5.2 Drop by Count....68
2.5.3 Drop All....68
2.5.4 Comprehensive Scenario....69
2.6 Usage Examples....70

2.6.1 Basic Write/Read 70
2.6.2 Drop Operation 70
2.7 Design Notes 72
2.7.1 FIFO Depth Requirements 72
2.7.2 Registered vs. Mux Mode 72
2.7.3 Drop Latency 72
2.7.4 Almost Full/Empty Margins 72
2.8 Related Modules 73
2.9 Testing 74
2.9.1 Running Tests 74
2.10 References 75
2.11 Navigation 76
3 gaxi_fifo_sync 77
3.1 Overview 78
3.1.1 Key Features 78
3.2 Parameters 79
3.3 Ports 80
3.4 Functional Description 81
3.4.1 Read Modes 81
3.4.2 Architecture 81
3.4.3 Dependencies 82
3.4.4 Resource Utilization 82
3.5 Timing Characteristics 83
3.6 Usage Examples 84

3.6.1 Example 1: Mux Mode (Low Latency)....84
3.6.2 Example 2: Flop Mode (Timing Closure)....84
3.7 Design Notes....85
3.8 Related Modules....86
3.9 Testing....87
3.10 References....88
3.11 Navigation....89
4 gaxi_regslice....90
4.1 Overview....91
4.1.1 Key Features....91
4.2 Parameters....92
4.3 Ports....93
4.4 Functional Description....94
4.4.1 Architecture....94
4.4.2 Ready/Valid Protocol....94
4.4.3 Transfer Scenarios....95
4.4.4 Storage Update Logic....95
4.5 Timing Characteristics....97
4.5.1 Latency Guarantee....97
4.6 Usage Examples....98
4.7 Design Notes....99
4.7.1 Port Compatibility....99
4.7.2 Status Signals....99
4.7.3 Assertions....99

4.7.4 Resource Utilization 99
4.8 Related Modules 100
4.8.1 gaxi_regslice vs gaxi_skid_buffer 100
4.8.2 gaxi_regslice vs gaxi_fifo_sync 100
4.9 Testing 101
4.9.1 Running Tests 101
4.10 References 102
4.11 Navigation 103
5 gaxi_skid_buffer 104
5.1 Overview 105
5.1.1 Key Features 105
5.2 Parameters 106
5.3 Ports 107
5.3.1 Clock and Reset 107
5.3.2 Write Interface 107
5.3.3 Read Interface 108
5.3.4 Status 108
5.4 Functional Description 109
5.4.1 Block diagram 109
5.4.2 Operation Modes 109
5.4.3 Internal State Machine 112
5.4.4 Ready/Valid Logic 113
5.5 Timing Characteristics 114
5.5.1 Latency 114

5.5.2 Throughput....114
5.5.3 Critical Paths....114
5.6 Waveforms....115
5.6.1 Scenario 1: Streaming Read/Write....115
5.6.2 Scenario 2: Burst Write Until Full....115
5.6.3 Scenario 3: Simultaneous Read/Write....116
5.6.4 Scenario 4: Burst Read Until Empty....117
5.6.5 Scenario 5: Fill Then Drain Pattern....118
5.6.6 Scenario 6: Alternating Read/Write....119
5.7 Usage Examples....121
5.7.1 Example 1: Basic Pipeline Stage....121
5.7.2 Example 2: Backpressure Absorption....121
5.7.3 Example 3: Monitoring Occupancy....121
5.8 Design Notes....123
5.8.1 Depth Selection....123
5.8.2 Timing Closure....123
5.8.3 Area vs. Performance Trade-offs....123
5.8.4 Error Checking....124
5.8.5 Issue 1: Timing Violation on the Read Path....124
5.8.6 Issue 2: Buffer Overflow....124
5.8.7 Issue 3: Unexpected Latency....124
5.9 Related Modules....126
5.10 Testing....127
5.10.1 Running Tests....127

5.11 References....128
5.12 Navigation....129
6 gaxi_skid_buffer_dbldrn....130
6.1 Overview....131
6.1.1 Key Features....131
6.2 Parameters....133
6.3 Ports....134
6.3.1 Clock and Reset....134
6.3.2 Write Interface....134
6.3.3 Read Interface....134
6.3.4 Status....135
6.4 Functional Description....136
6.4.1 Block diagram....136
6.4.2 Transfer Detection....136
6.4.3 Storage Update....137
6.4.4 Ready/Valid Logic....137
6.4.5 Output Assignments....138
6.4.6 Legality Assertion....138
6.5 Timing Characteristics....139
6.6 Usage Examples....140
6.7 Design Notes....141
6.8 Related Modules....142
6.9 Testing....143
6.10 References....144

6.11 Navigation....145
7 Shared Infrastructure Documentation Status....146
7.1 Completion Status....147
7.1.1 Completed Documentation....147
7.1.2 New shared infrastructure (no dedicated page, covered above)....148
7.1.3 Remaining Documentation (15 modules)....148
7.2 Documentation Template....150
7.3 Design Notes....152
7.3.1 [Important design aspect 1]....152
7.4 Related Modules....153
7.4.1 Used By....153
7.4.2 Uses....153
7.5 References....154
7.5.1 Specifications....154
7.5.2 Source Code....154
7.6 Navigation....155
7.6.1 Idle Signal Generation....165
7.6.2 Clock Gating Control Integration....165
7.6.3 Gating Threshold Behavior....166
7.7 Timing Characteristics....167
7.7.1 Gating Sequence (Activity → Idle)....167
7.7.2 Ungating Sequence (Idle → Activity)....167
7.8 Usage Examples....168
7.8.1 Basic Integration with AXI Interface....168

7.8.2 Multi-Master Arbitration with Clock Gating....169
7.8.3 Dynamic Threshold Adjustment....169
7.9 Design Notes....171
7.9.1 Wakeup Signal Registration....171
7.9.2 Activity Signal Selection....171
7.9.3 Integration with ICG Cells....171
7.9.4 Power Measurement Methodology....172
7.9.5 Performance Considerations....172
7.10 Related Modules....174
7.10.1 Used By....174
7.10.2 Uses....174
7.10.3 See Also....174
7.11 Testing....175
7.12 References....176
7.12.1 Source Code....176
7.12.2 Documentation....176
7.12.3 Industry Standards....176
7.13 Navigation....177
8 AXI4 DMA Slaves....178
8.1 Overview....179
8.1.1 Key Features....179
8.2 Parameters....180
8.3 Ports....182
8.3.1 Clock and Reset....182

8.3.2 Read-Side Telemetry....182
8.3.3 Write-Side Telemetry....182
8.3.4 AXI4 Read Slave (AR + R)....183
8.3.5 AXI4 Write Slave (AW + W + B)....183
8.3.6 Status....184
8.4 Functional Description....185
8.4.1 Block diagram....185
8.4.2 Structural Composition....185
8.4.3 Read Side (LFSR Source)....185
8.4.4 Write Side (CRC Sink)....186
8.4.5 Shared CRC vs Independent Resets....186
8.5 Timing Characteristics....187
8.6 Usage Examples....188
8.7 Design Notes....190
8.8 Related Modules....191
8.8.1 Used By....191
8.8.2 Uses....191
8.8.3 See Also....191
8.9 Testing....192
8.10 References....193
8.10.1 Source Code....193
8.10.2 Documentation....193
8.11 Navigation....194
9 AXI4 Master Read CRC Checker....195

9.1 Overview..... 196
9.1.1 Key Features..... 196
9.2 Parameters..... 197
9.3 Ports..... 199
9.3.1 Clock and Reset..... 199
9.3.2 Configuration (CSR) — same shape as the write generator..... 199
9.3.3 Telemetry..... 200
9.3.4 M-Side AXI4 Read (out to fabric/controller)..... 201
9.3.5 Debug Observability (active when DBG_FIFO_DEPTH > 0)..... 202
9.4 Functional Description..... 203
9.4.1 Block diagram..... 203
9.4.2 Workload FSM..... 203
9.4.3 Decoupled AR and R Address Generation..... 204
9.4.4 Expected Data: LFSR vs Address Hash..... 204
9.4.5 Pipelined Compare..... 204
9.4.6 CRC Accumulation..... 204
9.4.7 AR-ID Generation..... 204
9.4.8 Out-of-Order Completion — Known Limitation..... 205
9.4.9 Optional Debug FIFO..... 205
9.4.10 Standard Protocol Handler..... 205
9.5 Timing Characteristics..... 206
9.6 Usage Examples..... 207
9.7 Design Notes..... 209
9.8 Related Modules..... 210

9.8.1 Used By.....210
9.8.2 Uses.....210
9.8.3 See Also.....210
9.9 Testing.....211
9.10 References.....212
9.10.1 Source Code.....212
9.10.2 Documentation.....212
9.11 Navigation.....213
10 AXI4 Master Write Injector.....214
10.1 Overview.....215
10.1.1 Key Features.....215
10.2 Parameters.....217
10.3 Ports.....220
10.3.1 Clock and Reset.....220
10.3.2 Configuration (CSR).....220
10.3.3 Telemetry.....221
10.3.4 M-Side AXI4 Write (out to fabric/controller).....222
10.4 Functional Description.....223
10.4.1 Block diagram.....223
10.4.2 Workload FSM.....223
10.4.3 Decoupled AW and W Address Generation.....224
10.4.4 Data Path: LFSR vs Address Hash.....224
10.4.5 Hash Pipeline and Staging FIFO.....224
10.4.6 AW-ID Generation.....225

10.4.7 CRC Accumulation and Completion....225
10.4.8 Standard Protocol Handler....225
10.5 Timing Characteristics....226
10.6 Usage Examples....227
10.7 Design Notes....228
10.8 Related Modules....229
10.8.1 Used By....229
10.8.2 Uses....229
10.8.3 See Also....229
10.9 Testing....230
10.10 References....231
10.10.1 Source Code....231
10.10.2 Documentation....231
10.11 Navigation....232
11 AXI4 Slave Read Injector....233
11.1 Overview....234
11.1.1 Key Features....234
11.2 Parameters....235
11.3 Ports....237
11.3.1 Clock and Reset....237
11.3.2 Test Control....237
11.3.3 Per-Channel Telemetry....237
11.3.4 AXI4 Read Address Channel (AR)....237
11.3.5 AXI4 Read Data Channel (R)....238

11.3.6 Status....239
11.4 Functional Description....240
11.4.1 Block diagram....240
11.4.2 LFSR Pattern Generation....240
11.4.3 Data Replication to the Bus Width....241
11.4.4 Per-Channel CRC-32....241
11.4.5 Burst FSM and Gapless Back-to-Back Bursts....241
11.4.6 Standard Protocol Handler....241
11.5 Timing Characteristics....242
11.6 Usage Examples....243
11.7 Design Notes....244
11.8 Related Modules....245
11.8.1 Used By....245
11.8.2 Uses....245
11.8.3 See Also....245
11.9 Testing....246
11.10 References....247
11.10.1 Source Code....247
11.10.2 Documentation....247
11.11 Navigation....248
12 AXI4 Slave Write CRC Checker....249
12.1 Overview....250
12.1.1 Key Features....250
12.2 Parameters....251

12.3 Ports....253
12.3.1 Clock and Reset....253
12.3.2 Test Control....253
12.3.3 Per-Channel Telemetry....253
12.3.4 AXI4 Write Address Channel (AW)....253
12.3.5 AXI4 Write Data Channel (W)....254
12.3.6 AXI4 Write Response Channel (B)....255
12.3.7 Status....255
12.4 Functional Description....256
12.4.1 Block diagram....256
12.4.2 Data Slice Extraction....256
12.4.3 Per-Channel CRC-32....256
12.4.4 Write Burst FSM and Gapless Bursts....257
12.4.5 Back-to-Back B Response Id Latching....257
12.4.6 Standard Protocol Handler....257
12.5 Timing Characteristics....258
12.6 Usage Examples....259
12.7 Design Notes....260
12.8 Related Modules....261
12.8.1 Used By....261
12.8.2 Uses....261
12.8.3 See Also....261
12.9 Testing....262
12.10 References....263

12.10.1 Source Code....263
12.10.2 Documentation....263
12.11 Navigation....264
13 AXI Bus Meter....265
13.1 Overview....266
13.1.1 Key Features....266
13.2 Parameters....267
13.3 Ports....268
13.3.1 Clock and Reset....268
13.3.2 Window Control....268
13.3.3 AXI Bus Snoop....268
13.3.4 Aggregate Counters....269
13.3.5 Per-Channel Counters....269
13.4 Functional Description....270
13.4.1 Block diagram....270
13.4.2 Bucket Classification....270
13.4.3 Aggregate Counters....270
13.4.4 Per-Channel Counters and Overflow Stickies....271
13.4.5 Recommended Wiring....271
13.4.6 Clear and Freeze Semantics....271
13.5 Timing Characteristics....272
13.6 Usage Examples....273
13.7 Design Notes....274
13.7.1 Passive Observer....274

13.7.2 Counter Width Rationale....274
13.7.3 Verilator Output Drivers....274
13.7.4 Window Alignment....274
13.8 Related Modules....275
13.8.1 Used By....275
13.8.2 Uses....275
13.8.3 See Also....275
13.9 Testing....276
13.10 References....277
13.10.1 Source Code....277
13.10.2 Documentation....277
13.11 Navigation....278
14 AXI Burst Address Generator....279
14.1 Overview....280
14.1.1 Key Features....280
14.2 Parameters....281
14.3 Ports....282
14.4 Functional Description....283
14.4.1 Block diagram....283
14.4.2 Increment Calculation....283
14.4.3 Burst Type Handling....284
14.4.4 Aligned Address Output....284
14.5 Timing Characteristics....285
14.6 Usage Examples....286

14.7 Design Notes....287
14.7.1 Data Width Conversion....287
14.7.2 WRAP Burst Alignment....287
14.7.3 Pure Combinational....287
14.8 Related Modules....288
14.8.1 Used By....288
14.8.2 See Also....288
14.9 Testing....289
14.10 References....290
14.10.1 Specifications....290
14.10.2 Source Code....290
14.11 Navigation....291
15 AXI Master Read Splitter....292
15.1 Overview....293
15.1.1 Key Features....293
15.2 Parameters....294
15.3 Ports....295
15.3.1 Clock and Reset....295
15.3.2 Configuration Interface....295
15.3.3 Master AXI Read Interface (Downstream)....295
15.3.4 Slave AXI Read Interface (Upstream / FUB)....297
15.3.5 Split Information Interface....298
15.4 Functional Description....300
15.4.1 Block diagram....300

15.4.2 Transaction Splitting Overview....300
15.4.3 State Machine Architecture....301
15.4.4 Boundary Crossing Detection....301
15.4.5 AR Channel Forwarding Logic....301
15.4.6 Ready Signal Management....302
15.4.7 R Channel Handling....302
15.4.8 Split Information Tracking....302
15.5 Timing Characteristics....304
15.6 Usage Examples....305
15.6.1 Basic Integration (4KB Boundary Alignment)....305
15.6.2 Transaction Splitting Example Scenario....307
15.7 Design Notes....308
15.7.1 AXI Protocol Assumptions....308
15.7.2 Performance Characteristics....308
15.7.3 Error Handling....308
15.7.4 Integration Considerations....309
15.8 Related Modules....310
15.8.1 Used By....310
15.8.2 Uses....310
15.8.3 See Also....310
15.9 Testing....311
15.10 References....312
15.10.1 Source Code....312
15.10.2 Documentation....312

15.11 Navigation....313
16 AXI Master Write Splitter....314
16.1 Overview....315
16.1.1 Key Features....315
16.2 Parameters....317
16.3 Ports....318
16.3.1 Clock and Reset....318
16.3.2 Configuration Interface....318
16.3.3 Master AXI Write Interface (Downstream)....318
16.3.4 Slave AXI Write Interface (Upstream / FUB)....320
16.3.5 Split Information Interface....322
16.4 Functional Description....323
16.4.1 Block diagram....323
16.4.2 Write Transaction Splitting Overview....323
16.4.3 State Machine Architecture....324
16.4.4 Write Data Channel Management....324
16.4.5 Response Consolidation Logic....324
16.4.6 Boundary Crossing Detection....325
16.4.7 AW Channel Forwarding Logic....325
16.4.8 Ready Signal Management....326
16.5 Timing Characteristics....327
16.6 Usage Examples....328
16.6.1 Basic Integration (4KB Boundary, Response Consolidation)....328
16.6.2 Response Consolidation Example....329

16.7 Design Notes....331
16.7.1 AXI Protocol Assumptions....331
16.7.2 Response Consolidation Details....331
16.7.3 Performance Characteristics....331
16.7.4 Error Handling....332
16.7.5 Integration Considerations....332
16.8 Related Modules....333
16.8.1 Used By....333
16.8.2 Uses....333
16.8.3 See Also....333
16.9 Testing....334
16.10 References....335
16.10.1 Source Code....335
16.10.2 Documentation....335
16.11 Navigation....336
17 AXI Performance Latency Histogram....337
17.1 Overview....338
17.1.1 Key Features....338
17.2 Parameters....339
17.3 Ports....340
17.3.1 Clock and Reset....340
17.3.2 Window Control....340
17.3.3 Command Channel (AR for reads, AW for writes)....340
17.3.4 Read Data Channel (R) — used when IS_READ=1....340

17.3.5 Write Response Channel (B) — used when IS_READ=0 341
17.3.6 CSR-Indexed Readout 341
17.4 Functional Description 343
17.4.1 Block diagram 343
17.4.2 Metrics 343
17.4.3 Transaction Matching 344
17.4.4 Log2 Binning 344
17.4.5 Free-Running Timestamp 344
17.4.6 Four-Stage Update Pipeline 344
17.4.7 Window Control 345
17.4.8 Readout 345
17.5 Timing Characteristics 346
17.6 Usage Examples 347
17.7 Design Notes 348
17.7.1 Snoop-Only, Base Untouched 348
17.7.2 Timestamp Not Frozen 348
17.7.3 FIFO Depth vs Outstanding 348
17.7.4 Pipeline Drain Before Readback 348
17.8 Related Modules 349
17.8.1 Used By 349
17.8.2 Uses 349
17.8.3 See Also 349
17.9 Testing 350
17.10 References 351

17.10.1 Source Code....351
17.10.2 Documentation....351
17.11 Navigation....352
18 AXI Split Combinational Logic....353
18.1 Overview....354
18.1.1 Key Features....354
18.2 Parameters....355
18.3 Ports....356
18.3.1 Clock and Reset (For Assertions Only)....356
18.3.2 Input Signals....356
18.3.3 Output Signals....356
18.4 Functional Description....358
18.4.1 Block diagram....358
18.4.2 Boundary Crossing Detection Algorithm....358
18.4.3 Split Length Calculation....359
18.4.4 Remaining Length Calculation....359
18.4.5 State Machine Helper....360
18.5 Timing Characteristics....361
18.6 Usage Examples....362
18.6.1 Integration with Read Splitter....362
18.6.2 Example Calculation Walkthrough....363
18.7 Design Notes....365
18.7.1 Design Assumptions....365
18.7.2 Assumption 1: Address Alignment....365

18.7.3 Assumption 2: Fixed Transfer Size....365
18.7.4 Assumption 3: Incrementing Bursts Only....365
18.7.5 Assumption 4: No Address Wraparound....365
18.7.6 Synthesis Optimization....366
18.7.7 Assertion Coverage....366
18.7.8 Timing Considerations....366
18.7.9 Debug Support....367
18.8 Related Modules....368
18.8.1 Used By....368
18.8.2 Dependencies....368
18.8.3 See Also....368
18.9 Testing....369
18.10 References....370
18.10.1 Source Code....370
18.10.2 Documentation....370
18.11 Navigation....371
19 AXI-Stream Master Injector....372
19.1 Overview....373
19.1.1 Key Features....373
19.2 Parameters....374
19.3 Ports....376
19.3.1 Clock and Reset....376
19.3.2 Configuration / Control....376
19.3.3 Per-Channel Telemetry....377

19.3.4 AXI-Stream Master (m_axis) 377
19.4 Functional Description 378
19.4.1 Block diagram 378
19.4.2 Scheduling FSM 378
19.4.3 Channel Scheduling: Sequential or Interleaved 379
19.4.4 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks) 379
19.4.5 Stream Outputs and tlast Cadence 379
19.4.6 Backpressure Insensitivity 379
19.5 Timing Characteristics 381
19.6 Usage Examples 382
19.7 Design Notes 383
19.8 Related Modules 384
19.8.1 Used By 384
19.8.2 Uses 384
19.8.3 See Also 384
19.9 Testing 385
19.10 References 386
19.10.1 Source Code 386
19.10.2 Documentation 386
19.11 Navigation 387
20 AXI-Stream Slave Pattern Checker 388
20.1 Overview 389
20.1.1 Key Features 389
20.2 Parameters 390

20.3 Ports....392
20.3.1 Clock and Reset....392
20.3.2 Configuration / Control....392
20.3.3 Telemetry....392
20.3.4 AXI-Stream Slave (s_axis)....393
20.4 Functional Description....394
20.4.1 Block diagram....394
20.4.2 Readiness and Beat Handshake....394
20.4.3 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)....394
20.4.4 Per-Beat Compare and Sticky Error....395
20.4.5 Packet Counting....395
20.5 Timing Characteristics....396
20.6 Usage Examples....397
20.7 Design Notes....398
20.8 Related Modules....399
20.8.1 Used By....399
20.8.2 Uses....399
20.8.3 See Also....399
20.9 Testing....400
20.10 References....401
20.10.1 Source Code....401
20.10.2 Documentation....401
20.11 Navigation....402

21 AXIS Bus Meter....403
21.1 Overview....404
21.1.1 Key Features....404
21.2 Parameters....405
21.3 Ports....406
21.3.1 Clock and Reset....406
21.3.2 Window Control....406
21.3.3 AXIS Bus Snoop....406
21.3.4 Aggregate Cycle Buckets....406
21.3.5 Aggregate AXIS-Native Throughput (productive beats only)....407
21.3.6 Per-Channel Cycle Buckets (binned by tid)....407
21.4 Functional Description....409
21.4.1 Block diagram....409
21.4.2 Bucket Classification....409
21.4.3 Payload-Byte Popcount....410
21.4.4 AXIS-Native Throughput Counters....410
21.4.5 Per-Channel Cycle Buckets....410
21.4.6 Clear and Freeze Semantics....410
21.5 Timing Characteristics....411
21.6 Usage Examples....412
21.7 Design Notes....413
21.7.1 Why Byte/Packet Counters Are Window-Independent....413
21.7.2 Per-Channel Idle Is Not Attributed....413
21.7.3 tstrb Handling....413

21.7.4 Counter Widths....413
21.8 Related Modules....414
21.8.1 Used By....414
21.8.2 Uses....414
21.8.3 See Also....414
21.9 Testing....415
21.10 References....416
21.10.1 Source Code....416
21.10.2 Documentation....416
21.11 Navigation....417
22 Clock-Gated Variants Guide....418
22.1 Overview....419
22.2 Parameters....420
22.3 Ports....421
22.3.1 Configuration Inputs....421
22.3.2 Status Outputs....421
22.3.3 Ports That Do Not Exist....422
22.4 Functional Description....423
22.4.1 Gating Infrastructure....423
22.4.2 Activity Detection....423
22.4.3 Gating Conditions (All Must Be True)....424
22.4.4 Ready Forcing While Gated....424
22.5 Timing Characteristics....426
22.5.1 Counting Convention....426

22.5.2 Register Stages by Family....426
22.5.3 Gating Latency....428
22.5.4 Wake-Up Behavior....428
22.6 Usage Examples....429
22.6.1 Pattern 1: Aggressive Gating....429
22.6.2 Pattern 2: Runtime-Adjustable Threshold....429
22.6.3 Pattern 3: Gating Bypassed....430
22.6.4 Measuring Gated Cycles....430
22.7 Design Notes....431
22.7.1 Synthesis and Portability....431
22.7.2 Known Gaps....431
22.8 Related Modules....432
22.8.1 Available Clock-Gated Variants....432
22.8.2 Related Documentation....433
22.9 Testing....434
22.9.1 Functional Verification....434
22.9.2 Gating Verification....434
22.10 References....435
22.11 Navigation....436
23 CXS Pattern Injector and Checker....437
23.1 Overview....438
23.2 The Link....439
23.3 Parameters....440
23.4 Status Ports....441

23.5 Verification 442
24 SDPRAM Core 443
24.1 Overview 444
24.1.1 Key Features 444
24.2 Parameters 446
24.3 Ports 447
24.3.1 Clock and Reset 447
24.3.2 FUB Write Side (AW + W + B) 447
24.3.3 FUB Read Side (AR + R) 448
24.3.4 Bulk-Clear Control 449
24.3.5 Observation 449
24.4 Functional Description 450
24.4.1 Block diagram 450
24.4.2 FUB Interface Contract 450
24.4.3 Write Path — Burst Tracker 451
24.4.4 Read Path — Burst Tracker 451
24.4.5 Address Generation 451
24.4.6 BRAM Array 451
24.4.7 Bulk-Clear FSM 451
24.4.8 Observation Outputs 452
24.5 Timing Characteristics 453
24.6 Usage Examples 454
24.7 Design Notes 456
24.7.1 No Reset on the RAM Array 456

24.7.2 One Wire Format, Four Wrappers....456
24.7.3 WRAP Bursts....456
24.7.4 Single Outstanding Burst Per Side....456
24.8 Related Modules....457
24.8.1 Used By....457
24.8.2 Uses....457
24.8.3 See Also....457
24.9 Testing....458
24.10 References....459
24.10.1 Source Code....459
24.10.2 Documentation....459
24.11 Navigation....460
25 Simple Dual-Port BRAM Slave Family....461
25.1 Overview....462
25.2 Parameters....463
25.3 Ports....464
25.3.1 Debug / Observation Outputs....464
25.4 Functional Description....465
25.4.1 Why Four Wrappers + a Backend?....465
25.4.2 Burst Support....465
25.4.3 Bulk Clear....466
25.5 Usage Examples....467
25.5.1 Migration....467
25.6 Related Modules....468

25.6.1 Use in Ocelus....468
25.6.2 Family Relationships....468
25.7 Testing....469
25.8 References....470
25.9 Navigation....471
26 SDPRAM Slave — AXI4 Write / AXI4 Read....472
26.1 Overview....473
26.1.1 Key Features....473
26.2 Parameters....474
26.3 Ports....475
26.3.1 Clock and Reset....475
26.3.2 AXI4 Slave Write (AW + W + B)....475
26.3.3 AXI4 Slave Read (AR + R)....476
26.3.4 Bulk-Clear Control and Debug....478
26.4 Functional Description....479
26.4.1 Block diagram....479
26.4.2 Structure....479
26.4.3 Field Pass-Through and Tie-Offs....479
26.4.4 Memory Behavior....480
26.4.5 Debug Taps....480
26.4.6 WRAP Assertion....480
26.5 Timing Characteristics....481
26.6 Usage Examples....482
26.7 Design Notes....483

26.7.1 Native AXI4, No Emulation....483
26.7.2 One Backend, Four Wrappers....483
26.7.3 WRAP Support Is Sim-Gated....483
26.8 Related Modules....484
26.8.1 Used By....484
26.8.2 Uses....484
26.8.3 See Also....484
26.9 Testing....485
26.10 References....486
26.10.1 Source Code....486
26.10.2 Documentation....486
26.11 Navigation....487
27 SDPRAM Slave — AXI4 Write / AXIL Read....488
27.1 Overview....489
27.1.1 Key Features....489
27.2 Parameters....490
27.3 Ports....491
27.3.1 Clock and Reset....491
27.3.2 AXI4 Slave Write (AW + W + B)....491
27.3.3 AXIL Slave Read (AR + R)....492
27.3.4 Bulk-Clear Control and Debug....493
27.4 Functional Description....495
27.4.1 Block diagram....495
27.4.2 Structure....495

27.4.3 AXIL Read Bridge....495
27.4.4 Write Path and Tie-Offs....496
27.4.5 Memory Behavior....496
27.4.6 WRAP Assertion....496
27.5 Timing Characteristics....497
27.6 Usage Examples....498
27.7 Design Notes....499
27.7.1 Single-Beat Adaptation Lives in One Place....499
27.7.2 One Backend, Four Wrappers....499
27.7.3 Read Beat Size....499
27.8 Related Modules....500
27.8.1 Used By....500
27.8.2 Uses....500
27.8.3 See Also....500
27.9 Testing....501
27.10 References....502
27.10.1 Source Code....502
27.10.2 Documentation....502
27.11 Navigation....503
28 SDPRAM Slave — AXIL Write / AXI4 Read....504
28.1 Overview....505
28.1.1 Key Features....505
28.2 Parameters....506
28.3 Ports....507

28.3.1 Clock and Reset....507
28.3.2 AXIL Slave Write (AW + W + B)....507
28.3.3 AXI4 Slave Read (AR + R)....508
28.3.4 Bulk-Clear Control and Debug....509
28.4 Functional Description....511
28.4.1 Block diagram....511
28.4.2 Structure....511
28.4.3 AXIL Write Bridge....511
28.4.4 Read Path and Tie-Offs....512
28.4.5 Memory Behavior....512
28.4.6 WRAP Assertion....512
28.5 Timing Characteristics....513
28.6 Usage Examples....514
28.7 Design Notes....515
28.7.1 Descriptor-RAM Fit....515
28.7.2 Single-Beat Adaptation Lives in One Place....515
28.7.3 One Backend, Four Wrappers....515
28.8 Related Modules....516
28.8.1 Used By....516
28.8.2 Uses....516
28.8.3 See Also....516
28.9 Testing....517
28.10 References....518
28.10.1 Source Code....518

28.10.2 Documentation....518
28.11 Navigation....519
29 SDPRAM Slave — AXIL Write / AXIL Read....520
29.1 Overview....521
29.1.1 Key Features....521
29.2 Parameters....522
29.3 Ports....523
29.3.1 Clock and Reset....523
29.3.2 AXIL Slave Write (AW + W + B)....523
29.3.3 AXIL Slave Read (AR + R)....524
29.3.4 Bulk-Clear Control and Debug....524
29.4 Functional Description....526
29.4.1 Block diagram....526
29.4.2 Structure....526
29.4.3 Single-Beat Defaults on Both Sides....526
29.4.4 Memory Behavior....527
29.4.5 No WRAP Assertion....527
29.5 Timing Characteristics....528
29.6 Usage Examples....529
29.7 Design Notes....530
29.7.1 The Only “Fake” Fields, In One Place Per Side....530
29.7.2 Monitor-Bus Pairing....530
29.7.3 One Backend, Four Wrappers....530
29.8 Related Modules....531

29.8.1 Used By....531
29.8.2 Uses....531
29.8.3 See Also....531
29.9 Testing....532
29.10 References....533
29.10.1 Source Code....533
29.10.2 Documentation....533
29.11 Navigation....534
30 AMBA Protocol Shims....535
30.1 Overview....536
30.1.1 Key Features....536
30.1.2 Available Shims....536
30.2 Parameters....537
30.2.1 AXI to APB Bridge....537
30.2.2 PeakRDL Adapter....538
30.3 Timing....539
30.3.1 AXI4 to APB Bridge....539
30.3.2 PeakRDL Adapter....539
30.4 Usage Example....540
30.4.1 Quick Start: AXI4 to APB Bridge....540
30.4.2 Quick Start: PeakRDL Register Integration....540
30.4.3 Design Patterns....541
30.4.4 Common Use Cases....542
30.5 Design Notes....544

30.5.1 Design Tradeoffs....544
30.5.2 When to Use Shims....544
30.5.3 Clock Domain Crossing....544
30.5.4 Synthesis Notes....544
30.5.5 Migration Guide....545
30.5.6 Troubleshooting....546
30.6 Related Modules....547
30.7 Testing....548
30.8 References....549
30.8.1 Protocol Specifications....549
30.8.2 External Resources....549
30.8.3 Source Code....549
30.9 Navigation....551

1 GAXI (Generic AXI) Modules

Location: `rtl/amba/gaxi/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

GAXI is a lightweight valid/ready handshake protocol for streaming data between components. It's simpler than full AXI4-Stream but keeps the flow control that matters, which is why it shows up all over the RTL Design Sherpa infrastructure — elastic buffering, clock domain crossing, and timing closure all lean on these modules.

1.1.1 Key Features

- **Simple Handshake:** Valid/ready only - minimal overhead
- **Elastic Buffering:** Skid buffers absorb backpressure without stalling the producer
- **Clock Domain Crossing:** Async variants with proper CDC
- **Type-Safe Structs:** Struct-aware buffers for complex data
- **Drop Capability:** Special FIFO variant with drop-by-count/drop-all
- **Flexible Modes:** Mux/flop output modes for area/speed trade-offs

1.2 Modules

1.2.1 Skid Buffers (Elastic Buffering)

Module	Description	Documentation	Status
gaxi_skid_buffer	Synchronous elastic buffer, registered output	gaxi_skid_buffer.md	Documented
gaxi_skid_buffer_async	Asynchronous elastic buffer for CDC	gaxi_skid_buffer_async.md	Documented
gaxi_skid_buffer_dbldrn	Double-drain skid buffer variant	gaxi_skid_buffer_dbldrn.md	Documented

1.2.2 FIFOs (Deep Buffering)

Module	Description	Documentation	Status
gaxi_fifo_sync	Synchronous FIFO with mux/flop modes	gaxi_fifo_sync.md	Documented
gaxi_fifo_async	Asynchronous FIFO for clock domain crossing	gaxi_fifo_async.md	Documented
gaxi_drop_fifo_sync	FIFO with drop-by-count and drop-all capability	gaxi_drop_fifo_sync.md	Documented

1.2.3 Register Slices (Timing Isolation)

Module	Description	Documentation	Status
gaxi_regslice	Single-entry registered slice for breaking timing paths	gaxi_regslice.md	Documented

1.2.4 Test-Code Multi-Instance Helpers

`rtl/amba/testcode/` holds thin multi-instance wrappers used by the DV harnesses — each one instantiates a bank of the base GAXI primitive (and, for the `_sigmap` variant, presents a flattened array-signal port style). They're not documented individually; see the base module doc for the full behavior:

Test-code wrapper	Base module	Full documentation

1.3 Protocol

1.3.1 Interface Signals

Write Interface (Producer): - `wr_valid` - Data valid from source - `wr_ready` - Ready to accept data (backpressure) - `wr_data[N:0]` - Data from source

Read Interface (Consumer): - `rd_valid` - Data valid to sink - `rd_ready` - Ready to accept data from sink - `rd_data[N:0]` - Data to sink

Optional Monitoring: - `count` - Current occupancy. The skid buffers declare `[3:0]`; the FIFOs declare `[AW:0]` where `AW = $clog2(DEPTH)`, so a DEPTH=64 FIFO drives a 7-bit count.

1.3.2 Handshake Rules

1. **Transfer:** Occurs when `valid && ready` on rising clock edge
2. **Data Stability:** `data` must be stable when `valid` is high
3. **Backpressure:** `ready` may deassert to apply backpressure
4. **Valid Persistence:** `valid` must remain high until transfer completes

1.4 Usage Examples

1.4.1 Basic Skid Buffer

```
gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)           // 4 entries
) u_skid (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),

    // Write port (from producer)
    .wr_valid    (src_valid),
    .wr_ready    (src_ready),
    .wr_data     (src_data),

    // Read port (to consumer)
    .rd_valid    (dst_valid),
    .rd_ready    (dst_ready),
    .rd_data     (dst_data),

    // Status
    .count       (buf_count),
    .rd_count    ()
);
```

1.4.2 Synchronous FIFO (Deep Buffering)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(64),         // 64 entries
    .REGISTERED(0)      // 0=mux mode (fast), 1=flop mode (more area)
) u_fifo (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),

    .wr_valid    (wr_valid),
    .wr_ready    (wr_ready),
    .wr_data     (wr_data),

    .rd_valid    (rd_valid),
    .rd_ready    (rd_ready),
    .rd_data     (rd_data),

    .count       (fifo_level)
);
```

1.4.3 Asynchronous FIFO (Clock Domain Crossing)

Two depth rules in this directory, and they are not a contradiction. `gaxi_skid_buffer_dbldrn`) accept `DEPTH` in 2..8 inclusive (any integer). The first two store entries in a small unpacked array; `gaxi_skid_buffer_dbldrn` still uses the flat packed vector with dynamic part-selects the base module was refactored away from (the style its header blames for missing 100 MHz at wide data) – relevant when reasoning about timing at width. Among the FIFOs the rule differs per module: `gaxi_drop_fifo_sync` needs a power of 2, because its drop path adds `drop_count` to the read pointer and wraps at `2*MAX`, and the low `AW` bits only name the right entry when `DEPTH` is a power of 2. `gaxi_fifo_sync` increments by one through `counter_bin`, which wraps the low bits at `MAX-1` and toggles the MSB, so it accepts any `DEPTH <= 2**AW` – a power of 2 is optimal, not required. `gaxi_fifo_async` needs a power of 2 except with `USE_JOHNSON=1`, where Johnson pointers allow any depth (odd included). Check the module you are instantiating, not the directory.

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),          // Must be power of 2
    .N_FLOP_CROSS(3),    // CDC synchronizer stages
    .REGISTERED(0)
) u_async_fifo (
    // Write domain
    .axi_wr_aclk    (wr_clk),
    .axi_wr_aresetn (wr_rst_n),
    .wr_valid       (wr_valid),
    .wr_ready       (wr_ready),
    .wr_data        (wr_data),

    // Read domain
    .axi_rd_aclk    (rd_clk),
    .axi_rd_aresetn (rd_rst_n),
    .rd_valid       (rd_valid),
    .rd_ready       (rd_ready),
    .rd_data        (rd_data)
);
```

1.4.4 Type-Safe Struct Buffer

```
// Define AXI AR channel struct
typedef struct packed {
    logic [7:0]  arid;
```

```
    logic [31:0] araddr;
    logic [7:0]  arlen;
    logic [2:0]  arsize;
    logic [1:0]  arburst;
} axi_ar_t;

// Struct-aware buffer
    .STRUCT_TYPE(axi_ar_t),
    .DEPTH(4)
) u_ar_buf (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),

    .wr_valid    (master_arvalid),
    .wr_ready    (master_arready),
    .wr_data     (master_ar),      // Type-safe struct

    .rd_valid    (slave_arvalid),
    .rd_ready    (slave_arready),
    .rd_data     (slave_ar),       // Type-safe struct

    .count       (ar_count)
);
```

1.4.5 Drop FIFO (Packet Processing)

```
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),
    .DEPTH(64),
    .REGISTERED(1)      // Flop mode for 512-bit width
) u_drop_fifo (
    .axi_aclk        (clk),
    .axi_aresetn     (rst_n),

    .wr_valid        (pkt_valid),
    .wr_ready        (pkt_ready),
    .wr_data         (pkt_data),

    .rd_valid        (proc_valid),
    .rd_ready        (proc_ready),
    .rd_data         (proc_data),

    // Drop control
    .drop_valid      (drop_trigger),    // Request a drop (handshakes
with drop_ready)
    .drop_ready      (drop_ack),
    .drop_count      (drop_n_packets),  // Drop N oldest entries; must
```

```
be <= count
    .drop_all        (flush_fifo),      // Flush entire FIFO

    .count           (pkt_count)
);
```

1.5 Design Patterns

1.5.1 Pattern 1: Timing Closure (Pipeline Stage)

Use a skid buffer to break a combinatorial path that's too long to close:

```
// Without buffer: long combinatorial path
assign downstream_data = upstream_data;  // May not meet timing

// With skid buffer: registered boundary
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(2)) u_pipeline (
    .wr_valid(upstream_valid),
    .wr_ready(upstream_ready),
    .wr_data(upstream_data),
    .rd_valid(downstream_valid),
    .rd_ready(downstream_ready),
    .rd_data(downstream_data),
    ...
);
```

1.5.2 Pattern 2: Rate Adaptation

Use a FIFO to soak up burst traffic:

```
// Bursty writer, continuous reader
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(64)) u_rate_adapter (
    .wr_valid(bursty_valid),     // Intermittent bursts
    .wr_ready(bursty_ready),
    .wr_data(bursty_data),
    .rd_valid(continuous_valid),  // Always reading
    .rd_ready(continuous_ready),
    .rd_data(continuous_data),
    ...
);
```

1.5.3 Pattern 3: Clock Domain Crossing

Use an async FIFO for safe CDC:

```
// Domain A → Domain B
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3)      // 3-stage synchronizer
) u_cdc (
    .axi_wr_aclk(clk_a),
    .axi_wr_aresetn(rst_a_n),
    .wr_valid(domain_a_valid),
    .wr_ready(domain_a_ready),
```

```
    .wr_data(domain_a_data),
    .axi_rd_aclk(clk_b),
    .axi_rd_aresetn(rst_b_n),
    .rd_valid(domain_b_valid),
    .rd_ready(domain_b_ready),
    .rd_data(domain_b_data)
);
```

1.6 Performance Characteristics

1.6.1 Latency

Module	Empty Latency	Full Latency	Notes
gaxi_skid_buffer	1 cycle	1 cycle	Registered storage and valid
gaxi_fifo_sync (mux)	1 cycle	1 cycle	Combinatorial read
gaxi_fifo_sync (flop)	2 cycles	2 cycles	Registered read
gaxi_fifo_async	3-5 cycles	3-5 cycles	CDC synchronization overhead
gaxi_drop_fifo_sync (mux)	1 cycle	1 cycle	Plus 3 cycles for drop operation
gaxi_drop_fifo_sync (flop)	2 cycles	2 cycles	Registered read; the README's own wide-data examples use REGISTERED=1

1.6.2 Throughput

All modules support **1 transfer per clock cycle** when not applying backpressure.

1.6.3 Resource Utilization (Typical)

Module	Logic	Memory	Notes

1.7 Parameter Selection Guidelines

1.7.1 Buffer Depth (DEPTH)

Application	Recommended Depth	Rationale
Pipeline stage	2-4	Minimal latency, timing closure
Video line buffer	16-32	Line-sized bursts
Network packet buffer	32-64	Variable packet sizes
CDC buffer	8-16	Handle clock ratio variations

1.7.2 Output Mode (REGISTERED)

Mux Mode (REGISTERED=0): - Lower latency (1 cycle) - Smaller area - May not meet timing with wide data - **Use for:** Narrow data (≤64 bits), non-critical paths

Flop Mode (REGISTERED=1): - Better timing (registered output) - Supports wide data - Higher latency (2 cycles) - More area - **Use for:** Wide data (≥128 bits), critical paths

1.7.3 CDC Synchronizer Stages (N_FLOP_CROSS)

Stages	Use Case
2	Slow clocks (<100 MHz), low risk
3	**Recommended default**
4+	High-speed (>400 MHz) or high-reliability

1.8 Common Use Cases

1.8.1 Video Processing (Skid Buffer)

Characteristics: - High throughput (pixel streams) - Minimal latency required - Regular flow control

Recommended:

```
gaxi_skid_buffer #(
    .DATA_WIDTH(96),    // 4 pixels x 24-bit RGB
    .DEPTH(4)           // Shallow buffer
)
```

1.8.2 Network Packet Processing (Drop FIFO)

Characteristics: - Variable packet sizes - Need to drop oldest on overflow - High data rate

Recommended:

```
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),   // Wide data bus
    .DEPTH(64),         // Deep buffer
    .REGISTERED(1)      // Flop mode for timing
)
```

1.8.3 Clock Domain Crossing (Async FIFO)

Characteristics: - Independent clock domains - Metastability protection required - Variable clock ratios

Recommended:

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),         // Must be power of 2
    .N_FLOP_CROSS(3)    // 3-stage synchronizer
)
```

1.9 Design Notes

1.9.1 When to Use GAXI vs AXI4-Stream

Use GAXI For: - Internal buffering and timing closure - Simple data streaming - CDC with basic handshake - Infrastructure/glue logic

Use AXI4-Stream For: - External interfaces - Need TID/TDEST/TUSER signals - Complex routing requirements - Standards compliance

1.9.2 Buffer vs FIFO Selection

Use Skid Buffer (gaxi_skid_buffer) When: - Need to absorb short backpressure bursts - Shallow depth (2-8 entries) - Timing closure only - Minimal area

Use FIFO (gaxi_fifo_sync) When: - Need deeper buffering (16+ entries) - Rate adaptation required - Burst traffic handling - More predictable latency

Use Async FIFO (gaxi_fifo_async) When: - Clock domain crossing required - Independent clock domains - Need CDC-safe handshake

Use Drop FIFO (gaxi_drop_fifo_sync) When: - Need to discard oldest data on overflow - Packet-based processing - Flush/reset functionality required

---

1.10 Testing

Every GAXI module is verified with CocoTB-based testbenches:

```
# Run all GAXI tests
pytest val/amba/test_gaxi*.py -v

# Run specific module tests
pytest val/amba/test_gaxi_fifo_sync.py
val/amba/test_gaxi_skid_buffer.py -v      # Skid buffers + sync FIFOs
pytest val/cdc/test_gaxi_buffer_async.py -v     # Async FIFO
pytest val/amba/test_gaxi_drop_fifo_sync.py -v   # Drop FIFO

# Run with waveforms
pytest val/amba/test_gaxi_fifo_sync.py
val/amba/test_gaxi_skid_buffer.py --vcd=waves.vcd -v

# Generate WaveDrom timing diagrams
pytest val/amba/test_gaxi_wavedrom_example.py -v
cd val/amba && bash wd_cmd.sh  # Generate PNG/SVG
```

1.11 Common Issues and Debugging

1.11.1 Issue 1: Data Loss

Symptom: Missing data at output

Debug: 1. Check `count` signal - should not overflow (reach DEPTH) 2. Verify `wr_valid` stays high until `wr_ready` asserts 3. Increase DEPTH if necessary 4. Add backpressure handling upstream

1.11.2 Issue 2: Timing Violation

Symptom: Setup/hold violations in synthesis

Solutions: 1. Use flop mode: `.REGISTERED(1)` 2. Increase DEPTH for better buffering 3. Add pipeline stages with multiple skid buffers

1.11.3 Issue 3: CDC Metastability

Symptom: Incorrect data, simulation/silicon mismatch

Solutions: 1. Increase `N_FLOP_CROSS` (minimum 3 recommended) 2. Verify independent clock domains (no relationship) 3. Check reset synchronization in both domains 4. Use constraints for CDC paths

---

1.12 Related Documentation

1.12.1 Comprehensive Guide

- **GAXI Index (Detailed)** - Comprehensive guide with timing diagrams

1.12.2 Protocol Specifications

- ARM AMBA AXI4-Stream Protocol Specification (similar handshake)

1.12.3 RTL Design Sherpa Documentation

- **rtl-amba Overview** - Complete AMBA subsystem architecture
- **AXI4 Modules** - Full AXI4 protocol
- **AXIS4 Modules** - AXI4-Stream protocol
- **GAXI WaveDrom Tutorial** - Timing diagrams

1.12.4 Source Code

- RTL: `rtl/amba/gaxi/`
- Tests: `val/amba/test_gaxi*.py`
- Framework: `bin/TBClasses/components/gaxi/`

Last Updated: 2025-10-20

1.13 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9):1059–1076, Sept. 2001. DOI: 10.1109/43.945302 — relay stations (Sec. IV-B): the registered stage that lets a valid/ready channel be cut without changing behaviour.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC)*, pp. 657–662, 2006. DOI: 10.1145/1146909.1147077 — Property 3.1 (capacity >= forward + backward latency) is why a buffer with registered `rd_valid` and registered `wr_ready` needs `DEPTH >= 2` for one transfer per clock.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the (AW+1)-bit pointer scheme whose wrap bit separates full from empty, implemented here by `counter_bin` + `fifo_control` (indexed in `rtl/common/references/README.md`).
- Indexed in `rtl/amba/references/README.md`

1.14 Navigation

- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

2 gaxi_drop_fifo_sync

Module: `gaxi_drop_fifo_sync.sv` **Location:** `rtl/amba/gaxi/` **Status:** Production Ready

2.1 Overview

A synchronous GAXI FIFO with a trick most FIFOs don't have: a drop interface that removes entries without reading them. You get standard valid/ready handshaking on both the write and read sides, plus the ability to drop the N oldest entries or flush the whole buffer — the operation you want when stale packets are worse than no packets.

2.1.1 Key Features

- **Standard Handshake:** GAXI valid/ready on write and read
- **Configurable:** Data width and depth
- **Drop by Count:** Remove N oldest entries
- **Drop All:** Flush entire FIFO
- **I/O Blocking:** Writes and reads pause during drop operations (3-cycle latency)
- **Two Output Modes:** Registered or mux-based
- **Occupancy Output:** FIFO count output (`[AW:0]`); the almost-full/almost-empty flags stay internal

2.2 Parameters

Parameter	Type	Default	Description
MEM_STYLE	fifo_mem_t	FIFO_AUTO	Memory inference hint: FIFO_AUTO, FIFO_SRL (distributed/MLAB), or FIFO_BRAM (block RAM). FIFO_BRAM forces a synchronous memory read, adding a cycle on top of REGISTERED.
DATA_WIDTH	int	4	Width of data bus in bits
DEPTH	int	4	FIFO depth (must be power of 2)
REGISTERED	int	0	0 = mux mode, 1 = registered output
ALMOST_WR_MARGIN	int	1	Almost-full margin (internal only — see below)
ALMOST_RD_MARGIN	int	1	Almost-empty margin (internal only — see below)

The DATA_WIDTH/DEPTH defaults are 4 and 4, not 32 and 16. They are sized for a smoke test, so set both explicitly for anything real.

2.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	DATA_WIDTH
D	DEPTH

2.3 Ports

2.3.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock signal
axi_aresetn	input	1	Active-low asynchronous reset

2.3.2 Write Interface

Port	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	FIFO ready to accept write
wr_data	input	DATA_WIDTH	Write data

Handshake: Write occurs when `wr_valid && wr_ready` on rising edge of clock.

2.3.3 Read Interface

Port	Direction	Width	Description
rd_valid	output	1	Read data valid
rd_ready	input	1	Downstream ready to accept read
rd_data	output	DATA_WIDTH	Read data

Handshake: Read occurs when `rd_valid && rd_ready` on rising edge of clock.

2.3.4 Drop Interface

Port	Direction	Width	Description
drop_valid	input	1	Drop request valid
drop_ready	output	1	Drop operation complete
drop_count	input	$clog2(DEPT	Number of entries to drop.

Port	Direction	Width	Description
		H)+1	5 bits at DEPTH=16, 9 bits at DEPTH=256 — not a fixed 8.
drop_all	input	1	Drop all entries (ignore count)

Handshake: Drop completes when `drop_valid && drop_ready` on rising edge of clock.

Drop Latency: 3 clock cycles from `drop_valid` assertion to `drop_ready` assertion.

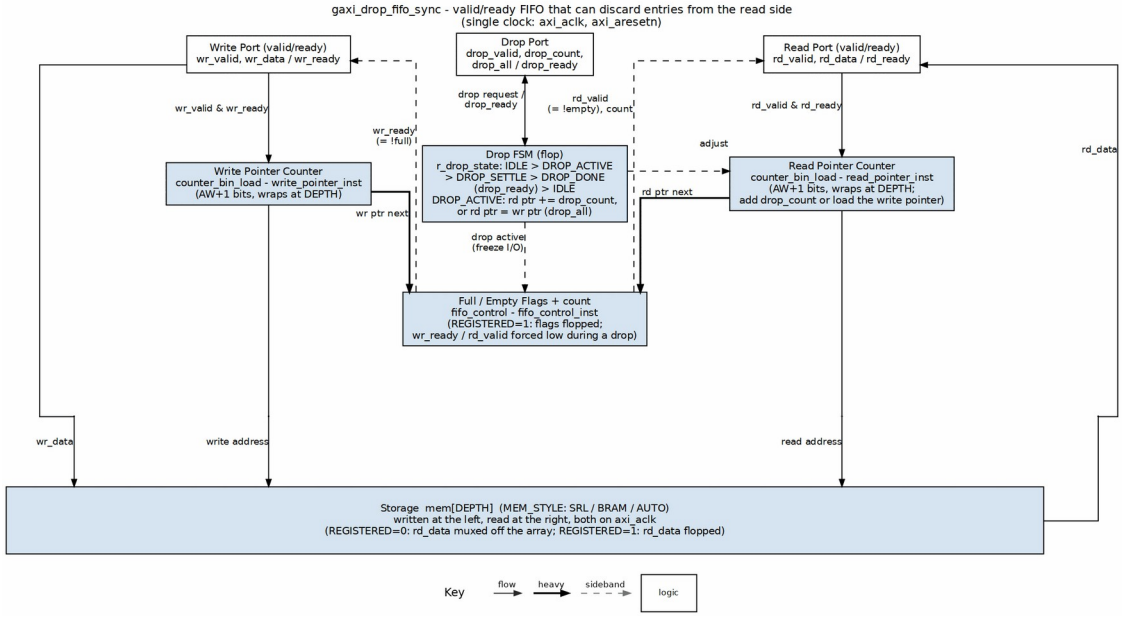
2.3.5 Status

Port	Direction	Width	Description
count	output	$clog2(DEPTH)+1	Current number of entries in FIFO

> **There are no `almost_full` / `almost_empty` ports.** The module's port list ends at `drop_all`. `fifo_control` computes both flags, but they land on internal wires (`r_wr_almost_full`, `r_rd_almost_empty`) that are not brought out, so `ALMOST_WR_MARGIN` and `ALMOST_RD_MARGIN` have no observable effect. Use `count` against your own threshold instead.

2.4 Functional Description

2.4.1 Block diagram



gaxi_drop_fifo_sync block diagram

The `gaxi_fifo_sync` structure with a drop port in the center column. A drop request starts the 4-state drop FSM; in `DROP_ACTIVE` it advances the read pointer by `drop_count` (or loads it with the write pointer for `drop_all`), and while the drop is in progress `wr_ready` and `rd_valid` are forced low. `drop_ready` returns once the pointers have settled.

2.4.2 Drop by Count

When `drop_valid=1` and `drop_all=0`, the FIFO removes `drop_count` oldest entries:

1. **Cycle 0**: Assert `drop_valid`, set `drop_count=N`
2. **Cycle 1-2**: Drop operation in progress, `drop_ready=0`, I/O blocked
3. **Cycle 3**: Drop complete, `drop_ready=1`
4. **Result**: the read pointer advances by exactly `N`

> **`drop_count` must not exceed `count`.** There is no clamp in hardware — `counter_bin_load` adds `drop_count` to the read pointer unconditionally. Dropping 5 from a FIFO holding 3 (DEPTH=16) leaves `rd_ptr=5` ahead of `wr_ptr=3`, and `fifo_control` then computes a count of 30 with `rd_empty` still low: the FIFO reports 30 entries of garbage and hands back memory that was never written. Simulation now catches this at the capture edge

with a `$error` (the check the RTL header had always promised); silicon does not — bound `drop_count` on your side.

I/O Blocking: During drop operation (cycles 1-2): - `wr_ready = 0` (writes blocked) - `rd_valid = 0` (reads blocked)

2.4.3 Drop All

When `drop_valid=1` and `drop_all=1`, the FIFO is completely flushed:

1. **Cycle 0**: Assert `drop_valid` and `drop_all=1`
2. **Cycle 1-2**: Drop operation in progress
3. **Cycle 3**: Drop complete, `drop_ready=1`
4. **Result**: FIFO count becomes 0

Note: `drop_count` is ignored when `drop_all=1`.

2.4.4 Simultaneous Operations

Write + **Drop**: Blocked - writes cannot proceed during drop operation.

Read + **Drop**: Blocked - reads cannot proceed during drop operation.

Drop + **Drop**: Not supported - wait for `drop_ready` before issuing next drop.

2.5 Timing Characteristics

2.5.1 Fill FIFO

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario shows writing 4 entries to an empty FIFO: - Write handshakes occur when `wr_valid && wr_ready` - FIFO count increments with each successful write - Data values: 0xA0, 0xA1, 0xA2, 0xA3

> *Timing diagram not yet captured.* The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_fill.json` into `docs/markdown/assets/WAVES/`, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.2 Drop by Count

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario demonstrates dropping 2 oldest entries: - `drop_valid` asserted with `drop_count=2` - 3-cycle latency until `drop_ready` assertion - FIFO count decreases by 2 - Normal I/O blocked during drop

> *Timing diagram not yet captured.* The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_drop_by_count.json` into `docs/markdown/assets/WAVES/`, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.3 Drop All

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario shows flushing the entire FIFO: - `drop_valid` asserted with `drop_all=1` - 3-cycle latency until `drop_ready` assertion - FIFO count becomes 0 - All entries discarded

> *Timing diagram not yet captured.* The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_drop_all.json` into `docs/markdown/assets/WAVES/`, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.4 Comprehensive Scenario

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario demonstrates mixed operations: - Read 1 entry from FIFO - Drop remaining entries with `drop_all=1` - Shows interaction between read and drop interfaces

> *Timing diagram not yet captured.* The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_comprehensive.json` into `docs/markdown/assets/WAVES/`, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.6 Usage Examples

2.6.1 Basic Write/Read

```
// Instantiate FIFO
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .REGISTERED(0)
) u_fifo (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (wr_valid),
    .wr_ready    (wr_ready),
    .wr_data     (wr_data),
    .rd_valid    (rd_valid),
    .rd_ready    (rd_ready),
    .rd_data     (rd_data),
    .drop_valid  (1'b0),        // No drop
    .drop_ready  (),
    .drop_count  (8'h0),
    .drop_all    (1'b0),
    .count       (fifo_count)
);

// Write logic
always_ff @(posedge clk) begin
    if (!rst_n) begin
        wr_valid <= 1'b0;
    end else if (wr_valid && wr_ready) begin
        wr_valid <= 1'b0;  // Deassert after handshake
    end else if (have_data_to_write && (fifo_count < DEPTH - 1)) begin
        wr_valid <= 1'b1;
        wr_data  <= next_data;
    end
end
```

2.6.2 Drop Operation

```
// Drop state machine
typedef enum logic [1:0] {
    IDLE,
    DROP_WAIT,
    DROP_DONE
} drop_state_t;

drop_state_t state;
```

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state      <= IDLE;
        drop_valid <= 1'b0;
    end else begin
        case (state)
            IDLE: begin
                if (trigger_drop_all) begin
                    drop_valid <= 1'b1;
                    drop_all   <= 1'b1;
                    drop_count <= 8'h0;  // Ignored when drop_all=1
                    state      <= DROP_WAIT;
                end else if (trigger_drop_count) begin
                    drop_valid <= 1'b1;
                    drop_all   <= 1'b0;
                    drop_count <= num_to_drop;
                    state      <= DROP_WAIT;
                end
            end

            DROP_WAIT: begin
                if (drop_ready) begin
                    drop_valid <= 1'b0;
                    state      <= DROP_DONE;
                end
            end

            DROP_DONE: begin
                // Single-cycle state
                state <= IDLE;
            end
        endcase
    end
end
```

2.7 Design Notes

2.7.1 FIFO Depth Requirements

MUST be a power of 2 for efficient address pointer implementation.

Valid: 8, 16, 32, 64, 128, 256, ...

Invalid: 10, 15, 20, 100, ...

2.7.2 Registered vs. Mux Mode

Mux Mode (REGISTERED=0): - Lower latency (combinatorial path from FIFO RAM to `rd_data`) - Better for timing-critical paths where output is registered downstream - Simpler implementation

Registered Mode (REGISTERED=1): - Extra register stage on `rd_data` output - Better timing closure for long data paths - Adds 1 cycle of read latency

2.7.3 Drop Latency

The 3-cycle drop latency is inherent to the implementation: 1. **Cycle 1**: Latch drop request 2. **Cycle 2**: Update read pointer 3. **Cycle 3**: Assert drop_ready

This latency ensures clean separation between normal FIFO operations and drop operations.

2.7.4 Almost Full/Empty Margins

`ALMOST_WR_MARGIN` and `ALMOST_RD_MARGIN` are accepted but have NO observable effect on this module – the almost-full/almost-empty flags exist only as internal wires and are never brought out (the port list ends at `drop_all`). Threshold logic belongs on YOUR side: compare `count` against your own watermark. (This section previously gave sizing guidance for margins that can never appear; the note earlier on this page was always the correct one.)

2.8 Related Modules

- `fifo_control.sv` - Core FIFO control logic
- `counter_bin.sv` - Binary counter for address generation
- `counter_bin_load.sv` - Loadable binary counter for drop pointer updates
- `gaxi_drop_fifo_async.sv` - Asynchronous clock domain version (future)

2.9 Testing

Comprehensive verification lives in `val/amba/test_gaxi_drop_fifo_sync.py`:

Test Scenario	Coverage
Basic FIFO Operation	Standard write/read without drops
Drop by Count	Partial FIFO flush with specific count
Drop All	Complete FIFO flush
Drop During I/O	Verify I/O blocking during drop
Wraparound with Drop	Drop across pointer wraparound boundary

Test Configurations: - Data widths: 8, 32, 64 bits - Depths: 8, 16, 64, 256 entries - Modes: Mux (REGISTERED=0) and Flop (REGISTERED=1)

Results: All 8 parameterized test cases pass

2.9.1 Running Tests

```
# Run all drop FIFO tests
pytest val/amba/test_gaxi_drop_fifo_sync.py -v

# Run specific test configuration
pytest
"val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_sync[8-8-0-
minimal-mux]" -v

# Run smoke test (quick verification)
pytest val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_smoke
-v

# Generate waveforms for documentation
env ENABLE_WAVEDROM=1 pytest val/amba/test_gaxi_drop_fifo_wavedrom.py
-v
```

Version: 1.0 **Last Updated:** 2025-10-17

2.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the (AW+1)-bit pointer scheme whose wrap bit separates full from empty, implemented here by `counter_bin` + `fifo_control` (indexed in `rtl/common/references/README.md`).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 3 “FIFO” — textbook treatment (secondary source).
- No paper: the drop/flush operation (advance the read pointer by a count or to the write pointer) is a standard technique.
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`)

2.11 Navigation

- ← Back to GAXI Index
- ← Back to rtl-amba Index

3 gaxi_fifo_sync

Module: `gaxi_fifo_sync.sv` **Location:** `rtl/amba/gaxi/` **Status:** Production Ready

3.1 Overview

Unlike the skid buffer, this module is a true FIFO built on read/write pointers, which is what lets it support larger depths efficiently. Any depth works; a power of 2 is recommended.

3.1.1 Key Features

- **Arbitrary Depth:** Any depth >= 2 (power of 2 optimal; DEPTH=1 does not elaborate – the pointer math needs at least one address bit)
- **Two Read Modes:** Mux mode (combinatorial) or Flop mode (registered)
- **Counter-Based:** Binary counters with wrapping
- **Occupancy Count:** `count` output, `[AW:0]` wide
- **Single Clock Domain:** Synchronous design

3.2 Parameters

Parameter	Type	Default	Description
MEM_STYLE	fifo_mem_t	FIFO_AUTO	Memory inference hint. FIFO_SRL targets distributed RAM / MLAB, FIFO_BRAM targets block RAM and forces a synchronous read regardless of REGISTERED. FIFO_AUTO lets the tool choose.
REGISTERED	int	0	0=mux mode (comb read), 1=flop mode (reg read)
DATA_WIDTH	int	4	Data bus width
DEPTH	int	4	FIFO depth (any value, power-of-2 optimal)
ALMOST_WR_MARGIN	int	1	Almost-full margin — internal only, no port
ALMOST_RD_MARGIN	int	1	Almost-empty margin — internal only, no port

The almost-full/almost-empty flags are computed inside fifo_control but are not brought out of gaxi_fifo_sync, so the two margins have no observable effect. Compare count against your own threshold instead.

3.3 Ports

```
module gaxi_fifo_sync #(
    parameter fifo_mem_t MEM_STYLE = FIFO_AUTO,  // FIFO_AUTO |
FIFO_SRL | FIFO_BRAM
    parameter int REGISTERED = 0,           // 0=mux mode, 1=flop mode
    parameter int DATA_WIDTH = 4,
    parameter int DEPTH = 4,
    parameter int ALMOST_WR_MARGIN = 1,
    parameter int ALMOST_RD_MARGIN = 1,
    parameter int DW = DATA_WIDTH,
    parameter int D  = DEPTH,
    parameter int AW = $clog2(DEPTH)
) (
    input  logic            axi_aclk,
    input  logic            axi_aresetn,
    input  logic            wr_valid,
    output logic            wr_ready,      // not full
    input  logic [DW-1:0]   wr_data,
    input  logic            rd_ready,
    output logic [AW:0]     count,
    output logic            rd_valid,      // not empty
    output logic [DW-1:0]   rd_data
);
```

Port	Direction	Width	Description
axi_aclk	input	1	
axi_aresetn	input	1	
wr_valid	input	1	
wr_ready	output	1	not full
wr_data	input	[DW-1:0]	
rd_ready	input	1	
count	output	[AW:0]	
rd_valid	output	1	not empty
rd_data	output	[DW-1:0]	

3.4 Functional Description

3.4.1 Read Modes

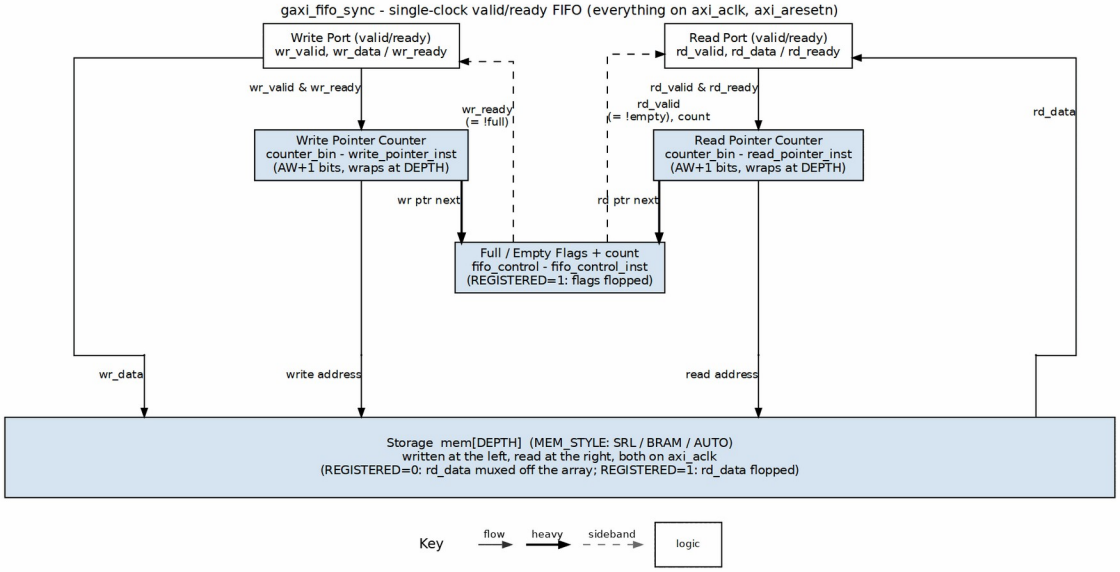
3.4.1.1 Mux Mode (REGISTERED=0)

- **Latency:** 1 cycle write → read
- **Read Path:** Combinatorial from memory
- **Use Case:** Low latency applications
- **Timing:** May create combinatorial path

3.4.1.2 Flop Mode (REGISTERED=1)

- **Latency:** 2 cycles write → read
- **Read Path:** Registered output
- **Use Case:** Timing closure, deep pipelines
- **Timing:** No combinatorial paths

3.4.2 Architecture



gaxi_fifo_sync block diagram

Write data drops down the left into the storage bar and read data comes back up the right. Two `counter_bin` pointers address the storage and both feed one `fifo_control`, which returns `wr_ready` (not full) and `rd_valid` (not empty) as dashed feedback to the ports. It is the valid/ready face of `fifo_sync`, with the same counters and flag block.

3.4.3 Dependencies

- `counter_bin.sv` - Binary counters for read/write pointers
- `fifo_control.sv` - Full/empty flag generation

3.4.4 Resource Utilization

DEPTH	Mode	Flops	LUTs	Memory Bits
16	Mux	16×DW + ~20	~80	16×DW
16	Flop	16×DW + DW + ~20	~80	16×DW
64	Mux	64×DW + ~30	~120	64×DW
64	Flop	64×DW + DW + ~30	~120	64×DW

3.5 Timing Characteristics

Mode	Write→Read Latency	Max Throughput	Read Path
Mux (REGISTERED=0)	1 cycle	1/cycle	Combinatorial
Flop (REGISTERED=1)	2 cycles	1/cycle	Registered

3.6 Usage Examples

3.6.1 Example 1: Mux Mode (Low Latency)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(32),
    .REGISTERED(0)        // Mux mode: 1-cycle latency
) u_low_latency_fifo (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (wr_valid),
    .wr_ready    (wr_ready),
    .wr_data     (wr_data),
    .rd_ready    (rd_ready),
    .rd_valid    (rd_valid),
    .rd_data     (rd_data),  // Combinatorial read
    .count       (fifo_level)
);
```

3.6.2 Example 2: Flop Mode (Timing Closure)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(128),
    .DEPTH(64),
    .REGISTERED(1)        // Flop mode: 2-cycle latency
) u_registered_fifo (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (wr_valid),
    .wr_ready    (wr_ready),
    .wr_data     (wr_data),
    .rd_ready    (rd_ready),
    .rd_valid    (rd_valid),
    .rd_data     (rd_data),  // Registered output
    .count       (fifo_level)
);
```

3.7 Design Notes

Depth need not be a power of two. `fifo_control` computes its flags through `counter_bin`'s MAX wrap, so any `DEPTH <= 2^ADDR_WIDTH` works. The header once claimed DEPTH must EQUAL 2^ADDR_WIDTH, which overstated the constraint and made non-power-of-two depths look illegal (common BUG-004, was COMMON-014).

Almost-full and almost-empty margins are in entries, not percent, and a margin larger than the depth makes the flag unreachable rather than always-asserted.

3.8 Related Modules

- gaxi_skid_buffer - Same 1-cycle latency, shallower elastic storage
- gaxi_fifo_async - Clock domain crossing version
- GAXI Index - Overview of all GAXI modules

3.9 Testing

One testbench covers both output modes:

```
# Test FIFO modes
pytest val/amba/test_gaxi_fifo_sync.py -k "fifo_mux" -v   # Mux mode
pytest val/amba/test_gaxi_fifo_sync.py -k "fifo_flop" -v  # Flop mode
```

Version: 1.0 **Last Updated:** 2025-10-06

3.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the (AW+1)-bit pointer scheme whose wrap bit separates full from empty, implemented here by `counter_bin` + `fifo_control` (indexed in `rtl/common/references/README.md`).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 3 “FIFO” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`)

3.11 Navigation

- ← Back to GAXI Index
- ← Back to rtl-amba Index

4 gaxi_regslice

Module: `gaxi_regslice.sv` **Location:** `rtl/amba/gaxi/` **Status:** Production Ready

4.1 Overview

This is the classic register slice: a 1-deep elastic buffer that exists for one reason — pipeline timing isolation. Data transfer is always registered, so you get a guaranteed, predictable 1-cycle latency and consistent throughput.

4.1.1 Key Features

- **Fixed 1-Cycle Latency:** Always registered, predictable timing
- **Full Throughput:** 1 beat/cycle in steady state
- **Elastic Buffering:** Absorbs single-cycle backpressure
- **Port Compatible:** Intentionally aligned with gaxi_skid_buffer
- **Ready/Valid Handshake:** Industry-standard AXI-like protocol
- **Minimal Resources:** Single register stage

4.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width (arbitrary)
DW	int	DATA_WIDTH	Derived parameter (internal use)

4.3 Ports

```
module gaxi_regslice #(
    parameter int DATA_WIDTH   = 32,
    parameter int DW           = DATA_WIDTH    // Derived
) (
    // Global Clock and Reset
    input  logic          axi_aclk,
    input  logic          axi_aresetn,

    // Write Interface (Input Side)
    input  logic          wr_valid,
    output logic          wr_ready,
    input  logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic          rd_valid,
    input  logic          rd_ready,
    output logic [DW-1:0] rd_data,

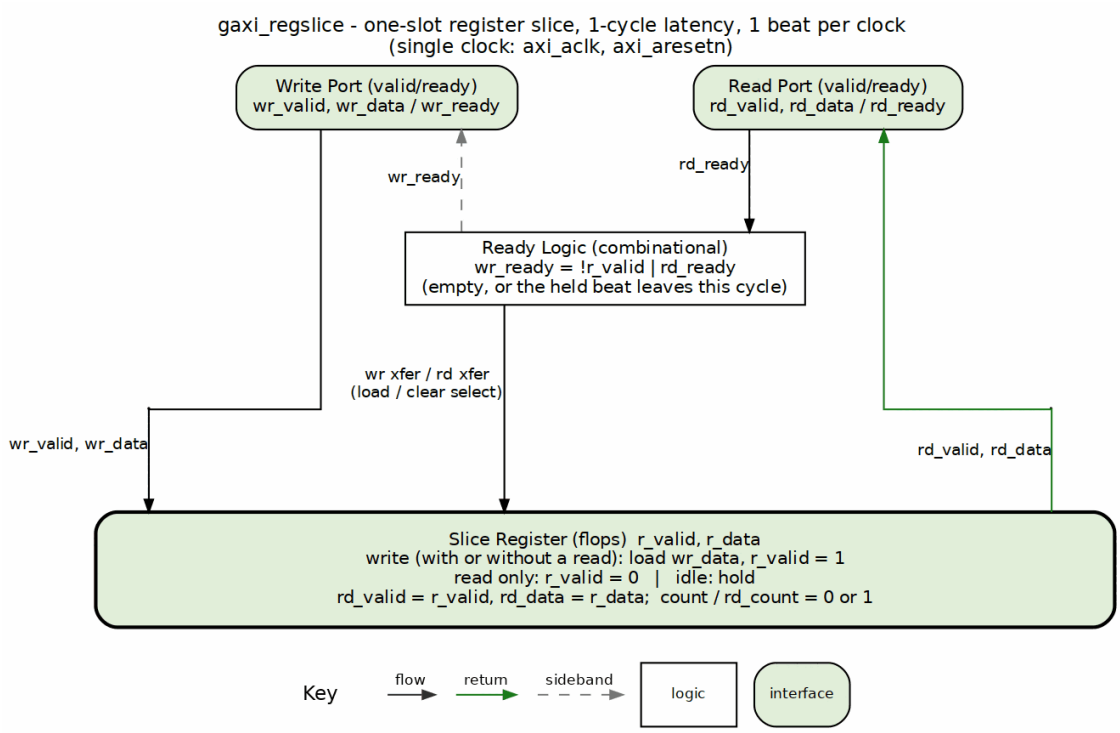
    // Status/Monitoring
    output logic [3:0]    count,      // 0 or 1
    output logic [3:0]    rd_count    // mirror of count
);
```

Port	Direction	Width	Description
axi_aclk	input	1	
axi_aresetn	input	1	
wr_valid	input	1	
wr_ready	output	1	
wr_data	input	[DW-1:0]	
rd_valid	output	1	
rd_ready	input	1	
rd_data	output	[DW-1:0]	
count	output	[3:0]	0 or 1
rd_count	output	[3:0]	mirror of count

4.4 Functional Description

4.4.1 Architecture

The register slice implements a single-entry elastic buffer with simultaneous push/pop capability:



gaxi_regslice block diagram

Write data drops down the left into the one-slot register and read data comes back up the right, one cycle later. Unlike the skid buffers, `wr_ready` is combinational: the slice accepts when it is empty or when the held beat leaves this cycle, which is what sustains one beat per clock.

4.4.2 Ready/Valid Protocol

Write Ready Conditions:

```
wr_ready = (!r_valid) || (r_valid && rd_ready)
```

- Accept when storage is **empty** (r_valid=0)
- Accept when storage is **full** AND downstream **consuming** (simultaneous transfer)

Read Valid:

```
rd_valid = r_valid
```

- Valid when storage is occupied

4.4.3 Transfer Scenarios

Scenario	wr_valid	wr_ready	rd_valid	rd_ready	Action
Idle	0	1 (empty)	0	X	Storage empty, ready to accept
Fill	1	1	0	X	Write data → storage, r_valid=1
Full	0	0	1	0	Storage occupied, downstream not consuming
Drain	0	1 (will empty)	1	1	Read consumes data, r_valid=0
Pass-through	1	1	1	1	Simultaneous write+read, data passes through

4.4.4 Storage Update Logic

```
unique case ({w_wxfer, w_rxfer})
    2'b10: begin  // Write only: fill
        r_valid <= 1'b1;
        r_data  <= wr_data;
    end
    2'b01: begin  // Read only: drain
        r_valid <= 1'b0;
    end
```

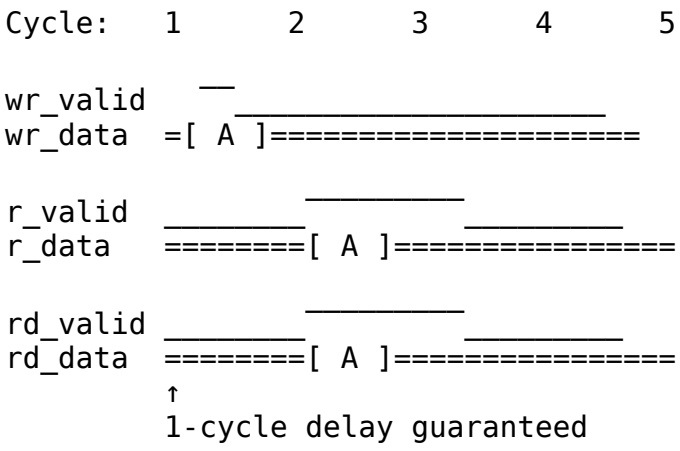
```
    2'b11: begin  // Simultaneous: pass through with register
        r_valid <= 1'b1;
        r_data  <= wr_data;
    end
    default: begin  // Idle: hold state
        r_valid <= r_valid;
        r_data  <= r_data;
    end
endcase
```

4.5 Timing Characteristics

Characteristic	Value	Notes
Write→Read Latency	**1 cycle**	Always registered, fixed latency
Max Throughput	**1 beat/cycle**	Sustained in steady state
Backpressure Absorption	**1 beat**	Single-entry elastic buffer
Read Path	**Registered**	No combinatorial paths
Write Path	**Combinatorial**	wr_ready depends on rd_ready

4.5.1 Latency Guarantee

The register slice introduces exactly 1 cycle of latency. So does `gaxi_skid_buffer` – the two differ in depth and backpressure absorption, not in minimum latency:



4.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
gaxi_regslice #(
    .DATA_WIDTH             (32)
) u_gaxi_regslice (
    .axi_aclk               (axi_aclk),
    .axi_aresetn            (axi_aresetn),
    .wr_valid               (wr_valid),
    .wr_ready               (wr_ready),
    .wr_data                (wr_data),
    .rd_valid               (rd_valid),
    .rd_ready               (rd_ready),
    .rd_data                (rd_data),
    .count                  (count),
    .rd_count               (rd_count)
);
```

4.7 Design Notes

4.7.1 Port Compatibility

Intentional alignment with gaxi_skid_buffer allows drop-in replacement:

```
// Both modules share identical port signatures
gaxi_regslice #(.DATA_WIDTH(64)) u_option1 (...);
gaxi_skid_buffer #(.DATA_WIDTH(64)) u_option2 (...);
```

This enables easy experimentation: - Start with **gaxi_regslice** when a single pipeline break is all you need - Swap to **gaxi_skid_buffer** when the consumer stalls and you want 2-8 entries of backpressure absorption – latency is identical either way

4.7.2 Status Signals

count and rd_count are 4-bit for interface compatibility: - Value 0: Storage empty - Value 1: Storage occupied - Both signals always equal (redundant, but matches skid buffer interface)

4.7.3 Assertions

The module contains exactly one check — an occupancy sanity check on a single-entry slice:

```
$error("[%m] count > 1 (=%0d) @ %0t", count, $time);
```

There are no backpressure or invalid-read checks, and the file carries no `translate_off` / `synopsys` pragmas, so nothing here is synthesis-guarded. Backpressure and handshake legality are checked in verification, by the GAXI BFMs in `val/amba/test_gaxi_regslice.py`.

4.7.4 Resource Utilization

FPGA resource usage, typical:

DATA_WIDTH	Flops	LUTs	Slice Registers
8	9	~6	9
32	33	~12	33
64	65	~18	65
128	129	~24	129

Scaling: Approximately DATA_WIDTH + 1 flops (data + valid flag)

4.8 Related Modules

4.8.1 gaxi_regslice vs gaxi_skid_buffer

Feature	gaxi_regslice	gaxi_skid_buffer
Depth	1 entry	DEPTH entries, 2..8 inclusive (any integer)
Latency	**1 cycle (fixed)**	1 cycle (fixed)
Bypass Path	No	No
Throughput	1 beat/cycle	1 beat/cycle
Use Case	**Timing isolation**	**Backpressure absorption**
Registered Output	Always	Always
Timing Closure	Good	Good

When to Choose: - **gaxi_regslice:** one entry is enough – you only need a pipeline break - **gaxi_skid_buffer:** you also need backpressure absorption (2-8 entries)

Latency does NOT differentiate them: both are fixed 1-cycle with registered outputs (the table above and the RTL agree). Choose on depth.

4.8.2 gaxi_regslice vs gaxi_fifo_sync

Feature	gaxi_regslice	gaxi_fifo_sync
Depth	1 entry (fixed)	Parameterized (N entries)
Latency	1 cycle	1-2 cycles (mode-dependent)
Resources	Minimal (1 register)	Scales with depth
Use Case	Pipeline breaks	Buffering, rate matching

- gaxi_skid_buffer - Same latency, deeper elastic storage
- gaxi_fifo_sync - Multi-entry FIFO version
- gaxi_fifo_async - Clock domain crossing version
- GAXI Index - Overview of all GAXI modules

4.9 Testing

Test File: `val/amba/test_gaxi_regslice.py`

Test Methods: - Simple incremental loops (fill/drain cycles) - Back-to-back transfers (sustained throughput) - Comprehensive randomizer sweep (varied timing patterns) - Stress test with random patterns

Test Levels: - **gate:** Quick smoke test (~30s, 4 loops) - **func:** Moderate coverage (~2min, expanded patterns) - **full:** Comprehensive validation (~5min, 100+ loops)

4.9.1 Running Tests

```
# Basic test (quick validation)
TEST_LEVEL=gate pytest val/amba/test_gaxi_regslice.py -v

# Medium test (normal CI)
TEST_LEVEL=func pytest val/amba/test_gaxi_regslice.py -v

# Full test (pre-release validation)
TEST_LEVEL=full pytest val/amba/test_gaxi_regslice.py -v
```

Version: 1.0 **Last Updated:** 2025-10-23

4.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC)*, pp. 657–662, 2006. DOI: 10.1145/1146909.1147077 — the Lf = 1, Lb = 0 elastic stage: valid/data registered, ready combinational, one entry. The paper notes Lb = 0 is legal locally but leaves the ready path combinational, which is the trade this slice makes.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9):1059–1076, Sept. 2001. DOI: 10.1109/43.945302 — relay stations, the fully registered alternative (`gaxi_skid_buffer`).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 2 “Handshake Protocols” (forward register slice) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`, `LatencyInsensitiveDesign__Carloni2001.pdf`)

4.11 Navigation

- ← Back to GAXI Index
- ← Back to rtl-amba Index

5 gaxi_skid_buffer

Module: `gaxi_skid_buffer.sv` **Location:** `rtl/amba/gaxi/` **Status:** Production Ready

5.1 Overview

The skid buffer is the simplest useful elastic buffer: it decouples a producer from a consumer and absorbs backpressure so neither side has to stall the other. Storage is a shift register and the output is registered, so the minimum latency is one clock.

5.1.1 Key Features

- **Registered Output:** Data appears one clock after an accepted write
- **Elastic Buffering:** Depth 2-8 entries prevent pipeline stalls
- **Single Clock Domain:** Synchronous design for simplicity
- **Parameterized:** Configurable data width and depth
- **Efficient:** Shift register implementation, minimal logic

5.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	2	Buffer depth: 2..8 inclusive (odd depths are legal), or 0 for a combinational pass-through (no register: `wr_ready = rd_ready`, `rd_valid = wr_valid`, `rd_data = wr_data`, counts read 0). Elaboration fails on 1 and above 8. Depth 0 serves the bridge's per-port latency modes.

Derived Parameters: - DW = DATA_WIDTH - BUF_WIDTH = DATA_WIDTH × DEPTH (total buffer storage) - BW = BUF_WIDTH

5.3 Ports

```
module gaxi_skid_buffer #(
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 2,        // 2..8 inclusive, or 0 = pass-
through
    parameter int DW = DATA_WIDTH,
    parameter int BUF_WIDTH = DATA_WIDTH * DEPTH,
    parameter int BW = BUF_WIDTH
) (
    // Global Clock and Reset
    input  logic          axi_aclk,
    input  logic          axi_aresetn,

    // Write Interface (Input Side)
    input  logic          wr_valid,
    output logic          wr_ready,
    input  logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic          rd_valid,
    input  logic          rd_ready,
    output logic [DW-1:0] rd_data,

    // Status/Monitoring
    output logic [3:0]    count,     // Current buffer occupancy
    output logic [3:0]    rd_count   // Same as count (for
compatibility)
);
```

5.3.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset

5.3.2 Write Interface

Port	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	Ready to accept write

Port	Direction	Width	Description
wr_data	input	DATA_WIDTH	Write data

5.3.3 Read Interface

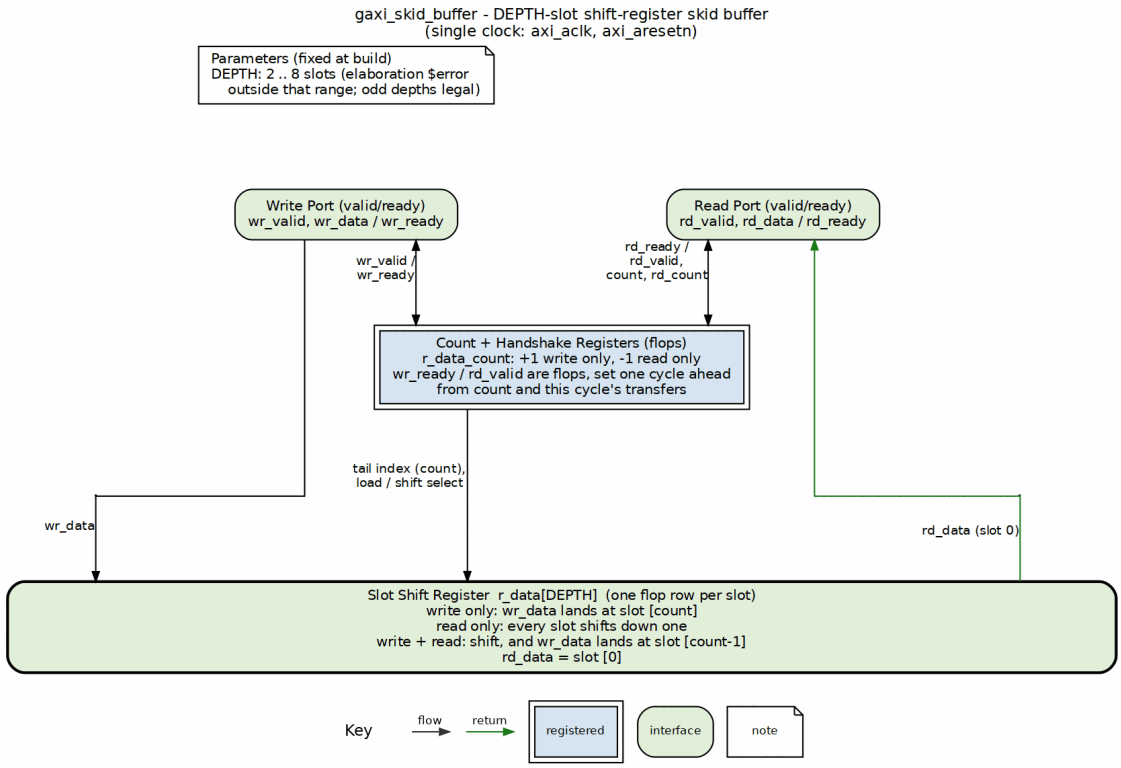
Port	Direction	Width	Description
rd_valid	output	1	Read data valid
rd_ready	input	1	Ready to accept read
rd_data	output	DATA_WIDTH	Read data

5.3.4 Status

Port	Direction	Width	Description
count	output	4	Current buffer occupancy (0 to DEPTH)
rd_count	output	4	Same as count (for compatibility)

5.4 Functional Description

5.4.1 Block diagram



gaxi_skid_buffer block diagram

Write beats drop down the left into the slot shift register; the head slot (slot 0) goes back up the right as `rd_data`. A write lands at the tail (slot `count`), a read shifts every slot down by one, and a simultaneous write and read shifts and loads slot `count-1`. The center block holds the slot count and computes `wr_ready` and `rd_valid` one cycle ahead, so both handshake outputs are registered.

5.4.2 Operation Modes

The skid buffer operates in different modes based on occupancy:

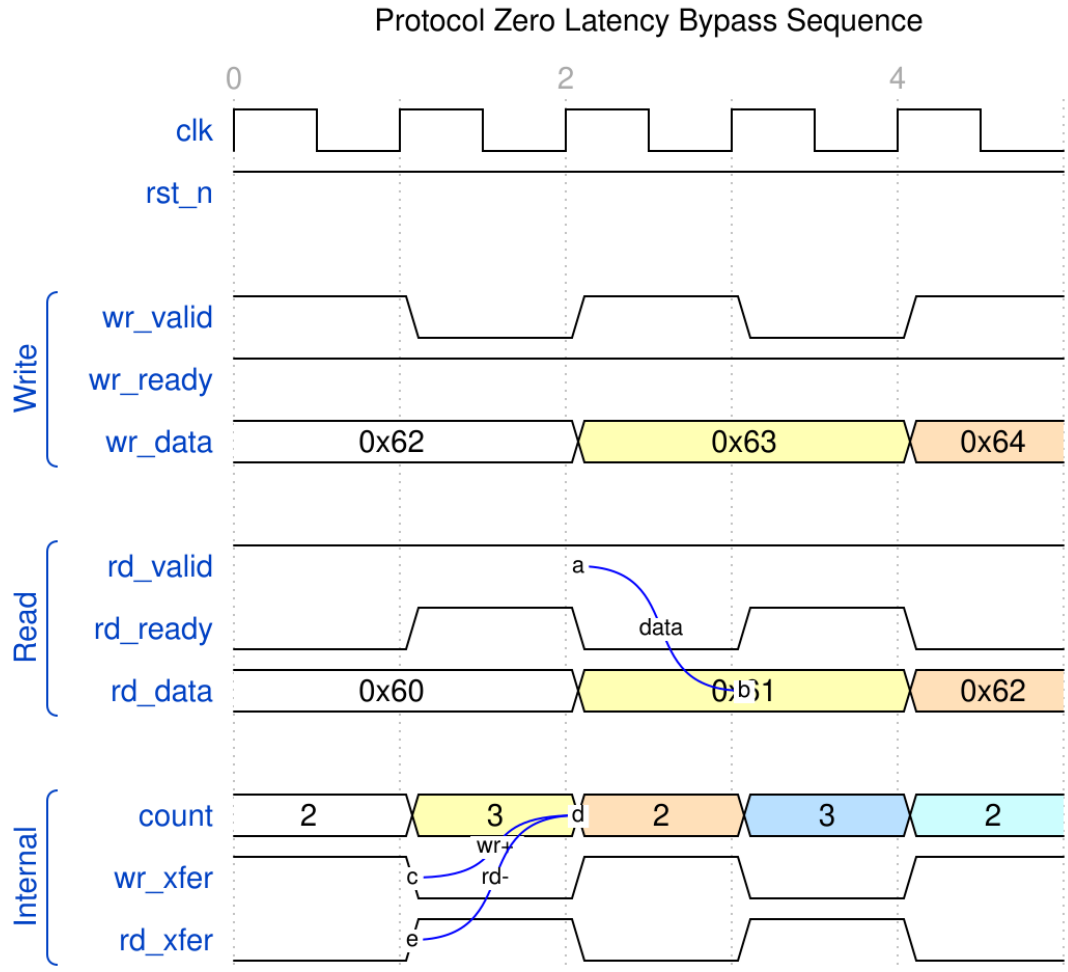
5.4.2.1 1. Empty Mode (count = 0)

One-Cycle Fill: - A write accepted at cycle N appears on `rd_data` at cycle N+1 - `rd_valid` asserts the cycle after `wr_valid` is accepted - There is no combinatorial `wr_data` → `rd_data` path

Both storage and valid are registered:

```
assign rd_data = r_data[0];   // r_data written only inside
ALWAYS_FF_RST
rd_valid <= (r_data_count >= 2) ||
```

```
        (r_data_count == 4'b0001 && (~w_rd_xfer || w_wr_xfer)) ||
        (r_data_count == 4'b0000 && w_wr_xfer);
```



Streaming read/write

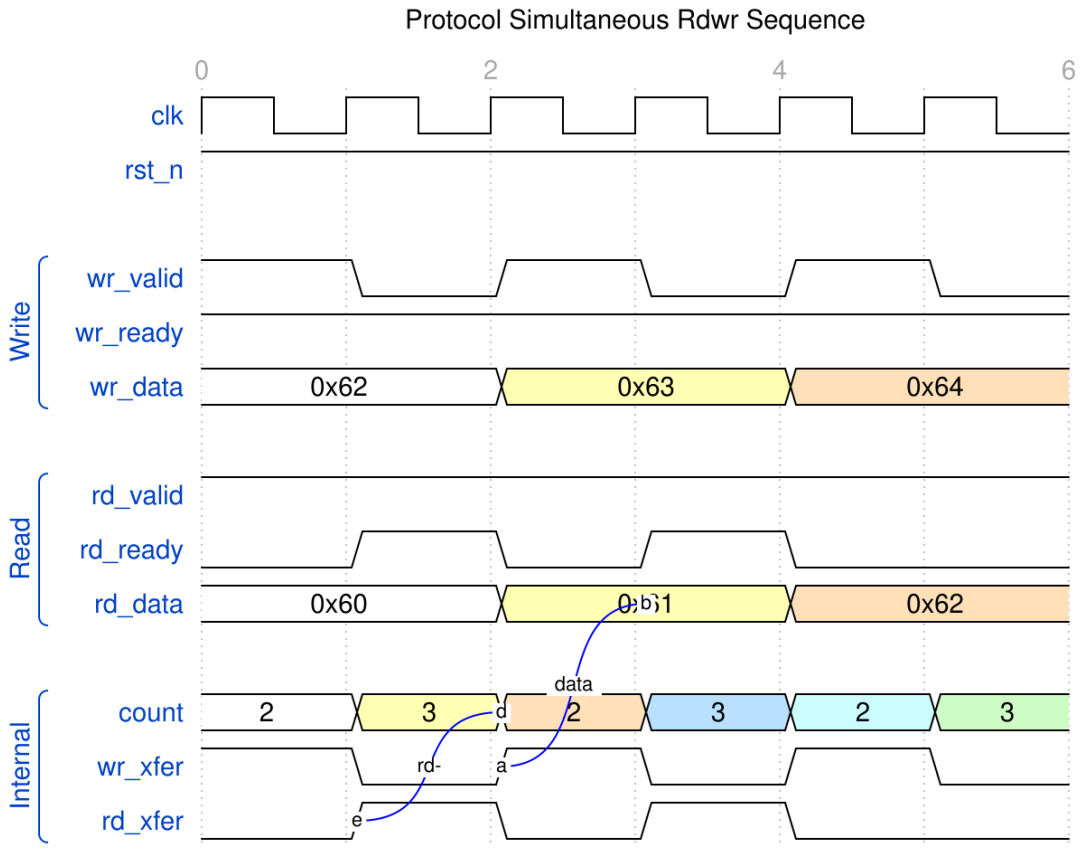
WaveJSON: zero_latency_bypass_001.json

Key Observations: - Write at cycle N → read valid at cycle N+1 - Minimum latency is 1 clock, not 0 - `count` increments when write occurs without simultaneous read

> The waveform file is still named `zero_latency_bypass` from the pre-refactor design and captures a partially-filled buffer streaming, not an empty-buffer bypass. The name is stale; the RTL it was captured from is current.

5.4.2.2 2. Partially Full Mode (0 < count < DEPTH)

Elastic Storage: - Data buffered in shift register - Both write and read can occur simultaneously - `count` tracks occupancy



Simultaneous Read/Write

WaveJSON: simultaneous_rdwr_001.json

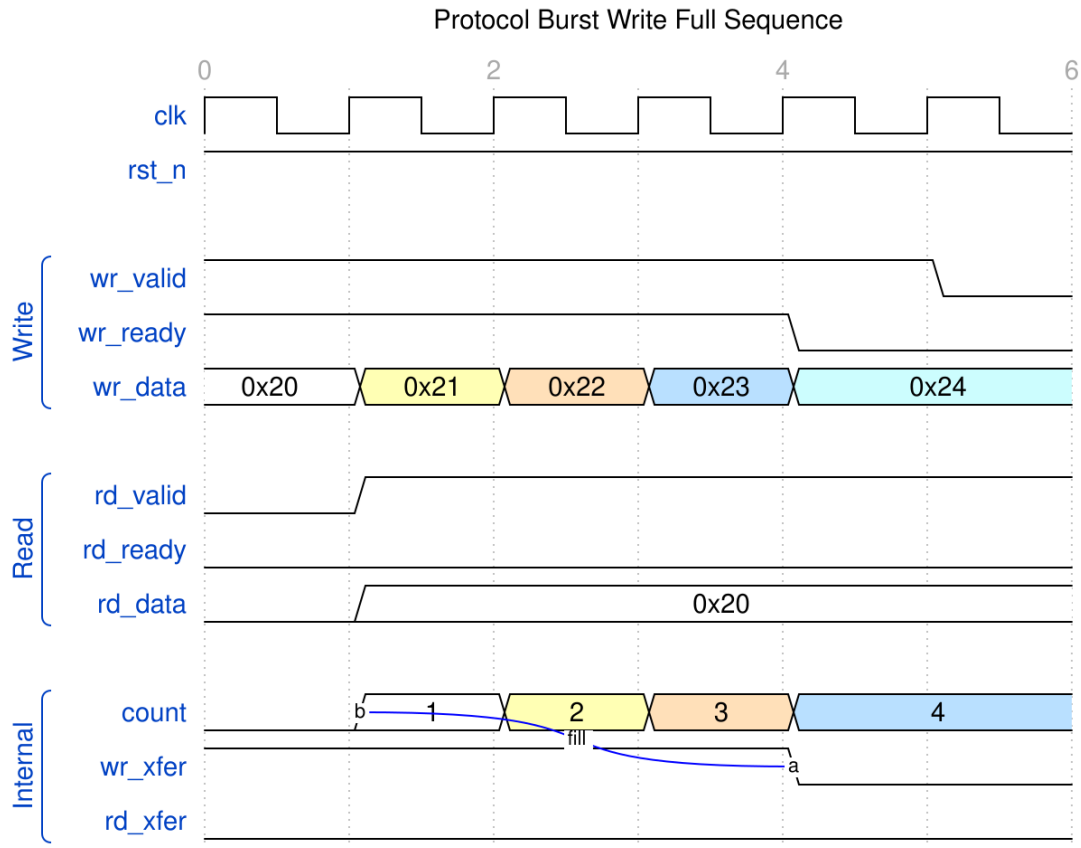
Key Observations: - Simultaneous operations: count stays constant - Write-only: count increments - Read-only: count decrements

5.4.2.3 3. Nearly Full Mode (count = DEPTH - 1)

Smart Backpressure: - `wr_ready` stays high if read occurring - Prevents false stalls

5.4.2.4 4. Full Mode (count = DEPTH)

Backpressure Applied: - `wr_ready` = 0 (cannot accept more writes) - Must read to make space



Burst Write Until Full

WaveJSON: burst_write_full_001.json

Key Observations: - `wr_ready` deasserts when buffer full - Subsequent writes blocked until space available - Reading creates space, `wr_ready` reasserts

5.4.3 Internal State Machine

The module implements a shift register for data storage:

```
// Data storage: shift register
logic [DATA_WIDTH-1:0] r_data [DEPTH];  // unpacked array, shifts on
read
logic [3:0]            r_data_count;  // Current occupancy

// Transfer detection
logic w_wr_xfer = wr_valid & wr_ready;
logic w_rd_xfer = rd_valid & rd_ready;
```

State Transitions:

Condition	wr_xfer	rd_xfer	Action	count Change
Write only	1	0	Shift data in	+1
Read only	0	1	Shift data out	-1
Simultaneous	1	1	Shift data in/out	0
Idle	0	0	No change	0

5.4.4 Ready/Valid Logic

```
// Write ready conditions
wr_ready <= (count <= DEPTH-2) ||
            (count == DEPTH-1 && (~wr_xfer || rd_xfer)) ||
            (count == DEPTH && rd_xfer);

// Read valid conditions
rd_valid <= (count >= 2) ||
            (count == 1 && (~rd_xfer || wr_xfer)) ||
            (count == 0 && wr_xfer);
```

5.5 Timing Characteristics

5.5.1 Latency

Condition	Latency	Cycles
Empty → Read	1	One clock (registered output)
Write → Read (buffered)	1	One clock cycle
Backpressure response	1	One clock after full

5.5.2 Throughput

- **Maximum:** 1 transfer per clock cycle
- **Sustained:** 1 transfer per clock (with proper flow control)

5.5.3 Critical Paths

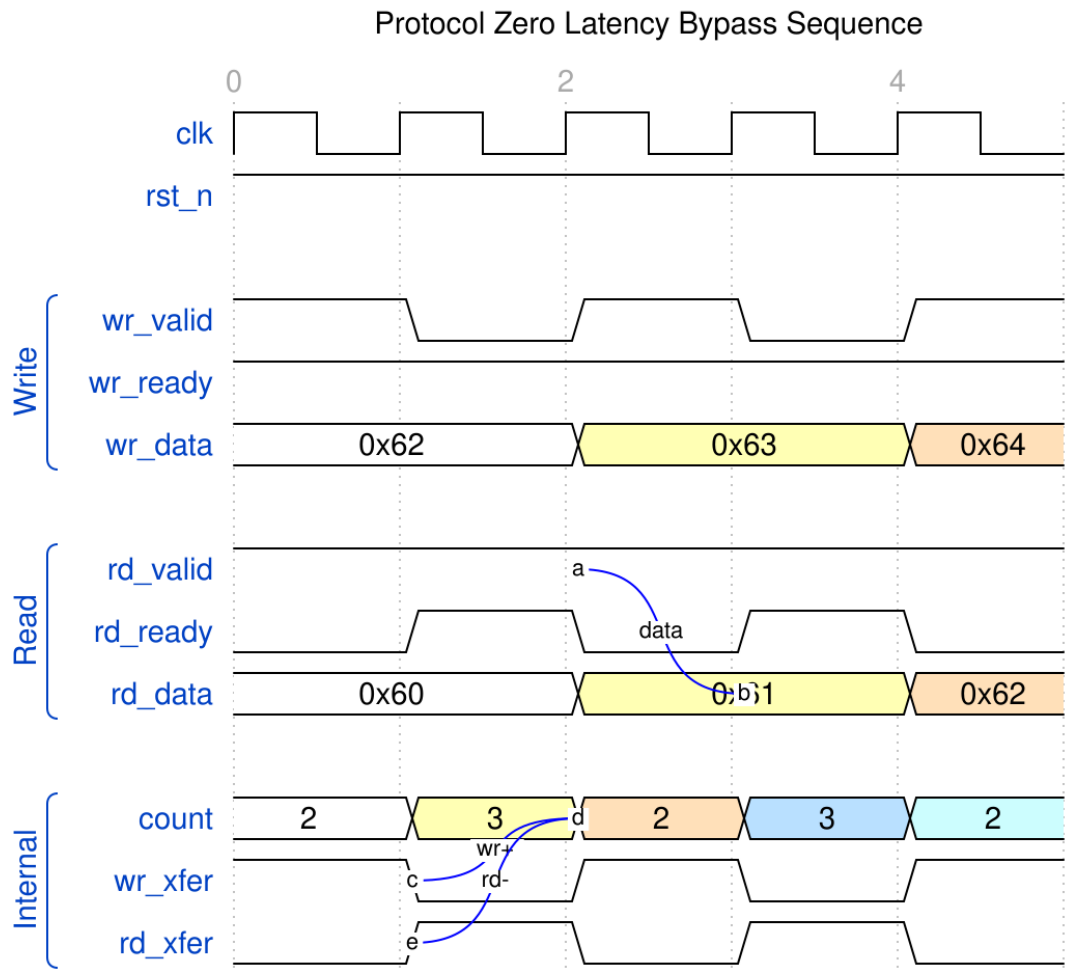
- **Write → Read (empty):** One clock. The path from `r_data[0]` to the consumer is the one to close, not a write-to-read bypass – there is none.
- **Ready/Valid Logic:** Single-cycle registered

5.6 Waveforms

Six captured scenarios cover the interesting corners. Each links its WaveJSON so you can re-render or zoom.

5.6.1 Scenario 1: Streaming Read/Write

A partially-filled buffer streaming, with `rd_data` trailing `wr_data`:

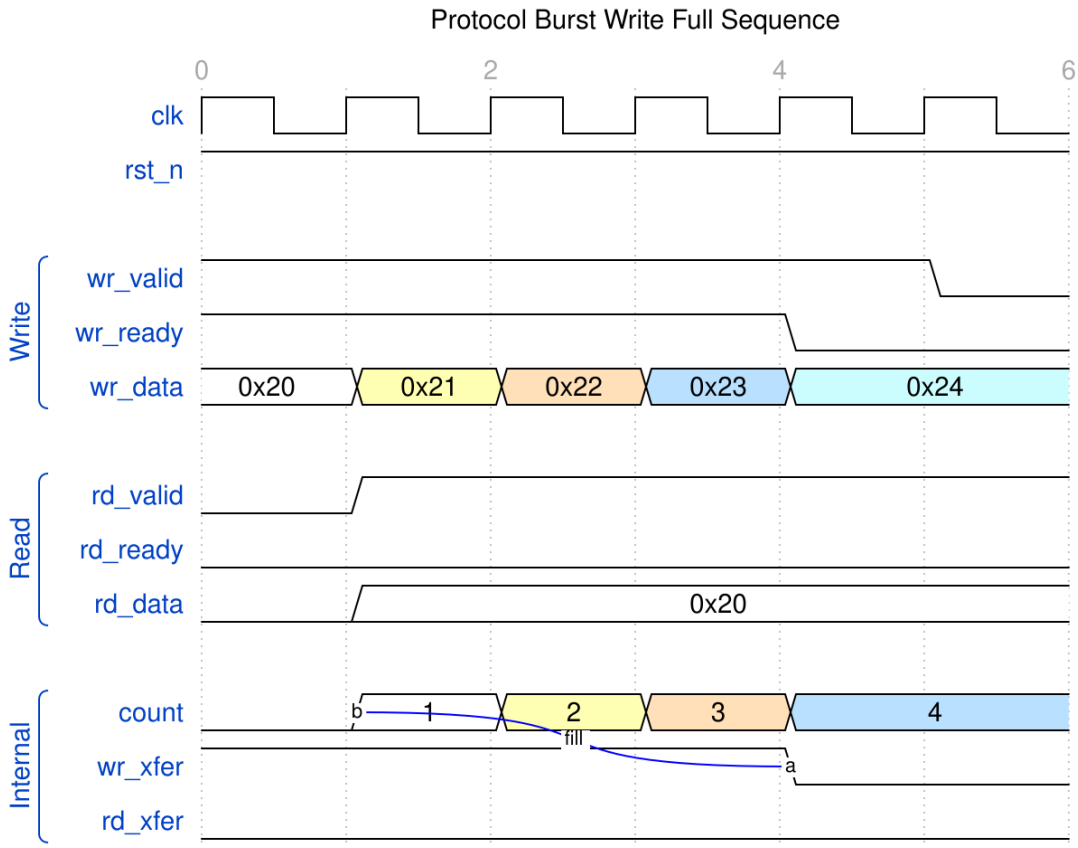


Streaming read/write

WaveJSON: zero_latency_bypass_001.json

5.6.2 Scenario 2: Burst Write Until Full

Demonstrates backpressure behavior:

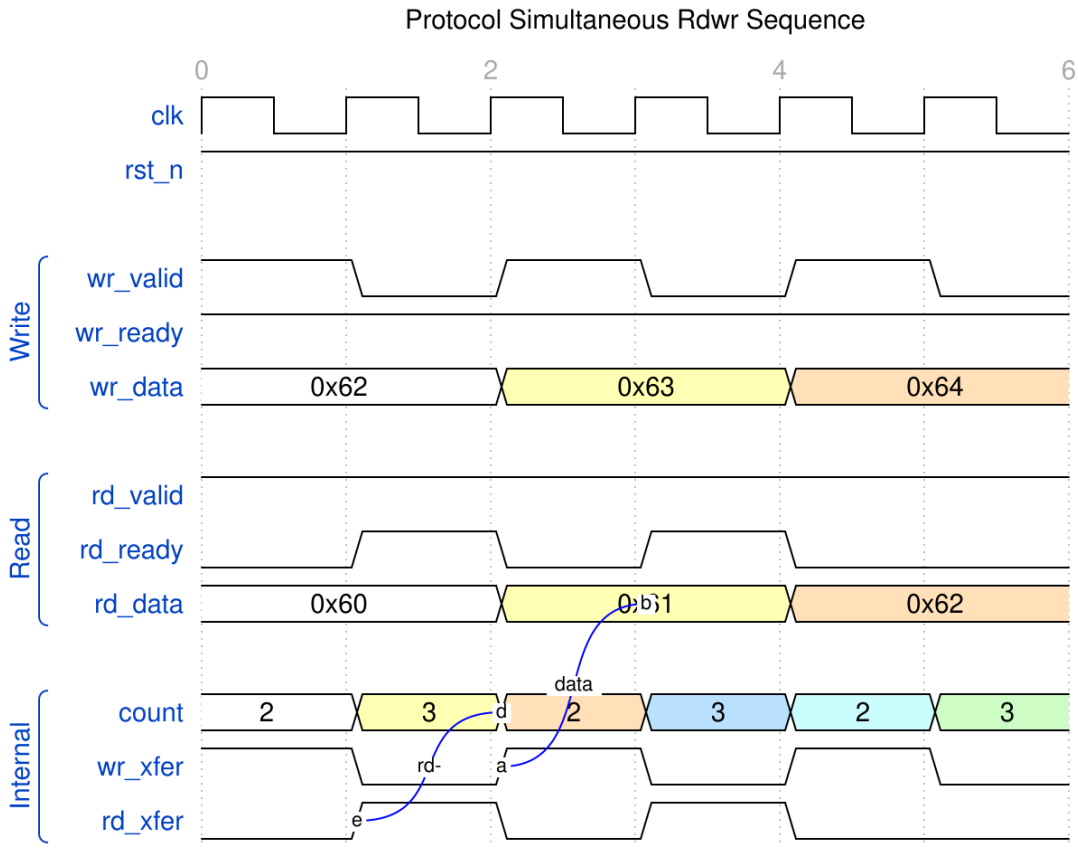


Burst Write Full

WaveJSON: burst_write_full_001.json

5.6.3 Scenario 3: Simultaneous Read/Write

Pass-through operation with constant occupancy:

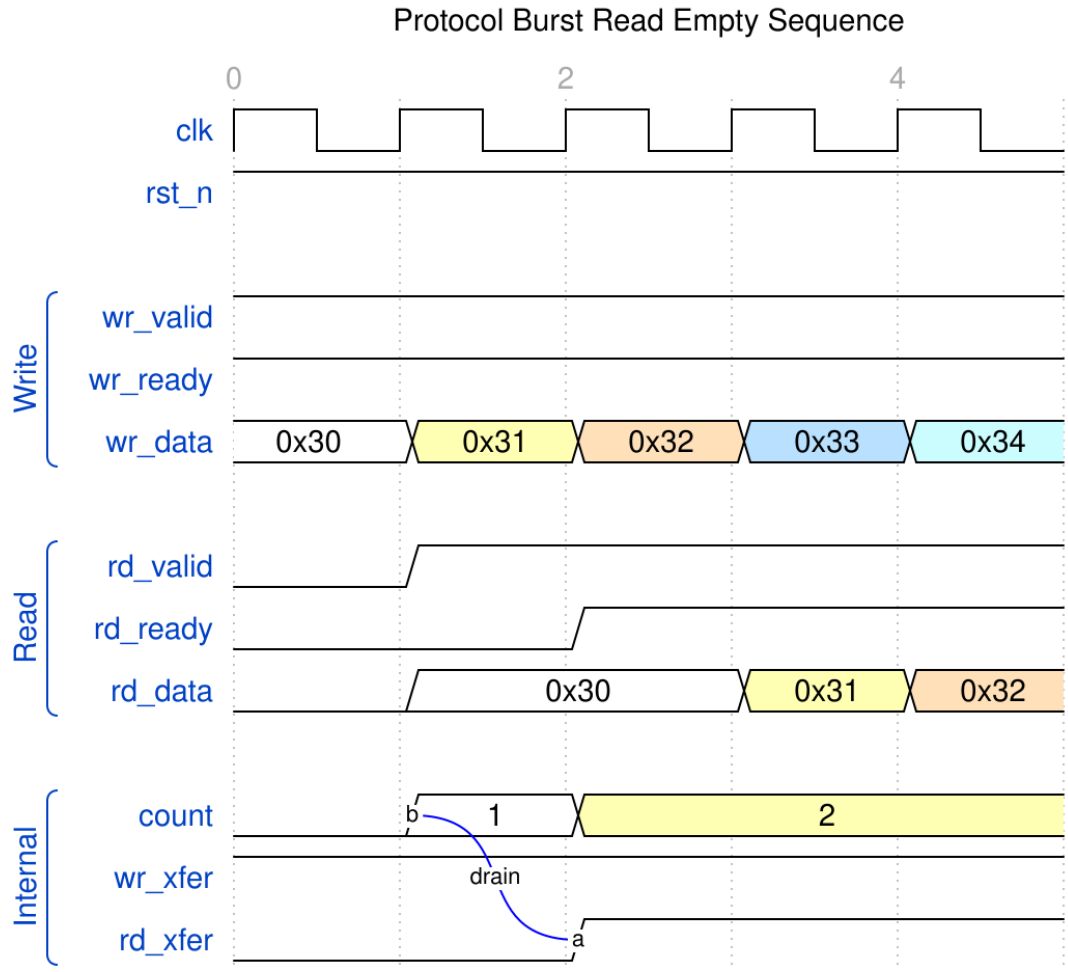


Simultaneous R/W

WaveJSON: simultaneous_rdwr_001.json

5.6.4 Scenario 4: Burst Read Until Empty

Draining the buffer:

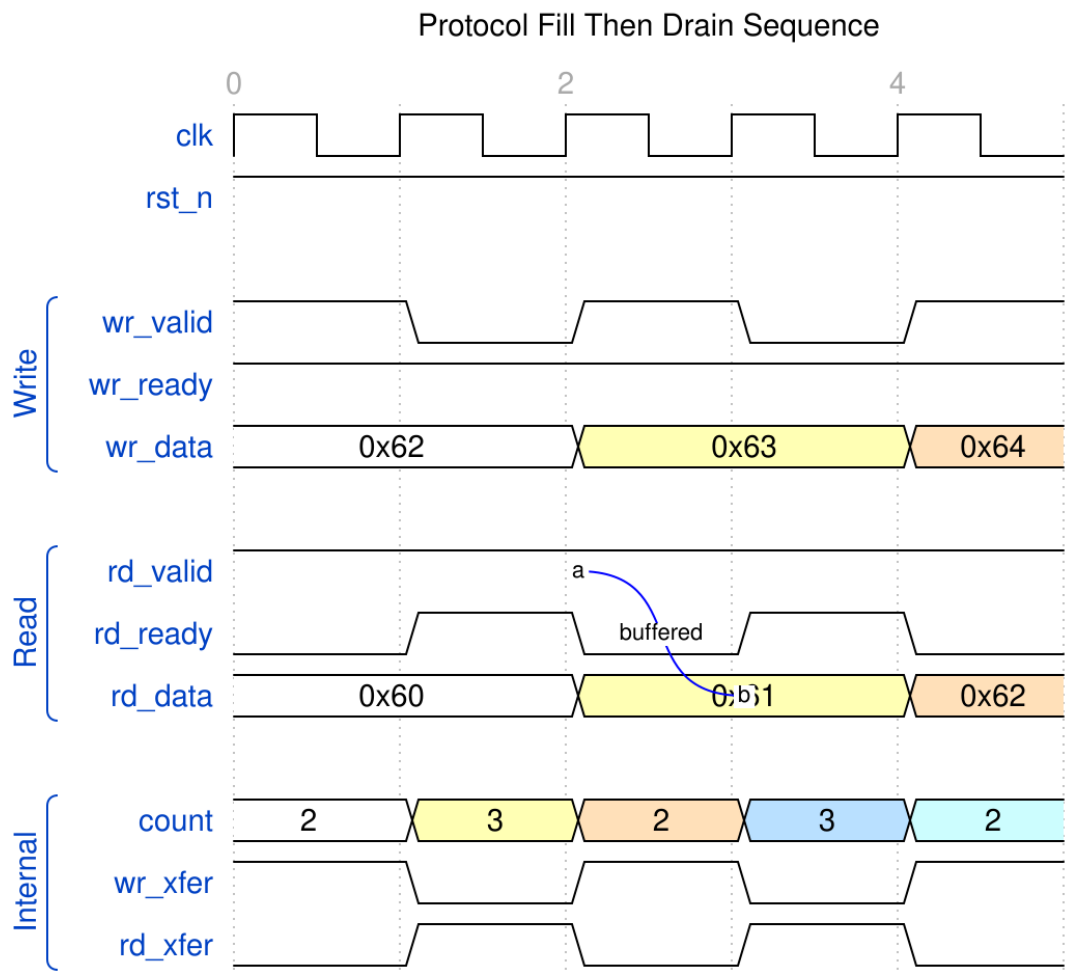


Burst Read Empty

WaveJSON: burst_read_empty_001.json

5.6.5 Scenario 5: Fill Then Drain Pattern

Complete fill phase followed by drain:

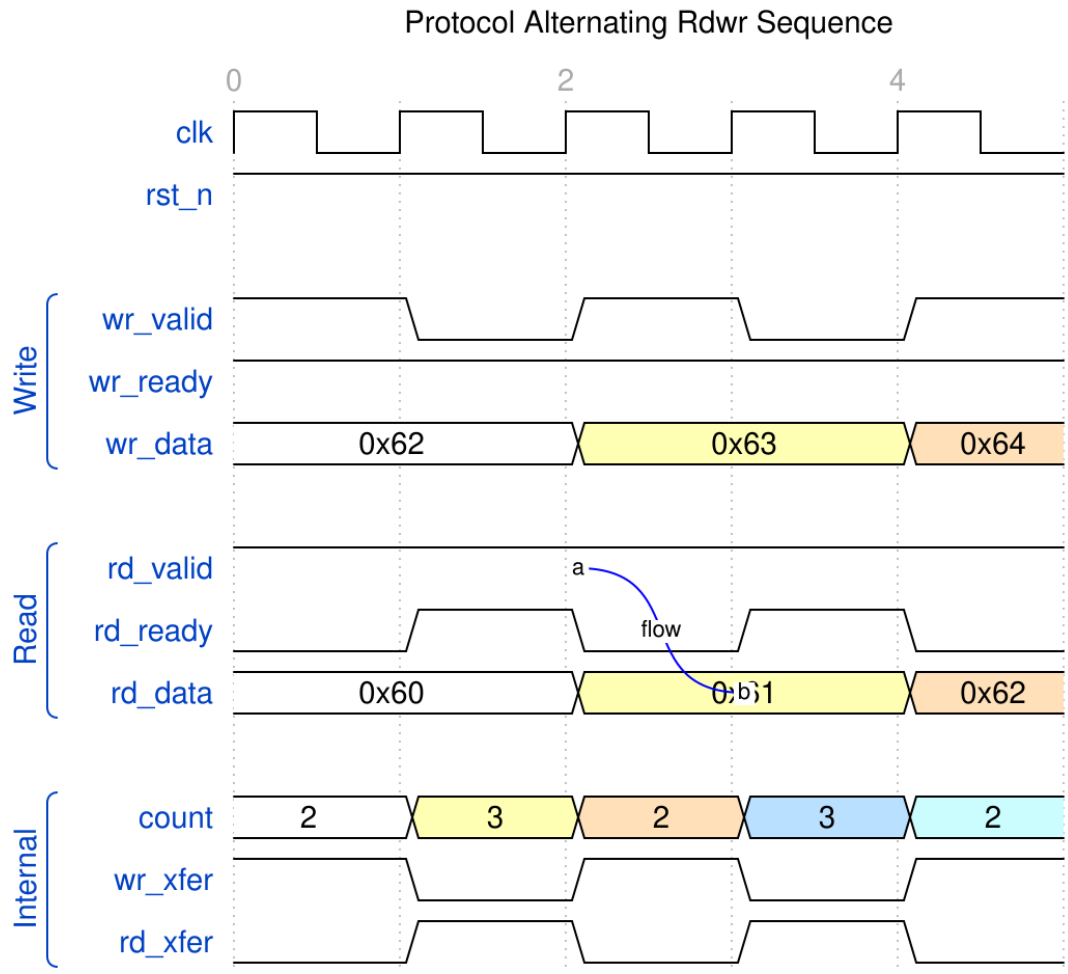


Fill Then Drain

WaveJSON: fill_then_drain_001.json

5.6.6 Scenario 6: Alternating Read/Write

Continuous interleaved operations:



Alternating R/W

WaveJSON: alternating_rdwr_001.json

5.7 Usage Examples

5.7.1 Example 1: Basic Pipeline Stage

```
// Break long combinatorial path
gaxi_skid_buffer #(
    .DATA_WIDTH(64),
    .DEPTH(4)
) u_pipeline_stage (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (stage1_valid),
    .wr_ready    (stage1_ready),
    .wr_data     (stage1_data),
    .rd_valid    (stage2_valid),
    .rd_ready    (stage2_ready),
    .rd_data     (stage2_data),
    .count       (stage_occupancy),
    .rd_count    ()
);
```

5.7.2 Example 2: Backpressure Absorption

```
// Absorb temporary backpressure from downstream
gaxi_skid_buffer #(
    .DATA_WIDTH(128),
    .DEPTH(8)               // Larger depth for more buffering
) u_backpressure_buffer (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (continuous_valid),  // Always producing
    .wr_ready    (continuous_ready),
    .wr_data     (continuous_data),
    .rd_valid    (bursty_valid),      // Intermittent consumer
    .rd_ready    (bursty_ready),
    .rd_data     (bursty_data),
    .count       (buffer_level),
    .rd_count    ()
);

// Monitor buffer level for flow control
assign almost_full = (buffer_level >= 6);  // Threshold
```

5.7.3 Example 3: Monitoring Occupancy

```
gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)
) u_monitored (
```

```
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    .wr_valid    (wr_valid),
    .wr_ready    (wr_ready),
    .wr_data     (wr_data),
    .rd_valid    (rd_valid),
    .rd_ready    (rd_ready),
    .rd_data     (rd_data),
    .count       (fifo_count),
    .rd_count    ()
);

// Performance monitoring
always_ff @(posedge clk) begin
    if (fifo_count > max_count_seen)
        max_count_seen <= fifo_count;

    if (fifo_count == 4'h0 && wr_valid && wr_ready)
        bypass_events <= bypass_events + 1;  // Track bypass usage
end
```

5.8 Design Notes

5.8.1 Depth Selection

DEPTH	Use Case	Latency	Buffering
2	Minimal buffering, tight timing	1 cycle	Limited backpressure tolerance
4	Standard pipeline stage	1 cycle	Moderate backpressure tolerance
6	Heavier buffering needs	1 cycle	Good backpressure tolerance
8	Maximum buffering	1 cycle	Excellent backpressure tolerance

Recommendation: Use DEPTH=4 for most cases - good balance of buffering and resource usage.

5.8.2 Timing Closure

If the memory-to-output path creates timing violations:

Option 1: Add pipeline stage after skid buffer

```
// Skid buffer (registered read)
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(4)) u_skid (...);

// Additional register stage
always_ff @(posedge clk) begin
    if (skid_rd_valid && skid_rd_ready) begin
        final_data <= skid_rd_data;
        final_valid <= 1'b1;
    end else if (final_ready) begin
        final_valid <= 1'b0;
    end
end
```

Option 2: Use FIFO with REGISTERED=1 instead (see gaxi_fifo_sync)

5.8.3 Area vs. Performance Trade-offs

Metric	DEPTH=2	DEPTH=4	DEPTH=8
Flops	2×DW	4×DW	8×DW
LUTs	~40	~50	~70

Metric	DEPTH=2	DEPTH=4	DEPTH=8
Minimum Latency	1 cycle	1 cycle	1 cycle
Backpressure Tolerance	Low	Moderate	High

5.8.4 Error Checking

The module contains no assertions. The 2026-04-23 storage refactor removed the checks this section used to quote, and the quoted code would not have caught anything anyway — both `if` bodies were empty.

Protocol violations are caught in verification instead, not by the RTL:

- `val/amba/test_gaxi_skid_buffer.py` drives the handshake through the GAXI BFMs, which flag a transfer asserted against a deasserted `ready`
- Occupancy is checked against a model on every beat

If you want in-RTL checking, add it at the integration level where a failure can be tied to a specific producer or consumer.

5.8.5 Issue 1: Timing Violation on the Read Path

Symptom: Setup time violation into the consumer's capture flop

Solution:

```
// Add register stage after skid buffer
gaxi_skid_buffer #(...) u_skid (...);

always_ff @(posedge clk) begin
    registered_data <= skid_rd_data;
    registered_valid <= skid_rd_valid;
end
```

5.8.6 Issue 2: Buffer Overflow

Symptom: Data loss, `count` exceeds DEPTH

Debug: 1. Check write logic respects `wr_ready` 2. Verify DEPTH parameter is sufficient 3. Monitor `count` signal - should never exceed DEPTH

5.8.7 Issue 3: Unexpected Latency

Symptom: Data takes multiple cycles to appear at output

Debug: 1. Check buffer was empty when write occurred 2. Verify `rd_ready` is asserted (prevents data from appearing) 3. Look at `count` - if non-zero, data is buffered

---

5.9 Related Modules

- gaxi_fifo_sync - Larger depth, optional registered output
- gaxi_fifo_async - Clock domain crossing
- gaxi_skid_buffer_async - Async version

5.10 Testing

Test File: `val/amba/test_gaxi_skid_buffer.py`

Test Modes: - `mode='skid'` → Tests this module specifically - `mode='fifo_mux'` → Tests gaxi_fifo_sync with REGISTERED=0 - `mode='fifo_flop'` → Tests gaxi_fifo_sync with REGISTERED=1

WaveDrom Test: `val/amba/test_gaxi_wavedrom_example.py` — generates comprehensive timing diagrams showing all 6 scenarios.

5.10.1 Running Tests

```
# Functional test (skid buffer mode)
pytest val/amba/test_gaxi_skid_buffer.py -k "skid" -v

# Generate waveforms
pytest val/amba/test_gaxi_wavedrom_example.py -v
cd val/amba && bash wd_cmd.sh
```

5.11 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9):1059–1076, Sept. 2001. DOI: 10.1109/43.945302 — relay stations (Sec. IV-B): the registered stage that lets a valid/ready channel be cut without changing behaviour.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC)*, pp. 657–662, 2006. DOI: 10.1145/1146909.1147077 — Property 3.1 (capacity >= forward + backward latency) is why a buffer with registered `rd_valid` and registered `wr_ready` needs `DEPTH >= 2` for one transfer per clock.
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 2 “Handshake Protocols” (forward and backward register slices) — textbook treatment (secondary source).
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 13 “Buffered Flow Control”, sec. 13.3 “Buffer Management and Backpressure” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

- **Tutorial:** GAXI WaveDrom Tutorial
- **Overview:** GAXI Index
- **Source:** `rtl/amba/gaxi/gaxi_skid_buffer.sv`
- **Tests:** `val/amba/test_gaxi_skid_buffer.py`, `val/amba/test_gaxi_wavedrom_example.py`

Version: 1.0 **Last Updated:** 2025-10-06 **Maintained By:** RTL Design Sherpa Project

---

5.12 Navigation

- ← Back to GAXI Index
- ← Back to rtl-amba Index

6 gaxi_skid_buffer_dbldrn

Module: `gaxi_skid_buffer_dbldrn.sv` **Location:** `rtl/amba/gaxi/` **Status:** Production Ready

6.1 Overview

gaxi_skid_buffer_dbldrn is the double-drain variant of the gaxi_skid_buffer. The elastic write side and the single-read output are unchanged; what's new is a **second read output** (rd_data2) and a **second drain request** (rd_ready2), so the consumer can pop **two entries in one clock** when at least two are buffered. That matters for consumers that can retire two items per cycle — a 2-wide unpack, a burst aligner — and would otherwise be throughput-limited by a one-per-cycle drain.

Like the base skid buffer, storage is a shift register of DEPTH entries and the read/write handshakes use the GAXI valid/ready convention. The base buffer drains at most one entry per cycle, which caps a fast consumer at one item per clock; this variant lets the consumer drain the lowest and next-lowest entries together, doubling drain throughput while still honoring GAXI backpressure and preserving in-order (lowest-position-first) delivery.

6.1.1 Key Features

- **Double-drain:** Assert rd_ready2 (legal only when rd_count >= 2) to pop two entries in a single cycle
- **Two read data outputs:** rd_data (lowest entry) and rd_data2 (next entry) presented simultaneously
- **Single-drain compatible:** With rd_ready2 low it behaves as an ordinary single-drain skid buffer
- **Priority-encoded update:** Write / single-read / double-read combinations handled by a one-hot case, double-drain prioritized over single-drain
- **Backpressure-aware ready/valid:** wr_ready and rd_valid account for both drain widths
- **Legality assertion:** Simulation $error fires if rd_ready2 is asserted with fewer than two entries buffered

Use Cases:

- Feeding a consumer that retires two entries per clock (2-wide unpack, dual-issue, burst aligner)
- Draining a buffer faster than one-per-cycle to relieve upstream backpressure
- Any place a gaxi_skid_buffer is used but the sink can opportunistically take a pair

Key Benefit: Doubles peak drain throughput (two entries per clock) when two or more are buffered, while degrading gracefully to ordinary single-drain skid-buffer behavior when only one entry is available.

6.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	4	Buffer depth in entries (must be 2..8 inclusive (any integer))
DW	int	DATA_WIDTH	Derived alias for DATA_WIDTH (do not override)
BUF_WIDTH	int	DATA_WIDTH * DEPTH	Derived total shift-register width (do not override)
BW	int	BUF_WIDTH	Derived alias for BUF_WIDTH (do not override)

6.3 Ports

6.3.1 Clock and Reset

Port	Direction	Width	Description
`axi_aclk`	input	1	Clock
`axi_aresetn`	input	1	Active-low asynchronous reset

6.3.2 Write Interface

Port	Direction	Width	Description
`wr_valid`	input	1	Write data valid
`wr_ready`	output	1	Ready to accept a write (registered)
`wr_data`	input	DATA_WIDTH	Write data

6.3.3 Read Interface

Port	Direction	Width	Description
`rd_valid`	output	1	Read data valid (registered)
`rd_ready`	input	1	Consumer ready to accept a read
`rd_ready2`	input	1	Double-drain request; legal only when `rd_count >= 2`
`rd_data`	output	DATA_WIDTH	First (lowest-position) entry
`rd_data2`	output	DATA_WIDTH	Second (next-position) entry, valid when double-draining

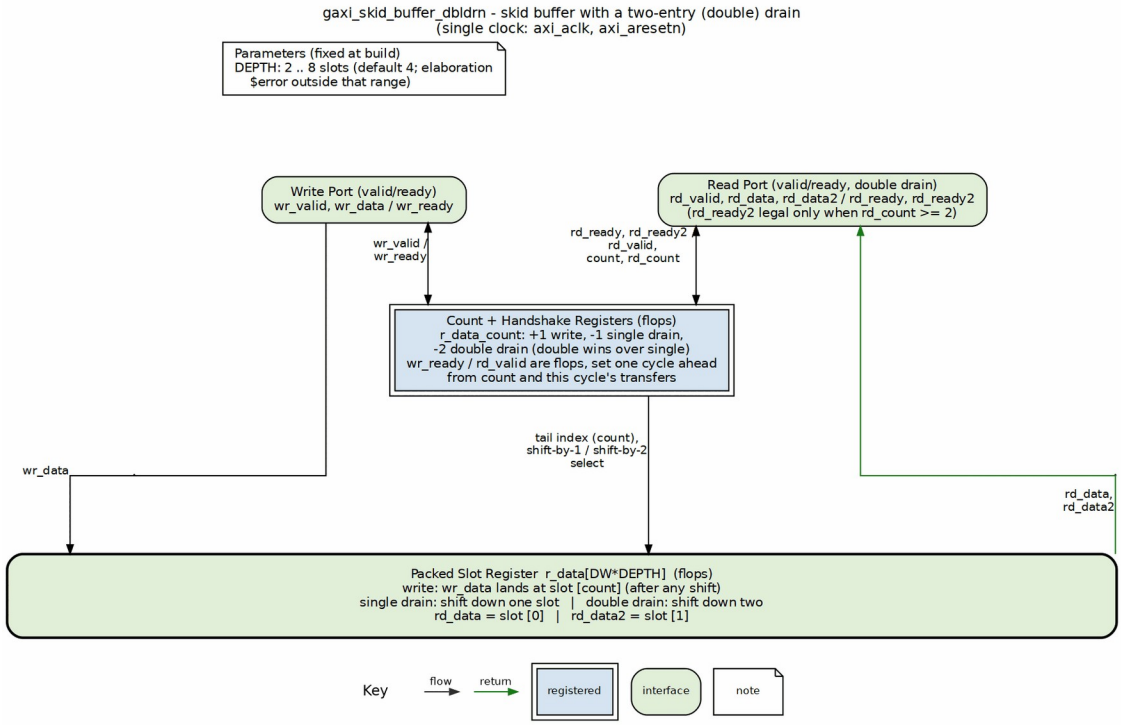
6.3.4 Status

Port	Direction	Width	Description
`count`	output	4	Current buffer

Port	Direction	Width	Description
			occupancy
`rd_count`	output	4	Same as `count` (read-side occupancy)

6.4 Functional Description

6.4.1 Block diagram



gaxi_skid_buffer_dbldrn block diagram

The same shift-register skid buffer as `gaxi_skid_buffer`, with a second drain. `rd_ready2` pops slots 0 and 1 together (`rd_data`, `rd_data2`) and shifts the packed register down by two; it is legal only when `rd_count >= 2`. The count register takes +1 for a write, -1 for a single drain and -2 for a double drain, and both handshake outputs stay registered.

6.4.2 Transfer Detection

Three transfer conditions are decoded combinationally. A single read is only recognized when `rd_ready2` is **not** asserted, and a double read additionally requires two entries present:

```
assign w_wr_xfer     = wr_valid & wr_ready;
assign w_rd_xfer     = rd_valid & rd_ready & ~rd_ready2;
// single drain
assign w_rd_dbl_xfer = rd_valid & rd_ready & rd_ready2 & (r_data_count
>= 2);// double drain
```

6.4.3 Storage Update

The shift register `r_data` and occupancy `r_data_count` are updated from a one-hot case over `{w_wr_xfer, w_rd_dbl_xfer, w_rd_xfer}`, with double-drain prioritized above single-drain:

Case	Meaning	r_data action	count change
3'b100	Write only	Load wr_data at the top of the occupied region	+1
3'b001	Single read only	Shift down by one entry	−1
3'b010	Double read only	Shift down by two entries	−2
3'b101	Write + single read	Shift down one, then insert wr_data at the new top	0
3'b110	Write + double read	Shift down two, then insert wr_data at the new top	−1
default	Idle / illegal (000, 011, 111)	No change	0

The 011 and 111 combinations are illegal (a single- and double-drain cannot both fire) and fall through to the no-change default. Reads always drain from the low positions (rd_data = entry 0, rd_data2 = entry 1), preserving in-order delivery.

6.4.4 Ready/Valid Logic

wr_ready and rd_valid are registered and computed one cycle ahead, accounting for both drain widths so that space freed by a double-drain is reflected in wr_ready:

```
wr_ready <= (r_data_count <= DEPTH-2) ||
            (r_data_count == DEPTH-1 && (~w_wr_xfer || w_rd_xfer ||
w_rd_dbl_xfer)) ||
            (r_data_count == DEPTH   && (w_rd_xfer || w_rd_dbl_xfer));

rd_valid <= (r_data_count >= 3) ||
            (r_data_count == 2 && (~w_rd_dbl_xfer || w_wr_xfer)) ||
            (r_data_count == 1 && (~w_rd_xfer || w_wr_xfer)) ||
            (r_data_count == 0 && w_wr_xfer);
```

The count == 2 term is the one the double drain makes load-bearing: the base buffer's plain count >= 2 is safe only for single drain (which always leaves at least one entry), but a double drain from exactly two leaves ZERO. The unfixed equation held rd_valid high for one cycle on the empty buffer, and a streaming consumer holding rd_ready through that cycle accepted a phantom entry – underflowing the count to 15 and wedging the module until reset. The double-drain-to-empty scenario in the TB is the regression for exactly this corner.

6.4.5 Output Assignments

The two lowest shift-register slots are presented on the two data outputs, and the occupancy drives both status ports:

```
assign rd_data  = r_data[DW-1:0];      // first item (lowest position)
assign rd_data2 = r_data[2*DW-1:DW];   // second item (next position)
assign rd_count = r_data_count;
assign count    = r_data_count;
```

6.4.6 Legality Assertion

A simulation-only check enforces the double-drain contract: asserting `rd_ready2` (with `rd_ready`) when fewer than two entries are buffered raises a `$error`. This catches consumers that request a double-drain the buffer cannot service.

6.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

6.6 Usage Examples

```
// Skid buffer feeding a consumer that can retire two entries per
clock.
gaxi_skid_buffer_dbldrn #(
    .DATA_WIDTH(32),
    .DEPTH(4)
) u_dbldrn (
    .axi_aclk    (clk),
    .axi_aresetn (rst_n),
    // write side
    .wr_valid    (src_valid),
    .wr_ready    (src_ready),
    .wr_data     (src_data),
    // read side
    .rd_valid    (dst_valid),
    .rd_ready    (dst_ready),
    .rd_ready2   (dst_take_two & (rd_occupancy >= 2)),  // only when
>= 2 buffered
    .rd_data     (dst_data0),
    .rd_data2    (dst_data1),
    // status
    .count       (rd_occupancy),
    .rd_count    ()
);
```

6.7 Design Notes

- **`rd_ready2` legality.** Only assert `rd_ready2` (together with `rd_ready`) when `rd_count >= 2`. The RTL guards the transfer with the same condition, and the simulation assertion flags misuse.
- **Priority.** When both a single- and double-drain could be decoded, double-drain wins; the illegal simultaneous cases (`011`, `111`) are ignored by the default branch.
- **In-order delivery.** Entry 0 is always the oldest; `rd_data`/`rd_data2` expose the two oldest entries, and drains shift the register down, so ordering is preserved.
- **Graceful degradation.** With `rd_ready2` tied low the module is functionally an ordinary single-drain skid buffer.
- **Reset macros.** The module uses the project `reset_defs.svh` `ALWAYS_FF_RST` / `RST_ASSERTED` macros for its registered state.

6.8 Related Modules

- gaxi_skid_buffer - The single-drain base skid buffer this variant extends
- gaxi_fifo_sync - Larger-depth synchronous FIFO with optional registered output

Self-contained: no RTL dependencies beyond the `reset_defs.svh` reset macros. Typical integrators are consumers that retire two GAXI entries per clock (2-wide unpack / aligner front-ends).

6.9 Testing

`val/amba/test_gaxi_skid_buffer_dbldrn.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_gaxi_skid_buffer_dbldrn.py -v
```

6.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” and A3.3.1 “Dependencies between channel handshake signals” — the valid/ready contract on both ports.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9):1059–1076, Sept. 2001. DOI: 10.1109/43.945302 — relay stations (Sec. IV-B): the registered stage that lets a valid/ready channel be cut without changing behaviour.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC)*, pp. 657–662, 2006. DOI: 10.1145/1146909.1147077 — Property 3.1 (capacity >= forward + backward latency) is why a buffer with registered `rd_valid` and registered `wr_ready` needs `DEPTH >= 2` for one transfer per clock.
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 2 “Handshake Protocols” (forward and backward register slices) — textbook treatment (secondary source).
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 13 “Buffered Flow Control”, sec. 13.3 “Buffer Management and Backpressure” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

- **Source:** `rtl/amba/gaxi/gaxi_skid_buffer_dbldrn.sv`
- **Base variant:** `rtl/amba/gaxi/gaxi_skid_buffer.sv`
- **Documentation:** `docs/markdown/rtl-amba/index.md`

Last Updated: 2026-07-15

6.11 Navigation

- ← Back to GAXI Index
- ← Back to rtl-amba Index

7 Shared Infrastructure Documentation Status

Generated: 2025-10-23 **Last updated:** 2026-06-15 (timing-closure refresh: 3-stage burst-writer pipeline, 2-stage compressor, per-template `delta_ts`, runtime `cfg_compress_en`, math_mod_3_compress helper, new axi4_dma_observer doc) **Location:** `/mnt/data/github/RTLDesignSherpa/docs/markdown/rtl-amba/shared/`

7.1 Completion Status

This is the inventory, not the documentation itself. Every module page stands on its own; this file exists so you can see at a glance what changed, when, and what the RTL notes underneath were extracted from. The entry numbers are the original inventory IDs — they're cross-referenced from elsewhere in this file, so they stay even though the list has gaps.

7.1.1 Completed Documentation

- **#1 — axi_monitor_base.md** — COMPLETE
  - Comprehensive module documentation with all sections
  - Usage examples provided
  - Design notes included
  - Cross-references added
- **#11 — axi_monitor_trans_mgr.md** — COMPLETE (rewritten 2026-06-08; updated 2026-07 for the cb29e226/95c9490a fixes)
  - Reflects CAM-backed revision (delegates to monitor_trans_cam)
  - Documents synthesis properties carried from the 2026-04-23 WNS fix
  - Same-ID slot separation, oldest-first attribution, command-entry cap, same-cycle AW+W bypass, in-RTL formal properties
  - (The legacy variant, its equivalence test, and the TRANS_MGR_VARIANT rollback knob were deleted in d246a72d)
- **#16 — sdpram_slave.md** — COMPLETE (new 2026-06-09)
  - Covers the full 5-file family: 1 backend + 4 protocol-specific wrappers (axi4_axi4 / axi4_axil / axil_axi4 / axil_axil).
  - Documents why the split exists (SystemVerilog cannot conditionally include/exclude ports in a single module declaration).
  - Migration recipe from bare `sdpram_slave` to the matching wrapper.
- **#17 — axi_monitor_reporter.md** — COMPLETE (rewritten 2026-06-11)
  - Reflects the 2026-06-06 sub-block refactor (thin dispatcher + 6 ENABLE_*_LOGIC-gated detection sub-blocks).
  - Lists the six sub-blocks (error / timeout / compl / threshold / perf / debug), their logic shapes, and their gate parameters.
  - Notes the bridge-case savings (ENABLE_ERROR_LOGIC=1, others 0 drops ~70% LUT/FF).
  - The six sub-block files (axi_monitor_reporter_*.sv) are explicitly covered here rather than as individual doc pages, since they are private to the reporter family.

- **#18 — axi4_dma_observer.md** — RETIRED 2026-08-14 with the module (replaced by axi4_intf_master/slave_observer in projects/components/utility-ip/misc)
  - Standalone, DMA-agnostic observability harness that wraps any AXI4-master DMA from outside the DMA (non-intrusive). Companion to the per-DMA axi_monitor_* family which wraps from inside.
  - Covers the full instantiation pattern: NUM_RD + NUM_WR axi4_master_*_mon taps, monbus_arbiter aggregator, monbus_axil4_axi4_group filter+dump, axi_bus_meter, and axi_perf_latency_hist per port.
  - Documents the runtime rid -> channel map (cfg_rd_rid_per_channel) for read-side per-channel attribution and, for writes, either the built-in AW->W awid order tracker (WR_CH_FROM_AWID=1, no DUT sideband) or the optional dma_wr_active_ch_* sideband.
  - The companion modules axi_bus_meter.sv and axi_perf_latency_hist.sv also have their own standalone pages (axi_bus_meter.md, axi_perf_latency_hist.md) — this doc covers their observer wiring.

7.1.2 New shared infrastructure (no dedicated page, covered above)

- `rtl/math/math_mod_3_compress.sv` — carry-save-compressor `X mod 3` for 16-bit operands; used by monbus_group_core's whole-record compression
- `rtl/amba/filelists/monbus_group.f` — canonical filelist enumerating the group core's dependency tree (math_adder_carry_save_nbit + math_mod_3_compress + monbus_cam + monbus_compressor + monbus_group_core).

7.1.3 Remaining Documentation (15 modules)

These modules follow the same pattern as axi_monitor_base.md:

7.1.3.1 Monitor Infrastructure

- **#2** — axi_monitor_filtered.md — DONE
- **#3** — axi_monitor_reporter.md — **COMPLETE** (rewritten 2026-06-11, see #17)
  - Now describes the dispatcher + 6 sub-blocks (error / timeout / compl / threshold / perf / debug). The six sub-block files

(axi_monitor_reporter_*.sv) are covered here, not as separate doc pages.

- **#4** — axi_monitor_timeout.md — DONE
- **#5** — axi_monitor_timer.md — DONE
- **#6** — axi_monitor_trans_mgr.md — **COMPLETE** (rewritten 2026-06-08)
  - See #11 in Completed Documentation above

7.1.3.2 Monitor Bus Delivery + Bulk-Trace Compression (NEW SECTION)

Nothing lives here anymore — the monitor-family pages (monbus_group, monbus_compressor, monbus_cam, monitor_trans_cam) moved to `../monitor/` with the monitor RTL. See the note at the bottom of this file.

7.1.3.3 Memory / BRAM Slave (NEW SECTION)

- sdpram_slave.md — **COMPLETE** (new 2026-06-09, see #16)
  - Covers backend (sdpram_core.sv) + 4 wrappers (axi4_axi4, axi4_axil, axil_axi4, axil_axil) in a single doc.

7.1.3.4 Monitor Bus Arbitration (4 modules)

- **#7** — arbiter_monbus_common.md — DONE
- **#8** — arbiter_rr_pwm_monbus.md — DONE
- **#9** — arbiter_wrr_pwm_monbus.md — DONE
- **#10** — monbus_arbiter.md — DONE

7.1.3.5 AXI Utilities (4 modules)

- **#11** — axi_gen_addr.md — DONE
- **#12** — axi_master_rd_splitter.md — DONE
- **#13** — axi_master_wr_splitter.md — DONE
- **#14** — axi_split_combi.md — DONE

7.1.3.6 Infrastructure (2 modules)

- **#15** — amba_clock_gate_ctrl.md — DONE

7.2 Documentation Template

Each documentation file should follow this structure (see axi_monitor_base.md as reference):

```
# [Module Name]

**Module:** `[module_name].sv`
**Location:** `rtl/amba/shared/`
**Status:** Production Ready

---

## Overview

[Brief description from RTL file header comments]

### Key Features

- Feature 1
- Feature 2
- Feature 3
- Feature 4

---

## Module Purpose

[Detailed purpose - why this module exists, what problem it solves]

---

## Parameters

| Parameter | Type | Default | Description |
|-----------|------|---------|-------------|
| ... | ... | ... | ... |

---

## Ports

### [Group 1 Name]

| Port | Direction | Width | Description |
```

```
|------|-----------|-------|-------------|
| ... | ... | ... | ... |

---

## Functional Description

[How the module works - key behavior, FSM states, protocol details]

---

## Usage Examples
```systemverilog
[Realistic instantiation example]
```

```

7.3 Design Notes

7.3.1 [Important design aspect 1]

[Details]

- All RTL source files have been read and analyzed
- Module purposes and key features extracted
- Parameter tables ready for documentation
- Port groupings identified
- Design notes captured
- No emojis used in technical documentation

7.4 Related Modules

7.4.1 Used By

- [List of parent modules]

7.4.2 Uses

- [List of child modules]

7.5 References

7.5.1 Specifications

- [ARM specs]
- [Internal references]

7.5.2 Source Code

- RTL: `rtl/amba/shared/[module_name].sv`
- Tests: `val/amba/test_[module_name].py`

Last Updated: 2025-10-23

7.6 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

```
---



::: {.pagebreak}
:::

## Key Information Extracted from RTL

### Monitor Infrastructure Modules

**axi_monitor_filtered.sv**
- Wraps axi_monitor_base with configurable packet filtering
- 2-level filtering: packet-type masking, then event-code masking
(error routing is NOT implemented; `err_select` is reserved)
- AXI protocol specific (protocol 3'b000)
- Optional pipeline stage for timing closure

**axi_monitor_reporter.sv**
- Reports events/errors through shared monitor bus
- Detects conditions from transaction table
- Formats 128-bit monitor packets (64-bit side-band timestamp)
- Supports error, completion, timeout, threshold, performance, debug
packets
- FIFO buffering with gaxi_fifo_sync
- Event reported feedback to trans_mgr (FIX-001)

**axi_monitor_timeout.sv**
- Monitors transaction table for timeout conditions
- Per-phase timeout detection (address, data, response)
- Uses timer tick from frequency invariant timer
- Configurable timeout thresholds per phase

**axi_monitor_timer.sv**
- Frequency invariant timer for timeout detection
- Uses counter_freq_invariant module
- Generates timing ticks based on frequency selection
- Maintains global timestamp counter

**axi_monitor_trans_mgr.sv**
- Manages transaction tracking table
```

- Tracks up to MAX_TRANSACTIONS concurrent transactions
- Handles out-of-order completions
- Supports data-before-address scenarios
- Event reported feedback input (FIX-001)

Monitor Bus Arbitration Modules

arbiter_monbus_common.sv
- Comprehensive monitoring for RR and WRR arbiters
- Silicon debug monitor with PROTOCOL_ARB events
- 3-bit protocol field [59:57]
- Event categories: error, timeout, completion, threshold,
performance, debug
- Per-client ACK timeout tracking
- Protocol violation detection
- Fairness deviation monitoring
- Grant efficiency tracking

arbiter_rr_pwm_monbus.sv
- Round-robin arbiter with PWM control
- Standardized fixed internal configurations
- PWM_WIDTH = 16 bits
- MON_FIFO_DEPTH = 16
- Uses arbiter_monbus_common for monitoring

arbiter_wrr_pwm_monbus.sv
- Weighted round-robin arbiter with PWM control
- Per-client weight thresholds
- Standardized fixed internal configurations
- Enhanced debug outputs for silicon debug

monbus_arbiter.sv
- Round-robin arbiter for monitor bus interfaces
- Optional input and output skid buffers
- ACK mode operation (grants held until acknowledged)
- 128-bit packet + 64-bit side-band timestamp, carried atomically
through a 192-bit skid
- Parameterizable number of clients

AXI Utilities Modules

axi_gen_addr.sv
- Address generation for AXI bursts
- Supports FIXED, INCR, WRAP burst types
- Handles data width conversions
- Calculates next address and aligned address

axi_master_rd_splitter.sv
- Splits AXI read transactions across boundary crossings
- Assumptions: aligned addresses, fixed transfer size, incrementing bursts
- No address wraparound handling (boundary arithmetic is AW+1 bits, TASK-095)
- Split information FIFO for tracking
- State machine: IDLE, SPLITTING

axi_master_wr_splitter.sv
- Splits AXI write transactions across boundary crossings
- Same assumptions as read splitter
- WLAST generation for split transactions
- Response consolidation (N split responses -> 1 upstream response)
- Error priority: DECERR > SLVERR > EXOKAY > OKAY

axi_split_combi.sv
- Pure combinational split decision logic
- Simplified boundary crossing detection
- No transaction wraparound handling; boundary arithmetic is AW+1 bits (TASK-095)
- Comprehensive assertions for validation
- Used by both read and write splitters

Infrastructure Modules

amba_clock_gate_ctrl.sv
- Wrapper for clock_gate_ctrl with AMBA-specific activity monitoring
- Monitors user_valid and axi_valid signals
- Configurable idle countdown
- Generates gated clock output

Monitor-family pages (monbus_group, monbus_compressor, monbus_cam, monitor_trans_cam) are NOT tracked here anymore — they live in `../monitor/` with the monitor RTL (`rtl/amba/ocelus/`).

::: {.pagebreak}
:::

Next Steps

All module pages exist. This file's remaining value is the RTL-extracted module notes above — when a module changes, update its page FIRST and this inventory second (or delete the module's entry here

rather than let it rot).

::: {.pagebreak}
:::

AMBA Shared Infrastructure

The 21 modules in `rtl/amba/shared/` are the protocol-adjacent utility layer: bus characterization blocks, performance meters, boundary splitters, a BRAM-backed slave family, address generation, and clock gating. Every module has its own page in this directory — this file is the map, not a second copy of the module documentation. The per-module pages are the reference; when this table and a module page disagree, trust the page (and fix the table).

An earlier version of this README carried full parameter tables, port lists, and usage examples for a subset of the modules. Those copies rotted — a review round found a `monbus_arbiter` example that could not elaborate, invented clock-gate ports, pre-widening "64-bit packet" claims, and a complete section for a CDC module that had moved to `rtl/cdc/` — so the catalog now lives only in the pages that track their RTL. One copy of the truth. It's the only way this stays honest.

::: {.pagebreak}
:::

Modules

Module	What it is	Page
`axi4_master_wr_injector`	Write-side characterization master: LFSR/hash traffic injector with running CRC	[page](axi4_master_wr_injector.md)
`axi4_master_rd_crc_check`	Read-side characterization master: regenerates the pattern, per-beat compare + running CRC	[page](axi4_master_rd_crc_check.md)
`axi4_slave_rd_injector`	Slave-side read-data injector (serves the expected stream)	[page](axi4_slave_rd_injector.md)
`axi4_slave_wr_crc_check`	Slave-side write CRC accumulator (no compare logic — integrity is an external CRC-vs-CRC check; 16-deep B FIFO)	[page](axi4_slave_wr_crc_check.md)
`axi4_dma_slaves`	Wrapper bundling the two slave-side blocks for DMA loopback	[page](axi4_dma_slaves.md)
`axis4_master_injector`	AXIS traffic injector (same LFSR family)

[page](axis4_master_injector.md) |
`axis4_slave_pattern_check`	AXIS pattern sink/checker	[page](axis4_slave_pattern_check.md)
`axi_bus_meter`	Always-on AXI bandwidth/beat/burst meter	[page](axi_bus_meter.md)
`axis_bus_meter`	AXIS variant of the bus meter	[page](axis_bus_meter.md)
`axi_perf_latency_hist`	Log2 latency histogram, one per metric (channels share it)	[page](axi_perf_latency_hist.md)
`axi_master_rd_splitter`	Boundary-crossing read splitter (RLAST consolidated to one per original burst)	[page](axi_master_rd_splitter.md)
`axi_master_wr_splitter`	Boundary-crossing write splitter with B consolidation	[page](axi_master_wr_splitter.md)
`axi_split_combi`	Pure combinational split decision used by both splitters	[page](axi_split_combi.md)
`sdpram_core`	BRAM glue + clear FSM backend of the slave family	[page](sdpram_core.md)
`sdpram_slave_axi4_axi4` / `_axi4_axil` / `_axil_axi4` / `_axil_axil`	Protocol-shaped wrappers over `sdpram_core`	[family page](sdpram_slave.md)
`axi_gen_addr`	AXI burst next-address generation (FIXED/INCR/WRAP)	[page](axi_gen_addr.md)
`amba_clock_gate_ctrl`	AMBA-side wrapper over `clock_gate_ctrl` (`clk_out`/`gating`/`idle` — no cycle counter)	[page](amba_clock_gate_ctrl.md)

The `_cg` clock-gated wrapper pattern is described in clock_gated_variants.md, and the per-module inventory with RTL-extracted notes is DOCUMENTATION_STATUS.md.

::: {.pagebreak}
:::

What is NOT here

- **CDC.** `cdc_2_phase_handshake`, `cdc_4_phase_handshake`, `cdc_open_loop` and `cdc_synchronizer` live in `rtl/cdc/`, documented in the [rtl-cdc book](../../rtl-cdc/overview.md). Nothing in this directory documents them anymore.
- **Monitor internals.** `monbus_arbiter`, `monbus_group_core`, `monbus_compressor` and the monitor taps live in `rtl/amba/ocelus/`, documented under [../monitor/](../monitor/monbus_group.md). One width fact worth pinning because the old copy here got it wrong twice: the monitor packet is **128 bits** with a **64-bit side-band timestamp**

(192 bits per client through the arbiter skids).

::: {.pagebreak}
:::

Known sharp edges

Recorded here because more than one page used to contradict them:

- The splitter defect cluster (intermediate RLASTs passing upstream, silent split-FIFO record drops, the write splitter's B consolidation missing the final split's error) is FIXED and closed in `vault/tracking/rtl/amba`: RLAST is consolidated to one per original transaction, a full FIFO sets the sticky `o_split_fifo_overflow` output, and the final split's error folds into the consolidated BRESP. A generic AXI master can sit upstream of either splitter directly; both serialize acceptance (one outstanding transaction at a time) — the read side fences on its owed-beat RLAST counter, the write side on open response consolidation.
- The interface observer produces no monbus traffic at its documented parameter defaults — the `TAP_ENABLE_*` parameters gate the tap logic off and perf packets are disabled; override them to get the dump path. It now lives at `projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv`; the `axi4_dma_observer` copy that used to sit here was retired 2026-08-14 (see below).
- `o_cfg_done_clear` on the sdpram family is a sticky level, not a pulse.

::: {.pagebreak}
:::

Testing

Every module here is covered from `val/amba/` (the characterization masters carry an independent software CRC cross-check in their TBs). Run the area with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

::: {.pagebreak}
:::

References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h — burst addressing and the 4KB rule (A3.4.1), response structure (A3.4.5), transaction IDs (A5.1).
- Könemann, B., Mucha, J., Zwiehoff, G. "Built-in Test for Complex Digital Integrated Circuits." *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: [10.1109/JSSC.1980.1051391](https://doi.org/10.1109/JSSC.1980.1051391) — lineage of the LFSR-source / signature-sink characterization blocks.
- Koopman, P. "32-Bit Cyclic Redundancy Codes for Internet Applications." *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: [10.1109/DSN.2002.1028931](https://doi.org/10.1109/DSN.2002.1028931) — the CRC-32 polynomial used for end-to-end signatures.
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`); clock-gating and LFSR/CRC references in `rtl/common/references/README.md`

::: {.pagebreak}
:::

Navigation

[Back to rtl-amba index](../index.md) · [Monitor infrastructure](../monitor/monbus_group.md) · [rtl-cdc book](../../rtl-cdc/overview.md)

::: {.pagebreak}
:::

AMBA Clock Gate Controller

Module: `amba_clock_gate_ctrl.sv`
Location: `rtl/amba/shared/`
Status: Production Ready

::: {.pagebreak}

:::

Overview

The AMBA clock gate controller adds dynamic clock gating to AMBA protocol interfaces. It watches transaction activity on both the user side and the AXI side, gates the clock during idle stretches, and wakes on new activity — cutting dynamic power without touching protocol behavior.

AMBA interfaces idle a lot in real systems, and an always-on clock burns power the whole time. This module does something about it:

1. Monitors transaction activity on both interfaces
2. Detects extended idle periods
3. Gates the clock automatically to save power
4. Ungates on new activity without waiting for the idle counter

Use cases:
- Power-critical AMBA interface implementations
- Battery-powered systems with intermittent bus activity
- Multi-master systems where individual masters idle frequently
- ASIC designs requiring aggressive dynamic power reduction

Key benefit: transparent power saving with no protocol impact — the activity detection guarantees the clock is there when a transfer needs it.

Key Features

- Automatic activity detection from user and AXI valid signals
- Configurable idle threshold before gating activation
- Integration with standard ICG (Integrated Clock Gate) cells
- Registered wakeup signal for metastability protection
- Global enable/disable control
- Idle status monitoring
- Zero added latency while the clock is already ungated (wake-up from gated costs 1 register stage here; see Performance Considerations)

::: {.pagebreak}
:::

Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (determines maximum idle threshold)
ICW	int	CG_IDLE_COUNT_WIDTH	Shorthand alias for idle count width

Idle Count Range:
- Minimum: 0 (immediate gating when idle)
- Maximum: 2^CG_IDLE_COUNT_WIDTH - 1
- Default (4-bit): 0 to 15 cycles

::: {.pagebreak}
:::

Ports

Clock and Reset

Port	Direction	Width	Description
clk_in	input	1	Input clock (always running, feeds ICG cell)
aresetn	input	1	Active-low asynchronous reset

Configuration Interface

Port	Direction	Width	Description
cfg_cg_enable	input	1	Global clock gating enable (1=allow gating, 0=always ungated)
cfg_cg_idle_count	input	ICW	Idle cycle threshold before gating activates

Activity Monitoring

Port	Direction	Width	Description
user_valid	input	1	User-side activity indicator (any valid signal from user interface)
axi_valid	input	1	AXI-side activity indicator (any valid

```
signal from AXI interface) |

### Clock Gating Outputs

| Port | Direction | Width | Description |
|------|-----------|-------|-------------|
| clk_out | output | 1 | Gated output clock (drives clocked logic) |
| gating | output | 1 | Gating active indicator (1=clock currently
gated) |
| idle | output | 1 | Idle status (1=no activity detected, 0=activity
present) |

---



::: {.pagebreak}
:::

## Functional Description

### Block diagram

![amba_clock_gate_ctrl block
diagram](/mnt/data/github/RTLDesignSherpa/docs/markdown/assets/rtl-
amba/graphviz/amba_clock_gate_ctrl.png){width=7.00in}

The clock runs straight down the left column, from `clk_in` through
the base
`clock_gate_ctrl` to `clk_out`. The two activity valids are ORed and
flopped
into `r_wakeup`, which resets to 1 so the block starts ungated.
`r_wakeup`
drives the base controller's wakeup input, and its inverse is the
`idle`
output. The base controller's idle countdown and ICG are drawn in its
own
diagram (rtl-common `clock_gate_ctrl`); the configuration enables
gating and
sets the countdown.

### Activity Detection Logic

The module ORs the activity signals from both interfaces into one
wakeup:
```

wakeup = user_valid OR axi_valid

```
**Why both interfaces?**
- User interface: catches upstream master activity
- AXI interface: catches downstream slave activity
- Combined: keeps the clock alive for bidirectional traffic

**Registered wakeup:**
```systemverilog
always_ff @(posedge clk_in or negedge aresetn) begin
    if (!aresetn)
        r_wakeup <= 1'b1;  // Default: Active (clock ungated at reset)
    else
        r_wakeup <= user_valid || axi_valid;
end
```

Registering the wakeup buys you three things:

1. Metastability protection, if the activity signals cross clock domains
2. A stable, glitch-free input to the downstream clock gate control
3. A default-active state during reset, so the clock is always available coming out of it

7.6.1 Idle Signal Generation

Idle is just the inverse of wakeup:

```
idle = ~r_wakeup
```

Reading it: - idle = 1: no activity for at least 1 cycle (eligible for gating) - idle = 0: recent activity detected (clock must stay ungated)

It's exported for monitoring and debug, so the system can observe per-interface idle status directly.

7.6.2 Clock Gating Control Integration

The real work is delegated to the base clock gate controller:

```
clock_gate_ctrl #(
    .IDLE_CNTR_WIDTH    (ICW)
) u_clock_gate_ctrl (
    .clk_in             (clk_in),
    .aresetn            (aresetn),
    .cfg_cg_enable      (cfg_cg_enable),
    .cfg_cg_idle_count  (cfg_cg_idle_count),
    .wakeup             (r_wakeup),
```
```

```
    .clk_out            (clk_out),
    .gating             (gating)
);
```

Base controller operation:

1. Monitors r_wakeup
2. Reloads the idle counter with cfg_cg_idle_count on any activity, and counts DOWN while wakeup=0
3. Gates the clock when the counter reaches 0
4. Ungates when wakeup=1 (activity), without waiting for the idle counter

7.6.3 Gating Threshold Behavior

cfg_cg_idle_count examples (4-bit counter):

Value	Behavior
0	Aggressive: gate in the SAME cycle the registered wakeup first reads 0 (2 clocks after the last bus activity on single-stage families, 3 on two-stage families)
1	Gate after 1 idle cycle
4	Conservative: Gate after 4 consecutive idle cycles
15	Maximum delay: Gate after 15 consecutive idle cycles

The trade-off: - Low threshold: maximum power saving, more gating/ungating churn - High threshold: less gating overhead, less power saved

Start with a moderate value (4-8) and tune against measured activity patterns. There's no universally right answer here — it depends on your burst shape.

7.7 Timing Characteristics

7.7.1 Gating Sequence (Activity → Idle)

1. Cycle N: user_valid=1, axi_valid=0 → r_wakeup=1 during cycle N+1 (registered)
2. Cycle N+1: user_valid=0, axi_valid=0 → r_wakeup=0 (idle)
3. Idle counter starts counting down from cfg_cg_idle_count
4. Cycle N+1+cfg_cg_idle_count: counter threshold reached
5. Clock gates on next cycle edge (gating=1)

7.7.2 Ungating Sequence (Idle → Activity)

1. Clock currently gated (gating=1)
2. Cycle M: user_valid=1 (new activity arrives)
3. Cycle M+1: r_wakeup=1 (registered); gating=0 combinationally
4. Cycle M+2: first usable gated-clock rising edge
5. Normal operation resumes with no lost cycles

The part that matters: ungating does not wait on the idle counter. It still costs 1 register stage here, so the first usable gated-clock edge arrives 2 clocks after activity asserts (3 on the two-stage APB, APB5, and AXI5-Stream wrappers).

7.8 Usage Examples

7.8.1 Basic Integration with AXI Interface

```
// Instantiate AMBA clock gate controller
amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH    (4)  // 0-15 cycle threshold
) u_clock_gate (
    // Input clock (always running)
    .clk_in                 (axi_aclk_raw),
    .aresetn                (axi_aresetn),

    // Configuration
    .cfg_cg_enable          (power_save_enable),     // From system
config
    .cfg_cg_idle_count      (4'd4),                  // Gate after 4
idle cycles

    // Activity monitoring (OR together all valid signals)
    .user_valid             (s_axi_awvalid | s_axi_wvalid |
s_axi_arvalid),
    .axi_valid              (m_axi_awvalid | m_axi_wvalid |
m_axi_arvalid |
                             m_axi_rvalid  | m_axi_bvalid),

    // Gated clock output
    .clk_out                (axi_aclk_gated),
    .gating                 (clock_gated_status),
    .idle                   (interface_idle)
);

// Use gated clock for all clocked logic
always_ff @(posedge axi_aclk_gated or negedge axi_aresetn) begin
    if (!axi_aresetn) begin
        // Reset logic
    end else begin
        // Normal operation with gated clock
    end
end

// Monitor gating status for debug/power measurement
assign power_gating_active = clock_gated_status;
assign bus_idle_indicator = interface_idle;
```

7.8.2 Multi-Master Arbitration with Clock Gating

```
// Individual clock gating per master interface
genvar i;
```

```
generate
    for (i = 0; i < NUM_MASTERS; i++) begin : gen_master_cg
        amba_clock_gate_ctrl #(
            .CG_IDLE_COUNT_WIDTH    (4)
        ) u_master_cg (
            .clk_in                 (system_clk),
            .aresetn                (aresetn),
            .cfg_cg_enable          (cg_enable[i]),
            .cfg_cg_idle_count      (cg_threshold[i]),

            // Monitor master i activity
            .user_valid             (master_valid[i]),
            .axi_valid              (1'b0),  // No AXI side for master
interface

            .clk_out                (master_clk_gated[i]),
            .gating                 (master_gating[i]),
            .idle                   (master_idle[i])
        );
    end
endgenerate

// Power monitoring: Count active masters
assign num_masters_active = NUM_MASTERS - $countones(master_idle);
```

7.8.3 Dynamic Threshold Adjustment

```
// Adjust gating threshold based on system load
logic [3:0] dynamic_threshold;

always_comb begin
    case (system_load_level)
        2'b00: dynamic_threshold = 4'd2;   // Light load: Aggressive
gating
        2'b01: dynamic_threshold = 4'd4;   // Medium load: Moderate
gating
        2'b10: dynamic_threshold = 4'd8;   // High load: Conservative
gating
        2'b11: dynamic_threshold = 4'd15;  // Critical load: Minimal
gating
    endcase
end

amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH    (4)
) u_adaptive_cg (
    .clk_in                 (clk),
    .aresetn                (rst_n),
```

```
    .cfg_cg_enable          (1'b1),
    .cfg_cg_idle_count      (dynamic_threshold),  // Runtime
adjustable
    .user_valid             (activity),
    .axi_valid              (1'b0),
    .clk_out                (clk_gated),
    .gating                 (gating),
    .idle                   (idle)
);
```

7.9 Design Notes

7.9.1 Wakeup Signal Registration

Design decision: the wakeup signal is registered before it reaches the clock gate controller.

Why: 1. **Metastability protection:** activity signals may originate from different clock domains 2. **Stable input:** the clock gate controller sees a glitch-free wakeup 3. **Reset safety:** defaults to active (1'b1), so the clock is available during reset

The cost: one register stage on activity detection, so the first usable gated-clock edge arrives 2 clocks after activity asserts (3 on the two-stage APB, APB5, and AXI5-Stream wrappers, which register activity again before this module). A clean, glitch-free gate enable is worth that cycle.

7.9.2 Activity Signal Selection

For the user interface (upstream):

```
// AXI Master Interface
user_valid = awvalid | wvalid | arvalid

// AXI Slave Interface
user_valid = awvalid | wvalid | arvalid | rvalid | bvalid
```

> **A peer's READY must never appear in the activity term.** A consumer that parks its response-ready high while idle is behaving correctly; folding that in pins the block permanently awake and defeats gating entirely, silently, because function is unaffected. Use the VALID your side drives. Ten _cg wrappers carried this defect until 2026-09-02.

For the AXI interface (downstream):

```
// AXI Master Interface
axi_valid = awready | wready | arready | rvalid | bvalid

// AXI Slave Interface
axi_valid = awvalid | wvalid | arvalid
```

Why not ready signals? - Valid signals indicate actual transaction progress - Ready signals may assert speculatively - Valid-based detection prevents premature gating

7.9.3 Integration with ICG Cells

The base clock_gate_ctrl expects a standard ICG cell underneath:

Typical ICG cell interface:

```
// Standard ICG cell instantiation (inside clock_gate_ctrl)
ICG u_icg (
    .CLK        (clk_in),
    .EN         (gate_enable),  // From controller FSM
    .CLK_OUT    (clk_out)
);
```

ASIC integration: - Use the library-specific ICG cell (vendor-provided) - The enable must be glitch-free (the controller provides that) - Consider test mode bypass for scan insertion

FPGA integration: - Use global clock mux primitives (BUFGMUX, BUFGCE) - Some FPGAs have dedicated clock gating resources - Alternative: fine-grained clock enable on the flip-flops themselves

7.9.4 Power Measurement Methodology

Estimating power savings:

1. **Measure idle time:**

```
logic [31:0] idle_cycle_count;
always_ff @(posedge clk_in) begin
    if (idle)
        idle_cycle_count <= idle_cycle_count + 1;
end
```

2. **Calculate gating efficiency:**

```
Gating Efficiency = (gating_active_cycles / total_cycles) × 100%
```

3. **Estimate power reduction:**

```
Dynamic Power Saving ≈ Gating Efficiency × Clock Tree Power
```

Typical results: - Low-activity interfaces: 60-80% gating efficiency - Burst-heavy traffic: 20-40% gating efficiency - Clock tree power: 20-30% of total dynamic power - Overall savings: 5-20% total chip dynamic power

7.9.5 Performance Considerations

Latency impact: - No added latency when the clock is already ungated (activity present) - Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The one exception is `apb4_slave_cdc_cg`, which drives this module combinationally (`assign pclk_user_valid = s_apb_PSEL || w_rsp_valid;`) and therefore registers once despite being APB — 2 clocks to the first usable gated edge, not 3. This module contributes the single `r_wakeup` stage; two-stage families add one more in the wrapper. - First usable gated-clock rising edge: 2 clocks (single-stage families) or 3 clocks (two-stage families) after activity asserts - Ungating does not wait on the idle counter

Throughput impact: - Zero: the clock is always available for valid transactions - Activity detection ungates before the transaction arrives

Area overhead: - Minimal: single register + base controller - Base controller: small FSM + counter - Typical: <100 gates total

---

7.10 Related Modules

7.10.1 Used By

- AXI interface wrappers requiring power optimization
- AMBA protocol bridges with intermittent activity
- Multi-master interconnect fabrics
- Power-critical peripheral interfaces

7.10.2 Uses

- **clock_gate_ctrl.sv** — base clock gating controller (idle counter, FSM)
- **reset_defs.svh** — standard reset macro definitions

7.10.3 See Also

- **clock_gate_ctrl.sv** — generic clock gating controller (rtl/common/)
- **icg.sv** — integrated clock gate cell wrapper

7.11 Testing

- Tests: `val/common/test_clock_gate_ctrl.py`

The module is also covered from `val/amba/` with the rest of the shared area — run it with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

7.12 References

- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design*, 15(6):630–643, June 1996. DOI: 10.1109/43.503933 — the latch-based glitch-free clock gate inside the wrapped `clock_gate_ctrl` (`icg`).
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3):299–316, June 2000. DOI: 10.1109/92.845896 — fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 3 “Sequential Logic Design”, sec. 3.2.5 (clock gating) and 3.5 (skew from gated clocks) — textbook treatment (secondary source).
- Indexed in `rtl/common/references/README.md` (local PDF: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

7.12.1 Source Code

- RTL: `rtl/amba/shared/amba_clock_gate_ctrl.sv`
- Base Controller: `rtl/common/clock_gate_ctrl.sv`

7.12.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Common Module: `docs/markdown/rtl-common/clock_gate_ctrl.md`
- Design Guide: `docs/markdown/rtl-amba/index.md`

7.12.3 Industry Standards

- IEEE 1801 (UPF) — Unified Power Format for power-aware design
- Arm AMBA Low Power Interface Specification (Q-Channel / P-Channel), ARM IHI 0068 — the AMBA clock-quiescence handshake; this wrapper does not implement it and gates on local valid activity instead

---

Last Updated: 2025-10-24

7.13 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

8 AXI4 DMA Slaves

Module: `axi4_dma_slaves.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

8.1 Overview

`axi4_dma_slaves` bundles a synthetic AXI4 read *source* and write *sink* into one block that a DMA or streaming engine can plug into for source/sink characterization without a real memory backend. The read side (`axi4_slave_rd_injector`) answers AR bursts with LFSR-generated data; the write side (`axi4_slave_wr_crc_check`) accepts AW/W bursts and CRCs the data. Both compute CRC-32 with the same configuration, so a master that reads the pattern and writes it straight back produces matching read/write CRCs when the datapath is clean.

Characterizing a DMA needs a memory model on both its read and write ports. Rather than instantiate a real RAM (and its checking logic) twice, this block wraps the two purpose-built synthetic slaves and exposes their combined port surface as a single AXI4 read+write slave. The read side manufactures a deterministic pattern; the write side verifies one. Because the CRC polynomial, init, xorout, and reflection settings are threaded identically into both children, the two CRCs are on the same footing — the master writes back the same LFSR data it read, and both sides compute against the same CRC.

Use cases: - `stream_char_harness` attaches this to STREAM's `m_axi_rd` / `m_axi_wr` to characterize DMA throughput, response-delay sweeps, and end-to-end CRC integrity - RAPIDS characterization: source/sink termination for an engine's AXI4 master ports - Any AXI4 master needing a fast, checkable, memory-free source/sink pair

Key benefit: a drop-in memory replacement that is both non-blocking (never the bottleneck) and self-checking (read and write CRCs must agree), all from a single instantiation.

8.1.1 Key Features

- One-instance source + sink slave pair for DMA / streaming-engine characterization
- Read side: LFSR-driven synthetic data source with per-channel CRC-32
- Write side: CRC-32-checking sink with per-channel accounting
- Shared CRC configuration across both sides — read and write CRCs are directly comparable
- LFSR configuration affects only the read (pattern-generation) side
- Independent read-LFSR and write-CRC resets so either side can be re-armed alone
- Per-channel and aggregate CRC / beat-count telemetry from both sides
- Per-side `busy_rd` / `busy_wr` status for harness-level stop triggers

8.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts on each side (channel = id low bits)
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
SKID_DEPTH_AR	int	2	Read AR skid depth
SKID_DEPTH_R	int	4	Read R skid depth
SKID_DEPTH_AW	int	2	Write AW skid depth
SKID_DEPTH_W	int	4	Write W skid depth
SKID_DEPTH_B	int	2	Write B skid depth
LFSR_WIDTH	int	32	LFSR width (read side only)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed (read side only)
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci taps (read side only)
CRC_WIDTH	int	32	CRC width (shared)
CRC_DATA_WIDTH	int	32	Bits per CRC update (shared)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial (shared)
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value (shared)
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR (shared)
CRC_REFIN	int	1	Reflect input (shared)
CRC_REFOUT	int	1	Reflect output (shared)
REPLICATION_FACT	int	(AXI_DATA_WID	Read-side pattern

Parameter	Type	Default	Description
OR		TH+31)/32	replication factor
CRC_SLICE_OFFSET	int	0	Write-side 32-bit CRC slice offset
CIW	int	(NUM_CHANNELS>1) ? $clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: the CRC parameters are passed unchanged to both children on purpose — that shared configuration is what makes read and write CRCs comparable.

8.3 Ports

8.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset
read_lfsr_reset	input	1	Re-arm the read-side LFSR + CRC (independent of write side)
write_crc_reset	input	1	Re-arm the write-side CRC (independent of read side)

8.3.2 Read-Side Telemetry

Port	Direction	Width	Description
read_crc_value	output	NUM_CHANNELS × 32	Per-channel read CRC-32
read_crc_valid	output	NUM_CHANNELS	Per-channel read CRC valid
read_beat_count	output	NUM_CHANNELS × 32	Per-channel read beats
read_beat_count_total	output	32	Aggregate read beat count

8.3.3 Write-Side Telemetry

Port	Direction	Width	Description
write_crc_value	output	NUM_CHANNELS × 32	Per-channel write CRC-32
write_crc_valid	output	NUM_CHANNELS	Per-channel write CRC valid
write_beat_count	output	NUM_CHANNELS × 32	Per-channel write beats
write_beat_count_total	output	32	Aggregate write beat count

8.3.4 AXI4 Read Slave (AR + R)

Port	Direction	Width	Description
s_axi_arid … s_axi_arvalid	input	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_arready	output	1	AR ready
s_axi_rid, s_axi_rdata, s_axi_rresp, s_axi_rlast, s_axi_ruser, s_axi_rvalid	output	—	R channel (LFSR pattern data, `OKAY` resp)
s_axi_rready	input	1	R ready

8.3.5 AXI4 Write Slave (AW + W + B)

Port	Direction	Width	Description
s_axi_awid … s_axi_awvalid	input	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_awready	output	1	AW ready
s_axi_wdata, s_axi_wstrb, s_axi_wlast, s_axi_wuse	input	—	W channel

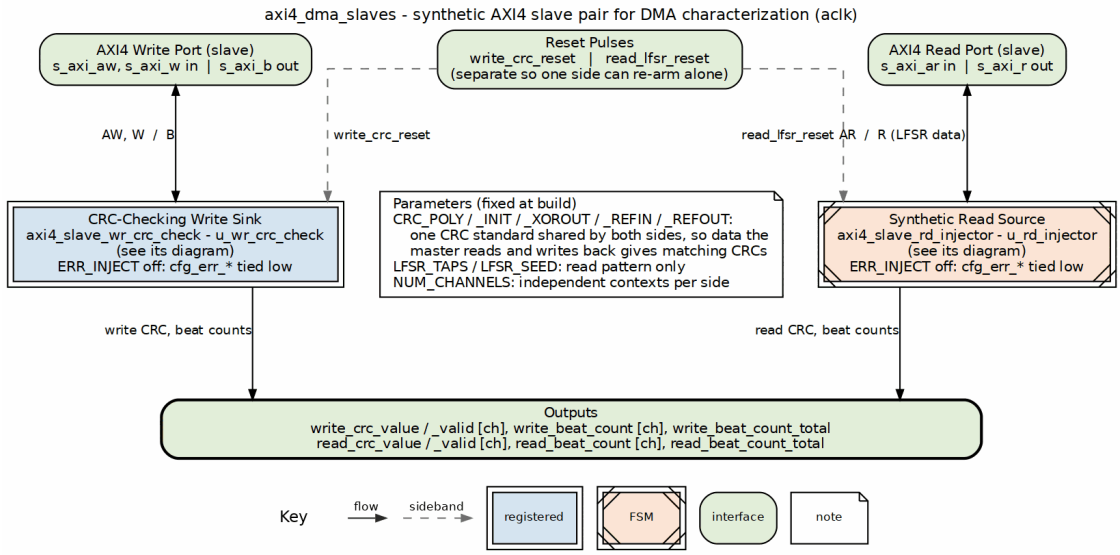
Port	Direction	Width	Description
r, s_axi_wvalid			
s_axi_wready	output	1	W ready
s_axi_bid, s_axi_bresp, s_axi_buser, s_axi_bvalid	output	—	B channel (`OKAY` resp)
s_axi_bready	input	1	B ready

8.3.6 Status

Port	Direction	Width	Description
busy_rd	output	1	Read-side busy (from the read pattern generator)
busy_wr	output	1	Write-side busy (from the write CRC checker)

8.4 Functional Description

8.4.1 Block diagram



axi4_dma_slaves block diagram

The master's write side plugs into the CRC-checking sink on the left, and its read side into the synthetic LFSR source on the right; both are drawn in their own diagrams. Both sides share one CRC standard. A master that reads the pattern and writes it back unchanged therefore ends with matching read and write CRCs. The two resets are separate, so either side can re-arm without disturbing the other. Error injection is not built here: the children's `cfg_err_*` inputs are tied low.

8.4.2 Structural Composition

The module is a thin structural wrapper: it instantiates one `axi4_slave_rd_injector` (`u_rd_injector`) and one `axi4_slave_wr_crc_check` (`u_wr_crc_check`) and fans the top-level ports out to them. There is no additional logic — all behavior lives in the two children, documented separately.

8.4.3 Read Side (LFSR Source)

`u_rd_injector` handles the AR/R channels. It returns per-channel LFSR-generated data (32-bit LFSR replicated to the bus width) and accumulates a per-channel CRC-32 over the emitted stream. Its `crc_lfsr_reset` is driven from the top-level `read_lfsr_reset`. LFSR configuration (`LFSR_WIDTH`, `LFSR_SEED`, `LFSR_TAPS`, `REPLICATION_FACTOR`) reaches only this side.

8.4.4 Write Side (CRC Sink)

u_wr_crc_check handles the AW/W/B channels. It CRCs the selected 32-bit slice of each accepted W beat per channel and returns OKAY B responses. Its crc_reset is driven from the top-level write_crc_reset, and CRC_SLICE_OFFSET selects the CRC'd lane.

8.4.5 Shared CRC vs Independent Resets

The CRC configuration (CRC_POLY, CRC_INIT, CRC_XOROUT, CRC_REFIN, CRC_REFOUT, widths) is threaded identically into both children so the read and write CRCs compute over the same algorithm — the intended usage is that the master writes back the LFSR data it read, and the two CRCs must then agree. The two reset inputs are kept separate so the harness can re-arm one side without disturbing the other; in practice both are usually driven from the same CSR clear pulse.

8.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

8.6 Usage Examples

```
// Source/sink pair on a DMA's read and write master ports.
axi4_dma_slaves #(
    .NUM_CHANNELS   (4),
    .AXI_ID_WIDTH   (8),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64)
) u_dma_slaves (
    .aclk            (aclk),
    .aresetn         (aresetn),
    .read_lfsr_reset (csr_clear_pulse),
    .write_crc_reset (csr_clear_pulse),

    // Read telemetry
    .read_crc_value        (rd_crc),   .read_crc_valid
(rd_crc_valid),
    .read_beat_count       (rd_beats), .read_beat_count_total
(rd_beats_total),
    // Write telemetry
    .write_crc_value       (wr_crc),   .write_crc_valid
(wr_crc_valid),
    .write_beat_count
(wr_beats), .write_beat_count_total(wr_beats_total),

    // Read slave <- DUT m_axi_rd
    .s_axi_arid (m_rd_arid), /* ... AR ...
*/ .s_axi_arready(m_rd_arready),
    .s_axi_rid  (m_rd_rid),  /* ... R  ... */ .s_axi_rready
(m_rd_rready),

    // Write slave <- DUT m_axi_wr
    .s_axi_awid (m_wr_awid), /* ... AW ...
*/ .s_axi_awready(m_wr_awready),
    .s_axi_wdata(m_wr_wdata),/* ... W  ... */ .s_axi_wready
(m_wr_wready),
    .s_axi_bid  (m_wr_bid),  /* ... B  ... */ .s_axi_bready
(m_wr_bready),

    .busy_rd (rd_busy),
    .busy_wr (wr_busy)
);

// End-to-end integrity: after a loopback run, compare per channel.
assign integrity_ok = (rd_crc[ch] == wr_crc[ch]);
```

8.7 Design Notes

- **Pure composition:** the block adds no logic of its own — it exists to give the harness one instance and one port list instead of two, and to enforce the shared-CRC contract at the parameter level.
- **Shared CRC config is intentional:** identical CRC parameters on both children are what let a loopback test compare read vs write CRCs directly.
- **LFSR is read-only:** the write side has no pattern generator; it only checks. LFSR parameters therefore reach only the read child.
- **Independent re-arm:** separate `read_lfsr_reset` / `write_crc_reset` allow re-seeding one side mid-experiment; typical harnesses drive both from one CSR pulse.
- **No memory backend:** addresses are ignored for data — the read side derives data from the LFSR phase (or, in the master-side blocks, from an address hash), not from stored contents.

8.8 Related Modules

8.8.1 Used By

- `projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv` — attaches to STREAM's `m_axi_rd` / `m_axi_wr`
- `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats/rtl/rapids_char_harness.sv` — source/sink termination

8.8.2 Uses

- **axi4_slave_rd_injector.sv** — the read-side LFSR data source + CRC
- **axi4_slave_wr_crc_check.sv** — the write-side CRC-checking sink

8.8.3 See Also

- **axi4_master_wr_injector.sv** / **axi4_master_rd_crc_check.sv** — the master-side counterparts (drive traffic instead of terminating it)
- **axi4_intf_master_observer.sv** — non-intrusive observability wrapper for a DMA's master ports (`projects/components/utility-ip/misc/rtl/`)

8.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

8.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — pattern generator plus signature compactor; the slave regenerates the LFSR stream and accumulates a CRC-32 signature in place of a MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the CRC-32 polynomial (0x04C11DB7).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A5.1 “AXI transaction identifiers” — the per-ID ordering that makes per-channel LFSR/CRC contexts demuxed by ID well defined.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Moon, T.K. *Error Correction Coding*, 2nd ed., Wiley, 2021, ch. 4 “Cyclic Codes, Rings, and Polynomials”, sec. 4.13 “Binary CRC Codes” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

8.10.1 Source Code

- RTL: `rtl/amba/shared/axi4_dma_slaves.sv`

8.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Read child: `docs/markdown/rtl-amba/shared/axi4_slave_rd_injector.md`
- Write child: `docs/markdown/rtl-amba/shared/axi4_slave_wr_crc_check.md`

Last Updated: 2026-07-15

8.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

9 AXI4 Master Read CRC Checker

Module: `axi4_master_rd_crc_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

9.1 Overview

`axi4_master_rd_crc_check` is a CSR-programmed AXI4 read *master* and integrity checker for memory-controller characterization. It walks the *same* algorithmic address mix (via `dma_address_gen`) and the *same* LFSR / hash schedule as `axi4_master_wr_injector`, so each returned R beat can be compared bit-for-bit against a locally regenerated pattern. It also accumulates a CRC-32 over the regenerated stream, so the harness can compare `o_actual_crc` against the writer's `o_expected_crc`.

The read half of a memory-controller integrity loop has to know what it *should* receive. This block regenerates the writer's exact data locally — either from the same LFSR phase counter or from the same address hash — and compares every returned R beat against it, latching any mismatch. It also rolls the regenerated data into a CRC-32 that should equal the writer's expected CRC when the wire is clean. Together those give you both a per-beat pinpoint (which beat disagreed) and a whole-run summary (the CRC compare).

Use cases: - Reading back a DDR / memory controller and verifying the data written by `axi4_master_wr_injector` - Address-pattern integrity sweeps sharing one descriptor with the write generator - Multi-id / out-of-order read validation using hash (address-derived) expected data - On-chip (FPGA) read checker in the DDR2 characterization harness

Key benefit: end-to-end integrity in one block — the per-beat compare localizes corruption while the CRC-32 gives a single comparable summary, both derived from exactly the writer's algorithm.

9.1.1 Key Features

- CSR-programmed read workload with the *same descriptor shape* as the write pattern generator (one CSR word drives both)
- Fully decoupled AR and R paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
- Two expected-data sources: 32-bit Fibonacci LFSR (phase counter) or address-derived Murmur3-fmix hash (out-of-order-safe)
- Pipelined compare (two isolated 32-bit multiplies) so the mode-1 hash closes 100 MHz; returned data rides the same stages to align the compare
- Three AR-ID generation modes: FIXED, COUNTER, LFSR
- Sticky `o_data_error` on any per-beat data mismatch, mismatch counter, and sticky `o_rresp_error` on non-OKAY R beats
- CRC-32 over the regenerated LFSR stream (`o_actual_crc`) comparable to the writer's expected CRC
- Optional debug FIFO capturing `(actual, expected, mismatch)` per R beat for ground-truth disagreement logging

9.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR skid depth
SKID_DEPTH_R	int	4	R skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
LFSR_WIDTH	int	32	LFSR width (must match writer)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	LFSR seed default (must match writer)
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci taps (library-table primitive set)
BURST_LEN_MULTIPLE	int	1	Sim-only guard: cfg_start errors if `cfg_burst_len % BURST_LEN_MULTIPLE != 0` (set to the DRAM burst multiple)
CRC_REFIN	int	1	CRC input reflection – MUST match the slave-side blocks or writer-vs-slave CRC compares can never match
CRC_REFOUT	int	1	CRC output reflection (same constraint)
CRC_WIDTH	int	32	CRC width (must match writer)
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [CRC_WIDTH-1:0]	32'h04C11DB7	CRC polynomial

Parameter	Type	Default	Description
CRC_POLY_INIT	logic [CRC_WIDTH-1:0]	'1	CRC initial value
CRC_XOROUT	logic [CRC_WIDTH-1:0]	'1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of `cfg_txn_count`
INDEX_WIDTH	int	16	`dma_address_gen` index width
STRIDE_WIDTH	int	24	`dma_address_gen` signed stride width
DBG_FIFO_DEPTH	int	0	>0 elaborates a debug FIFO capturing per-beat records; 0 ties `dbg_*` off
IW / AW / DW / UW	int	—	Aliases for id/addr/data/user widths

Note: LFSR + CRC parameters must match `axi4_master_wr_injector` or the compare and CRC roll-up are meaningless. Internally `REPLICATION_FACTOR = (DW+31)/32` and `HSTAGES = 4` compare-pipeline stages.

9.3 Ports

9.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

9.3.2 Configuration (CSR) — same shape as the write generator

Port	Direction	Width	Description
cfg_start_addr	input	AW	Base address for `dma_address_gen`
cfg_addr_stride_0	input	signed STRIDE_WIDTH	Address stride, dimension 0
cfg_addr_stride_1	input	signed STRIDE_WIDTH	Address stride, dimension 1
cfg_addr_wrap_mask_0	input	AW	Address wrap mask, dimension 0
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_len	input	8	Beats per burst (1..255; the port is 8 bits, so 256 is not expressible – it truncates to 0); `arlen = len - 1`
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AR-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR

Port	Direction	Width	Description
cfg_axi_size	input	3	`arsize`
cfg_axi_burst	input	2	`arburst`
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address hash (OOO-safe)
cfg_hash_seed0/1/2	input	32 each	Murmur3-fmix mixer seeds
cfg_rd_gap	input	4	Inter-burst idle cycles (0..15), independent of the writer's gap
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High when all bursts complete and the compare pipeline drains

9.3.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	CRC_WIDTH	Running CRC-32 over the regenerated LFSR stream
o_actual_crc_valid	output	1	High with `cfg_done` (LFSR mode)
o_data_error	output	1	Sticky on any per-beat data mismatch
o_rresp_error	output	1	Sticky on any non-OKAY R beat
o_beats_mismatched	output	TXN_COUNT_WIDTH	Count of mismatching R beats
o_stray_beat_error	output	1	Sticky: an R beat arrived outside any expected transaction (drained and

Port	Direction	Width	Description
			counted, never wedges the engine). Detection rides the FUB side of the internal skid – the pre-2026-08-13 pin-side detect counted the stray but PARKED it in the skid, poisoning the next run's first compare
o_stray_beats	output	TXN_COUNT_WIDTH	Count of stray beats drained

9.3.4 M-Side AXI4 Read (out to fabric/controller)

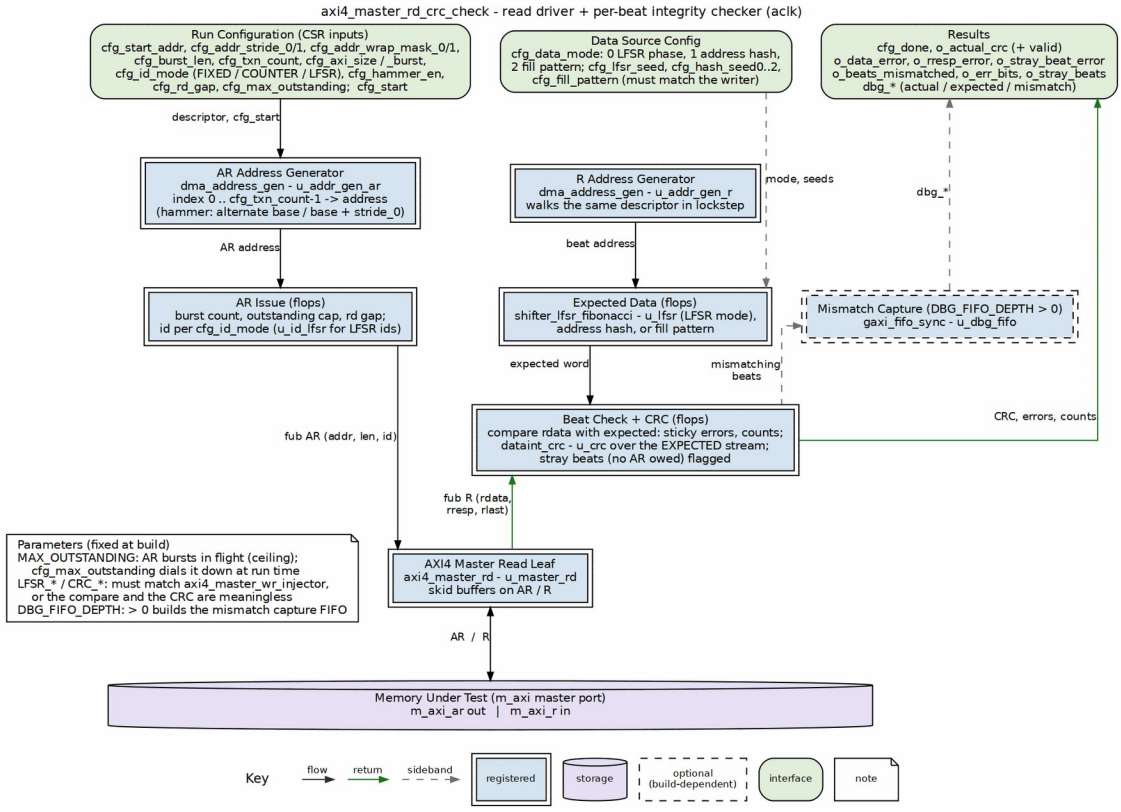
Port	Direction	Width	Description
m_axi_arid … m_axi_arvalid	output	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_arready	input	1	AR ready
m_axi_rid, m_axi_rdata, m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid	input	—	R channel
m_axi_rready	output	1	R ready

9.3.5 Debug Observability (active when DBG_FIFO_DEPTH > 0)

Port	Direction	Width	Description
dbg_valid	output	1	A captured record is available
dbg_ready	input	1	Bench pops a record
dbg_actual	output	DW	Returned data for this beat
dbg_expected	output	DW	Locally regenerated expected data
dbg_mismatch	output	1	This beat disagreed

9.4 Functional Description

9.4.1 Block diagram



axi4_master_rd_crc_check block diagram

This block is a read master; the memory under test is the bottom bar. The AR address generator walks the programmed descriptor, and AR issue applies the outstanding cap, the inter-burst gap and the ID mode before `axi4_master_rd` sends each burst. R beats come back up into the beat checker. A second address generator walks the same descriptor in lockstep so the expected-data source (LFSR phase, address hash or fill pattern) produces each beat's expected word. Each beat is compared, mismatches are counted and optionally captured, and `dataint_crc` rolls up the expected stream. `o_actual_crc` therefore matches the writer's `o_expected_crc` when the memory returned what was written.

9.4.2 Workload FSM

A four-state FSM (`S_IDLE`, `S_RUN`, `S_GAP`, `S_DONE`) mirrors the write generator. `cfg_start` latches the CSR program and enters `S_RUN` (or `S_DONE` if `cfg_txn_count == 0`). `S_RUN` runs the AR and R paths concurrently; `S_GAP` inserts `cfg_rd_gap` idle cycles between bursts (pausing both AR and R together); `S_DONE` holds until re-armed. Because the config shape is identical to the writer, one harness CSR word drives both blocks.

9.4.3 Decoupled AR and R Address Generation

Two independent dma_address_gen instances (u_addr_gen_ar, u_addr_gen_r) walk the same descriptor. The AR path issues one AR per index as fast as arready and the address pipeline allow; arvalid stays asserted from its first cycle to the last AR handshake at cfg_rd_gap = 0. The R path produces the current burst's base address (popped on rlast) so the hash-mode expected-data regen has the right anchor for the following beat. R beats are absorbed only after the AR for that burst is on the wire and the R address generator has produced the base.

9.4.4 Expected Data: LFSR vs Address Hash

Muxed by cfg_data_mode:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR advances on every accepted R beat — the same logic as the writer, so with the same total beat count the two streams stay phase-aligned and match bit-for-bit. Replicated across the bus for the compare.
- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix function of the beat's byte address and the three hash seeds — a pure function of the address, so out-of-order returns still validate.

9.4.5 Pipelined Compare

The mode-1 hash chains two 32-bit multiplies — the same ~25 ns / 4-DSP cone as the writer, too slow to feed the per-beat compare combinationally at 100 MHz. Each multiply is isolated in its own register stage (HSTAGES = 4); the returned rdata rides through the same stages so the compare happens at the pipeline output, aligned with the delayed expected value. Beat/burst accounting stays keyed on the R-beat handshake — only the data compare is delayed. A per-beat mismatch at the pipeline output sets o_data_error and increments o_beats_mismatched; rresp is checked immediately at the R beat (no hash dependency) into o_rresp_error. cfg_done waits for the compare pipeline to drain (!(|r_cp_valid)) so no trailing mismatch is missed.

9.4.6 CRC Accumulation

A dataint_crc accumulates over the regenerated LFSR stream (not the returned rdata) — matching the writer's accounting so o_actual_crc equals the writer's o_expected_crc on a clean wire. o_actual_crc_valid sets with cfg_done in LFSR mode; hash mode relies on the per-beat compare (o_data_error).

9.4.7 AR-ID Generation

arid is muxed by cfg_id_mode exactly like the writer's awid: FIXED, an 8-bit COUNTER, or an 8-bit Fibonacci LFSR (taps {7,6,5,1}, seeded cfg_axi_id[7:0] | 1).

9.4.8 Out-of-Order Completion — Known Limitation

Here's the part that gets everyone. The v1 LFSR mirror advances on *arrival* index, so it is only correct with a single outstanding AR, or when all ARs share one ID (AXI4 mandates in-order R per id). With multiple outstanding ARs at distinct IDs the controller may return bursts interleaved / fully OOO, which reorders R beats versus the writer's W phase and breaks both the per-beat compare and the CRC roll-up. The header documents the v2 plan (per-address hash expected function + commutative CRC roll-up). Until then, the harness CSR must keep the read block single-outstanding (or all-same-id) when the controller has OOO enabled — or use hash data mode, which is inherently OOO-safe for the per-beat compare.

9.4.9 Optional Debug FIFO

When `DBG_FIFO_DEPTH > 0` a `gaxi_fifo_sync` captures `(actual, expected, mismatch)` at each compare-pipeline output so the bench can walk the exact disagreements instead of inferring from `o_data_error`. When depth is 0 the generate `else` arm ties the `dbg_*` outputs off and the FIFO is not built.

9.4.10 Standard Protocol Handler

AR/R skid buffering and AXI compliance are delegated to an `axi4_master_rd` instance; the FSM/LFSR/hash/compare logic drives its FUB side and the `m_axi_*` ports pass straight out to the fabric.

9.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

9.6 Usage Examples

```
// Read checker paired with the write generator on a DDR2 sweep.
axi4_master_rd_crc_check #(
    .AXI_ID_WIDTH   (8),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64),
    .DBG_FIFO_DEPTH (0)
) u_rd_chk (
    .aclk    (aclk),
    .aresetn (aresetn),

    // SAME descriptor word as the write generator
    .cfg_start_addr       (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id           (8'd0),
    .cfg_id_mode          (2'd0),      // FIXED / single-outstanding-
safe
    .cfg_axi_size         (3'd3),
    .cfg_axi_burst        (2'd1),
    .cfg_lfsr_seed        (32'd0),
    .cfg_data_mode        (1'b0),      // LFSR (CRC comparable to
writer)
    .cfg_hash_seed0       (32'd0),
    .cfg_hash_seed1       (32'd0),
    .cfg_hash_seed2       (32'd0),
    .cfg_rd_gap           (4'd0),
    .cfg_start            (csr_start_pulse),
    .cfg_done             (rd_done),

    .o_actual_crc         (rd_actual_crc),
    .o_actual_crc_valid   (rd_actual_crc_valid),
    .o_data_error         (rd_data_error),
    .o_rresp_error        (rd_rresp_error),
    .o_beats_mismatched   (rd_beats_bad),

    // M-side AXI from the controller
    .m_axi_arid (arid), /* ... AR ... */ .m_axi_arready(arready),
    .m_axi_rid  (rid),  /* ... R  ... */ .m_axi_rready (rready),

    // Debug FIFO tied off (DBG_FIFO_DEPTH == 0)
    .dbg_valid (), .dbg_ready (1'b0),
```

```
    .dbg_actual(), .dbg_expected(), .dbg_mismatch()
);

// Clean-wire check: CRCs agree and no per-beat mismatch.
assign integrity_ok = (rd_actual_crc == wr_expected_crc) && !
rd_data_error;
```

9.7 Design Notes

- **Mirror the writer exactly:** LFSR seed schedule, CRC config, address descriptor, and ID modes are all mirror images of `axi4_master_wr_injector` — that symmetry is what makes the CRCs and per-beat expected values comparable.
- **Per-beat compare + CRC are complementary:** the compare localizes the failing beat; the CRC gives a single-register whole-run summary. Both are provided so a failure can be both detected and pinpointed.
- **Compare pipeline aligns delayed expected with delayed rdata:** the returned data is intentionally delayed the same `HSTAGES` as the hash so the compare is apples-to-apples, and `cfg_done` waits for the pipeline to drain.
- **OOO is a real v1 limitation:** with multi-id OOO completion, use hash data mode (`cfg_data_mode = 1`) or hold the read block single-outstanding — the header spells out the v2 per-address-hash fix.
- **Debug FIFO is opt-in:** `DBG_FIFO_DEPTH > 0` adds ground-truth per-beat capture at essentially no cost when disabled.

9.8 Related Modules

9.8.1 Used By

- `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/build-perf/rtl/ddr2_char_harness.sv` — on-chip read checker
- DDR2 characterization macro / harness CSR blocks under `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/`

9.8.2 Uses

- **axi4_master_rd.sv** — standard AXI4 read master protocol handler (AR/R skid + compliance)
- **dma_address_gen.sv** — algorithmic address sequence generator (×2, decoupled AR/R)
- **shifter_lfsr_fibonacci.sv** — expected-data LFSR and 8-bit AR-ID LFSR
- **dataint_crc.sv** — CRC-32 accumulator
- **gaxi_fifo_sync.sv** — optional debug capture FIFO

9.8.3 See Also

- **axi4_master_wr_injector.sv** — the matching write-side driver (same LFSR/CRC/hash/descriptor config)
- **axi4_slave_rd_injector.sv** — the slave-side read pattern source

9.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`. The characterization masters carry an independent software CRC cross-check in their TBs.

9.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — the built-in-test architecture of an LFSR pattern generator paired with a signature compactor; this block plays the generator role and keeps a CRC-32 signature instead of a MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the IEEE 802.3 CRC-32 polynomial (0x04C11DB7) used for the signature.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” — the burst rules the issued AW/AR sequence follows.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Moon, T.K. *Error Correction Coding*, 2nd ed., Wiley, 2021, ch. 4 “Cyclic Codes, Rings, and Polynomials”, sec. 4.13 “Binary CRC Codes” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

9.10.1 Source Code

- RTL: `rtl/amba/shared/axi4_master_rd_crc_check.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_master_rd.sv`

9.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/README.md`

Last Updated: 2026-07-15

9.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

10 AXI4 Master Write Injector

Module: `axi4_master_wr_injector.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

10.1 Overview

`axi4_master_wr_injector` is a CSR-programmed AXI4 write *master* for memory-controller characterization. On a start pulse it walks an algorithmic address mix (via `dma_address_gen`) and streams LFSR-pattern data through `axi4_master_wr`, accumulating a CRC-32 over the data it writes. It pairs with `axi4_master_rd_crc_check`, which regenerates the same pattern on the read side so the two CRCs (and per-beat compares) validate end-to-end data integrity through a real DRAM controller.

Bringing up a memory controller means driving it with realistic, deterministic write traffic and proving the data survives the round trip. This block drives the writes: it walks a programmable address pattern and emits reproducible data, folding that data into a CRC-32 that becomes the "expected" value for the read-side checker. The write data can be a simple LFSR phase counter (fast, but order-sensitive) or an address-derived hash (each beat's data is a pure function of its byte address, so multi-id / out-of-order completion still validates).

Use cases: - Driving a DDR / memory-controller's AXI4 write port during characterization sweeps - Generating the "golden" CRC / data that `axi4_master_rd_crc_check` compares against - Address-pattern stress (incremental, row-major, column-major page attacks) via the `dma_address_gen` descriptor - On-chip (FPGA) write stimulus in the DDR2 characterization harness

Key benefit: a CSR-driven, deterministic write generator whose data can be regenerated bit-for-bit anywhere — turning memory-controller bring-up into a single expected-vs-actual CRC comparison.

10.1.1 Key Features

- CSR-programmed write workload: start address, address-generator strides/wrap masks, burst length, transaction count, AXI id/size/burst attributes
- Fully decoupled AW and W paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
- Two data sources: a 32-bit Fibonacci LFSR (phase-counter) or an address-derived Murmur3-fmix hash (out-of-order-safe)
- Pipelined hash datapath (two isolated 32-bit multiplies) to close 100 MHz, decoupled from the AXI W handshake by a staging FIFO
- Three AW-ID generation modes: FIXED, COUNTER, LFSR
- CRC-32 accumulator over the written LFSR stream (`o_expected_crc`) for the read side to compare against
- Configurable inter-burst idle gap (`cfg_wr_gap`) for throughput-stress sweeps

- Sticky `o_bresp_error` on any non-OKAY write response; direct re-arm from the done state

---

10.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW skid depth
SKID_DEPTH_W	int	4	W skid depth
SKID_DEPTH_B	int	2	B skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (derived)
LFSR_WIDTH	int	32	LFSR width; matches the slave side. CRC reflection is parameterized (CRC_REFIN/CRC_REFOUT, default 1) to the same standard CRC-32 convention as the slave blocks, so writer and slave CRCs interchange – the old hardcoded REFIN/REFOUT=0 here broke that silently
LFSR_SEED	logic [31:0]	32'hDEADBEEF	LFSR seed default
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci taps (library-table primitive set)
BURST_LEN_MULTIPLE	int	1	Sim-only guard: cfg_start errors if `cfg_burst_len % BURST_LEN_MULTIPLE != 0` (set to the DRAM burst multiple)
CRC_REFIN	int	1	CRC input reflection –

Parameter	Type	Default	Description
			MUST match the slave-side blocks
CRC_REFOUT	int	1	CRC output reflection (same constraint)
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update (fixed 32 to match slave side)
CRC_POLY	logic [CRC_WIDTH-1:0]	32'h04C11DB7	CRC polynomial
CRC_POLY_INIT	logic [CRC_WIDTH-1:0]	'1	CRC initial value
CRC_XOROUT	logic [CRC_WIDTH-1:0]	'1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of `cfg_txn_count` (up to 64K bursts)
INDEX_WIDTH	int	16	`dma_address_gen` index width
STRIDE_WIDTH	int	24	`dma_address_gen` signed stride width
GAP_WIDTH	int	4	Width of `cfg_wr_gap`; 4 gives 0..15 idle cycles. `char_gen_unit` builds 11 (`gap << gap_shift`, up to 1920)
IW / AW / DW / UW / SW	int	—	Aliases for id/addr/data/user/strobe widths

Note: LFSR + CRC parameters mirror `axi4_master_rd_crc_check` exactly so the two blocks' CRC values are interchangeable. Internally `REP = (DW+31)/32` 32-bit slices per beat and `HSTAGES = 4` hash-pipeline stages.

10.3 Ports

10.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

10.3.2 Configuration (CSR)

Port	Direction	Width	Description
cfg_start_addr	input	AW	Base address handed to `dma_address_gen`
cfg_addr_stride_0	input	signed STRIDE_WIDTH	Address stride, dimension 0
cfg_addr_stride_1	input	signed STRIDE_WIDTH	Address stride, dimension 1 (held at index 0 here)
cfg_addr_wrap_mask_0	input	AW	Address wrap mask, dimension 0
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_len	input	8	Beats per burst (1..255; the port is 8 bits, so 256 is not expressible – it truncates to 0); `awlen = len - 1`
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AW-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR

Port	Direction	Width	Description
cfg_axi_size	input	3	`awsize`
cfg_axi_burst	input	2	`awburst` (INCR=1)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use `LFSR_SEED` param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address-derived hash (OOO-safe)
cfg_hash_seed0/1/2	input	32 each	Murmur3-fmix mixer seeds (hash mode)
cfg_wr_gap	input	GAP_WIDTH	Inter-burst idle cycles (0..2^GAP_WIDTH-1; 0..15 at the default width)
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High once all B responses received

10.3.3 Telemetry

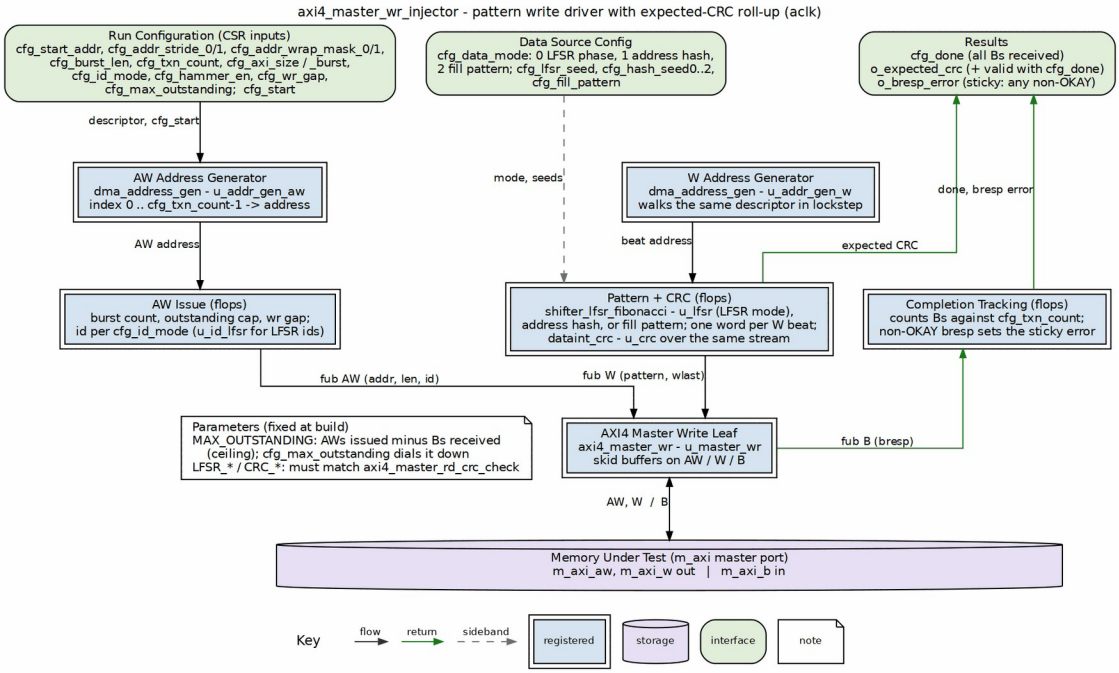
Port	Direction	Width	Description
o_expected_crc	output	CRC_WIDTH	End-of-run CRC-32 over the written LFSR stream, captured two cycles after the final W beat (holds the previous run's value, 0 after reset, until then); qualified by o_expected_crc_valid
o_expected_crc_valid	output	1	High with `cfg_done` (LFSR mode only)
o_bresp_error	output	1	Sticky on any non-OKAY BRESP

10.3.4 M-Side AXI4 Write (out to fabric/controller)

Port	Direction	Width	Description
m_axi_awid … m_axi_awvalid	output	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_awready	input	1	AW ready
m_axi_wdata, m_axi_wstrb, m_axi_wlast, m_axi_wuser, m_axi_wvalid	output	—	W channel (strobes tied full-beat)
m_axi_wready	input	1	W ready
m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid	input	—	B channel
m_axi_bready	output	1	B ready

10.4 Functional Description

10.4.1 Block diagram



axi4_master_wr_injector block diagram

This block is a write master; the memory under test is the bottom bar. The AW address generator walks the programmed descriptor, and AW issue applies the outstanding cap, the inter-burst gap and the ID mode. A second address generator walks the same descriptor in lockstep, so the data source (LFSR phase, address hash or fill pattern) emits one word per W beat. `dataint_crc` rolls those words into `o_expected_crc`. AW and W go out through `axi4_master_wr`. B responses come back up into completion tracking, which raises `cfg_done` when every burst has answered and latches any non-OKAY `bresp`.

10.4.2 Workload FSM

A four-state FSM (`S_IDLE`, `S_RUN`, `S_GAP`, `S_DONE`) sequences a run. On `cfg_start` the CSR program is latched (so software can change the `cfg_*` inputs on the next cycle without disturbing the run) and the block enters `S_RUN` (or `S_DONE` immediately if `cfg_txn_count == 0`). In `S_RUN` the AW and W paths run concurrently; `S_GAP` inserts `cfg_wr_gap` idle cycles between bursts; `S_DONE` awaits the final B responses. `S_DONE` also accepts a fresh `cfg_start` for a direct re-arm without passing back through `S_IDLE`.

10.4.3 Decoupled AW and W Address Generation

Two independent `dma_address_gen` instances (`u_addr_gen_aw`, `u_addr_gen_w`) walk the *same* descriptor with the same indices, so both produce the identical address sequence. The AW path

issues one AW per index at awready rate; the W path produces the current burst's base address for the hash datapath, popped on WLAST. awvalid stays continuously asserted from its first cycle to the last AW handshake when cfg_wr_gap = 0 — the address generator produces one result per cycle once warmed. With a non-zero gap an AW never runs ahead of the previous burst's W: the next AW issues only after the gap and after every earlier burst's last beat has left the generator. An AW pipelined ahead of a gap would reach the slave and wait there for W data the gap is holding, and since AXI4 W follows AW order at the slave, every other master's write behind it would wait too. Outstanding depth is bounded by the slave's awready throttling, not by this block. Only index_0 is walked (index_1 held at 0); a second instance with a different cfg_addr_stride_1 would exercise the 2D path.

10.4.4 Data Path: LFSR vs Address Hash

Two data sources, muxed by cfg_data_mode:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR (shifter_lfsr_fibonacci, taps {23,3,2,1}) advances on every accepted W beat, replicated across the data bus (REP copies). The data stream is a deterministic function of (seed, total_beats_so_far). This is order-sensitive and breaks under multi-id / out-of-order completion.
- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix-style function of its byte address and the three cfg_hash_seed values (xor-shift + odd multiplies). Because each beat's data depends only on its address, reorder does not perturb the per-beat compare on the read side.

The per-beat byte address is anchored on the W address-generator output and stepped by 2**awsize bytes per beat. Full-beat writes only — wstrb is tied all-ones.

10.4.5 Hash Pipeline and Staging FIFO

The mode-1 hash chains two 32-bit multiplies — combinationally a ~25 ns / 4-DSP cone that misses 100 MHz. Each multiply is isolated in its own register stage (HSTAGES = 4), with the WLAST/mode/LFSR-data riding alongside so the output stays beat-aligned. The pipeline is latency-insensitive: per-beat data values and order are unchanged, so the CRC / per-beat-compare contract still holds. A show-ahead staging FIFO (gaxi_fifo_sync, depth 16) between the pipeline output and the AXI W handshake lets W stream back-to-back independent of pipeline fill or master backpressure; a beat is admitted only when the generator has one and the FIFO has room for it plus the HSTAGES beats already in flight.

10.4.6 AW-ID Generation

awid is muxed by cfg_id_mode: FIXED passes cfg_axi_id through; COUNTER is an 8-bit counter seeded at cfg_axi_id[7:0], +1 per AW; LFSR is an 8-bit maximal-length Fibonacci LFSR (taps {7,6,5,1}) seeded with cfg_axi_id[7:0] | 1 to avoid the all-zero seed. Internal counter/LFSR are 8-bit and zero-extended/truncated to IW.

10.4.7 CRC Accumulation and Completion

A `dataint_crc` instance accumulates over the same LFSR stream sent on W (not over the hash), latched into `o_expected_crc` two cycles after the last W beat of the last burst (the CRC accumulator and its conditioned output register each lag one cycle) — meaningful only in LFSR mode, where `o_expected_crc_valid` sets. In hash mode the CRC is not load-bearing and validity is gated low; the harness uses the read side's per-beat compare instead. B responses are counted independently of FSM phase (except never accepted pre-start); `o_bresp_error` sticks on any non-OKAY BRESP. `cfg_done` asserts once in `S_DONE` with all `cfg_txn_count` B's received.

10.4.8 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to an `axi4_master_wr` instance; the FSM/LFSR/hash logic drives its FUB side and the `m_axi_*` ports pass straight out to the fabric.

10.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

10.6 Usage Examples

```
// Write driver for a DDR2 characterization sweep (LFSR data mode).
axi4_master_wr_injector #(
    .AXI_ID_WIDTH   (8),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64)
) u_wr_gen (
    .aclk    (aclk),
    .aresetn (aresetn),

    // CSR program (from the harness register block)
    .cfg_start_addr       (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id           (8'd0),
    .cfg_id_mode          (2'd0),      // FIXED
    .cfg_axi_size         (3'd3),      // 8 bytes/beat
    .cfg_axi_burst        (2'd1),      // INCR
    .cfg_lfsr_seed        (32'd0),     // use param default
    .cfg_data_mode        (1'b0),      // LFSR (golden CRC valid)
    .cfg_hash_seed0       (32'd0),
    .cfg_hash_seed1       (32'd0),
    .cfg_hash_seed2       (32'd0),
    .cfg_wr_gap           (4'd0),
    .cfg_start            (csr_start_pulse),
    .cfg_done             (wr_done),

    // Golden CRC for the read side to compare against
    .o_expected_crc       (wr_expected_crc),
    .o_expected_crc_valid (wr_expected_crc_valid),
    .o_bresp_error        (wr_bresp_error),

    // M-side AXI to the controller
    .m_axi_awid (awid), /* ... AW ... */ .m_axi_awready(awready),
    .m_axi_wdata(wdata),/* ... W  ... */ .m_axi_wready (wready),
    .m_axi_bid  (bid),  /* ... B  ... */ .m_axi_bready (bready)
);
```

10.7 Design Notes

- **Two data modes for two threat models:** LFSR mode is the fast default with a single golden CRC; hash mode trades the CRC for OOO-safe per-beat verification when multi-id traffic reorders completions.
- **Hash pipelining was a timing fix:** the two-multiply Murmur cone was isolated into DSP-register stages specifically to close 100 MHz on FPGA; the staging FIFO then hides the added latency from the AXI handshake.
- **CRC follows the LFSR, not the wire:** the CRC absorbs the LFSR words (32-bit), keeping it interchangeable with the read checker and independent of bus width; it is only latched valid in LFSR mode.
- **Decoupled AW/W:** separate address generators mean AW can race ahead at `awready` rate while W streams at `wready` rate, exposing controller reorder/backpressure behavior a lock-stepped generator would mask.
- **Direct re-arm:** `S_DONE` re-latches on `cfg_start` so back-to-back sweeps don't require a return to idle.

10.8 Related Modules

10.8.1 Used By

- `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/build-perf/rtl/ddr2_char_harness.sv` — on-chip write driver
- DDR2 characterization macro / harness CSR blocks under `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/`

10.8.2 Uses

- **axi4_master_wr.sv** — standard AXI4 write master protocol handler (AW/W/B skid + compliance)
- **dma_address_gen.sv** — algorithmic address sequence generator (×2, decoupled AW/W)
- **shifter_lfsr_fibonacci.sv** — LFSR data source and 8-bit AW-ID LFSR
- **dataint_crc.sv** — CRC-32 accumulator
- **gaxi_fifo_sync.sv** — show-ahead staging FIFO decoupling the hash pipeline from W

10.8.3 See Also

- **axi4_master_rd_crc_check.sv** — the matching read-side driver + checker (same LFSR/CRC/hash config)
- **axi4_slave_wr_crc_check.sv** — the slave-side write CRC sink

---

10.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`. The characterization masters carry an independent software CRC cross-check in their TBs.

10.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — the built-in-test architecture of an LFSR pattern generator paired with a signature compactor; this block plays the generator role and keeps a CRC-32 signature instead of a MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the IEEE 802.3 CRC-32 polynomial (0x04C11DB7) used for the signature.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” — the burst rules the issued AW/AR sequence follows.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Moon, T.K. *Error Correction Coding*, 2nd ed., Wiley, 2021, ch. 4 “Cyclic Codes, Rings, and Polynomials”, sec. 4.13 “Binary CRC Codes” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

10.10.1 Source Code

- RTL: `rtl/amba/shared/axi4_master_wr_injector.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_master_wr.sv`

10.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/README.md`

Last Updated: 2026-07-15

10.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

11 AXI4 Slave Read Injector

Module: `axi4_slave_rd_injector.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

11.1 Overview

`axi4_slave_rd_injector` is a read-only AXI4 slave that answers AR bursts with deterministic, LFSR-generated data and accumulates a running CRC-32 over the stream it emits. It exists to serve as a synthetic data *source* for DMA / streaming-engine characterization: a master under test issues read bursts, the slave returns a reproducible pseudo-random pattern, and the same pattern (and CRC) can be regenerated anywhere else in the system to prove end-to-end data integrity.

Characterizing a DMA read path needs a slave that is both *fast* (never the bottleneck) and *checkable* (the returned data is a known function of the request). A real memory backend gives you neither cheaply. This module instead generates its read data from a Fibonacci LFSR, so every beat is a deterministic function of `(seed, beat_index)`, and folds that same data into a CRC-32 the harness can compare against an independently computed golden value.

Use cases: - Feeding a DMA / streaming engine's read (`m_axi_rd`) port during throughput characterization - End-to-end integrity checks where read data is looped back to a write-CRC sink - Per-channel multi-context validation, where each channel's data must be independent of another channel's interleave on the shared AXI port - On-chip (FPGA) stimulus in the STREAM / RAPIDS characterization harnesses

Key benefit: a memory-free, deterministic read slave whose per-channel CRC-32 is bit-identical to the write-side checker — integrity is proven with a single register compare, and the slave never starves the master.

11.1.1 Key Features

- Read-only AXI4 slave (AR + R channels) built on the standard `axi4_slave_rd` protocol handler
- Per-channel independent LFSR + CRC-32 state, demuxed off the low bits of `arid`
- 32-bit maximal-length Fibonacci LFSR (seed `0xDEADBEEF`, taps `{23,3,2,1}`) replicated to fill the data bus
- CRC-32/Ethernet accumulator per channel (poly `0x04C11DB7`, reflected in/out) matching the write-side checker bit-for-bit
- Gapless back-to-back bursts (AR accepted on the last beat) so `rvalid` never drops mid-stream
- Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers
- Single `crc_lfsr_reset` pulse re-arms all channel LFSRs and CRCs together

11.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent LFSR/CRC contexts; channel index is `arid[CIW-1:0]`
SKID_DEPTH_AR	int	2	AR channel skid buffer depth
SKID_DEPTH_R	int	4	R channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width (may be wider; must be handled by replication)
AXI_USER_WIDTH	int	1	AXI user signal width
LFSR_WIDTH	int	32	LFSR width (fixed at 32 for timing/simplicity)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses `LFSR_SEED ^ N`
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci tap indices
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits processed per CRC update (the 32-bit LFSR output)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output

Parameter	Type	Default	Description
REPLICATION_FACTOR	int	(AXI_DATA_WIDTH+31)/32	Derived: number of 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS>1) ? $clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: `CRC_*` must match `axi4_slave_wr_crc_check` (and the AXIS pattern blocks) exactly, or cross-block CRC comparison is meaningless.

11.3 Ports

11.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

11.3.2 Test Control

Port	Direction	Width	Description
crc_lfsr_reset	input	1	Pulse to reload all channel LFSR seeds and clear all CRCs / beat counts

11.3.3 Per-Channel Telemetry

Port	Direction	Width	Description
read_crc_value	output	NUM_CHANNELS × 32	Running CRC-32 per channel
read_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid (set after first accepted beat)
read_beat_count	output	NUM_CHANNELS × 32	Per-channel beats emitted
read_beat_count_total	output	32	Sum of per-channel beat counts (harness stop trigger)

11.3.4 AXI4 Read Address Channel (AR)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read address ID (low bits select the channel context)
s_axi_araddr	input	AXI_ADDR_WIDTH	Read address (unused for data generation)
s_axi_arlen	input	8	Burst length minus 1 (beats − 1)

Port	Direction	Width	Description
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock type
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection attributes
s_axi_arqos	input	4	Quality of service
s_axi_arregion	input	4	Region identifier
s_axi_aruser	input	AXI_USER_WIDTH	User sideband (echoed on R)
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready

11.3.5 AXI4 Read Data Channel (R)

Port	Direction	Width	Description
s_axi_rid	output	AXI_ID_WIDTH	Read data ID (echoes captured `arid`)
s_axi_rdata	output	AXI_DATA_WIDTH	LFSR pattern data (32-bit LFSR replicated to fill the bus)
s_axi_rresp	output	2	Response, always `OKAY`
s_axi_rlast	output	1	Last beat of burst
s_axi_ruser	output	AXI_USER_WIDTH	User sideband (echoes captured `aruser`)
s_axi_rvali	output	1	R valid

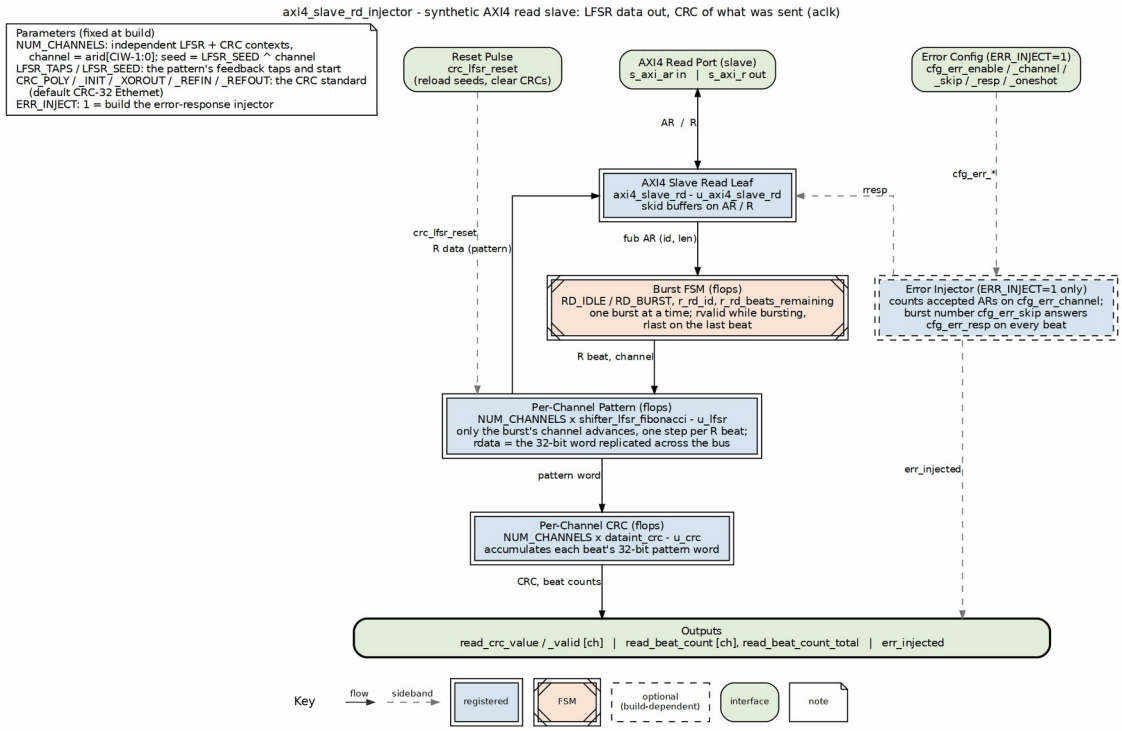
Port	Direction	Width	Description
d			
s_axi_rready	input	1	R ready

11.3.6 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from `axi4_slave_rd`)

11.4 Functional Description

11.4.1 Block diagram



axi4_slave_rd_injector block diagram

`axi4_slave_rd` handles the AR/R protocol and its skid buffers. The burst FSM serves one AR at a time, and the low bits of `arid` pick the channel. On each R beat only that channel's LFSR advances, and its 32-bit word, replicated across the bus, becomes the R data that returns up to the port. The same channel's CRC accumulates that word, so `read_crc_value` is the CRC of what was actually sent. `crc_lfsr_reset` reloads every seed and clears the CRCs. With `ERR_INJECT=1`, a chosen burst on a chosen channel answers `cfg_err_resp` on every beat.

11.4.2 LFSR Pattern Generation

Each channel owns a `shifter_lfsr_fibonacci` instance (32-bit, taps `{23,3,2,1}`) seeded with `LFSR_SEED ^ channel_index`. XOR-ing the channel index into the seed guarantees each channel produces a distinct stream, so channel interleave at the shared AXI port cannot corrupt any one channel's sequence. Only the channel currently being served advances on a given R-beat handshake; all other channel LFSRs hold their state. On the global `crc_lfsr_reset` pulse every channel reloads its seed simultaneously.

11.4.3 Data Replication to the Bus Width

The 32-bit LFSR output is expanded to `AXI_DATA_WIDTH` by replication. A generate block picks the right path: a direct assignment when the bus is exactly 32 bits, a whole-number replication when

the width is a multiple of 32, or a replicate-and-truncate for non-aligned widths (`REPLICATION_FACTOR = (AXI_DATA_WIDTH+31)/32` copies, sliced down). The active channel's replicated word drives `s_axi_rdata`.

11.4.4 Per-Channel CRC-32

Each channel has its own `dataint_crc` instance (CRC-32/Ethernet, `REFIN`/`REFOUT` = 1). The CRC absorbs one 32-bit LFSR word per accepted beat for that channel (`load_from_cascade` gated by "R beat AND active channel == this channel", `cascade_sel = 4'b1000`). A per-channel valid flag sets on the first accepted beat and a per-channel beat counter increments alongside. `crc_lfsr_reset` clears the CRC (via `load_crc_start`), the valid flag, and the counter. The `read_beat_count_total` output is a combinational sum across all channels, used by the harness as a workload-completion trigger.

11.4.5 Burst FSM and Gapless Back-to-Back Bursts

A small two-state FSM (`RD_IDLE`, `RD_BURST`) drives the FUB-side AR/R handshake. AR is accepted when idle *or* on the last beat of the current burst — the latter (`w_rd_last_beat`) lets the next burst reload `r_rd_id` / `r_rd_beats_remaining` on the same cycle the current one finishes, keeping `rvalid` continuously asserted. The original idle-only accept forced a one-cycle dead gap per burst, which surfaced as ~1 starvation cycle per burst on the master's R channel (a slave-model artifact, not a DUT limitation); accepting the AR on `rlast` removes it. `rresp` is hardwired to `OKAY` and `rlast` asserts when `r_rd_beats_remaining` reaches 0.

11.4.6 Standard Protocol Handler

All AR/R skid buffering and AXI protocol compliance are delegated to a `axi4_slave_rd` instance. The pattern-gen FSM and LFSR/CRC logic ride on that module's FUB-side (`fub_axi_ar*` / `fub_axi_r*`) interface, so the external `s_axi_*` ports are fully AXI4-compliant. (A header TODO notes that hand-rolled FUB logic should eventually wrap `axi4_slave_rd_mon` instead — tracked as task #78 — but the current handler is the standard `axi4_slave_rd`.)

11.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

11.6 Usage Examples

```
// Synthetic read source for a 4-channel DMA characterization run.
axi4_slave_rd_injector #(
    .NUM_CHANNELS   (4),
    .AXI_ID_WIDTH   (8),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64),
    .LFSR_SEED      (32'hDEADBEEF)
) u_rd_src (
    .aclk                   (aclk),
    .aresetn                (aresetn),
    .crc_lfsr_reset         (csr_clear_pulse),   // re-arm all channels

    // Telemetry to the harness CSR block
    .read_crc_value         (rd_crc_value),
    .read_crc_valid         (rd_crc_valid),
    .read_beat_count        (rd_beat_count),
    .read_beat_count_total (rd_beats_total),

    // AR/R wired to the DUT's read master port
    .s_axi_arid    (dut_arid),   .s_axi_araddr  (dut_araddr),
    .s_axi_arlen   (dut_arlen),  .s_axi_arsize  (dut_arsize),
    .s_axi_arburst (dut_arburst),.s_axi_arlock  (dut_arlock),
    .s_axi_arcache (dut_arcache),.s_axi_arprot  (dut_arprot),
    .s_axi_arqos   (dut_arqos),  .s_axi_arregion(dut_arregion),
    .s_axi_aruser  (dut_aruser), .s_axi_arvalid (dut_arvalid),
    .s_axi_arready (dut_arready),

    .s_axi_rid   (dut_rid),   .s_axi_rdata  (dut_rdata),
    .s_axi_rresp (dut_rresp), .s_axi_rlast  (dut_rlast),
    .s_axi_ruser (dut_ruser), .s_axi_rvalid (dut_rvalid),
    .s_axi_rready(dut_rready),

    .busy (rd_busy)
);
```

11.7 Design Notes

- **Seed-per-channel independence:** `LFSR_SEED ^ N` is deliberately simple but sufficient — a maximal-length LFSR seeded at distinct points produces streams that stay decorrelated over the characterization window, so per-channel CRCs are independently checkable.
- **CRC over the LFSR word, not the replicated bus:** only the 32-bit LFSR output feeds the CRC (`cascade_sel = 4'b1000`), so the CRC value is bus-width independent and interchangeable with the 32-bit-wide write checker and AXIS blocks.
- **Gapless accept is a measurement fix:** the back-to-back AR accept was added specifically so the read model wouldn't inject a false ~6% starvation artifact into the master's utilization numbers.
- **Reset separation:** `crc_lfsr_reset` is distinct from `aresetn` so the harness can re-arm the pattern between runs without a full block reset.
- **Standards note:** the AR/R glue is hand-rolled on top of `axi4_slave_rd`; the header flags a future refactor to the monitored `axi4_slave_rd_mon` wrapper (task #78).

11.8 Related Modules

11.8.1 Used By

- `axi4_dma_slaves` — bundles this read source with `axi4_slave_wr_crc_check` into a single source/sink slave pair
- `projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv` — on-chip read stimulus
- `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip read stimulus

11.8.2 Uses

- **axi4_slave_rd.sv** — standard AXI4 read slave protocol handler (AR/R skid + compliance)
- **shifter_lfsr_fibonacci.sv** — per-channel maximal-length Fibonacci LFSR
- **dataint_crc.sv** — per-channel CRC-32 accumulator

11.8.3 See Also

- **axi4_slave_wr_crc_check.sv** — the matching write-side CRC sink (same CRC config)
- **axi4_dma_slaves.sv** — the source/sink bundle
- **axis4_master_injector.sv** — AXIS equivalent of this generator

---

11.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

11.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — pattern generator plus signature compactor; the slave regenerates the LFSR stream and accumulates a CRC-32 signature in place of a MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the CRC-32 polynomial (0x04C11DB7).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A5.1 “AXI transaction identifiers” — the per-ID ordering that makes per-channel LFSR/CRC contexts demuxed by ID well defined.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Moon, T.K. *Error Correction Coding*, 2nd ed., Wiley, 2021, ch. 4 “Cyclic Codes, Rings, and Polynomials”, sec. 4.13 “Binary CRC Codes” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

11.10.1 Source Code

- RTL: `rtl/amba/shared/axi4_slave_rd_injector.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_slave_rd.sv`

11.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/README.md`

Last Updated: 2026-07-15

11.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

12 AXI4 Slave Write CRC Checker

Module: `axi4_slave_wr_crc_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

12.1 Overview

`axi4_slave_wr_crc_check` is a write-only AXI4 slave that accepts AW/W bursts, folds the written data into a per-channel CRC-32, and returns B responses. It is the *sink* counterpart to `axi4_slave_rd_injector`: a master under test writes data, this slave CRCs it, and the resulting CRC can be compared against an independently computed golden value to prove the write path carried the data intact.

The write half of a DMA characterization loop needs a slave that accepts write traffic at full rate and reports whether the data arrived correctly. This module CRCs the incoming write data per channel using exactly the same CRC-32 configuration as the read pattern generator, so if a master reads the LFSR pattern and writes it straight back, the write-side CRC must equal the read-side CRC. Any mismatch localizes a data-path corruption to the write path.

Use cases: - Terminating a DMA / streaming engine's write (`m_axi_wr`) port during characterization - End-to-end integrity checks where read data is looped back and re-verified on write - Per-channel multi-context validation on a shared write port - On-chip (FPGA) write sink in the STREAM / RAPIDS characterization harnesses

Key benefit: a memory-free write slave that produces a per-channel CRC-32 directly comparable to the read generator — a single register compare confirms the write path is clean, and the sink never backpressures the master unnecessarily.

12.1.1 Key Features

- Write-only AXI4 slave (AW + W + B channels) built on the standard `axi4_slave_wr` protocol handler
- Per-channel independent CRC-32 state, demuxed off the low bits of the captured `awid`
- CRC-32/Ethernet (poly `0x04C11DB7`, reflected in/out) — bit-identical to `axi4_slave_rd_injector`
- Configurable 32-bit data slice (`CRC_SLICE_OFFSET`) selects which 32-bit lane of a wide bus is CRC'd
- Gapless back-to-back bursts (AW accepted on the last W beat) so `wready` never drops mid-stream
- Separately-latched B id/user so back-to-back bursts return the correct response id
- Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers

12.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent CRC contexts; channel index is `awid[CIW-1:0]`
SKID_DEPTH_AW	int	2	AW channel skid buffer depth
SKID_DEPTH_W	int	4	W channel skid buffer depth
SKID_DEPTH_B	int	2	B channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits processed per CRC update
CRC_POLY	logic [31:0]	32’h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32’hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32’hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output
CRC_SLICE_OFFSET	int	0	Which 32-bit slice of `wdata` to CRC (in 32-bit units)
CIW	int	(NUM_CHANNELS>1) ? $clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: `CRC_*` must match `axi4_slave_rd_injector` exactly. A compile-time `$error` fires if `CRC_SLICE_OFFSET` selects a slice beyond `AXI_DATA_WIDTH`.

12.3 Ports

12.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

12.3.2 Test Control

Port	Direction	Width	Description
crc_reset	input	1	Pulse to clear all channel CRCs, valid flags, and beat counts

12.3.3 Per-Channel Telemetry

Port	Direction	Width	Description
write_crc_value	output	NUM_CHANNELS × 32	Running CRC-32 per channel
write_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid (set after first accepted W beat)
write_beat_count	output	NUM_CHANNELS × 32	Per-channel W beats absorbed
write_beat_count_total	output	32	Sum of per-channel beat counts (harness stop trigger)

12.3.4 AXI4 Write Address Channel (AW)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write address ID (low bits select the channel context)
s_axi_awaddr	input	AXI_ADDR_WIDTH	Write address (unused for CRC)

Port	Direction	Width	Description
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Burst size
s_axi_awburst	input	2	Burst type
s_axi_awlock	input	1	Lock type
s_axi_awcache	input	4	Cache attributes
s_axi_awprot	input	3	Protection attributes
s_axi_awqos	input	4	Quality of service
s_axi_awregion	input	4	Region identifier
s_axi_awuser	input	AXI_USER_WIDTH	User sideband (echoed on B)
s_axi_awvalid	input	1	AW valid
s_axi_awready	output	1	AW ready

12.3.5 AXI4 Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	input	AXI_DATA_WIDTH	Write data (the selected 32-bit slice is CRC'd)
s_axi_wstrb	input	AXI_DATA_WIDTH/8	Write byte strobes (not used to gate the CRC)
s_axi_wlast	input	1	Last beat of burst
s_axi_wuser	input	AXI_USER_WIDTH	User sideband
s_axi_wval	input	1	W valid

Port	Direction	Width	Description
id			
s_axi_wready	output	1	W ready

12.3.6 AXI4 Write Response Channel (B)

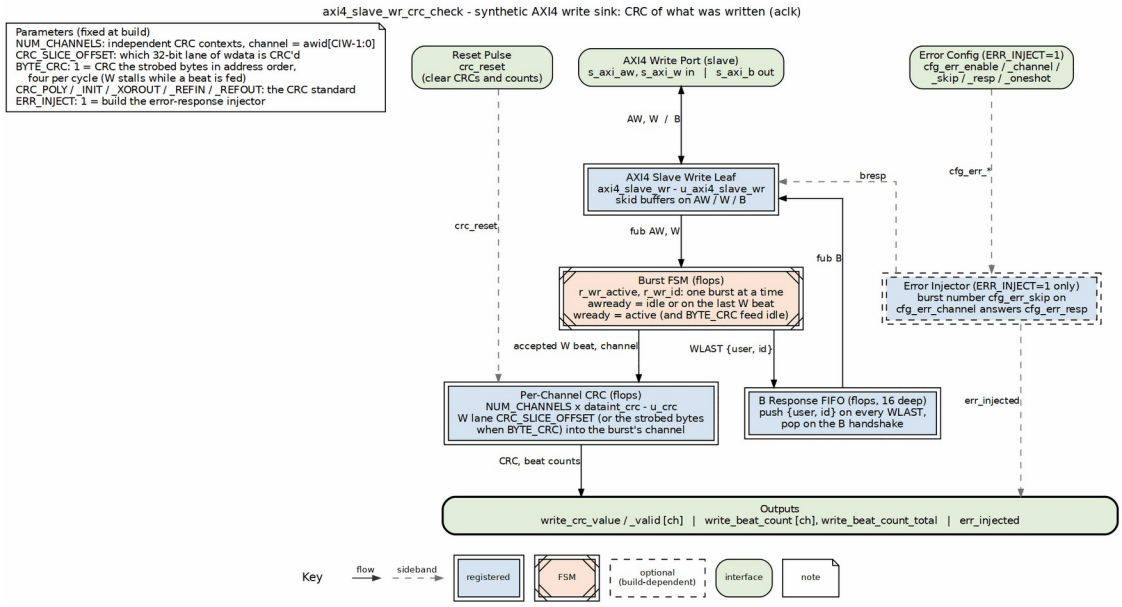
Port	Direction	Width	Description
s_axi_bid	output	AXI_ID_WIDTH	Response ID (separately latched at WLAST)
s_axi_bresp	output	2	Response, always `OKAY`
s_axi_buser	output	AXI_USER_WIDTH	User sideband
s_axi_bvalid	output	1	B valid
s_axi_bready	input	1	B ready

12.3.7 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from `axi4_slave_wr`)

12.4 Functional Description

12.4.1 Block diagram



axi4_slave_wr_crc_check block diagram

`axi4_slave_wr` handles the AW/W/B protocol and its skid buffers. The burst FSM accepts one AW at a time, and the low bits of `awid` pick the channel. Each accepted W beat feeds that channel's CRC: one 32-bit lane of `wdata`, or, with `BYTE_CRC=1`, the strobed bytes in address order, four per cycle, while W is held. Every WLAST pushes the burst's {user, id} into a 16-deep B FIFO, which returns responses up to the port, so back-to-back bursts never drop a B. `crc_reset` clears the CRCs and counts. With `ERR_INJECT=1`, a chosen burst answers `cfg_err_resp`.

12.4.2 Data Slice Extraction

A wide AXI bus carries several 32-bit lanes; only one is CRC'd so the CRC stays interchangeable with the 32-bit read generator. A generate block assigns `data_slice` directly when the bus is 32 bits wide, and otherwise selects `wdata[CRC_SLICE_OFFSET*32 +: 32]`. A compile-time assertion rejects a `CRC_SLICE_OFFSET` that would read past the end of the bus.

12.4.3 Per-Channel CRC-32

Each channel owns a `dataint_crc` instance (CRC-32/Ethernet, `REFIN`/`REFOUT` = 1, `cascade_sel = 4'b1000`) fed the 32-bit `data_slice`. The channel selector `w_active_ch` is the low `CIW` bits of the captured `awid` (`r_wr_id`) — because the AW FSM accepts one burst at a time and W is in-order with AW, the active burst's id identifies the channel during its W beats. A CRC absorbs one word per accepted W beat for its channel (`ch_load_from_cascade` = "accepted W beat AND active channel == this channel"). Per-channel valid flags and beat counters track alongside;

crc_reset clears them all. write_beat_count_total is a combinational sum across channels for the harness stop trigger.

12.4.4 Write Burst FSM and Gapless Bursts

A compact FSM tracks a single active burst via r_wr_active. awready asserts when not active *or* on the last W beat of the current burst (w_wr_last_beat), letting the next burst's AW be accepted back-to-back so wready (= r_wr_active) never drops between bursts. This mirrors the read-slave gapless fix — without it, a one-cycle !active gap per burst would become the write-side throughput limiter once the read slave feeds data gaplessly.

12.4.5 Back-to-Back B Response Id Latching

When bursts run back-to-back, r_wr_id is reloaded with the *next* burst's id on the same cycle the current burst's WLAST lands — so the B channel cannot be driven from it. Instead an inline 16-deep B-response FIFO (BFIFO_DEPTH = 16) pushes the completing burst's {user, id} on every WLAST and pops on the B handshake. Multiple outstanding B responses queue cleanly; gapless multi-channel bursts never drop one. (An earlier single-outstanding r_b_pending design did drop them, which is exactly why the FIFO replaced it.)

12.4.6 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to a axi4_slave_wr instance; the CRC and FSM logic ride on its FUB-side (fub_axi_aw* / fub_axi_w* / fub_axi_b*) interface. (A header TODO tracks a future refactor to wrap the monitored axi4_slave_wr_mon — task #79.)

12.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

12.6 Usage Examples

```
// Synthetic write sink for a 4-channel DMA characterization run.
axi4_slave_wr_crc_check #(
    .NUM_CHANNELS   (4),
    .AXI_ID_WIDTH   (8),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64),
    .CRC_SLICE_OFFSET(0)
) u_wr_sink (
    .aclk                   (aclk),
    .aresetn                (aresetn),
    .crc_reset              (csr_clear_pulse),

    // Telemetry to the harness CSR block
    .write_crc_value        (wr_crc_value),
    .write_crc_valid        (wr_crc_valid),
    .write_beat_count       (wr_beat_count),
    .write_beat_count_total (wr_beats_total),

    // AW/W/B wired to the DUT's write master port
    .s_axi_awid   (dut_awid),   .s_axi_awaddr  (dut_awaddr),
    .s_axi_awlen  (dut_awlen),  .s_axi_awsize  (dut_awsize),
    .s_axi_awburst(dut_awburst),.s_axi_awlock  (dut_awlock),
    .s_axi_awcache(dut_awcache),.s_axi_awprot  (dut_awprot),
    .s_axi_awqos  (dut_awqos),  .s_axi_awregion(dut_awregion),
    .s_axi_awuser (dut_awuser), .s_axi_awvalid (dut_awvalid),
    .s_axi_awready(dut_awready),

    .s_axi_wdata (dut_wdata), .s_axi_wstrb (dut_wstrb),
    .s_axi_wlast (dut_wlast), .s_axi_wuser (dut_wuser),
    .s_axi_wvalid(dut_wvalid),.s_axi_wready(dut_wready),

    .s_axi_bid   (dut_bid),   .s_axi_bresp (dut_bresp),
    .s_axi_buser (dut_buser), .s_axi_bvalid(dut_bvalid),
    .s_axi_bready(dut_bready),

    .busy (wr_busy)
);
```

12.7 Design Notes

- **CRC config must match the source:** the whole point is a comparable value. `CRC_POLY`, `CRC_INIT`, `CRC_XOROUT`, `CRC_REFIN`, `CRC_REFOUT`, and the 32-bit slice width are all fixed to mirror `axi4_slave_rd_injector`.
- **Channel demux by AW id, not W sideband:** because W is in-order behind AW and only one burst is active at a time, the captured `awid` unambiguously names the channel for that burst's W beats.
- **B responses queue through a 16-deep FIFO** (`BFIFO_DEPTH`): up to 16 completed bursts can await their B handshake without loss, so a slow B-drain or gapless multi-channel bursts are safe up to that depth.
- **Gapless accept is a measurement fix:** the back-to-back AW accept prevents the sink from injecting a false ~1-cycle-per-burst starvation into write-side utilization numbers.
- **Standards note:** the AW/W/B glue is hand-rolled on top of `axi4_slave_wr`; a future refactor to `axi4_slave_wr_mon` is tracked as task #79.

12.8 Related Modules

12.8.1 Used By

- `axi4_dma_slaves` — bundles this write sink with `axi4_slave_rd_injector` into a single source/sink slave pair
- `projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv` — on-chip write sink
- `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip write sink

12.8.2 Uses

- **axi4_slave_wr.sv** — standard AXI4 write slave protocol handler (AW/W/B skid + compliance)
- **dataint_crc.sv** — per-channel CRC-32 accumulator

12.8.3 See Also

- **axi4_slave_rd_injector.sv** — the matching read-side pattern source (same CRC config)
- **axi4_dma_slaves.sv** — the source/sink bundle
- **axis4_slave_pattern_check.sv** — AXIS equivalent checker

12.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

12.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — pattern generator plus signature compactor; the slave regenerates the LFSR stream and accumulates a CRC-32 signature in place of a MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the CRC-32 polynomial (0x04C11DB7).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A5.1 “AXI transaction identifiers” — the per-ID ordering that makes per-channel LFSR/CRC contexts demuxed by ID well defined.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Moon, T.K. *Error Correction Coding*, 2nd ed., Wiley, 2021, ch. 4 “Cyclic Codes, Rings, and Polynomials”, sec. 4.13 “Binary CRC Codes” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

12.10.1 Source Code

- RTL: `rtl/amba/shared/axi4_slave_wr_crc_check.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_slave_wr.sv`

12.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/ddr2-characterization/README.md`

Last Updated: 2026-07-15

12.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

13 AXI Bus Meter

Module: `axi_bus_meter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

13.1 Overview

The AXI bus meter is a per-cycle valid/ready classifier for a single AXI data channel. Every clock cycle it inspects the `valid`/`ready` handshake pair and attributes that cycle to one of four buckets — productive, backpressure, starvation, or idle — accumulating both aggregate (32-bit) and per-channel (16-bit) counts. It's a pure passive observer that drives nothing back onto the bus, so you can drop it onto any live AXI R or W channel to characterize datapath utilization.

AXI datapath performance comes down to how often data actually moves versus how often a handshake stalls. A raw throughput number can't distinguish a slow producer (master not offering data) from a congested consumer (slave withholding `ready`). The meter separates those root causes by classifying every cycle into one of four mutually-exclusive buckets, so post-run analysis can compute utilization and — this is the part that matters — attribute lost cycles to the correct side of the interface.

The block is instantiated one per bus to be measured. On a read engine it drops onto the R channel (aggregate plus per-channel via `rid`); on a write engine it drops onto the W channel (aggregate plus per-channel via an engine-side sideband, since AXI4 W beats carry no id).

Use cases: - Metering read R-bus and write W-bus utilization on the stream/rapids characterization engines - Root-causing throughput shortfalls (backpressure-bound vs starvation-bound datapaths) - Per-channel utilization breakdown in multi-channel DMA engines - On-silicon performance characterization runs driven from a host CSR interface

Key benefit: turns a single throughput figure into an attributable four-way cycle budget, so lost bandwidth can be pinned to producer starvation or consumer backpressure per channel.

13.1.1 Key Features

- Four-bucket per-cycle cycle classification (productive / backpressure / starvation / idle)
- Aggregate 32-bit counters (~42.9 s at 100 MHz before wrap) — always attributed based on bus state
- Per-channel 16-bit counters, `NUM_CHANNELS` deep, binned by a caller-supplied channel id
- Per-channel 4-bit sticky overflow mask flags counters that wrapped
- Synchronous one-cycle `i_clear` for atomic reset with the surrounding measurement substrate
- `i_freeze` window control to close the measurement window the instant the workload finishes
- Passive snoop — no protocol interaction, zero backpressure onto the metered bus

13.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of per-channel bins (depth of the per-channel counter arrays)
CW	int	`(NUM_CHANNELS > 1) ? $clog2(NUM_CHANNELS) : 1`	Derived channel-id width; do not override

13.3 Ports

13.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

13.3.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes all counters and overflow stickies
i_freeze	input	1	When high, every counter and overflow sticky holds its value (window closed)

13.3.3 AXI Bus Snoop

Port	Direction	Width	Description
i_valid	input	1	Snooped channel `valid` (e.g. `m_axi_rvalid` / `m_axi_wvalid`)
i_ready	input	1	Snooped channel `ready` (e.g. `m_axi_rready` / `m_axi_wready`)
i_channel_id	input	CW	Channel index selecting the per-channel bin to increment this cycle
i_channel_valid	input	1	Gates per-channel accumulation; high when a channel is on the hook this cycle

13.3.4 Aggregate Counters

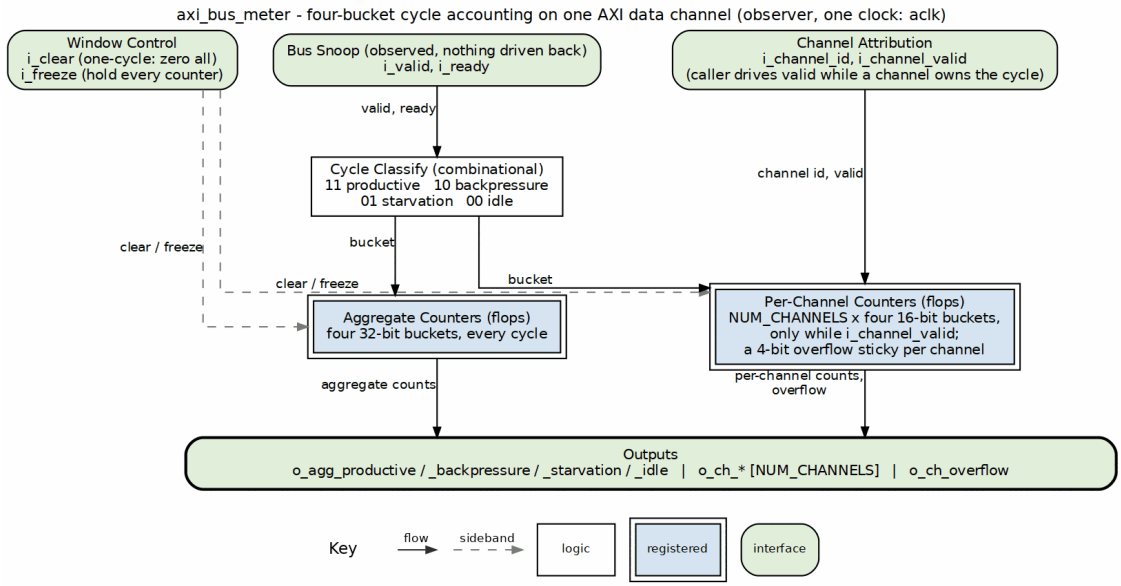
Port	Direction	Width	Description
o_agg_productive	output	32	Cycles with `valid && ready` (data delivered)
o_agg_backpressure	output	32	Cycles with `valid && !ready` (master offering, slave stalling)
o_agg_starvation	output	32	Cycles with `!valid && ready` (slave ready, master not producing)
o_agg_idle	output	32	Cycles with `!valid && !ready` (both sides quiet)

13.3.5 Per-Channel Counters

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive-cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure-cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation-cycle counts
o_ch_idle	output	16 × NUM_CHANNELS	Per-channel idle-cycle counts
o_ch_overflow	output	NUM_CHANNELS*4	Sticky overflow mask, packed `{prod, bp, starv, idle}` per channel

13.4 Functional Description

13.4.1 Block diagram



axi_bus_meter block diagram

The meter only observes; it drives nothing back onto the bus. Each cycle's valid/ready pair picks one of four buckets: productive, backpressure, starvation or idle. The four 32-bit aggregate counters count every cycle. The 16-bit per-channel counters count only while `i_channel_valid` names the channel that owns the cycle, and a sticky bit per counter records any wrap. `i_clear` zeroes everything and `i_freeze` holds the window closed.

13.4.2 Bucket Classification

The four buckets decode combinationally from the handshake pair and are mutually exclusive — exactly one asserts every cycle:

```
w_prod  =  i_valid &&  i_ready   // productive   — data delivered
w_bp    =  i_valid && !i_ready   // backpressure — master wants to
send, slave stalls
w_starv = !i_valid &&  i_ready   // starvation   — slave ready, master
not producing
w_idle  = !i_valid && !i_ready   // idle         — both sides quiet
```

This is the reference methodology documented in `DMA_UTILIZATION_MEASUREMENT.md` §3.

13.4.3 Aggregate Counters

The four 32-bit aggregate counters always increment based on the raw bus state (subject only to `i_clear` and `i_freeze`). At 100 MHz a 32-bit counter lasts ~42.9 s before overflow — ample for a

single characterization run. These provide the whole-bus utilization figure regardless of which channel was active.

13.4.4 Per-Channel Counters and Overflow Stickies

The per-channel arrays are 16 bits wide and `NUM_CHANNELS` deep. Only the bin selected by `i_channel_id` is incremented, and only when `i_channel_valid` is high. The caller asserts `i_channel_valid` whenever some channel is on the hook for the current cycle (for example mid-burst); for cycles where the engine has no work on any channel, `i_channel_valid` is left low and only the aggregate idle/starvation counts are attributed.

Because 16 bits wraps after 65 536 cycles (~655 µs at 100 MHz), each per-channel counter has a companion sticky overflow bit. When a counter is at `16'hFFFF` and would increment again, its overflow bit latches high. The four bits per channel are packed `{prod, bp, starv, idle}` into `o_ch_overflow`. Software checks this mask after stopping the meter; any set bit means the burst outran 16-bit per-channel resolution and the per-channel numbers for that channel should be discarded (the aggregate 32-bit figures remain valid).

13.4.5 Recommended Wiring

- **Read engine:** `i_valid = m_axi_rvalid`, `i_ready = m_axi_rready`, `i_channel_id = m_axi_rid[CW-1:0]`, `i_channel_valid = m_axi_rvalid` (rid is only meaningful when rvalid is high).
- **Write engine:** `i_valid = m_axi_wvalid`, `i_ready = m_axi_wready`, `i_channel_id` = the engine's internal write-channel-id sideband, `i_channel_valid` = a "W beats in flight on this channel" indicator (W beats carry no id in AXI4).

13.4.6 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse: on the cycle it is high, all counters and all overflow stickies reset to zero. Wire it to the same control bit that clears the surrounding measurement state (debug SRAM, harness CRC) so a single CSR write atomically resets the entire measurement substrate.

`i_freeze` holds every counter and sticky frozen while high — no bucket increments, no overflow flips. It is driven from the characterization timer's `done` so the window closes the moment the workload finishes. Without it, the lifetime starvation count would drift upward at one bit per cycle during post-burst host polling, contaminating the in-window utilization math. Hold `i_freeze` low for unbounded free-running measurement.

13.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

13.6 Usage Examples

```
// Meter the read engine's R channel: aggregate + per-channel by rid.
axi_bus_meter #(
    .NUM_CHANNELS   (8)
) u_r_meter (
    .aclk               (aclk),
    .aresetn            (aresetn),

    // Window control (share with the char timer + CRC clear)
    .i_clear            (perf_run_rising),   // one-cycle pulse on RUN
rising edge
    .i_freeze           (~perf_run),         // freeze when the window
is closed

    // R channel snoop
    .i_valid            (m_axi_rvalid),
    .i_ready            (m_axi_rready),
    .i_channel_id       (m_axi_rid[2:0]),
    .i_channel_valid    (m_axi_rvalid),      // rid valid only when
rvalid

    // Aggregate readback (to CSR / host)
    .o_agg_productive   (r_agg_prod),
    .o_agg_backpressure (r_agg_bp),
    .o_agg_starvation   (r_agg_starv),
    .o_agg_idle         (r_agg_idle),

    // Per-channel readback
    .o_ch_productive    (r_ch_prod),
    .o_ch_backpressure  (r_ch_bp),
    .o_ch_starvation    (r_ch_starv),
    .o_ch_idle          (r_ch_idle),
    .o_ch_overflow      (r_ch_overflow)
);

// Utilization = productive / (productive + backpressure + starvation
+ idle)
// Backpressure-heavy => consumer bound; starvation-heavy => producer
bound.
```

13.7 Design Notes

13.7.1 Passive Observer

The meter drives nothing back onto the AXI bus — it only reads `valid`/`ready`. You can add it to or remove it from a design without any protocol impact, and multiple meters can snoop different channels independently.

13.7.2 Counter Width Rationale

Aggregate counters are 32-bit because they must survive a full-length run (~42.9 s at 100 MHz). Per-channel counters are deliberately kept at 16 bits to bound area across `NUM_CHANNELS` bins; the sticky overflow mask makes the resulting ~655 µs resolution limit self-diagnosing rather than silently wrong.

13.7.3 Verilator Output Drivers

The per-channel register arrays are copied to the unpacked output arrays through explicit `always_comb` loops, and the overflow stickies are packed into `o_ch_overflow` with a `c*4 +: 4` slice, because Verilator cannot infer unpacked-array output assignments directly from a register array.

13.7.4 Window Alignment

For correct in-window math, drive `i_clear` and `i_freeze` from the same window controller used by the companion `axi_perf_latency_hist` block so all meters open and close on the same cycle boundaries.

13.8 Related Modules

13.8.1 Used By

- `stream_char` datapath utilization metering (read R bus, write W bus)
- `rapids_char` AXI datapath meters (see `projects/NexysA7/.../rapids_char_harness.sv`)
- Host-driven on-silicon performance characterization harnesses

13.8.2 Uses

- **reset_defs.svh** — `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

13.8.3 See Also

- **axis_bus_meter.sv** — AXIS analogue with byte/packet throughput counters
- **axi_perf_latency_hist.sv** — per-channel latency histogram, same window-control convention

13.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

13.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.2.1 “Handshake process” — the four valid/ready combinations that define the productive, backpressure, starvation and idle buckets. No paper: the classification is a standard technique.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 13 “Buffered Flow Control”, sec. 13.3 “Buffer Management and Backpressure” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

13.10.1 Source Code

- RTL: `rtl/amba/shared/axi_bus_meter.sv`

13.10.2 Documentation

- Methodology: `docs/.../DMA_UTILIZATION_MEASUREMENT.md` (window semantics, four-bucket definition)
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2026-07-15

13.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

14 AXI Burst Address Generator

Module: `axi_gen_addr.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

14.1 Overview

The AXI burst address generator calculates the next address for AXI burst transactions, supporting FIXED, INCR, and WRAP burst types. It handles data width conversions and provides both next-address and aligned-address outputs for boundary-aware transaction processing.

14.1.1 Key Features

- Support for all AXI burst types (FIXED, INCR, WRAP)
- Data width conversion handling (DW != ODW)
- Next address calculation with proper increment
- Aligned address generation for boundary checking
- Pure combinational logic for zero-latency operation

14.2 Parameters

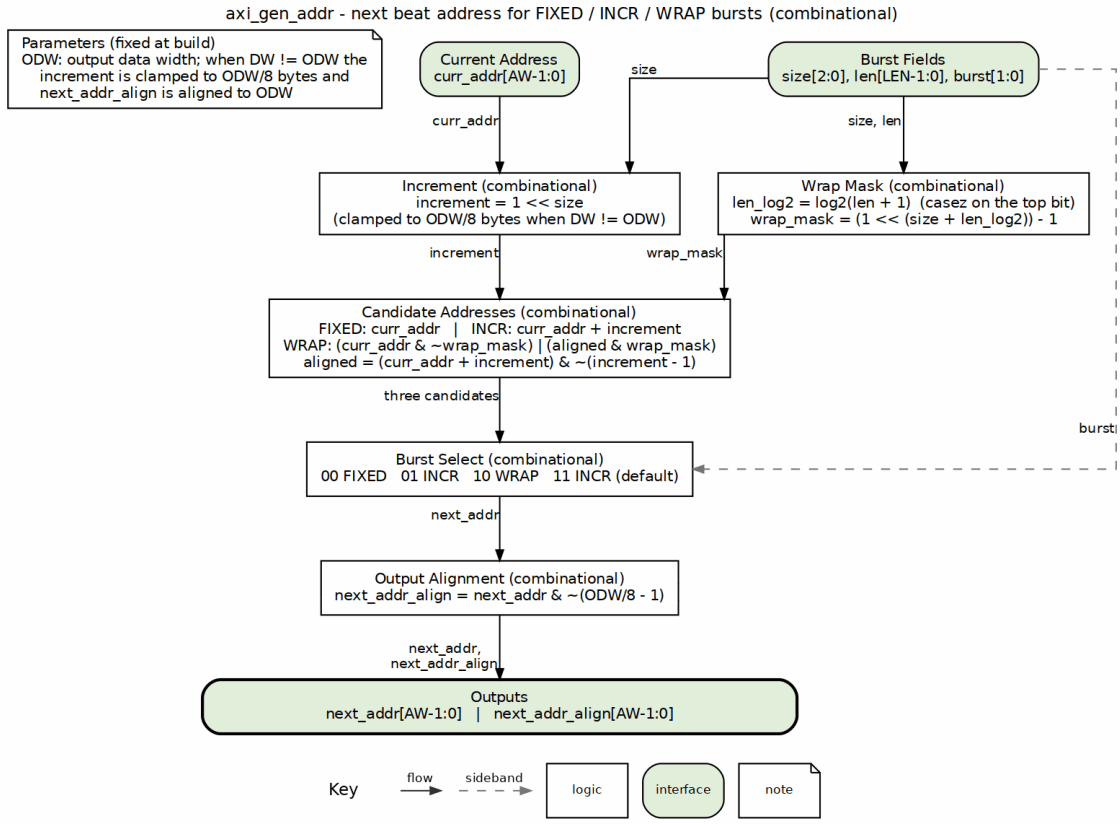
Parameter	Type	Default	Description
AW	int	32	Address bus width
DW	int	32	Input data width (internal)
ODW	int	32	Output data width (external bus)
LEN	int	8	Burst length field width

14.3 Ports

Port	Direction	Width	Description
curr_addr	input	AW	Current address
size	input	3	AXI size encoding (2^size bytes per beat)
burst	input	2	AXI burst type (00=FIXED, 01=INCR, 10=WRAP)
len	input	LEN	AXI burst length (beats - 1)
next_addr	output	AW	Next transaction address
next_addr_align	output	AW	Next address aligned to ODW

14.4 Functional Description

14.4.1 Block diagram



axi_gen_addr block diagram

Everything is combinational. `size` sets the byte increment, which is clamped to ODW/8 bytes when the internal and output widths differ. `size` and `len` together set the WRAP mask. From these the block forms all three candidate next addresses, and `burst` selects one: FIXED holds the address, INCR adds the increment, and WRAP keeps the high bits and wraps the low bits inside the mask. The result is also aligned down to the output data width as `next_addr_align`.

14.4.2 Increment Calculation

The address increment comes from the size field:

```
increment_pre = (1 << size)  // 2^size bytes per beat
```

When the data widths differ (DW != ODW), the increment is clamped:

```
if (increment_pre > ODWBYTES)
    increment = ODWBYTES;
```

14.4.3 Burst Type Handling

FIXED burst (burst = 2'b00): - Address stays constant for all beats - `next_addr = curr_addr`

INCR burst (burst = 2'b01): - Address increments by size each beat - `next_addr = curr_addr + increment`

WRAP burst (burst = 2'b10): - Address wraps at the burst boundary - Wrap mask = (1 << (size + log2(len+1))) - 1 - Aligned address = (curr_addr + increment) & ~(increment - 1) - Wrap address = (curr_addr & ~wrap_mask) | (aligned_addr & wrap_mask)

14.4.4 Aligned Address Output

The aligned output aligns the next address to the output data width:

```
alignment_mask = ODWBYTES - 1
next_addr_align = next_addr & ~alignment_mask
```

That's what the boundary-crossing detection in the transaction splitters consumes.

14.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

14.6 Usage Examples

```
// Generate next address for AXI4 64-bit bus
axi_gen_addr #(
    .AW   (32),
    .DW   (64),   // Internal 64-bit
    .ODW  (64),   // External 64-bit
    .LEN  (8)
) u_addr_gen (
    .curr_addr       (32'h1000),
    .size            (3'b011),    // 8 bytes per beat
    .burst           (2'b01),     // INCR
    .len             (8'd3),      // 4 beats total
    .next_addr       (addr_next),
    .next_addr_align (addr_aligned)
);

// addr_next = 0x1000 + 8 = 0x1008
// addr_aligned = 0x1008 (already aligned to 8-byte boundary)
```

14.7 Design Notes

14.7.1 Data Width Conversion

When DW != ODW, the module handles the mismatch: - If increment > output bus bytes, clamp to the output bus width - This prevents address increments larger than the physical bus

14.7.2 WRAP Burst Alignment

WRAP bursts need careful address math: - The wrap boundary is determined by size and length - The wrapped address stays within the wrap region - Example: 4-beat wrap (len=3), 8 bytes/beat, at 0x0FF8: wrap_mask = (1<<(3+2))-1 = 0x1F, aligned next = 0x1000, so next_addr = (0x0FF8 & ~0x1F) | (0x1000 & 0x1F) = **0x0FE0** — the address wraps to the start of the 32-byte-aligned region, not past 0x1000

14.7.3 Pure Combinational

The module is purely combinational, for zero latency: - No clock, no reset - Instant address generation - Drop it into address calculation pipelines

---

14.8 Related Modules

14.8.1 Used By

- sdpram_core.sv (both BRAM address trackers)
- axi4_to_apb4_convert.sv (APB beat addressing)
- (The splitters do NOT use this module — their boundary math is inline in axi_split_combi)

14.8.2 See Also

- axi_split_combi.sv (uses aligned addresses for split decisions)

14.9 Testing

Covered from `val/amba/` with the rest of the shared area — run everything with `make -C val/amba clean-all && make -C val/amba run-all-func-parallel`.

14.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (FIXED/INCR/WRAP burst addressing, wrapping-burst rules, 4KB boundary) and A3.4.2 “Pseudocode description of the transfers” — the address sequence this block computes.
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, ch. 7 “Common On-Chip Bus Protocols” (FIXED, INCR and WRAP bursts) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

14.10.1 Specifications

- ARM IHI 0022, Issue H: AMBA AXI and ACE Protocol Specification (sec. A3.4.1 “Address structure”)
- Internal: docs/markdown/rtl-amba/index.md

14.10.2 Source Code

- RTL: `rtl/amba/shared/axi_gen_addr.sv`

Last Updated: 2025-10-24

14.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

15 AXI Master Read Splitter

Module: `axi_master_rd_splitter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

15.1 Overview

The AXI master read splitter takes a single upstream AXI read transaction and splits it into multiple downstream transactions when it crosses a boundary. That's what enables address range interleaving, memory-mapped I/O access, and heterogeneous memory system integration. The R-side return path consolidates RLAST to one pulse per original transaction, so a generic AXI4 master can sit upstream directly — see the consumer contract under R Channel Handling.

In modern SoC designs, memory systems are often partitioned across multiple address ranges, memory controllers, or protection domains. This module solves the fundamental problem of transactions that cross those architectural boundaries.

The problem: - Upstream master issues single AXI read (ADDR=0x0FC0, LEN=7, 8 beats total, 64-byte beats) - Transaction crosses 4KB boundary at 0x1000 - Downstream slaves require separate transactions per boundary region

The solution: - The module detects the boundary crossing using combinational split logic - Automatically generates N split transactions (where N >= 1) - First split: ADDR=0x0FC0, LEN=0 (1 beat to boundary) - Second split: ADDR=0x1000, LEN=6 (7 beats remaining) - All response data aggregated transparently

Key invariants: - Upstream sees exactly ONE transaction request/acceptance - Downstream sees N split transactions (boundary-aligned) - Total response beats equals the original transaction beat count - All AXI signaling (ID, USER, PROT, QOS) preserved across splits

15.1.1 Key Features

- Automatic boundary crossing detection and transaction splitting
- Configurable alignment boundary via the 12-bit `alignment_mask` port (4KB maximum — see Integration)
- AR channel forwarding with address/length calculation
- R channel pass-through with transparent ID handling
- Split transaction tracking via dedicated FIFO interface
- Zero added latency for non-split transactions
- Burst/ID/user fields carried through on the AR side; the R side CONSOLIDATES RLAST to one pulse per original transaction (owed-beat counter — see Consumer contract below), so a generic AXI4 master can sit upstream directly

15.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (ARID, RID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (ARADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (RDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (ARUSER, RUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

15.3 Ports

15.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low asynchronous reset

15.3.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_ready	input	1	Backpressure from error monitor (suppresses fub_aready when asserted)

15.3.3 Master AXI Read Interface (Downstream)

AR Channel (Address Read):

Port	Direction	Width	Description
m_axi_arid	output	IW	Transaction ID for split read transactions
m_axi_araddr	output	AW	Calculated address for current split (aligned to boundaries)
m_axi_arlen	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_arsize	output	3	Transfer size (preserved from original transaction)
m_axi_arb	output	2	Burst type (preserved

Port	Direction	Width	Description
urst			from original, typically INCR)
m_axi_arlock	output	1	Lock type (preserved from original)
m_axi_arcache	output	4	Cache attributes (preserved from original)
m_axi_arprot	output	3	Protection attributes (preserved from original)
m_axi_arqos	output	4	Quality of Service (preserved from original)
m_axi_arregion	output	4	Region identifier (preserved from original)
m_axi_aruser	output	UW	User-defined extension (preserved from original)
m_axi_arvalid	output	1	Valid signal for current split transaction
m_axi_arready	input	1	Ready signal from downstream slave

R Channel (Read Data):

Port	Direction	Width	Description
m_axi_rid	input	IW	Response transaction ID (matches ARID)
m_axi_rdata	input	DW	Read data payload
m_axi_rresp	input	2	Response status (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	input	1	Last beat indicator for current split
m_axi_ruser	input	UW	User-defined extension for response
m_axi_rval	input	1	Valid signal for response

Port	Direction	Width	Description
id			data
m_axi_rready	output	1	Ready signal to downstream slave (pass-through from fub)

15.3.4 Slave AXI Read Interface (Upstream / FUB)

AR Channel (Address Read):

Port	Direction	Width	Description
fub_arid	input	IW	Original transaction ID from upstream master
fub_araddr	input	AW	Original starting address (may cross boundaries)
fub_arlen	input	8	Original burst length (may span multiple boundaries)
fub_arsize	input	3	Transfer size per beat
fub_arburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_arlock	input	1	Lock type for atomic operations
fub_arcache	input	4	Cache attributes
fub_arprot	input	3	Protection attributes
fub_arqos	input	4	Quality of Service priority
fub_arregion	input	4	Region identifier
fub_aruser	input	UW	User-defined extension
fub_arvalid	input	1	Valid signal for original transaction
fub_arready	output	1	Ready to the upstream master, asserted at ADMISSION (the cycle the first split goes downstream) so read data

Port	Direction	Width	Description
			cannot precede acceptance – AXI A3.3.1

R Channel (Read Data):

Port	Direction	Width	Description
fub_rid	output	IW	Response transaction ID (pass-through from m_axi)
fub_rdata	output	DW	Read data payload (pass-through from m_axi)
fub_rresp	output	2	Response status (pass-through from m_axi)
fub_rlast	output	1	Last beat indicator (consolidated: one per ORIGINAL transaction via the owed-beat counter)
fub_ruser	output	UW	User-defined extension (pass-through from m_axi)
fub_rvalid	output	1	Valid signal for response (pass-through from m_axi)
fub_rready	input	1	Ready signal from upstream master

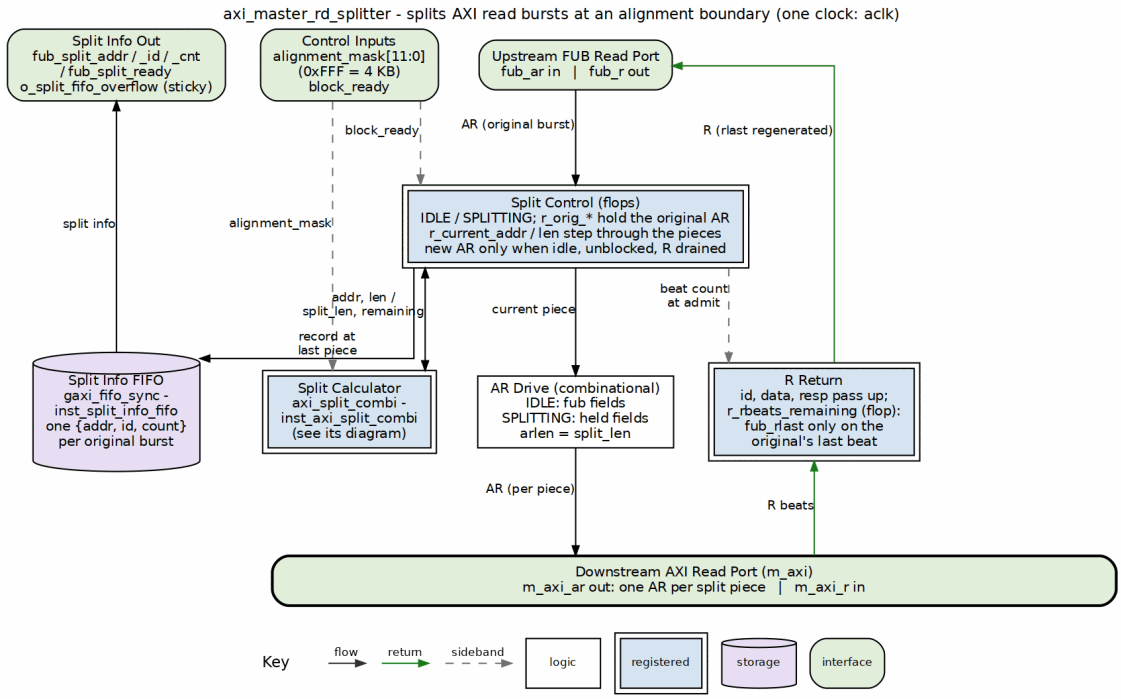
15.3.5 Split Information Interface

Port	Direction	Width	Description
fub_split_addr	output	AW	Original transaction starting address
fub_split_id	output	IW	Original transaction ID
fub_split_cnt	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_valid	output	1	Valid signal for split information
fub_split_r	input	1	Ready signal from split

Port	Direction	Width	Description
eady			information consumer
o_split_fifo_overflow	output	1	STICKY: a split-info record was dropped because the FIFO was full; stays high until reset

15.4 Functional Description

15.4.1 Block diagram



axi_master_rd_splitter block diagram

The original AR comes down the center into split control, which holds its fields and steps `r_current_addr`/len through the pieces. Each step asks `axi_split_combi` where the alignment boundary falls, and the AR drive sends one downstream AR per piece. A new AR is accepted only when the control is idle, `block_ready` is low, and the previous burst's R beats have drained. R beats come back up the right lane and pass through unchanged, except that `fub_rlast` is regenerated from a beat count so the upstream master sees one burst. One split-info record {addr, id, piece count} per original burst goes out through the split-info FIFO. If that FIFO overflows, the sticky `o_split_fifo_overflow` flag reports it.

15.4.2 Transaction Splitting Overview

The module operates as a transparent pass-through for non-split transactions and as an intelligent splitter for boundary-crossing ones.

Non-split path (fast): - Transaction fully contained within a single boundary region - AR signals pass directly from fub to m_axi - fub_aready = m_axi_aready (immediate acceptance) - R channel completely transparent - Split count = 1

Split path (multi-transaction): - Transaction crosses one or more boundary regions - AR signals buffered and regenerated for each split - fub_aready asserts on the FIRST split (admission), not the last: the R channel is a passthrough, so beats for split 1 return as soon as split 1 is accepted

downstream, and AXI A3.3.1 requires the upstream AR handshake to complete first - R channel remains transparent (ID-based aggregation) - Split count = N (number of boundary crossings + 1)

15.4.3 State Machine Architecture

The module implements a simple two-state FSM:

IDLE state: - Awaits new transactions on the fub_ar interface - Combinational split logic evaluates the boundary crossing - If no split: immediate pass-through with fub_aready = m_axi_aready - If split: accept upstream as well, buffer the transaction and transition to SPLITTING to issue the remaining splits from the buffered copy

SPLITTING state: - Issue the current split using the buffered address/length - Update next_addr and remaining_len for the subsequent split - If more splits needed: stay in SPLITTING, increment split_count - If final split: return to IDLE and report the split record with its true count. fub_aready stays low throughout SPLITTING – the original was already accepted

15.4.4 Boundary Crossing Detection

A combinational logic module (axi_split_combi) provides the real-time splitting decisions:

Inputs: - current_addr: transaction start address - current_len: total beats remaining (AXI encoding) - ax_size: bytes per beat - alignment_mask: boundary size (e.g., 0xFFF for 4KB)

Outputs: - split_required: transaction crosses a boundary - split_len: beats to consume in the current split (AXI encoding) - next_boundary_addr: address at next boundary alignment - remaining_len_after_split: beats remaining after the current split

Key calculation:

```
next_boundary_ext = (current_addr | alignment_mask) + 1   // AW+1
bits, so 2**AW is representable
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
split_len = beats_to_boundary - 1  // AXI encoding
```

15.4.5 AR Channel Forwarding Logic

Address/length calculation: - IDLE state: use live fub_araddr and calculated split_len - SPLITTING state: use buffered r_current_addr and split_len - Always output the minimum of (split_len, current_len) for safety

Control signals (ID, PROT, QOS, etc.): - IDLE state: pass through from the fub_ar* signals - SPLITTING state: use the buffered r_orig_ar* signals (captured at acceptance) - This keeps every split transaction carrying identical attributes

Valid signal generation: - IDLE: m_axi_arvalid = fub_arvalid - SPLITTING: m_axi_arvalid = 1 (always asserting the next split)

15.4.6 Ready Signal Management

The protocol constraint that drives everything here: - AXI A3.3.1: a slave must not assert RVALID until the AR handshake it answers has COMPLETED. On the fub port this module IS that slave - The upstream master expects exactly ONE handshake for its request - Suppressing fub_aready until the last split (the behaviour before TASK-094) broke the first rule for every split read: data for split 1 reached the requester while the requester's request was still unaccepted

Ready logic: - IDLE: fub_aready = m_axi_aready && !block_ready && !r_rbeats_active – split or not, acceptance happens at admission, alongside the first downstream split - SPLITTING: fub_aready = 0 (already accepted) - The NEXT original is fenced by r_rbeats_active, not by fub_aready: a second admission while beats are still owed would reload the single owed-beat counter

15.4.7 R Channel Handling

The R channel is completely transparent:

Pass-through signals: - fub_rid = m_axi_rid - fub_rdata = m_axi_rdata - fub_rresp = m_axi_rresp - fub_rlast = owed-beat consolidation: `r_rbeats_active ? (r_rbeats_remaining == 1) : m_axi_rlast` - fub_ruser = m_axi_ruser - fub_rvalid = m_axi_rvalid - m_axi_rready = fub_rready

Consumer contract (read this before integrating): - Split transactions use the same ID as the original; downstream slaves respond with a matching ID and the upstream sees the concatenated beat stream - `fub_rlast` is CONSOLIDATED to one pulse per ORIGINAL transaction: an owed-beat counter (`r_rbeats_remaining`, loaded with the original burst length at acceptance) retires one beat per upstream R handshake and asserts `fub_rlast` only on the final owed beat; intermediate splits' `m_axi_rlast` pulses are absorbed - A standards-compliant generic AXI master can sit upstream directly — it sees exactly one RLAST per burst it issued — because acceptance is SERIALIZED: the owed-beat counter is single-transaction state, so `r_rbeats_active` fences admission (fub_aready, the FSM capture and m_axi_arvalid in IDLE all carry `! r_rbeats_active`) until the previous burst's final beat has returned. One outstanding upstream read at a time is the throughput cost of the consolidation; mutation-proven by the back-to-back scenario in the rd splitter TB. (The pre-fix behavior passed every intermediate RLAST through and required a beat-counting consumer; that defect was closed with the splitter cluster in vault/tracking/rtl/amba)

15.4.8 Split Information Tracking

A dedicated FIFO captures metadata for each completed transaction:

FIFO write conditions: - Written when fub_arvalid && fub_aready (transaction accepted) - Contains: {original_addr, transaction_id, split_count}

Split count values: - No split: split_cnt = 1 - Split occurred: split_cnt = N (number of splits generated)

Use cases: - Performance monitoring (detect excessive splitting) - Error correlation (map responses to original requests) - Debug visibility (trace transaction decomposition)

15.5 Timing Characteristics

This module is **sequential**: it contains 1 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

15.6 Usage Examples

15.6.1 Basic Integration (4KB Boundary Alignment)

```
axi_master_rd_splitter #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .SPLIT_FIFO_DEPTH   (16)
) u_rd_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
    .block_ready        (error_detected),

    // Upstream AXI read interface (from master)
    .fub_arid           (master_arid),
    .fub_araddr         (master_araddr),
    .fub_arlen          (master_arlen),
    .fub_arsize         (master_arsize),
    .fub_arburst        (master_arburst),
    .fub_arlock         (master_arlock),
    .fub_arcache        (master_arcache),
    .fub_arprot         (master_arprot),
    .fub_arqos          (master_arqos),
    .fub_arregion       (master_arregion),
    .fub_aruser         (master_aruser),
    .fub_arvalid        (master_arvalid),
    .fub_arready        (master_arready),

    .fub_rid            (master_rid),
    .fub_rdata          (master_rdata),
    .fub_rresp          (master_rresp),
    .fub_rlast          (master_rlast),
    .fub_ruser          (master_ruser),
    .fub_rvalid         (master_rvalid),
    .fub_rready         (master_rready),

    // Downstream AXI read interface (to memory system)
    .m_axi_arid         (mem_arid),
    .m_axi_araddr       (mem_araddr),
    .m_axi_arlen        (mem_arlen),
```

```
    .m_axi_arsize       (mem_arsize),
    .m_axi_arburst      (mem_arburst),
    .m_axi_arlock       (mem_arlock),
    .m_axi_arcache      (mem_arcache),
    .m_axi_arprot       (mem_arprot),
    .m_axi_arqos        (mem_arqos),
    .m_axi_arregion     (mem_arregion),
    .m_axi_aruser       (mem_aruser),
    .m_axi_arvalid      (mem_arvalid),
    .m_axi_arready      (mem_arready),

    .m_axi_rid          (mem_rid),
    .m_axi_rdata        (mem_rdata),
    .m_axi_rresp        (mem_rresp),
    .m_axi_rlast        (mem_rlast),
    .m_axi_ruser        (mem_ruser),
    .m_axi_rvalid       (mem_rvalid),
    .m_axi_rready       (mem_rready),

    // Split information for monitoring
    .fub_split_addr     (split_addr),
    .fub_split_id       (split_id),
    .fub_split_cnt      (split_count),
    .fub_split_valid    (split_valid),
    .fub_split_ready    (split_ready)
);

// Connect split information to monitoring infrastructure
gaxi_fifo_sync #(
    .DATA_WIDTH         (32 + 8 + 8),  // addr + id + count
    .DEPTH              (64)
) u_split_fifo (
    .axi_aclk           (axi_clk),
    .axi_aresetn        (axi_rst_n),
    .wr_valid           (split_valid),
    .wr_data            ({split_addr, split_id, split_count}),
    .wr_ready           (split_ready),
    .rd_valid           (monitor_split_valid),
    .rd_data            (monitor_split_data),
    .rd_ready           (monitor_split_ready),
    .count              (split_fifo_level)
);
```

15.6.2 Transaction Splitting Example Scenario

Original transaction: - Address: 0x0FC0 (4032 bytes into 4KB region) - Length: 7 (8 beats total in AXI encoding) - Size: 3'b011 (8 bytes per beat = 64 bits) - Total bytes: 8 beats × 8 bytes = 64 bytes - End address: 0x0FC0 + 64 - 1 = 0x0FFF

Boundary analysis: - 4KB boundary at 0x1000 - Bytes to boundary: 0x1000 - 0x0FC0 = 64 bytes - Beats to boundary: 64 / 8 = 8 beats - Transaction crosses boundary: NO (ends exactly at boundary) - Split required: NO

Modified scenario (crosses boundary): - Address: 0x0FC0 - Length: 8 (9 beats) - Total bytes: 9 × 8 = 72 bytes - End address: 0x0FC0 + 72 - 1 = 0x1007

Split sequence: 1. First split: ADDR=0x0FC0, LEN=7 (8 beats to reach 0x1000) 2. Second split: ADDR=0x1000, LEN=0 (1 remaining beat) 3. Split count = 2

15.7 Design Notes

15.7.1 AXI Protocol Assumptions

The module enforces several assumptions you need to hold for correct operation:

Assumption 1: Address alignment - All AXI addresses aligned to the data bus width - If DATA_WIDTH = 512 bits, ARADDR is always 64-byte aligned - Verified by assertions in axi_split_combi

Assumption 2: Fixed transfer size - All transfers use the maximum size equal to the bus width - ARSIZE always matches log2(DATA_WIDTH/8) - Example: 64-bit bus → ARSIZE = 3'b011 (8 bytes)

Assumption 3: Incrementing bursts only - Only INCR burst type supported (ARBURST = 2'b01) - No FIXED or WRAP burst handling - Simplifies the address calculation logic

Assumption 4: No address wraparound - Transactions never wrap 0xFFFFFFFF → 0x00000000 - Guaranteed by system software / memory allocators - Enables the simplified boundary crossing logic - The BOUNDARY above the transaction may still land at `2**AW`; `axi_split_combi` carries it in AW+1 bits so the top alignment window does not produce a spurious split (TASK-095)

15.7.2 Performance Characteristics

Latency: - No split: zero added cycles (combinational pass-through) - Split required: 1 cycle per split transaction - Example: 3-way split adds 2 cycles total latency

Throughput: - Full AXI bandwidth for non-split traffic - Sustained split traffic: limited by the split calculation (1 cycle/split) - R channel: full bandwidth (pass-through)

Area: - Minimal overhead: state machine + transaction buffer - Major component: split information FIFO - Configurable via the SPLIT_FIFO_DEPTH parameter

15.7.3 Error Handling

Split-FIFO overflow (observable, sticky): - The split-info FIFO's `wr_ready` is connected; a push attempted while the FIFO is full sets the STICKY output `o_split_fifo_overflow`, which stays high until reset — integration logic should treat it as a fatal configuration error (a split record was lost) - Sizing the FIFO >= the true maximum outstanding split transactions is still a CORRECTNESS requirement; the sticky flag makes a violation observable rather than silent

Protocol violations: - The module assumes well-formed AXI transactions - Assertions validate the assumptions (address alignment, ax_size, no-WRAP — in `axi_split_combi`; burst TYPE itself is not asserted) - Synthesis-time checks enforce parameter constraints

Backpressure propagation: - `block_ready` gates the full acceptance path: `fub_arready`, the split-FSM capture, AND `m_axi_arvalid` in IDLE (`m_axi_arvalid = fub_arvalid && ! block_ready`), so an AR presented while `block_ready=1` neither issues downstream nor is captured — blocking, not duplication (the TASK-061 duplication defect is fixed and mutation-proven)

15.7.4 Integration Considerations

Alignment mask configuration: - The mask port is 12 bits and zero-extended, so the largest expressible boundary is 0xFFF = 4KB; smaller powers of two (0x3F = 64B … 0x7FF = 2KB) are the other legal values - Driving a wider value (e.g. 0xFFFF hoping for 64KB) silently truncates to 0xFFF and splits at 4KB - Must match downstream memory region alignment

FIFO sizing: - SPLIT_FIFO_DEPTH should accommodate burst traffic - Typical value: 16 entries for general-purpose use - High-performance: 32-64 entries

Clock domain constraints: - Single clock domain (aclk) for all interfaces - No CDC required within the module - Ensure timing closure on the split calculation path

---

15.8 Related Modules

15.8.1 Used By

- Memory controller wrappers requiring boundary-aligned transactions
- Multi-bank DRAM controllers with interleaving requirements
- Bridge modules interfacing heterogeneous memory regions
- DMA engines with address range constraints

15.8.2 Uses

- **axi_split_combi.sv** — combinational splitting logic (boundary detection, length calculation)
- **gaxi_fifo_sync.sv** — split information tracking FIFO
- **reset_defs.svh** — standard reset macro definitions

15.8.3 See Also

- **axi_master_wr_splitter.sv** — write channel splitting (AW + W channels)
- **axi_split_combi.sv** — combined read/write splitting wrapper

15.9 Testing

- Tests: `val/amba/test_axi_master_rd_splitter.py`
- Testbench: `bin/TBClasses/amba/axi_master_rd_splitter_tb.py`

The back-to-back scenario in the testbench mutation-proves the RLAST consolidation, and the TASK-061 `block_ready` duplication fix is mutation-proven as well.

15.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (“A burst must not cross a 4KB address boundary”), A3.3.1 “Dependencies between channel handshake signals”, and A3.4.5 “Read and write response structure” — the boundary rule the splitter enforces and the response it consolidates.
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, ch. 7 “Common On-Chip Bus Protocols” (AXI burst length, size and type) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

15.10.1 Source Code

- RTL: `rtl/amba/shared/axi_master_rd_splitter.sv`

15.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- AXI Specification: AMBA AXI and ACE Protocol Specification, ARM IHI 0022, Issue H (sec. A3.4.1, 4KB boundary)
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2025-10-24

15.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

16 AXI Master Write Splitter

Module: `axi_master_wr_splitter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

16.1 Overview

The write half of the splitter pair earns its keep. Splitting a boundary-crossing read is mostly bookkeeping — the responses flow back upstream as they arrive. Splitting a write is harder: the downstream slave returns one B-channel response per split, but the upstream master issued exactly one transaction and expects exactly one response. This module detects the crossing, splits the transaction, keeps the AW and W channels synchronized while it does it, and folds N responses into one consolidated response with proper error priority handling.

16.1.1 Key Features

- Automatic boundary crossing detection for write transactions
- AW + W channel forwarding with synchronized address/data flow
- B channel response consolidation (N responses → 1 consolidated response)
- Error priority handling (DECERR > SLVERR > EXOKAY > OKAY)
- The consolidated response combines the registered fold with the IN-FLIGHT final response combinationally (`fub_bresp = w_is_final_response ?  w_resp_with_current : OKAY`), so an error on the LAST split is included – the historical error-on-last-reads-as-success defect was fixed and mutation-proven with the splitter cluster (vault/tracking/rtl/amba)
- WLAST regeneration for split boundaries
- Zero added latency for non-split transactions
- Full AXI4 protocol compliance with user extensions

Write transactions couple three channels — address (AW), data (W), and response (B) — that handshake on independent timing. That coupling is what makes write splitting a genuinely different problem from read splitting, and it's the problem this module solves without bending the protocol.

Problem Statement: - Upstream master issues write (ADDR=0x0FC0, LEN=7, 8 data beats, 64-byte beats – AXI_DATA_WIDTH=512; at the default 32-bit width this burst would not cross) - Transaction crosses 4KB boundary at 0x1000 - Downstream requires separate transactions per boundary - Each split generates independent response - Upstream expects single consolidated response

Solution: - Module detects boundary crossing using combinational split logic - AW channel: Generates N split address transactions - W channel: Regenerates WLAST at split boundaries - B channel: Consolidates N responses into 1 (error priority) - Example split: - Split 1: ADDR=0x0FC0, LEN=0 (1 beat) → Response: OKAY - Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: OKAY - Consolidated: OKAY (worst error across all splits)

Key Invariants: - Upstream sees exactly ONE write transaction with ONE response - Downstream sees N split transactions with N responses - Consolidated response reflects worst error status -

Write data flows through unchanged (WLAST regenerated) - All AXI signaling preserved across splits

---

16.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (AWID, BID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (AWADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (WDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (AWUSER, WUSER, BUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

16.3 Ports

16.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low asynchronous reset

16.3.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_ready	input	1	Backpressure from error monitor (suppresses fub_awready when asserted)

16.3.3 Master AXI Write Interface (Downstream)

AW Channel (Address Write):

Port	Direction	Width	Description
m_axi_awid	output	IW	Transaction ID for split write transactions
m_axi_awaddr	output	AW	Calculated address for current split (boundary-aligned)
m_axi_awlen	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_awsize	output	3	Transfer size (preserved from original transaction)
m_axi_awb	output	2	Burst type (preserved

Port	Direction	Width	Description
urst			from original, typically INCR)
m_axi_awlock	output	1	Lock type (preserved from original)
m_axi_awcache	output	4	Cache attributes (preserved from original)
m_axi_awprot	output	3	Protection attributes (preserved from original)
m_axi_awqos	output	4	Quality of Service (preserved from original)
m_axi_awregion	output	4	Region identifier (preserved from original)
m_axi_awuser	output	UW	User-defined extension (preserved from original)
m_axi_awvalid	output	1	Valid signal for current split transaction
m_axi_awready	input	1	Ready signal from downstream slave

W Channel (Write Data):

Port	Direction	Width	Description
m_axi_wdata	output	DW	Write data payload (pass-through from fub)
m_axi_wstrb	output	DW/8	Write strobes (pass-through from fub)
m_axi_wlast	output	1	Last beat indicator (regenerated at split boundaries)
m_axi_wuser	output	UW	User-defined extension (pass-through from fub)
m_axi_wvalid	output	1	Valid signal for write data (fub_wvalid gated on r_aw_issued)

Port	Direction	Width	Description
m_axi_wready	input	1	Ready signal from downstream slave

B Channel (Write Response):

Port	Direction	Width	Description
m_axi_bid	input	IW	Response transaction ID (matches AWID)
m_axi_bresp	input	2	Response status (OKAY=00, EXOKAY=01, SLVERR=10, DECERR=11)
m_axi_buser	input	UW	User-defined extension for response
m_axi_bvalid	input	1	Valid signal for response
m_axi_bready	output	1	Ready signal to downstream slave (controlled during consolidation)

16.3.4 Slave AXI Write Interface (Upstream / FUB)

AW Channel (Address Write):

Port	Direction	Width	Description
fub_awid	input	IW	Original transaction ID from upstream master
fub_awaddr	input	AW	Original starting address (may cross boundaries)
fub_awlen	input	8	Original burst length (may span multiple boundaries)
fub_awsize	input	3	Transfer size per beat
fub_awburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_awlock	input	1	Lock type for atomic operations

Port	Direction	Width	Description
fub_awcache	input	4	Cache attributes
fub_awprot	input	3	Protection attributes
fub_awqos	input	4	Quality of Service priority
fub_awregion	input	4	Region identifier
fub_awuser	input	UW	User-defined extension
fub_awvalid	input	1	Valid signal for original transaction
fub_awready	output	1	Ready signal to upstream master (suppressed until all splits complete)

W Channel (Write Data):

Port	Direction	Width	Description
fub_wdata	input	DW	Write data payload from upstream master
fub_wstrb	input	DW/8	Write strobes from upstream master
fub_wlast	input	1	Last beat indicator from upstream master
fub_wuser	input	UW	User-defined extension
fub_wvalid	input	1	Valid signal for write data
fub_wready	output	1	Ready signal to upstream master (m_axi_wready gated on r_aw_issued)

B Channel (Write Response):

Port	Direction	Width	Description
fub_bid	output	IW	Consolidated response transaction ID (original AWID)

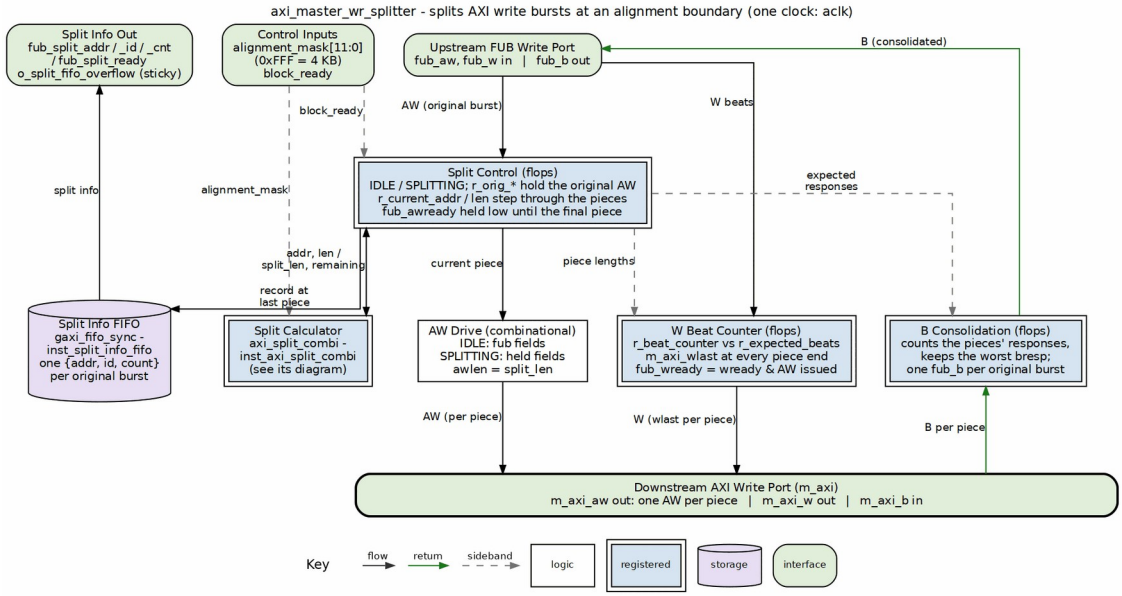
Port	Direction	Width	Description
fub_bresp	output	2	Consolidated response status (worst error across all splits)
fub_buser	output	UW	User-defined extension (from original transaction)
fub_bvalid	output	1	Valid signal for consolidated response (only after all splits received)
fub_bready	input	1	Ready signal from upstream master

16.3.5 Split Information Interface

Port	Direction	Width	Description
fub_split_addr	output	AW	Original transaction starting address
fub_split_id	output	IW	Original transaction ID
fub_split_cnt	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_valid	output	1	Valid signal for split information
fub_split_ready	input	1	Ready signal from split information consumer
o_split_fifo_overflow	output	1	STICKY: a split-info record was dropped because the FIFO was full; stays high until reset

16.4 Functional Description

16.4.1 Block diagram



axi_master_wr_splitter block diagram

The original AW comes down the center into split control, which uses `axi_split_combi` to issue one downstream AW per piece and holds `fub_awready` low until the final piece issues. W beats go through the beat counter, which raises `m_axi_wlast` at the end of every piece. `fub_wready` follows `m_axi_wready`, but only once the current AW has been issued. B responses come back up the right lane into the consolidation logic. It counts one response per piece, keeps the worst `bresp`, and sends a single `fub_b` for the original burst. One split-info record per original burst goes out through the split-info FIFO, with a sticky overflow flag.

16.4.2 Write Transaction Splitting Overview

Write splitting is a juggling act: three independent AXI channels with different timing characteristics, one transaction that has to stay coherent. The module has two paths through it.

Non-Split Path (Fast): - Transaction fully contained within single boundary - AW/W signals pass directly from fub to m_axi - fub_awready = m_axi_awready (immediate acceptance) - B channel passes through directly - Split count = 1

Split Path (Multi-Transaction + Response Consolidation): - Transaction crosses one or more boundaries - AW signals buffered and regenerated for each split - W channel WLAST regenerated at split boundaries - B channel: Collect N responses, consolidate into 1 - fub_awready suppressed until final split accepted - Split count = N (number of splits)

16.4.3 State Machine Architecture

Same skeleton as the read splitter, with extra bookkeeping for the W channel:

IDLE State: - Awaits new transactions on fub_aw interface - Combinational split logic evaluates boundary crossing - If no split: pass-through mode (single response) - If split: buffer transaction, setup response consolidation, transition to SPLITTING

SPLITTING State: - Issue current split using buffered address/length - Track W channel beats to regenerate WLAST correctly - Collect B channel responses (consolidate errors) - Update next_addr and remaining_len for next split - Return to IDLE when final split accepted

16.4.4 Write Data Channel Management

WLAST Regeneration:

This is the part that bites people. When a burst gets cut at a boundary, the WLAST that arrived with the original stream lands in the wrong place for every split except the last — so the module regenerates it.

Beat Counter Tracking: - r_expected_beats: Beats required for current split (from split_len + 1) - r_beat_counter: Current beat being transferred - w_split_boundary_reached: Counter matches expected beats

WLAST Logic:

```
If splitting:
    WLAST = (beat_counter + 1 >= expected_beats)
Else:
    WLAST = fub_wlast (pass-through)
```

Example (8 beats split into 1 + 7): - Original: 8 beats with WLAST on beat 8 - Split 1: Beat 1 → Generate WLAST (boundary reached) - Split 2: Beats 2-8 → Original WLAST on beat 8

16.4.5 Response Consolidation Logic

CRITICAL FEATURE: the N split responses have to come back upstream as one.

Consolidation State: - r_expected_response_count: Total splits generated - r_received_response_count: Responses received so far - r_waiting_for_responses: Consolidation active flag - r_consolidated_resp_status: Aggregated error status - r_original_txn_id: Transaction ID for consolidated response

Response Priority (Worst Error Wins):

```
Priority: DECERR (11) > SLVERR (10) > EXOKAY (01) > OKAY (00)

if (m_axi_bresp > r_consolidated_resp_status):
    r_consolidated_resp_status = m_axi_bresp
```

Consolidation Flow:

1. **Setup (at transaction start):**
   - Capture original AWID
   - Initialize consolidated_resp_status = OKAY
   - Set expected_response_count = estimated splits
   - Assert waiting_for_responses flag
2. **Collection (each B response):**
   - Accept response even if fub_bready not asserted
   - OR error status into consolidated_resp_status
   - Increment received_response_count
   - Keep fub_bvalid suppressed
3. **Completion (final response):**
   - Check: received_response_count >= expected_response_count
   - Assert fub_bvalid with consolidated status
   - Wait for fub_bready handshake
   - Clear waiting_for_responses flag

Pass-Through Mode (No Split): - waiting_for_responses = 0 - Direct connection: fub_b* = m_axi_b* - Single response, no aggregation needed

16.4.6 Boundary Crossing Detection

Same combinational engine the read splitter uses (axi_split_combi) — one way to compute a boundary, shared by both.

Inputs: - current_addr, current_len, ax_size, alignment_mask - Same calculation as read splitter

Outputs: - split_required, split_len, next_boundary_addr, remaining_len_after_split - Used for both AW channel splitting and W channel beat tracking

16.4.7 AW Channel Forwarding Logic

Address/Length Calculation: - IDLE state: Use live fub_awaddr and calculated split_len - SPLITTING state: Use buffered r_current_addr and split_len

Control Signals: - IDLE state: Pass through from fub_aw* signals - SPLITTING state: Use buffered r_orig_aw* signals

Valid Signal: - IDLE: m_axi_awvalid = fub_awvalid - SPLITTING: m_axi_awvalid = 1 (asserting next split)

16.4.8 Ready Signal Management

fub_awready Logic: - IDLE + split needed: fub_awready = 0 (suppress) - IDLE + no split: fub_awready = m_axi_awready && !block_ready && !r_waiting_for_responses – the `r_waiting_for_responses` term is the ACCEPTANCE FENCE: no new transaction is accepted while a previous one's response consolidation is still open (one consolidation state set exists; unfenced acceptance was one of the splitter-cluster defects) - SPLITTING + intermediate: fub_awready = 0 - SPLITTING + final split: fub_awready = w_is_final_split && m_axi_awready && ! block_ready

W Channel Ready: - Gated on address issue: `fub_wready = m_axi_wready && r_aw_issued` (and `m_axi_wvalid = fub_wvalid && r_aw_issued`) – W data is HELD until its AW has been accepted downstream, enforcing the repo-wide AW-before-W rule so WLAST regeneration can never run ahead of the address stream - Backpressure otherwise propagates naturally through the AXI protocol

B Channel Ready: - Consolidation mode: m_axi_bready = fub_bready || !w_is_final_response – intermediate split responses are always accepted, but the FINAL split's response waits on the upstream's fub_bready - Pass-through mode: m_axi_bready = fub_bready - Intermediate responses are accepted even if the upstream is not ready; the final response is not

---

16.5 Timing Characteristics

This module is **sequential**: it contains 1 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

16.6 Usage Examples

16.6.1 Basic Integration (4KB Boundary, Response Consolidation)

```
axi_master_wr_splitter #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .SPLIT_FIFO_DEPTH   (16)
) u_wr_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
    .block_ready        (error_detected),

    // Upstream AXI write interface (from master)
    .fub_awid           (master_awid),
    .fub_awaddr         (master_awaddr),
    .fub_awlen          (master_awlen),
    .fub_awsize         (master_awsize),
    .fub_awburst        (master_awburst),
    .fub_awlock         (master_awlock),
    .fub_awcache        (master_awcache),
    .fub_awprot         (master_awprot),
    .fub_awqos          (master_awqos),
    .fub_awregion       (master_awregion),
    .fub_awuser         (master_awuser),
    .fub_awvalid        (master_awvalid),
    .fub_awready        (master_awready),

    .fub_wdata          (master_wdata),
    .fub_wstrb          (master_wstrb),
    .fub_wlast          (master_wlast),
    .fub_wuser          (master_wuser),
    .fub_wvalid         (master_wvalid),
    .fub_wready         (master_wready),

    .fub_bid            (master_bid),
    .fub_bresp          (master_bresp),
    .fub_buser          (master_buser),
    .fub_bvalid         (master_bvalid),
    .fub_bready         (master_bready),
```

```
    // Downstream AXI write interface (to memory)
    .m_axi_awid         (mem_awid),
    .m_axi_awaddr       (mem_awaddr),
    .m_axi_awlen        (mem_awlen),
    .m_axi_awsize       (mem_awsize),
    .m_axi_awburst      (mem_awburst),
    .m_axi_awlock       (mem_awlock),
    .m_axi_awcache      (mem_awcache),
    .m_axi_awprot       (mem_awprot),
    .m_axi_awqos        (mem_awqos),
    .m_axi_awregion     (mem_awregion),
    .m_axi_awuser       (mem_awuser),
    .m_axi_awvalid      (mem_awvalid),
    .m_axi_awready      (mem_awready),

    .m_axi_wdata        (mem_wdata),
    .m_axi_wstrb        (mem_wstrb),
    .m_axi_wlast        (mem_wlast),
    .m_axi_wuser        (mem_wuser),
    .m_axi_wvalid       (mem_wvalid),
    .m_axi_wready       (mem_wready),

    .m_axi_bid          (mem_bid),
    .m_axi_bresp        (mem_bresp),
    .m_axi_buser        (mem_buser),
    .m_axi_bvalid       (mem_bvalid),
    .m_axi_bready       (mem_bready),

    // Split information tracking
    .fub_split_addr     (split_addr),
    .fub_split_id       (split_id),
    .fub_split_cnt      (split_count),
    .fub_split_valid    (split_valid),
    .fub_split_ready    (split_ready)
);
```

16.6.2 Response Consolidation Example

Scenario: 2-way split with error in second split

Original Transaction: - ADDR=0x0FC0, LEN=7 (8 beats), ID=0x42

Split Transactions: 1. Split 1: ADDR=0x0FC0, LEN=0 (1 beat) → Response: OKAY (00) 2. Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: SLVERR (10)

Consolidation Logic:

```
Initial: consolidated_resp = OKAY (00)
Split 1 response: OKAY (00)
    → 00 > 00? No → Keep OKAY
Split 2 response: SLVERR (10)
    → 10 > 00? Yes → Update to SLVERR

Final consolidated response: SLVERR (10)
```

Upstream View: - Master issues 1 write transaction (ADDR=0x0FC0, LEN=7, ID=0x42) - Receives 1 response (ID=0x42) with BRESP=SLVERR: the last split's error is combined combinationally with the accumulated fold (`w_resp_with_current`), so error-on-last upstreams correctly (the historical one-cycle-late fold was the closed splitter-cluster defect) - Behavior: error propagated despite partial success – the final split's in-flight BRESP is combined combinationally, so error-on-last upstreams correctly (the historical loss was TASK-063, closed)

16.7 Design Notes

16.7.1 AXI Protocol Assumptions

Same assumptions as read splitter:

Assumption 1: Address Alignment - All addresses aligned to data bus width

Assumption 2: Fixed Transfer Size - AWSIZE matches data width

Assumption 3: Incrementing Bursts Only - Only INCR burst type supported

Assumption 4: No Address Wraparound - Transactions never wrap address space - The boundary above a transaction may still land at `2**AW`; `axi_split_combi` carries it in AW+1 bits so the top alignment window does not split spuriously (TASK-095)

16.7.2 Response Consolidation Details

Why Consolidation is Critical: - AXI protocol: 1 address transaction → 1 response - Splitting creates N address transactions → N responses - Upstream expects exactly 1 response - Must aggregate N responses into 1 consolidated response

Error Priority Rationale:

```
DECERR (Decode Error):     Transaction to invalid address
SLVERR (Slave Error):      Valid address, processing error
EXOKAY (Exclusive Okay):   Exclusive access succeeded
OKAY:                      Normal successful completion

Worst error wins → System sees most severe failure
```

Consolidation Timing: - Must wait for ALL split responses before asserting fub_bvalid - Prevents partial success reports - Ensures atomicity from upstream perspective

16.7.3 Performance Characteristics

Latency: - No split: Zero added cycles (pass-through) - Split required: 1 cycle per split + response consolidation - Response consolidation adds latency equal to slowest split

Throughput: - AW channel: Limited by split calculation (1 cycle/split) - W channel: Full bandwidth (pass-through with WLAST regeneration) - B channel: Responses accumulated (no upstream throughput impact)

Area: - Overhead: State machine + transaction buffer + response consolidation logic - Major components: Split FIFO + response tracking registers

16.7.4 Error Handling

Split Response Timeout: - Module waits indefinitely for all split responses - No timeout mechanism (relies on downstream protocol compliance) - Recommendation: Implement system-level timeout monitor

FIFO Overflow: - Split FIFO can fill if consumer stalls - block_ready suppresses fub_awready; the split-info FIFO does not backpressure acceptance – instead a push to a full FIFO sets the sticky `o_split_fifo_overflow` output (a record was lost; treat as fatal). Sizing the FIFO for the true maximum outstanding splits remains a correctness requirement - Size FIFO >= max outstanding writes

Data Channel Synchronization: - W channel must deliver exact beat count per split - WLAST regeneration critical for correctness - Assertions validate beat counter operation

16.7.5 Integration Considerations

Response Consolidation Requirements: - Downstream must return exactly 1 response per split - Response IDs must match split transaction IDs - All responses must eventually arrive

WLAST Timing: - Generated combinationally based on beat counter - Timing-critical path in high-speed designs - Consider registering if timing closure issues

Backpressure Handling: - B channel: intermediate split responses accepted unconditionally; the final one handshakes with the upstream (can backpressure the downstream B channel) - Upstream backpressure (fub_bready=0) only delays final consolidated response - Internal buffering for responses: Single register (r_consolidated_resp_status)

16.8 Related Modules

16.8.1 Used By

- Memory controller wrappers with boundary constraints
- Multi-bank DRAM controllers requiring aligned writes
- Bridge modules interfacing heterogeneous memory systems
- DMA engines with address range restrictions

16.8.2 Uses

- **axi_split_combi.sv** - Combinational splitting logic (shared with read splitter)
- **gaxi_fifo_sync.sv** - Split information tracking FIFO
- **reset_defs.svh** - Standard reset macro definitions

16.8.3 See Also

- **axi_master_rd_splitter.sv** - Read channel splitting (AR + R channels)
- **axi_split_combi.sv** - Combined read/write splitting wrapper

16.9 Testing

`val/amba/test_axi_master_wr_splitter.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi_master_wr_splitter.py -v
```

16.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (“A burst must not cross a 4KB address boundary”), A3.3.1 “Dependencies between channel handshake signals”, and A3.4.5 “Read and write response structure” — the boundary rule the splitter enforces and the response it consolidates.
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, ch. 7 “Common On-Chip Bus Protocols” (AXI burst length, size and type) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

16.10.1 Source Code

- RTL: `rtl/amba/shared/axi_master_wr_splitter.sv`
- Tests: `val/amba/test_axi_master_wr_splitter.py`
- Testbench: `bin/TBClasses/amba/axi_master_wr_splitter_tb.py`

16.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- AXI Specification: AMBA AXI and ACE Protocol Specification, ARM IHI 0022, Issue H (sec. A3.4.1, 4KB boundary)
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2025-10-24

16.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

17 AXI Performance Latency Histogram

Module: `axi_perf_latency_hist.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

17.1 Overview

The AXI Performance Latency Histogram captures per-transaction latency on an AXI master bus and bins it into a log2 histogram, surfaced through a CSR-indexed readout — no MonBus packets. It's a self-contained snoop block: it doesn't gate or otherwise touch the AXI datapath, it sits alongside the datapath monitors, and it deliberately leaves the shared `axi_monitor_base` alone. For reads it measures both AR→first-R and AR→RLAST latency; for writes it measures AW→B latency.

17.1.1 Key Features

- Log2 latency histogram, one per metric (default 16 bins: bin `b` counts latencies in `[2^b, 2^(b+1))`). There is NO channel dimension in the histogram storage – all channels fold into one histogram per metric; NUM_CHANNELS sizes only the per-channel command-timestamp FIFOs
- Read mode (`IS_READ=1`): two metrics — AR→first-R beat and AR→RLAST
- Write mode (`IS_READ=0`): one metric — AW→B response
- Per-channel command-timestamp FIFO matches completions to oldest outstanding command (same-ID in-order)
- Four-stage histogram update pipeline for FPGA timing closure
- Free-running (non-frozen) timestamp so latencies straddling the window boundary stay correct
- CSR-indexed readout of any bin count plus each metric's transaction total
- Window control (`i_clear` / `i_freeze`) mirroring `axi_bus_meter`

Average latency hides the distribution that actually matters for QoS analysis — a bus can have a fine mean while a tail of slow transactions strangles a real workload. So this block bins each transaction's measured latency into a log2 histogram, and the shape of the distribution (and its tail) becomes visible from a handful of CSR reads. It snoops the command and completion channels, timestamps each command as it is accepted, and on completion subtracts to get latency and increments the matching log2 bin. It's purely observational and adds no load to the AXI path, so you can drop it in for characterization without perturbing the design under measurement.

Use Cases: - Read/write latency distribution characterization on AXI master ports (perfmon Stage D) - Tail-latency analysis for QoS and arbitration tuning - Parity latency metrics alongside `axi4_intf_master_observer` per-port histograms - Host-driven on-silicon performance runs (CSR readout, no MonBus traffic)

Key Benefit: Exposes the full latency distribution (including the tail) through cheap CSR reads, without touching the shared monitor base or adding any MonBus packet traffic. Distributions are aggregated across channels – per-channel histograms would need one instance per channel.

17.2 Parameters

Parameter	Type	Default	Description
ID_WIDTH	int	8	AXI id width on the snooped bus
NUM_CHANNELS	int	8	Number of per-channel command-timestamp FIFOs (channel = `id[CW-1:0]`); does NOT create per-channel histogram bins
MAX_OUTSTANDING	int	8	Per-channel command-timestamp FIFO depth
NUM_BINS	int	16	Number of log2 histogram bins; bin `b` counts `[2^b, 2^(b+1))`
IS_READ	bit	1’b1	1 = read build (2 metrics), 0 = write build (1 metric)
CNT_W	int	32	Histogram bin and total counter width
CW	int	derived	`$clog2(NUM_CHANNELS)` — channel index width
PW	int	derived	`$clog2(MAX_OUTSTANDING)` — FIFO pointer width
PW1	int	derived	`PW + 1` — occupancy width (extra bit for full)
BINW	int	derived	`$clog2(NUM_BINS)` — bin index width

17.3 Ports

17.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

17.3.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	One-cycle pulse (perf RUN rising edge): resets histogram + FIFOs
i_freeze	input	1	Hold high to freeze the histogram (counters stop; readback stable)

17.3.3 Command Channel (AR for reads, AW for writes)

Port	Direction	Width	Description
cmd_valid	input	1	Command-channel valid
cmd_ready	input	1	Command-channel ready (handshake timestamps the command)
cmd_id	input	ID_WIDTH	Command id; low CW bits select the channel

17.3.4 Read Data Channel (R) — used when IS_READ=1

Port	Direction	Width	Description
data_valid	input	1	R-channel valid
data_ready	input	1	R-channel ready
data_last	input	1	R-channel `rlast` (marks the RLAST metric / FIFO pop)
data_id	input	ID_WIDTH	R-channel id; low CW bits

Port	Direction	Width	Description
			select the channel

17.3.5 Write Response Channel (B) — used when IS_READ=0

Port	Direction	Width	Description
resp_valid	input	1	B-channel valid
resp_ready	input	1	B-channel ready
resp_id	input	ID_WIDTH	B-channel id; low CW bits select the channel

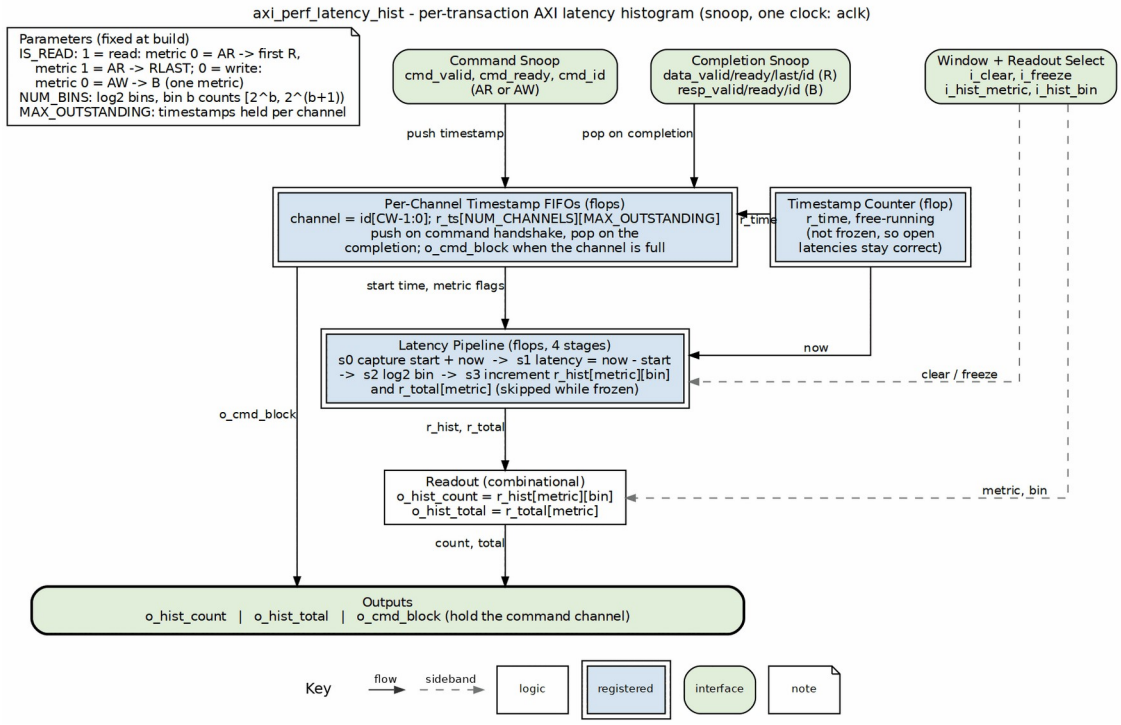
17.3.6 CSR-Indexed Readout

Port	Direction	Width	Description
i_hist_metric	input	1	Metric select: 0 = first-R/B, 1 = RLAST (reads only)
i_hist_bin	input	BINW	Bin index to read
o_hist_count	output	CNT_W	Selected bin's count for the selected metric
o_hist_total	output	CNT_W	Selected metric's total transaction count
o_cmd_block	output	1	High while the addressed channel's timestamp FIFO is full. AND this into the command-channel ready (it is combinational in cmd_id). Without it the module degrades SILENTLY: a command that handshakes at a full channel is never timestamped, is missing from o_hist_total, and its completion later pops some OTHER command's timestamp – surviving latencies are misattributed as well as

Port	Direction	Width	Description
			undercounted

17.4 Functional Description

17.4.1 Block diagram



axi_perf_latency_hist block diagram

Every command handshake pushes the free-running timestamp into the FIFO for the command's channel (`id[CW-1:0]`). The matching completion pops the oldest entry; AXI keeps same-ID responses in order, so the oldest is the right one. A four-stage pipeline captures the start and completion times, subtracts them, bins the latency by log2, and increments that bin and the metric's total. Reads track two metrics (first R beat and RLAST) and writes one (B). Readout is a combinational select by metric and bin. When a channel's FIFO is full, `o_cmd_block` lets the caller hold commands instead of losing the sample.

17.4.2 Metrics

- **`IS_READ=1` (read build), `NUM_METRICS=2`:** metric 0 = AR handshake → first R beat; metric 1 = AR handshake → RLAST beat.
- **`IS_READ=0` (write build), `NUM_METRICS=1`:** metric 0 = AW handshake → B response.

The histogram storage is always sized for two metrics for uniform indexing; the write build uses only metric 0.

17.4.3 Transaction Matching

AXI requires same-ID responses to return in order, and that guarantee buys us something useful: completions can be matched to commands with a simple per-channel FIFO of command-phase timestamps rather than a CAM. On a command handshake (`cmd_valid && cmd_ready`) the current free-running time is pushed into the channel's FIFO (bounded by `MAX_OUTSTANDING`). On completion the oldest timestamp for that channel (`r_ts[ch][head]`) is the start time, and the FIFO is popped when the completing beat is `last` (RLAST for reads; B is always "last" for writes). A single AXI data/response bus carries at most one beat per cycle, so at most one push and one pop occur per cycle — there is no multi-writer hazard on the shared histogram.

For reads, a per-channel `r_burst_active` flag distinguishes the first R beat (metric 0, AR→first-R) from subsequent beats of the same burst; it is set on the first beat and cleared when the burst pops on RLAST.

17.4.4 Log2 Binning

`latency_bin()` returns `floor(log2(lat))` clamped to `NUM_BINS-1`; latencies of 0 or 1 map to bin 0. Bin `b` therefore counts latencies in `[2^b, 2^(b+1))`.

17.4.5 Free-Running Timestamp

`r_time` is a 32-bit counter that increments every cycle and is **not** frozen by `i_freeze`. Keeping it running means a latency measurement that straddles the window-close boundary still subtracts two consistent absolute timestamps and yields the correct value. Only the histogram accumulation is frozen.

17.4.6 Four-Stage Update Pipeline

The chain FIFO-read → latency subtract → log2 bin → indexed histogram increment is far too deep for a single 100 MHz cycle (it was the routed critical path), so it is split into four register stages:

- **Stage 0:** capture the event — `{start_ts, time, metric flags m0/m1, valid}`. Gated by `i_freeze` (a frozen window captures no new events).
- **Stage 1:** latency = completion time − start time.
- **Stage 2:** log2 bin from the latency.
- **Stage 3:** increment the selected histogram bin(s) and the per-metric total(s).

The increment is delayed a few cycles, but the host reads the window long after RUN drops to 0, by which time the pipeline has drained — `i_freeze` gates stage 0 while stages 1–3 keep advancing to flush. Because a single AXI bus completes at most one transaction per cycle and metrics m0/m1 target different histograms, consecutive same-bin increments are hazard-free (the cross-cycle NBA read-after-write is correct).

17.4.7 Window Control

`i_clear` (a one-cycle pulse on the perf RUN rising edge) resets the histograms, totals, and all per-channel FIFOs and state. `i_freeze` (~RUN) holds the histogram so a closed window can be read back, while the free-running timestamp keeps counting.

17.4.8 Readout

Readout is a combinational mux: `o_hist_count` returns `r_hist[metric][bin]` for the CSR-selected metric and bin, and `o_hist_total` returns that metric's running transaction total (useful for normalizing bin counts into a distribution).

Sizing MAX_OUTSTANDING is a correctness requirement, not a tuning note. If a channel's FIFO can fill under real traffic, either wire `o_cmd_block` into the command ready path or size `MAX_OUTSTANDING` to the true per-channel outstanding depth – otherwise the histogram silently undercounts and misattributes.

17.5 Timing Characteristics

This module is **sequential**: it contains 1 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

17.6 Usage Examples

```
// Read-latency histogram on an AXI master read port (2 metrics:
first-R, RLAST).
axi_perf_latency_hist #(
    .ID_WIDTH        (8),
    .NUM_CHANNELS    (8),
    .MAX_OUTSTANDING (8),
    .NUM_BINS        (16),
    .IS_READ         (1'b1)
) u_rd_lat_hist (
    .aclk           (aclk),
    .aresetn        (aresetn),

    // Window control (share with axi_bus_meter)
    .i_clear        (perf_run_rising),
    .i_freeze       (~perf_run),

    // AR command channel
    .cmd_valid      (m_axi_arvalid),
    .cmd_ready      (m_axi_arready),
    .cmd_id         (m_axi_arid),

    // R data channel
    .data_valid     (m_axi_rvalid),
    .data_ready     (m_axi_rready),
    .data_last      (m_axi_rlast),
    .data_id        (m_axi_rid),

    // B channel unused for reads
    .resp_valid     (1'b0),
    .resp_ready     (1'b0),
    .resp_id        ('0),

    // CSR readout
    .i_hist_metric  (csr_metric),   // 0 = AR->first-R, 1 = AR->RLAST
    .i_hist_bin     (csr_bin),
    .o_hist_count   (csr_bin_count),
    .o_hist_total   (csr_metric_total)
);

// Write build: set IS_READ=0, wire AW to cmd_* and B to resp_*, leave
data_* tied off.
```

17.7 Design Notes

17.7.1 Snoop-Only, Base Untouched

The block is instantiated next to the datapath monitors but does not modify `axi_monitor_base`; it snoops the AXI channels read-only and emits no MonBus packets. Results leave exclusively through the CSR-indexed readout.

17.7.2 Timestamp Not Frozen

Only histogram accumulation freezes on `i_freeze`; `r_time` keeps counting so a transaction whose command was accepted before the window closed and whose completion arrives just after still yields a correct latency.

17.7.3 FIFO Depth vs Outstanding

`MAX_OUTSTANDING` bounds the per-channel timestamp FIFO. A command handshake only pushes when the channel FIFO is not full, and a completion only pops when the FIFO is non-empty, so an over-run on one channel cannot corrupt another. Size `MAX_OUTSTANDING` to the real per-channel outstanding depth of the bus.

17.7.4 Pipeline Drain Before Readback

Because increments land four cycles after the completing beat (capture, subtract, bin, increment – one register stage each), always read the histogram after the window has been closed (`i_freeze` high) long enough for the pipeline to drain — the host reads long after RUN falls, so this is automatic in the intended usage.

17.8 Related Modules

17.8.1 Used By

- Perfmon Stage D characterization (see RFC below)
- Datapath-monitor instantiations needing latency distributions alongside `axi_bus_meter`

17.8.2 Uses

- **reset_defs.svh** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

17.8.3 See Also

- **axi_bus_meter.sv** - Four-bucket utilization meter, shares the window-control convention
- **axi4_intf_master_observer.sv** - interface observability wrapper with per-port latency histograms (`projects/components/utility-ip/misc/rtl/`)
- **axi_monitor_base.sv** - The shared monitor scaffold this block deliberately does not touch

---

17.9 Testing

`val/amba/test_axi_perf_latency_hist.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi_perf_latency_hist.py -v
```

17.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A5.1 “AXI transaction identifiers” and ch. A6 “AXI ordering model” — same-ID responses return in request order, which is what lets a per-ID FIFO of issue timestamps match each completion to its command. Log2 binning has no paper (standard technique).
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, ch. 7 “Common On-Chip Bus Protocols” (AXI IDs and out-of-order completion) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

17.10.1 Source Code

- RTL: `rtl/amba/shared/axi_perf_latency_hist.sv`

17.10.2 Documentation

- RFC: `docs/markdown/rtl-amba/index.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2026-07-15

17.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

18 AXI Split Combinational Logic

Module: `axi_split_combi.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

18.1 Overview

This is the arithmetic engine both splitters share: pure combinational boundary-crossing detection and split-length calculation for AXI transactions. No state machine, no cycles of latency — the split decision is ready the same cycle the address shows up, which is exactly what the read and write splitters need to make real-time calls.

18.1.1 Key Features

- Pure combinational logic (zero latency)
- Boundary crossing detection for arbitrary alignment boundaries
- Automatic split length calculation (AXI encoding)
- Optimized for synthesis (bit shifts instead of division)
- Comprehensive assertion-based validation
- Configurable address and data widths
- Transactions are assumed not to wrap; boundary arithmetic is exact even in the top window

AXI transaction splitting needs three questions answered, fast: 1. Does this transaction cross a boundary? 2. How many beats fit before the boundary? 3. What address/length for the next split?

This module packs all of that arithmetic into one reusable combinational block, so the read and write splitters behave identically by construction.

Design Philosophy: - Separation of concerns: Calculation logic separate from state management - Reusability: Single module used by both read and write splitters - Performance: Combinational path enables single-cycle decisions - Correctness: Extensive assertions validate assumptions

---

18.2 Parameters

Parameter	Type	Default	Description
AW	int	32	Address width in bits (typically 32 or 64)
DW	int	32	Data width in bits, must be power of 2 (32, 64, 128, 256, 512, 1024)

Derived Constants (localparam):

Constant	Calculation	Description
BYTES_PER_BEAT	DW / 8	Bytes transferred per beat (e.g., 8 for 64-bit bus)
LOG2_BYTES_PER_BEAT	$clog2(BYTES_PER_BEAT)	Bit shift amount for beat calculations
EXPECTED_AX_SIZE	LOG2_BYTES_PER_BEAT	Expected value of ax_size input (for assertions)
ADDR_ALIGN_MASK	BYTES_PER_BEAT - 1	Mask for address alignment checking

18.3 Ports

18.3.1 Clock and Reset (For Assertions Only)

Port	Direction	Width	Description
aclk	input	1	Clock for assertion evaluation (not used in functional logic)
aresetn	input	1	Active-low reset for assertion gating (not used in functional logic)

18.3.2 Input Signals

Port	Direction	Width	Description
current_addr	input	AW	Current transaction starting address (may be mid-split)
current_len	input	8	Current transaction length in AXI encoding (beats - 1)
ax_size	input	3	AXI size field (log2 of bytes per beat)
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB)
is_idle_state	input	1	Indicates whether state machine is in IDLE (first transaction)
transaction_valid	input	1	Valid transaction present on inputs

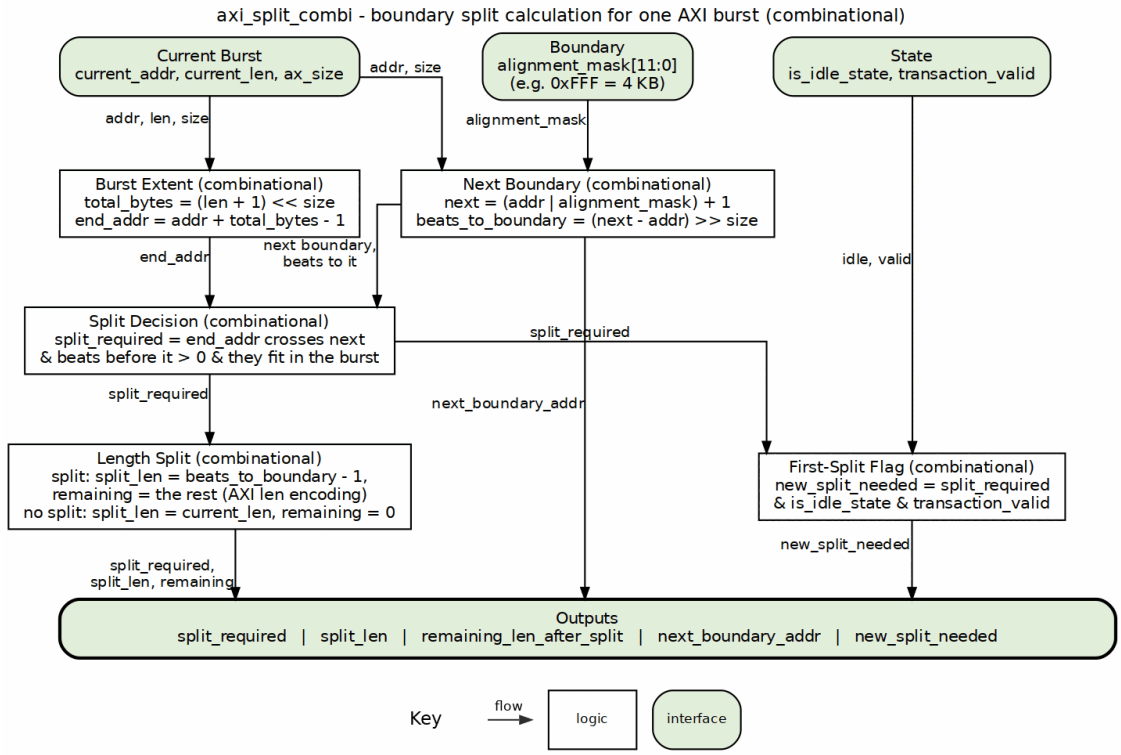
18.3.3 Output Signals

Port	Direction	Width	Description
split_required	output	1	Transaction crosses boundary and requires splitting
split_len	output	8	Length of current split in AXI encoding (beats - 1)

Port	Direction	Width	Description
next_boundary_addr	output	AW	Address at next boundary alignment
remaining_len_after_split	output	8	Remaining length after current split (AXI encoding)
new_split_needed	output	1	Helper signal: split_required AND is_idle_state AND transaction_valid

18.4 Functional Description

18.4.1 Block diagram



axi_split_combi block diagram

Everything is combinational; `aclk` and `aresetn` feed only simulation checks. The burst's extent gives its last byte address. The alignment mask gives the next boundary above `current_addr` and how many beats fit before it. A split is required when the burst crosses that boundary and at least one beat, but not the whole burst, fits before it. The length split then gives the first piece's `split_len` and the `remaining_len_after_split`, both in AXI len encoding. `new_split_needed` flags the first split of a new transaction (idle and valid).

18.4.2 Boundary Crossing Detection Algorithm

The module implements a simplified boundary detection algorithm based on key AXI assumptions:

Step 1: Calculate Transaction Parameters

```
total_bytes = (current_len + 1) << ax_size
transaction_end_addr = current_addr + total_bytes - 1
```

Step 2: Calculate Next Boundary

```
next_boundary_ext = (current_addr | alignment_mask) + 1   // AW+1
bits, so 2**AW is representable
```

Example (4KB boundary, ADDR=0x0FC0, mask=0xFFF):

```
current_addr      = 0x0FC0
alignment_mask    = 0x0FFF
OR result         = 0x0FFF
next_boundary     = 0x0FFF + 1 = 0x1000
```

Step 3: Calculate Space to Boundary

```
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
```

Example (continuing above, ax_size=3 for 8-byte beats):

```
bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats
```

Step 4: Determine Split Requirement

Transaction requires splitting if: 1. It ends at or past the next boundary (crosses_boundary) 2. There is space for at least one beat before boundary (has_beats_before_boundary) 3. The beats to boundary fit within total transaction (beats_fit_before_boundary)

```
crosses_boundary = (transaction_end_addr >= next_boundary_addr)
has_beats_before_boundary = (beats_to_boundary > 0)
beats_fit_before_boundary = (beats_to_boundary <= (current_len + 1))

split_required = crosses_boundary AND has_beats_before_boundary
                 AND beats_fit_before_boundary
```

18.4.3 Split Length Calculation

If splitting is required, the split length represents how many beats to consume before the boundary:

```
split_len = split_required ? (beats_to_boundary - 1) : current_len
```

AXI Encoding: Length is always encoded as (beats - 1): - 1 beat → LEN=0 - 2 beats → LEN=1 - 8 beats → LEN=7

18.4.4 Remaining Length Calculation

After consuming the current split, calculate remaining beats:

```
original_beats_total = current_len + 1
split_beats_actual = split_required ? beats_to_boundary :
original_beats_total
remaining_beats_actual = split_required ? (original_beats_total -
```

```
split_beats_actual) : 0

// Convert back to AXI encoding
remaining_len_after_split = split_required ?
                            ((remaining_beats_actual > 0) ?
(remaining_beats_actual - 1) : 0)
                            : 0
```

18.4.5 State Machine Helper

The new_split_needed output simplifies state machine control:

```
new_split_needed = split_required AND is_idle_state AND
transaction_valid
```

This signal indicates a new split sequence is starting (as opposed to continuing an existing split).

18.5 Timing Characteristics

This module is **sequential**: it contains 3 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

18.6 Usage Examples

18.6.1 Integration with Read Splitter

```
// Instantiate split logic within read splitter
axi_split_combi #(
    .AW             (32),
    .DW             (64)
) u_split_logic (
    // Clock/reset for assertions only
    .aclk                       (aclk),
    .aresetn                    (aresetn),

    // Transaction parameters
    .current_addr               (w_current_addr),       // From state
machine
    .current_len                (w_current_len),        // From state
machine
    .ax_size                    (w_current_size),       // ARSIZE
    .alignment_mask             (12'hFFF),              // 4KB
boundary
    .is_idle_state              (r_state == IDLE),
    .transaction_valid          (fub_arvalid),

    // Splitting decision outputs
    .split_required             (w_split_required),
    .split_len                  (w_split_len),
    .next_boundary_addr         (w_next_boundary_addr),
    .remaining_len_after_split  (w_remaining_len_after_split),
    .new_split_needed           (w_new_split_needed)
);

// Use outputs in state machine
always_comb begin
    case (r_state)
        IDLE: begin
            if (fub_arvalid && m_axi_arready) begin
                if (w_new_split_needed) begin
                    // Split required - start sequence
                    next_state = SPLITTING;
                    // Buffer next split parameters
                    buffer_addr = w_next_boundary_addr;
                    buffer_len = w_remaining_len_after_split;
                end else begin
                    // No split - pass through
                    next_state = IDLE;
                end
            end
```

```
        end

        SPLITTING: begin
            if (m_axi_arvalid && m_axi_arready) begin
                if (w_split_required) begin
                    // More splits needed
                    buffer_addr = w_next_boundary_addr;
                    buffer_len = w_remaining_len_after_split;
                end else begin
                    // Final split complete
                    next_state = IDLE;
                end
            end
        end
    endcase
end

// Use split_len for address channel
assign m_axi_araddr = w_current_addr;
assign m_axi_arlen = w_split_required ? w_split_len : w_current_len;
```

18.6.2 Example Calculation Walkthrough

Scenario: 4KB boundary, 64-bit data bus

Inputs: - current_addr = 0x0FC0 - current_len = 7 (8 beats in AXI encoding) - ax_size = 3 (8 bytes per beat) - alignment_mask = 0xFFF

Calculations:

Step 1: Transaction parameters

```
total_bytes = (7 + 1) << 3 = 8 << 3 = 64 bytes
transaction_end_addr = 0x0FC0 + 64 - 1 = 0x0FFF
```

Step 2: Next boundary

```
next_boundary_addr = (0x0FC0 | 0x0FFF) + 1
                   = 0x0FFF + 1
                   = 0x1000
```

Step 3: Space to boundary

```
bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats
```

Step 4: Split decision

```
crosses_boundary = (0x0FFF >= 0x1000) = False
split_required = False
```

Outputs: - split_required = 0 (transaction ends exactly at boundary, no split) - split_len = 7 (pass through original length) - remaining_len_after_split = 0

Modified Scenario: Transaction crosses boundary

Inputs: - current_addr = 0x0FC0 - current_len = 8 (9 beats) - ax_size = 3 - alignment_mask = 0xFFF

Calculations:

Step 1: Transaction parameters

```
total_bytes = (8 + 1) << 3 = 72 bytes
transaction_end_addr = 0x0FC0 + 72 - 1 = 0x1007
```

Step 4: Split decision

```
crosses_boundary = (0x1007 >= 0x1000) = True
has_beats_before_boundary = (8 > 0) = True
beats_fit_before_boundary = (8 <= 9) = True
split_required = True
```

Outputs: - split_required = 1 - split_len = 7 (8 beats - 1 in AXI encoding) - next_boundary_addr = 0x1000 - remaining_len_after_split = 0 (1 beat - 1 in AXI encoding)

18.7 Design Notes

18.7.1 Design Assumptions

Four assumptions make the simplified, high-performance logic possible, and the module enforces every one of them:

18.7.2 Assumption 1: Address Alignment

Assumption: All AXI addresses are aligned to the data bus width.

Implication: - If DATA_WIDTH = 512 bits (64 bytes), address is always 64-byte aligned - Low-order address bits are always zero

Verification:

```
assert ((current_addr & ADDR_ALIGN_MASK) == '0)
```

18.7.3 Assumption 2: Fixed Transfer Size

Assumption: All AXI transfers use maximum transfer size equal to bus width.

Implication: - ax_size always matches log2(DATA_WIDTH/8) - Example: 64-bit bus → ax_size = 3'b011 (8 bytes)

Verification:

```
assert (ax_size == EXPECTED_AX_SIZE)
```

18.7.4 Assumption 3: Incrementing Bursts Only

Assumption: All bursts use incrementing address mode (AxBURST = 2'b01).

Implication: - No FIXED or WRAP burst handling - Address calculation simplified

Rationale: - Covers 95%+ of real-world use cases - FIXED bursts rarely cross boundaries - WRAP bursts require complex modulo arithmetic

18.7.5 Assumption 4: No Address Wraparound

Assumption: Transactions never wrap around top of address space (0xFFFFFFFF → 0x00000000).

Implication: - No wraparound detection or handling for the TRANSACTION itself - Simplified comparison logic

The BOUNDARY is a separate matter (TASK-095). For a transaction in the final alignment window the next boundary is `2**AW`, one past the top of the address space – and the transaction itself does not have to wrap for that to happen. A one-beat read at `0xFFFC` (AW=16, mask `0xFFF`) ends exactly at the top, crosses nothing, and still had `(addr | mask) + 1` overflow to `0` in `AW` bits, so `transaction_end_addr >= next_boundary_addr` compared against zero, read TRUE,

and the transaction was split anyway. The boundary is now carried in AW+1 bits, so the comparison is against the real value. The next_boundary_addr port stays AW wide and truncates that boundary to 0, which is harmless: both splitters read it only on the split path, and split_required is false in that window.

Verification:

```
assert (transaction_end_addr >= current_addr)       // Transaction
doesn't wrap
assert (w_next_boundary_ext > {1'b0, current_addr}) // Boundary math
doesn't wrap
```

Rationale: - Top of address space typically reserved for device registers - Memory allocators avoid near-boundary allocations - Real systems never allow this condition

18.7.6 Synthesis Optimization

Bit Shifts Instead of Division:

```
// Original (expensive):
beats_to_boundary = bytes_to_boundary / (1 << ax_size)

// Optimized (single barrel shifter):
beats_to_boundary = bytes_to_boundary >> ax_size
```

Fixed Alignment Assumptions: - Removes modulo arithmetic - Enables simple OR-based boundary calculation - Critical for timing closure in high-speed designs

18.7.7 Assertion Coverage

The module includes comprehensive assertions for validation:

Input Validation (Triggered on transaction_valid && is_idle_state): - Address alignment to data width - ax_size matches expected value for data width - No address wraparound in transaction - No boundary calculation wraparound (checked in AW+1 bits)

Output Validation (Triggered on transaction_valid): - Beats to boundary within reasonable range - Split length conservation (split_beats + remaining_beats = original_beats) - Boundary alignment verification - Comparison logic consistency

18.7.8 Timing Considerations

Critical Path: 1. Address OR with alignment_mask (next_boundary_addr) 2. Subtraction (bytes_to_boundary) 3. Variable barrel shift by ax_size (beats_to_boundary) 4. Comparison and output muxing (split_required, split_len)

Optimization Strategies: - Pipeline if timing closure issues (adds 1 cycle latency) - Use registered alignment_mask if it's static - Consider ax_size as constant parameter if fixed data width

18.7.9 Debug Support

DEBUG_AXI_SPLIT Macro:

Enable detailed transaction logging:

```
`define DEBUG_AXI_SPLIT

// Generates console output (one line per accepted transaction):
=== AXI SPLIT DEBUG === addr=... len=... split_required=...
split_len=... remaining_after=...
```

18.8 Related Modules

18.8.1 Used By

- **axi_master_rd_splitter.sv** - Read transaction splitting (AR channel)
- **axi_master_wr_splitter.sv** - Write transaction splitting (AW channel)

18.8.2 Dependencies

- None (pure combinational logic)

18.8.3 See Also

- **axi_master_rd_splitter.sv** - Full read splitter with state machine
- **axi_master_wr_splitter.sv** - Full write splitter with response consolidation

---

18.9 Testing

`val/amba/test_axi_split_combi.py` exercises this module. It collects 16 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi_split_combi.py -v
```

18.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (“A burst must not cross a 4KB address boundary”), A3.3.1 “Dependencies between channel handshake signals”, and A3.4.5 “Read and write response structure” — the boundary rule the splitter enforces and the response it consolidates.
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, ch. 7 “Common On-Chip Bus Protocols” (AXI burst length, size and type) — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

18.10.1 Source Code

- RTL: `rtl/amba/shared/axi_split_combi.sv`
- Tests: Indirectly tested via splitter tests (`test_axi_master_rd_splitter.py`, `test_axi_master_wr_splitter.py`)

18.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- AXI Specification: AMBA AXI and ACE Protocol Specification, ARM IHI 0022, Issue H (sec. A3.4.1, 4KB boundary)
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2025-10-24

18.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

19 AXI-Stream Master Injector

Module: `axis4_master_injector.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

19.1 Overview

axis4_master_injector is an AXI-Stream master that emits a deterministic, per-channel LFSR data pattern for characterization and verification. It is the AXIS *sink* stimulus for the RAPIDS characterization harness, and it is pattern/CRC-consistent with the AXI4 pattern blocks: the per-channel 32-bit CRC-32 it computes is bit-identical to what axi4_slave_wr_crc_check produces for the same emitted stream (self-check path: axis_gen → m_axi_wr → wr_crc_check).

19.1.1 Key Features

- AXI-Stream master (m_axis_*) with no address channel — simpler than the AXI4 pattern generators
- Per-channel independent LFSR (seed ^ ch) and CRC-32, copied verbatim from axi4_slave_rd_injector
- Two channel schedules: sequential (finish one channel's beats, then the next) or, with cfg_interleave, round-robin beat by beat across the active channels so every channel holds data in the sink at once; per-channel data, CRC and packet boundaries are identical in both
- Channel mask (cfg_channel_mask) selects active channels (0 → all)
- Programmable beats-per-channel and tlast cadence (cfg_beats_per_pkt)
- Per-channel expected-CRC / beat-count telemetry plus an aggregate total
- LFSR/CRC advance only on accepted beats, so beat N of channel C is a deterministic function of (seed ^ C, N) independent of tready stalls

Characterizing a stream-consuming engine (a "sink") needs a deterministic source that a downstream checker can predict. This block streams a known LFSR pattern per channel and computes the CRC-32 that the same data will produce when it lands in a CRC-checking sink. Because the LFSR advances only on accepted beats, backpressure never perturbs the sequence — the emitted data is a pure function of channel and beat index, and the exported per-channel CRC is the golden value for end-to-end integrity.

Use Cases: - Driving a RAPIDS sink engine's AXIS input during characterization - Sink self-check: stream into a DMA write path terminated by axi4_slave_wr_crc_check and compare CRCs - Per-channel throughput / backpressure sweeps with deterministic, reproducible data - On-chip (FPGA) stream stimulus in the RAPIDS characterization harness

Key Benefit: A memory-free, backpressure-insensitive stream source whose per-channel CRC-32 matches the AXI4 CRC blocks bit-for-bit, so a stream integrity test reduces to a single register compare.

19.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts; channel index drives `tid`
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of `LFSR_WIDTH`)
AXIS_ID_WIDTH	int	8	`tid` width
AXIS_DEST_WIDTH	int	4	`tdest` width
AXIS_USER_WIDTH	int	1	`tuser` width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the AXI4 blocks)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses `seed ^ N`
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: `tstrb` width
REP	int	AXIS_DATA_WIDTH/	Derived: 32-bit LFSR copies per beat

Parameter	Type	Default	Description
CIW	int	LFSR_WIDTH (NUM_CHANNELS>1) ? $clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: LFSR + CRC parameters must match `axi4_slave_rd_injector` / `axi4_slave_wr_crc_check` for cross-block CRC consistency.

19.3 Ports

19.3.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

19.3.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to begin a run (seeds all channels)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use `LFSR_SEED` param)
cfg_channel_mask	input	NUM_CHANNELS	Active channels (0 → all channels active)
cfg_num_beats	input	BEAT_COUNT_WIDTH	Beats to send per channel
cfg_beats_per_pkt	input	BEAT_COUNT_WIDTH	`tlast` cadence (0 → one packet per channel)
cfg_interleave	input	1	1 → round-robin the active channels one beat at a time; 0 → sequential (latched with the mask on `cfg_start`)
cfg_tdest	input	AXIS_DEST_WIDTH	Value driven on `tdest`
cfg_busy	output	1	High while running (state != IDLE)
cfg_done	output	1	One-cycle pulse at end of run

19.3.3 Per-Channel Telemetry

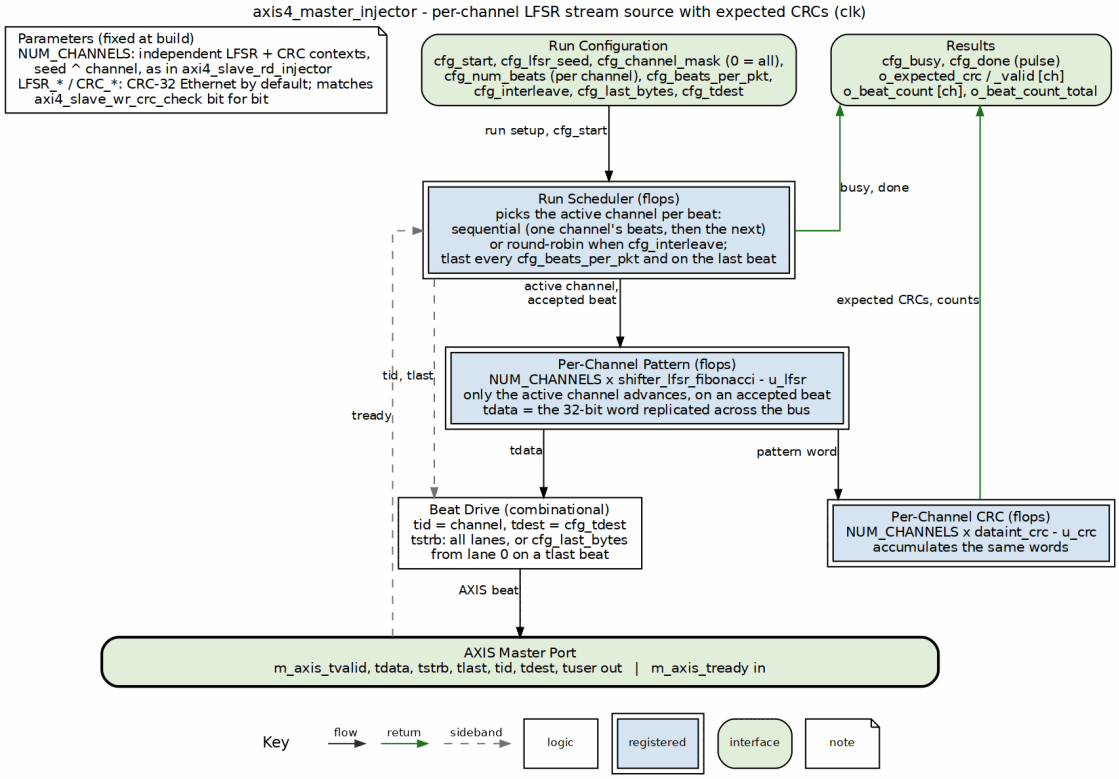
Port	Direction	Width	Description
o_expected_crc	output	NUM_CHANNELS × 32	Per-channel expected CRC-32
o_expected_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid
o_beat_count	output	NUM_CHANNELS × 32	Per-channel beats emitted
o_beat_count_total	output	32	Sum across channels (harness stop trigger)

19.3.4 AXI-Stream Master (m_axis)

Port	Direction	Width	Description
m_axis_tvalid	output	1	Beat valid (asserted in the RUN state)
m_axis_tready	input	1	Downstream ready
m_axis_tdata	output	AXIS_DATA_WIDTH	`{REP{lfsr_out[ch]}}` — 32-bit LFSR replicated to fill the bus
m_axis_tstrb	output	STRB_WIDTH	All-ones (full beat)
m_axis_tlast	output	1	Packet last (per `cfg_beats_per_pkt`, and each channel's final beat)
m_axis_tid	output	AXIS_ID_WIDTH	Channel index of the current beat
m_axis_tdest	output	AXIS_DEST_WIDTH	Driven from `cfg_tdest`
m_axis_tuser	output	AXIS_USER_WIDTH	Tied to 0

19.4 Functional Description

19.4.1 Block diagram



axis4_master_injector block diagram

After `cfg_start`, the run scheduler picks the active channel for each beat. It runs either sequentially (all of one channel's beats, then the next) or round-robin when `cfg_interleave` is set, and it places tlast. Only that channel's LFSR advances, and only on an accepted beat, so beat N of channel C depends only on the seed, C and N, not on stalls or scheduling order. The 32-bit word, replicated across the bus, becomes `tdata`, with `tid` = channel and a short `tstrb` on tlast beats when `cfg_last_bytes` is set. The channel's CRC accumulates the same words and matches `axi4_slave_wr_crc_check` bit for bit.

19.4.2 Scheduling FSM

A three-state FSM (`IDLE`, `RUN`, `DONE`) sequences a run. On `cfg_start` the effective channel mask, beats-per-channel, and `tlast` cadence are latched; if there are no beats or no active channel it drops straight to `DONE`, otherwise it selects the first active channel and enters `RUN`. `DONE` pulses `cfg_done` for one cycle and returns to `IDLE`. `m_axis_tvalid` is asserted exactly in `RUN`.

19.4.3 Channel Scheduling: Sequential or Interleaved

In the sequential mode (`cfg_interleave = 0`) channels are streamed one at a time in ascending index order among the masked set. A `f_next_active_after` function priority-scans the mask for the lowest active channel index strictly greater than the current one (or the first active channel at start). When the current channel's beats are exhausted (`w_ch_last_beat`), the FSM advances to the next active channel and reloads its beat count; when none remain it goes to `DONE`. Because channels run to completion in turn, each channel's LFSR sequence is contiguous and uninterrupted.

In the interleaved mode (`cfg_interleave = 1`, latched on `cfg_start`) the same scan hands the bus to the next active channel after every accepted beat, wrapping to the first active channel when the scan finds none above the current one. Every active channel takes exactly one beat per round, so a single round counter (`r_beats_remaining`) and a single packet counter serve all channels: both advance when the last active channel of a round has taken its beat, and the run ends when the final round completes. A channel's LFSR and CRC still advance only on its own beats, so beat N of channel C carries the same data as in a sequential run and the exported CRCs do not change; only the order in which channels occupy the bus differs. This is the schedule that puts data in every sink channel at once, which is what an aggregate-window measurement needs (the sequential schedule only ever fills one sink channel).

19.4.4 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks)

Each channel owns a `shifter_lfsr_fibonacci` (32-bit, taps `{23,3,2,1}`, seed `w_seed ^ ch`) and a `dataint_crc` (CRC-32/Ethernet, `cascade_sel = 4'b1000`). The active channel advances on its accepted beat (`ch_beat = w_beat && r_ch == ch`); all channels reload their seed / clear their CRC on the global load pulse (`cfg_start` in IDLE). The seed, taps, replication, CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_injector` so the per-channel 32-bit CRC is bit-identical. Per-channel valid flags and beat counters track alongside, and `o_beat_count_total` is a combinational sum for the harness stop trigger.

19.4.5 Stream Outputs and tlast Cadence

`m_axis_tdata` is the active channel's LFSR output replicated `REP` times; `tid` carries the channel index; `tstrb` is all-ones (full beats); `tdest` is driven from `cfg_tdest`; `tuser` is 0. `tlast` asserts on each channel's final beat, and additionally every `cfg_beats_per_pkt` beats when that cadence is non-zero (0 → one packet spanning the channel's whole run). A per-channel packet counter (`r_pkt_cnt`) resets at each `tlast` and at channel boundaries; in the interleaved mode every channel shares it, since all channels sit at the same beat index within their packets.

19.4.6 Backpressure Insensitivity

Because the LFSR and CRC advance only on accepted beats (`w_beat = tvalid && tready`), a downstream `tready` stall simply holds the current beat — the emitted value for beat N of channel C is always `f(seed ^ C, N)` regardless of stall pattern. This is what lets the exported per-channel CRC serve as a stable golden value.

19.5 Timing Characteristics

This module is **sequential**: it contains 2 `always_ff` block(s), clocked on `clk` with active-low asynchronous reset `rst_n`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

19.6 Usage Examples

```
// AXIS stimulus for a 4-channel RAPIDS sink characterization run.
axis4_master_injector #(
    .NUM_CHANNELS    (4),
    .AXIS_DATA_WIDTH (512),
    .AXIS_ID_WIDTH   (8),
    .LFSR_SEED       (32'hDEADBEEF)
) u_axis_gen (
    .clk (clk),
    .rst_n (rst_n),

    // Control
    .cfg_start         (csr_start_pulse),
    .cfg_lfsr_seed     (32'd0),        // use param default
    .cfg_channel_mask  (4'b1111),      // all channels
    .cfg_num_beats     (32'd1024),     // per channel
    .cfg_beats_per_pkt (32'd16),       // tlast every 16 beats
    .cfg_tdest         (4'd0),
    .cfg_busy          (gen_busy),
    .cfg_done          (gen_done),

    // Golden CRCs for the downstream checker
    .o_expected_crc       (gen_crc),
    .o_expected_crc_valid (gen_crc_valid),
    .o_beat_count         (gen_beats),
    .o_beat_count_total   (gen_beats_total),

    // Stream to the DUT sink
    .m_axis_tvalid (s_tvalid), .m_axis_tready (s_tready),
    .m_axis_tdata  (s_tdata),  .m_axis_tstrb  (s_tstrb),
    .m_axis_tlast  (s_tlast),  .m_axis_tid    (s_tid),
    .m_axis_tdest  (s_tdest),  .m_axis_tuser  (s_tuser)
);
```

19.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_injector`, so a stream emitted here and re-CRC'd by `axi4_slave_wr_crc_check` (or checked by `axis4_slave_pattern_check`) yields identical per-channel CRCs.
- **Either schedule keeps sequences contiguous per channel:** the checker demultiplexes by `tid` and regenerates each channel's LFSR from its own beats, so sequential and interleaved runs verify the same way. Interleaving exists for the sink-side measurement (rapids ISSUE-006): sequential streaming leaves only one sink channel holding data, so the measured window was one channel's, not the DUT's.
- **No address channel:** as a pure stream the generator is simpler than the AXI4 pattern gens — no `dma_address_gen`, no burst FSM, just per-channel beat counting.
- **Backpressure-safe:** advancing only on accepted beats decouples the data sequence from `tready` timing.
- **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since `tdata = REP × lfsr_out`.

19.8 Related Modules

19.8.1 Used By

- `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS sink stimulus
- RAPIDS sink-path characterization flows

19.8.2 Uses

- **shifter_lfsr_fibonacci.sv** — per-channel maximal-length Fibonacci LFSR
- **dataint_crc.sv** — per-channel CRC-32 accumulator

19.8.3 See Also

- **axis4_slave_pattern_check.sv** — the matching AXIS checker (same LFSR/CRC config)
- **axi4_slave_rd_injector.sv** — the AXI4 read pattern source this is copied from
- **axi4_slave_wr_crc_check.sv** — CRC sink whose values match this generator

---

19.9 Testing

`val/amba/test_axis4_pattern_pair.py` covers this module. It drives the generator and checker as a pair through a `tb_` wrapper.

The test is named for the PAIR rather than the module, which is why an earlier audit looking only for `val/**/test_axis4_master_injector.py` reported it as having no coverage. It does.

Treat any behaviour described on this page as unverified by simulation.

19.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — pattern generator plus signature compactor; a CRC-32 replaces the MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the CRC-32 polynomial (0x04C11DB7).
- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051, Issue B, 2021. developer.arm.com/documentation/ihi0051/b, sec. 2.2 “Handshake signaling”, 2.6 “Packet boundaries” (TLAST) and 2.7 “Source and destination signaling” (TID, the per-channel demux key).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

19.10.1 Source Code

- RTL: `rtl/amba/shared/axis4_master_injector.sv`

19.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `vault/tracking/projects/components/dma-ip/rapids/CONTROL_ENGINE_INTEGRATION.md`

Last Updated: 2026-07-15

19.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

20 AXI-Stream Slave Pattern Checker

Module: `axis4_slave_pattern_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

20.1 Overview

`axis4_slave_pattern_check` is an AXI-Stream slave that consumes a stream and checks it against a deterministic, per-channel LFSR pattern, computing a per-channel CRC-32 as it goes. It is the AXIS *source* checker for the RAPIDS characterization harness and is pattern/CRC-consistent with `axi4_slave_rd_injector`: the per-channel CRC-32 it computes is bit-identical to `axi4_slave_rd_injector.read_crc_value` for the same stream (self-check path: `rd_gen → m_axis → axis_check`).

20.1.1 Key Features

- AXI-Stream slave (`s_axis_*`), pure sink with `tready` driven by a `ready_en` input to model backpressure
- Per-channel independent LFSR (seed `^ ch`) and CRC-32, copied verbatim from `axi4_slave_rd_injector`
- Incoming beats demuxed by `s_axis_tid[CIW-1:0]` into the matching channel context
- Per-beat compare against the locally regenerated pattern; sticky `o_data_error` on any mismatch
- Per-channel actual-CRC / beat-count telemetry plus aggregate beat and packet (`tlast`) counters
- LFSR advances only on accepted beats, so the check is independent of upstream stalls and cross-channel interleave

Characterizing a stream-*producing* engine (a “source”) needs a checker that knows what the engine should emit. This block seeds every channel identically to the generator and, on each accepted beat, regenerates the expected pattern for that channel and compares it against the received `tdata`. It also folds the regenerated data into a per-channel CRC-32 for a whole-run summary. Because the LFSR advances only on accepted beats and is demuxed by `tid`, the check is insensitive to backpressure and to arbitrary interleave of channels on the shared stream.

Use Cases: - Terminating and verifying a RAPIDS source engine’s AXIS output during characterization - Source self-check: check a stream produced from `axi4_slave_rd_injector` and compare CRCs - Per-channel integrity checking under backpressure (via `ready_en`) - On-chip (FPGA) stream checker in the RAPIDS characterization harness

Key Benefit: A memory-free stream checker that both pinpoints (sticky `o_data_error` per beat) and summarizes (per-channel CRC-32 identical to the generator) integrity, immune to stalls and channel interleave.

20.2 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts; channel = `tid` low bits
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of `LFSR_WIDTH`)
AXIS_ID_WIDTH	int	8	`tid` width
AXIS_DEST_WIDTH	int	4	`tdest` width
AXIS_USER_WIDTH	int	1	`tuser` width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the generator)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses `seed ^ N`
LFSR_TAPS	logic [47:0]	{12'd23, 12'd3, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: `tstrb` width
REP	int	AXIS_DATA_WIDTH/	Derived: 32-bit LFSR copies per beat

Parameter	Type	Default	Description
CIW	int	LFSR_WIDTH (NUM_CHANNELS>1) ? $clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: LFSR + CRC parameters must match `axi4_slave_rd_injector` / `axis4_master_injector` for cross-block CRC consistency.

20.3 Ports

20.3.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

20.3.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to arm and seed all channels (also clears errors/counters)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use `LFSR_SEED` param)
ready_en	input	1	Drives `s_axis_tready` (tie high for a pure sink; deassert to model backpressure)

20.3.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	NUM_CHANNELS × 32	Per-channel actual CRC-32
o_actual_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid
o_data_error	output	1	Sticky: any beat mismatched the expected pattern
o_beat_count	output	NUM_CHANNELS × 32	Per-channel beats received
o_beat_count_total	output	32	Sum across channels (harness stop trigger)
o_pkt_coun	output	BEAT_COUN	Aggregate `tlast` beats

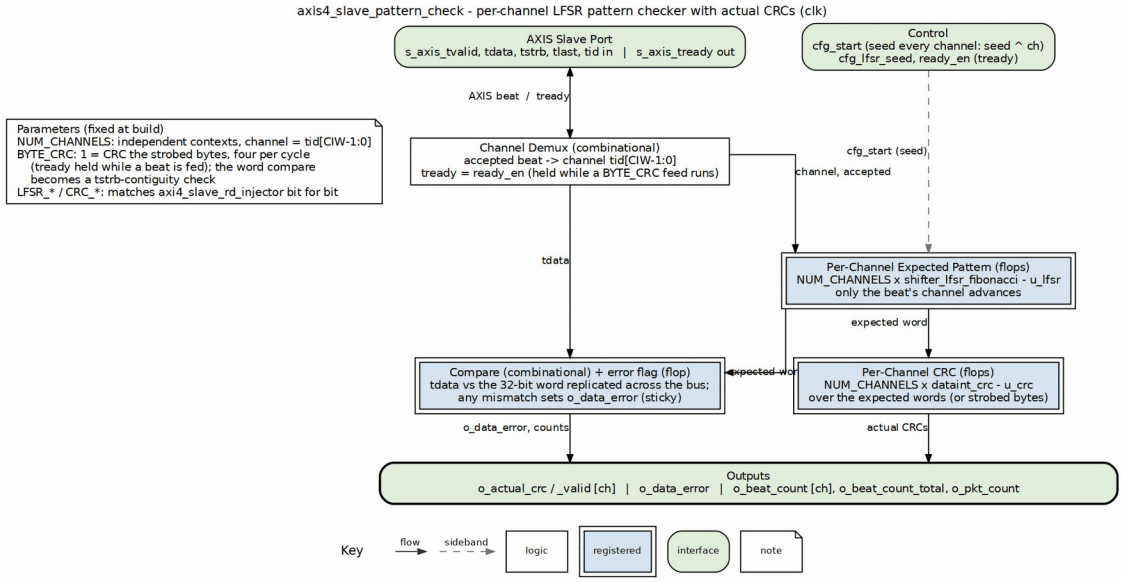
Port	Direction	Width	Description
t		T_WIDTH	received

20.3.4 AXI-Stream Slave (s_axis)

Port	Direction	Width	Description
s_axis_tvalid	input	1	Beat valid
s_axis_tready	output	1	Ready (= `ready_en`)
s_axis_tdata	input	AXIS_DATA_WIDTH	Received data (compared to `{REP{lfsr_out[ch]}}`)
s_axis_tstrb	input	STRB_WIDTH	Byte strobes (not used by the check)
s_axis_tlast	input	1	Packet last (counted into `o_pkt_count`)
s_axis_tid	input	AXIS_ID_WIDTH	Channel selector (`tid[CIW-1:0]`)
s_axis_tdest	input	AXIS_DEST_WIDTH	Destination (unused)
s_axis_tuser	input	AXIS_USER_WIDTH	User sideband (unused)

20.4 Functional Description

20.4.1 Block diagram



axis4_slave_pattern_check block diagram

Each accepted beat is routed by `tid` to its channel's context. That channel's LFSR, seeded at `cfg_start` exactly as the generator seeds it, regenerates the expected word; the received `tdata` is compared with it replicated across the bus, and any mismatch sets the sticky `o_data_error`. The channel's CRC accumulates the expected words and matches `axi4_slave_rd_injector` bit for bit. With `BYTE_CRC=1` it CRCs the strobed bytes instead. `tready` follows `ready_en`, so a test can model backpressure.

20.4.2 Readiness and Beat Handshake

`s_axis_tready` is driven directly from `ready_en` — tie it high for a pure, always-ready sink, or deassert it to model downstream backpressure. An accepted beat is `w_beat = tvalid && tready`. Because the same handshake gates both the compare and the LFSR/CRC advance, the checker and the upstream generator advance in lockstep. The active channel for a beat is `s_axis_tid[CIW-1:0]`.

20.4.3 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)

Each channel owns a `shifter_lfsr_fibonacci` (32-bit, taps `{23,3,2,1}`, seed `w_seed ^ ch`) and a `dataint_crc` (CRC-32/Ethernet, `cascade_sel = 4'b1000`). On `cfg_start` (the arm/load pulse) every channel reloads its seed and clears its CRC / counters. A channel advances only on an accepted beat carrying its `tid` (`ch_beat = w_beat && w_ch == ch`). The seed, taps, replication, CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_injector`, so the per-channel CRC is bit-identical. Each channel exposes `o_actual_crc`, `o_actual_crc_valid`, and `o_beat_count`; `o_beat_count_total` is a combinational sum.

20.4.4 Per-Beat Compare and Sticky Error

The expected data for the active channel is its current LFSR value replicated to the bus width (`expected_data_per_ch[w_ch] = {REP{lfsr_out[ch]}}`), sampled *before* the LFSR advances on the same edge. A mismatch (`w_beat && tdata != expected`) latches `o_data_error` sticky until the next `cfg_start`. Because the compare uses the pre-advance value and the LFSR steps only on accepted beats, the check is independent of upstream stalls and of how channels interleave on the shared stream.

20.4.5 Packet Counting

`o_pkt_count` increments on every accepted beat that carries `tlast`, giving an aggregate packet count across channels. Both `o_data_error` and `o_pkt_count` are cleared by `cfg_start`.

20.5 Timing Characteristics

This module is **sequential**: it contains 2 `always_ff` block(s), clocked on `clk` with active-low asynchronous reset `rst_n`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

20.6 Usage Examples

```
// AXIS checker terminating a 4-channel RAPIDS source engine.
axis4_slave_pattern_check #(
    .NUM_CHANNELS    (4),
    .AXIS_DATA_WIDTH (512),
    .AXIS_ID_WIDTH   (8),
    .LFSR_SEED       (32'hDEADBEEF)
) u_axis_chk (
    .clk (clk),
    .rst_n (rst_n),

    // Control
    .cfg_start     (csr_arm_pulse),
    .cfg_lfsr_seed (32'd0),       // use param default
    .ready_en      (sink_ready),  // 1 = always ready; toggle to
backpressure

    // Telemetry to the harness CSR block
    .o_actual_crc       (chk_crc),
    .o_actual_crc_valid (chk_crc_valid),
    .o_data_error       (chk_data_error),
    .o_beat_count       (chk_beats),
    .o_beat_count_total (chk_beats_total),
    .o_pkt_count        (chk_pkts),

    // Stream from the DUT source
    .s_axis_tvalid (m_tvalid), .s_axis_tready (m_tready),
    .s_axis_tdata  (m_tdata),  .s_axis_tstrb  (m_tstrb),
    .s_axis_tlast  (m_tlast),  .s_axis_tid    (m_tid),
    .s_axis_tdest  (m_tdest),  .s_axis_tuser  (m_tuser)
);

// Clean check: no mismatch and per-channel CRC matches the
generator's.
assign stream_ok = !chk_data_error && (chk_crc[ch] ==
gen_expected_crc[ch]);
```

20.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_injector`, so a stream produced by that generator (or by `axis4_master_injector`) yields identical per-channel CRCs here.
- **Compare uses the pre-advance LFSR value:** the expected data is sampled before the channel's LFSR steps on the same clock edge, so the beat is compared against the value the generator emitted, not the next one.
- **Two integrity signals:** sticky `o_data_error` pinpoints that *some* beat disagreed, while the per-channel CRC lets the harness confirm the exact stream matched end-to-end.
- **Backpressure via `ready_en`:** exposing readiness as a config input lets sweeps drive the checker as an always-ready sink or as a throttling one without extra logic.
- **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since the expected data is `REP × lfsr_out`.

20.8 Related Modules

20.8.1 Used By

- `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS source checker
- RAPIDS source-path characterization flows

20.8.2 Uses

- **shifter_lfsr_fibonacci.sv** — per-channel maximal-length Fibonacci LFSR
- **dataint_crc.sv** — per-channel CRC-32 accumulator

20.8.3 See Also

- **axis4_master_injector.sv** — the matching AXIS generator (same LFSR/CRC config)
- **axi4_slave_rd_injector.sv** — the AXI4 read pattern source this is copied from
- **axi4_master_rd_crc_check.sv** — the AXI4 read-side per-beat compare + CRC counterpart

---

20.9 Testing

`val/amba/test_axis4_pattern_pair.py` covers this module. It drives the generator and checker as a pair through a `tb_` wrapper.

The test is named for the PAIR rather than the module, which is why an earlier audit looking only for `val/**/test_axis4_slave_pattern_check.py` reported it as having no coverage. It does.

Treat any behaviour described on this page as unverified by simulation.

20.10 References

- Könemann, B., Mucha, J., Zwiehoff, G. “Built-in Test for Complex Digital Integrated Circuits.” *IEEE J. Solid-State Circuits*, 15(3):315–319, June 1980. DOI: 10.1109/JSSC.1980.1051391 — pattern generator plus signature compactor; a CRC-32 replaces the MISR.
- Koopman, P. “32-Bit Cyclic Redundancy Codes for Internet Applications.” *Proc. Int. Conf. Dependable Systems and Networks (DSN 2002)*, pp. 459–468, 2002. DOI: 10.1109/DSN.2002.1028931 — error-detection properties of the CRC-32 polynomial (0x04C11DB7).
- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051, Issue B, 2021. developer.arm.com/documentation/ihi0051/b, sec. 2.2 “Handshake signaling”, 2.6 “Packet boundaries” (TLAST) and 2.7 “Source and destination signaling” (TID, the per-channel demux key).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 4 “Basic Sequential Circuits”, sec. 4.5 “Pseudo Noise Sequence Generator” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `Crc32InternetApplications__Koopman2002.pdf`)

20.10.1 Source Code

- RTL: `rtl/amba/shared/axis4_slave_pattern_check.sv`

20.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Index: `docs/markdown/rtl-amba/index.md`
- Harness: `vault/tracking/projects/components/dma-ip/rapids/CONTROL_ENGINE_INTEGRATION.md`

Last Updated: 2026-07-15

20.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

21 AXIS Bus Meter

Module: `axis_bus_meter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

21.1 Overview

The AXIS Bus Meter is the AXI-Stream analogue of `axi_bus_meter`. It performs the same four-bucket per-cycle valid/ready classification (productive / backpressure / starvation / idle), and adds AXIS-native throughput counters that are **window-independent**: a payload-byte count derived from `tstrb` popcount and a packet count derived from `tlast`. Because bytes and packets are counted only on productive beats, they measure exactly how much data moved regardless of how long the measurement window stays open — so throughput computed as bytes / busy-time is immune to the backpressure and idle padding that would distort a pure cycle-utilization figure. Like its AXI cousin, it is a pure observer and drives nothing back onto the bus.

21.1.1 Key Features

- Four-bucket per-cycle classification identical to `axi_bus_meter`
- Window-independent AXIS throughput counters: 64-bit bytes (via `tstrb` popcount), 32-bit beats, 32-bit packets (via `tlast`)
- Aggregate 32-bit cycle-bucket counters (~42.9 s at 100 MHz before wrap)
- Per-channel 16-bit cycle buckets binned by `tid`, with per-channel 4-bit sticky overflow
- Synchronous one-cycle `i_clear` and `i_freeze` window control
- Passive snoop of `tvalid`/`tready`/`tlast`/`tstrb`/`tid` — no bus interaction

Cycle-utilization alone is a fragile throughput proxy on a stream bus: if a window is held open for host polling after the last beat, idle cycles inflate and the utilization ratio drops even though the same number of bytes moved. The AXIS Bus Meter fixes this by counting the actual payload transferred. Every productive beat contributes `popcount(tstrb)` bytes to a 64-bit accumulator and, when `tlast` is set, one packet to a 32-bit accumulator. These are byte-exact and independent of window length, so the honest throughput figure is `bytes / busy_time`. The four cycle buckets are retained alongside for root-causing where non-productive time went.

The block is instantiated one per AXIS bus to be measured, snooping the stream signals as a passive observer.

Use Cases: - Metering AXIS datapath throughput in the `rapids_char` characterization harness - Byte-exact throughput measurement immune to window-hold artifacts - Per-stream (per-`tid`) cycle-utilization breakdown on multi-stream buses - Packet-rate measurement via `tlast` on framed streams

Key Benefit: Window-independent byte and packet counts give a throughput number that cannot be distorted by how long the host holds the measurement window open.

21.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	512	Stream data width in bits (sets `tstrb`/byte-lane width)
NUM_CHANNELS	int	8	Number of per-channel (`tid`) bins
TID_WIDTH	int	`(NUM_CHANNELS > 1) ? $clog2(NUM_CHANNELS) : 1`	`tid` bus width; derived, do not override
SW	int	`DATA_WIDTH / 8`	`tstrb` width (byte lanes); derived
CW	int	`(NUM_CHANNELS > 1) ? $clog2(NUM_CHANNELS) : 1`	Channel-bin index width; derived

21.3 Ports

21.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

21.3.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes every counter and sticky
i_freeze	input	1	Hold high to close the measurement window (no counter moves)

21.3.3 AXIS Bus Snoop

Port	Direction	Width	Description
i_tvalid	input	1	Stream `tvalid`
i_tready	input	1	Stream `tready`
i_tlast	input	1	Stream `tlast` (increments packet count on a productive beat)
i_tstrb	input	SW	Byte-lane strobes; popcount gives this beat's payload byte count
i_tid	input	TID_WIDTH	Stream id; its low CW bits select the per-channel bin

21.3.4 Aggregate Cycle Buckets

Port	Direction	Width	Description
o_agg_productive	output	32	Cycles with `tvalid && tready` (beat transferred)

Port	Direction	Width	Description
o_agg_backpressure	output	32	Cycles with `tvalid && !tready`
o_agg_starvation	output	32	Cycles with `!tvalid && tready`
o_agg_idle	output	32	Cycles with `!tvalid && !tready`

21.3.5 Aggregate AXIS-Native Throughput (productive beats only)

Port	Direction	Width	Description
o_agg_bytes	output	64	Sum of `popcount(tstrb)` over productive beats — exact payload bytes moved
o_agg_beats	output	32	Productive beat count (equals `o_agg_productive`; provided for convenience)
o_agg_packets	output	32	Productive beats with `tlast` asserted (stream packets)

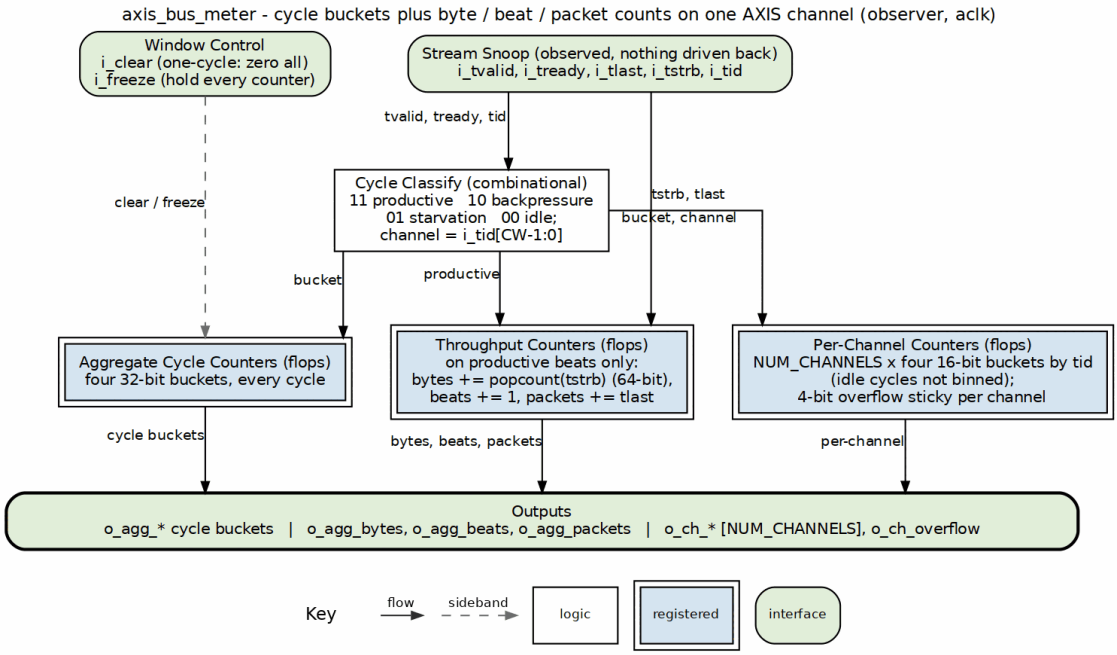
21.3.6 Per-Channel Cycle Buckets (binned by tid)

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive-cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure-cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation-cycle counts
o_ch_idle	output	16 × NUM_CHANNELS	Per-channel idle-cycle counts (always zero — see Design Notes)

Port	Direction	Width	Description
o_ch_overflow	output	NUM_CHANNELS*4	Sticky overflow mask, packed `{prod, bp, starv, idle}` per channel

21.4 Functional Description

21.4.1 Block diagram



axis_bus_meter block diagram

The meter only observes. The same four-bucket cycle classification as `axi_bus_meter` feeds the aggregate cycle counters and, binned by `tid`, the per-channel counters (idle cycles are not binned). On every productive beat the throughput counters add the payload bytes (the popcount of `tstrb`), one beat, and one packet if `tlast` is set. These counts do not depend on how long the window stays open. `i_clear` zeroes everything and `i_freeze` holds it.

21.4.2 Bucket Classification

Identical to `axi_bus_meter`: the handshake pair decodes combinationally into four mutually-exclusive buckets.

```
w_prod  =  i_tvalid &&  i_tready   // productive   — beat transferred
w_bp    =  i_tvalid && !i_tready   // backpressure — master wants to
send, slave stalls
w_starv = !i_tvalid &&  i_tready   // starvation   — slave ready,
master not producing
w_idle  = !i_tvalid && !i_tready   // idle         — both sides quiet
```

See `DMA_UTILIZATION_MEASUREMENT.md` §3 for the reference methodology.

21.4.3 Payload-Byte Popcount

Each beat's byte count is the number of asserted `tstrb` lanes, computed combinationally by summing the strobe bits into `w_beat_bytes`. If the stream has no meaningful `tstrb`, tie it all-ones so bytes accumulate as `beats × (DATA_WIDTH/8)`.

21.4.4 AXIS-Native Throughput Counters

On every productive beat (`tvalid && tready`, window open):

- `o_agg_bytes` += this beat's `popcount(tstrb)` — a 64-bit accumulator that will not wrap in any realistic run.
- `o_agg_beats` += 1 — a convenience mirror of `o_agg_productive`.
- `o_agg_packets` += 1 when `i_tlast` is set — counts framed stream packets.

Because these advance only on productive beats, they are window-length independent: holding `i_freeze` low longer only accumulates more idle cycles in the aggregate buckets, never more bytes or packets.

21.4.5 Per-Channel Cycle Buckets

Per-channel buckets are 16-bit, `NUM_CHANNELS` deep, binned by the low `CW` bits of `tid` (`w_ch`). AXIS carries `tid` on every valid cycle, so the productive, backpressure, and starvation buckets are attributed to `w_ch` whenever the bus is active. Each per-channel counter has a companion sticky overflow bit (`{prod, bp, starv, idle}` packed into `o_ch_overflow`) that latches when the counter would advance past `16'hFFFF`, exactly as in `axi_bus_meter`. Software discards the per-channel numbers for any channel whose overflow mask is nonzero.

21.4.6 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse that zeroes every counter and sticky. `i_freeze` holds all counters frozen while high and is driven from the characterization window controller so the window closes the moment the workload finishes.

21.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

21.6 Usage Examples

```
// Meter an AXIS bus: cycle buckets + byte/packet throughput, per-tid
bins.
axis_bus_meter #(
    .DATA_WIDTH     (512),
    .NUM_CHANNELS   (8)
) u_axis_meter (
    .aclk               (aclk),
    .aresetn            (aresetn),

    .i_clear            (perf_run_rising),
    .i_freeze           (~perf_run),

    // Stream snoop (all inputs — passive)
    .i_tvalid           (s_axis_tvalid),
    .i_tready           (s_axis_tready),
    .i_tlast            (s_axis_tlast),
    .i_tstrb            (s_axis_tstrb),      // tie all-ones if the
stream has no tstrb
    .i_tid              (s_axis_tid),

    // Aggregate cycle buckets
    .o_agg_productive   (agg_prod),
    .o_agg_backpressure (agg_bp),
    .o_agg_starvation   (agg_starv),
    .o_agg_idle         (agg_idle),

    // Window-independent throughput
    .o_agg_bytes        (agg_bytes),
    .o_agg_beats        (agg_beats),
    .o_agg_packets      (agg_packets),

    // Per-channel (by tid)
    .o_ch_productive    (ch_prod),
    .o_ch_backpressure  (ch_bp),
    .o_ch_starvation    (ch_starv),
    .o_ch_idle          (ch_idle),
    .o_ch_overflow      (ch_overflow)
);

// Honest throughput = o_agg_bytes / busy_time  (immune to window-hold
padding)
```

21.7 Design Notes

21.7.1 Why Byte/Packet Counters Are Window-Independent

A pure cycle-utilization ratio drops if the window is held open past the last beat (host polling adds idle cycles). Byte and packet counters advance only on productive beats, so they capture the true transferred payload no matter how long the window is held. Reporting `bytes / busy_time` sidesteps the padding-sensitivity of `productive / total_cycles`.

21.7.2 Per-Channel Idle Is Not Attributed

Idle cycles (`!tvalid && !tready`) cannot be assigned to any stream — no `tid` is meaningful when nothing is on the bus — so `r_ch_idle` is never incremented and `o_ch_idle` stays zero. Idle cycles land only in the aggregate `o_agg_idle`. The per-channel idle output and its overflow bit are kept for structural symmetry with `axi_bus_meter`.

21.7.3 tstrb Handling

If the source does not drive a meaningful `tstrb`, tie it all-ones so `o_agg_bytes` equals `beats × (DATA_WIDTH/8)`. Otherwise `o_agg_bytes` reflects sparse/partial beats exactly.

21.7.4 Counter Widths

Cycle buckets are 32-bit (aggregate) / 16-bit (per-channel) matching `axi_bus_meter`. Bytes are 64-bit to guarantee no wrap; beats and packets are 32-bit.

21.8 Related Modules

21.8.1 Used By

- `rapids_char` AXIS datapath meters (characterization harness)
- Host-driven AXIS throughput characterization runs

21.8.2 Uses

- **reset_defs.svh** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

21.8.3 See Also

- **axi_bus_meter.sv** - AXI channel four-bucket meter (this module's origin)
- **axi_perf_latency_hist.sv** - Latency histogram sharing the same window-control convention

---

21.9 Testing

`val/amba/test_axis_bus_meter.py` exercises this module. It collects 10 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis_bus_meter.py -v
```

21.10 References

- Arm. *AMBA AXI-Stream Protocol Specification*, ARM IHI 0051, Issue B, 2021. developer.arm.com/documentation/ihi0051/b, sec. 2.2 “Handshake signaling”, 2.5 “Byte qualifiers” (TSTRB byte counts) and 2.6 “Packet boundaries” (TLAST packet counts). No paper: the four-bucket classification is a standard technique.
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 13 “Buffered Flow Control”, sec. 13.3 “Buffer Management and Backpressure” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

21.10.1 Source Code

- RTL: `rtl/amba/shared/axis_bus_meter.sv`

21.10.2 Documentation

- Methodology: `docs/.../DMA_UTILIZATION_MEASUREMENT.md` (four-bucket definition, window semantics)
- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Design Guide: `docs/markdown/rtl-amba/index.md`

Last Updated: 2026-07-15

21.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

22 Clock-Gated Variants Guide

Category: Infrastructure Documentation **Location:** `rtl/amba/shared/amba_clock_gate_ctrl.sv`, `rtl/common/clock_gate_ctrl.sv` **Applies To:** All AMBA modules with a `_cg` suffix **Status:** Production Ready (transport modules) — see Known Gaps for monitor variants

---

22.1 Overview

Most AMBA protocol modules have a clock-gated (_cg) variant that wraps the base module, drives it from a gated clock, and gates that clock after a programmable idle period. The gating decision is made entirely at runtime through configuration inputs — there is no compile-time enable parameter on the wrapper.

Naming Convention: {base_module}_cg.sv

Examples: - axi4_master_rd.sv → axi4_master_rd_cg.sv - apb4_slave.sv → apb4_slave_cg.sv - axis4_master.sv → axis4_master_cg.sv

The key principle: **clock-gated variants preserve the functional behavior of the base module and add runtime power management.**

```
Base Module + amba_clock_gate_ctrl + activity detection = _cg Variant
```

All base-module parameters and ports are preserved. The wrapper adds one parameter, two configuration inputs, and one or two status outputs.

One thing to understand before you wire one up: a _cg variant is not cycle-identical to its base module. Most _cg wrappers force the relevant *ready signals low while the clock is gated, so the first transfer that arrives out of a gated period is backpressured for the wake-up cycle. Transfers are delayed, never dropped. See Wake-Up Behavior.

22.2 Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle counter. Maximum idle count is 2**CG_IDLE_COUNT_WIDTH - 1 (15 cycles at the default width). Sets the width of cfg_cg_idle_count.

CG_IDLE_COUNT_WIDTH is a counter **width**, not a cycle count. To change the idle threshold at runtime, drive cfg_cg_idle_count; widen CG_IDLE_COUNT_WIDTH only when the threshold you need exceeds the current counter range.

All other parameters are identical to the base module.

22.3 Ports

22.3.1 Configuration Inputs

Port	Direction	Width	Description
`cfg_cg_enable`	Input	1	Enable clock gating. `0` = clock always runs (runtime bypass).
`cfg_cg_idle_count`	Input	`CG_IDLE_COUNT_WIDTH`	Idle countdown value. Gating engages `cfg_cg_idle_count + 1` clocks after the internal wakeup deasserts, which is `cfg_cg_idle_count + 2` clocks after the last bus activity on single-stage families and `+ 3` on two-stage families. See Gating Latency.

22.3.2 Status Outputs

Status port naming differs by protocol family. Check the module port list before wiring.

Family	Status Ports
AXI4, AXI5, AXI4-Lite, AXI-Stream (`axis4`)	`cg_gating`, `cg_idle`
AXI5 monitor (`*_mon_cg`)	`cg_gating`, `cg_idle`
AXI5-Stream (`axis5`)	`cg_gating`
APB, APB5 (non-CDC)	`cg_gating` (the controller `idle` output is left unconnected)
APB5 CDC (`apb5_slave_cdc_cg`)	`cg_gating`
APB CDC (`apb4_slave_cdc_cg`)	`cg_gating`, `cg_idle`, `cg_gating`, `cg_idle` (two gating domains, two controller instances)

All other ports are identical to the base module.

22.3.3 Ports That Do Not Exist

These are common misconceptions; none of them are present in the RTL:

- **No `cg_clk_count` / gated-cycle counter.** The wrappers expose instantaneous state only. Accumulate `cg_gating` externally if a power metric is needed — see Measuring Gated Cycles.
- **No scan or test-bypass port.** For DFT, hold `cfg_cg_enable` low (`.cfg_cg_enable(~scan_mode)`).
- **No `ENABLE_CLOCK_GATING`, `CG_IDLE_CYCLES`, or `CG_GATE_*` parameters** on any transport-level `_cg` module. Those names appear only in the legacy AXI4 and AXI4-Lite `*_mon_cg` wrappers described under Known Gaps.

22.4 Functional Description

22.4.1 Gating Infrastructure

The gating logic lives in two shared modules, not in the wrappers themselves.

22.4.1.1 amba_clock_gate_ctrl (rtl/amba/shared/)

The AMBA-facing adapter. It ORs the two activity inputs, registers the result into `r_wakeup`, exposes `idle = ~r_wakeup`, and passes `r_wakeup` to the generic controller.

Port	Direction	Description
`clk_in`	Input	Ungated clock
`aresetn`	Input	Asynchronous active-low reset
`cfg_cg_enable`	Input	Global gating enable (0 = clock always runs)
`cfg_cg_idle_count`	Input	Idle countdown value, `CG_IDLE_COUNT_WIDTH` bits
`user_valid`	Input	Any user-side valid/activity signal
`axi_valid`	Input	Any bus-side valid/activity signal
`clk_out`	Output	Gated clock
`gating`	Output	Clock currently gated
`idle`	Output	No activity seen in the previous cycle

22.4.1.2 clock_gate_ctrl (rtl/common/)

The generic countdown controller. It loads `cfg_cg_idle_count` on reset, on `wakeup`, and whenever `cfg_cg_enable` is low; otherwise it decrements to zero and holds. The gate condition is:

```
w_gate_enable = cfg_cg_enable && !wakeup && (r_idle_counter == 0);
```

`w_gate_enable` drives the `icg` cell (inverted, since the cell enable is active-high) and is also driven out as `gating`.

22.4.2 Activity Detection

Each wrapper builds `user_valid` and `axi_valid` from the valid signals (and, on the AXI transport wrappers, the base module's `busy` output) of the channels it owns. For example, `axi4_master_rd_cg`:

```
assign user_valid = fub_axi_arvalid || fub_axi_rvalid || int_busy;
assign axi_valid  = m_axi_arvalid   || m_axi_rvalid;
```

> **A peer's READY must never appear in the activity term.** A consumer that parks its response-ready high while idle is behaving correctly; folding that in pins the block permanently awake and defeats gating entirely, silently, because function is unaffected. Use the VALID your side drives. Ten _cg wrappers carried this defect until 2026-09-02.

22.4.3 Gating Conditions (All Must Be True)

1. `cfg_cg_enable = 1`
2. The internal `r_wakeup` is deasserted (no activity in the previous cycle)
3. The idle counter has decremented to zero

22.4.4 Ready Forcing While Gated

The following wrappers drive the relevant `*ready` outputs to zero while gated, so the first beat out of a gated period is backpressured rather than lost:

- All AXI4, AXI5, and AXI4-Lite transport _cg modules
- `axis4_master_cg`, `axis4_slave_cg`
- All four AXI5 `*_mon_cg` modules
- `apb4_slave_cdc_cg` (both clock domains)

`apb4_master_cg`, `apb4_slave_cg`, the APB5 variants, and the AXI5-Stream variants do not force ready.

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> COUNTING : wakeup deasserted
    COUNTING --> RUNNING : activity detected<br/>(counter reloads)
    COUNTING --> GATED : counter reaches 0<br/>&& cfg_cg_enable
    GATED --> RUNNING : activity detected<br/>(1 stage / 2 clocks to
first edge;<br/>2 stages / 3 clocks on APB, APB5, AXI5-Stream)

    state RUNNING {
        note right of RUNNING : Clock running, counter reloaded
    }

    state COUNTING {
        note right of COUNTING : Counter decrementing to 0
    }
```

```
    state GATED {
        note right of GATED : Clock stopped, ready forced low
    }
```

22.5 Timing Characteristics

22.5.1 Counting Convention

Two numbers are quoted throughout this book, and they must not be confused:

- **Register stages** — how many flops an activity edge passes through before it reaches the ICG enable.
- **First usable gated-clock edge** — how many clocks after activity asserts before the gated block sees a rising edge it can work on. This is always one more than the register stage count, because the ICG enable itself is combinational and the released clock is only usable on the following edge.

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. `apb4_slave_cdc_cg` is the one exception – it drives `amba_clock_gate_ctrl` combinationally and so registers once despite being APB. The first gated-clock rising edge available to the block therefore arrives **2 clocks** (single-stage families) or **3 clocks** (two-stage families) after activity asserts.

`clock_gate_ctrl` itself adds **no** flop on the wake-up path: `w_gate_enable` is a combinational function of `wakeup`, feeding the `icg` enable directly. Its header comment "Wakeup: 1 clock from wakeup assertion to clock restoration" describes the resulting clock edge, not a register stage.

22.5.2 Register Stages by Family

Family	Wrapper flop	`amba_clock_gate_ctrl` flop	Stages	First usable edge
AXI4, AXI5, AXI4-Lite `_cg`	No (combinational)	Yes	1	2 clocks
`axis4_master_cg`, `axis4_slave_cg` (AXI4-Stream)	No (combinational)	Yes	1	2 clocks
All `*_mon_cg`	No (combinational)	Yes	1	2 clocks

Family	Wrapper flop	`amba_clock_gate_ctrl` flop	Stages	First usable edge
monitor wrappers				
`apb4_slave_cdc_cg`	No (combinational)	Yes	1	2 clocks
`apb4_master_cg`, `apb4_slave_cg`	Yes	Yes	2	3 clocks
`apb5_master_cg`, `apb5_slave_cg`, `apb5_slave_cdc_cg`	Yes	Yes	2	3 clocks
`axis5_master_cg`, `axis5_slave_cg` (AXI5-Stream)	Yes	Yes	2	3 clocks

Note: the AXI5-Stream `_cg` wrappers register activity locally and so behave like the APB family, not like the AXI4-Stream wrappers. Conversely `apb4_slave_cdc_cg` drives the activity terms combinationally and so behaves like the AXI family. On both CDC wrappers the cross-domain activity term additionally pays the usual two-flop synchronizer delay in the receiving domain, on top of the stages counted above.

22.5.3 Gating Latency

`gating` asserts `cfg_cg_idle_count + 1` clocks after the last cycle in which the internal `wakeup` was high. Measured from the last bus activity, that is:

- `cfg_cg_idle_count + 2` clocks on the single-stage families (AXI4, AXI5, AXI4-Lite, AXI4-Stream, the monitor `_cg` wrappers, `apb4_slave_cdc_cg`)
- `cfg_cg_idle_count + 3` clocks on the two-stage families (APB, APB5, AXI5-Stream)

The extra clock in each case is the time activity takes to appear on `r_wakeup`.

With `cfg_cg_idle_count = 0`, gating engages on the next clock after wakeup deasserts.

22.5.4 Wake-Up Behavior

Wake-up latency is not zero. Activity is registered before it can release the gate.

AXI4, AXI5, AXI4-Lite, AXI4-Stream: the wrapper combines the valid signals combinationally, so the only flop in the path is `r_wakeup` inside `amba_clock_gate_ctrl` — **1 register stage, first usable clock edge 2 clocks** after activity asserts:

```
Cycle N:   Clock gated (cg_gating = 1). ARVALID asserts.
           user_valid = 1 combinationally; ARREADY forced low, so no
handshake.
Cycle N+1: r_wakeup = 1 -> gate released combinationally, cg_gating =
0.
Cycle N+2: First usable gated-clock rising edge. ARREADY reflects the
base
           module; the transfer can complete.
```

APB, APB5, AXI5-Stream: these wrappers register activity into their own `r_wakeup` flop before handing it to `amba_clock_gate_ctrl`, which registers it again — **2 register stages, first usable clock edge 3 clocks** after activity asserts.

22.6 Usage Examples

22.6.1 Pattern 1: Aggressive Gating

Use Case: Bursty traffic, low duty cycle, power-constrained systems.

```
axi4_master_rd_cg #(
    // Base module parameters (same as the non-CG variant)
    .AXI_ID_WIDTH        (8),
    .AXI_ADDR_WIDTH      (32),
    .AXI_DATA_WIDTH      (64),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)          // counter range 0-15
) u_cg_aggressive (
    .aclk                (clk),
    .aresetn             (rst_n),

    .cfg_cg_enable       (1'b1),
    .cfg_cg_idle_count   (4'd1),      // gate 2 clocks after wakeup
deasserts

    // ... all other ports same as the base module ...

    .cg_gating           (rd_gated),
    .cg_idle             (rd_idle)
);
```

22.6.2 Pattern 2: Runtime-Adjustable Threshold

Use Case: A power-management controller trades wake-up cost against gated time.

```
logic [3:0] idle_threshold;

always_comb begin
    case (power_mode)
        POWER_HIGH_PERF: idle_threshold = 4'd15;  // conservative
        POWER_BALANCED:  idle_threshold = 4'd5;   // moderate
        POWER_LOW:       idle_threshold = 4'd1;   // aggressive
        default:         idle_threshold = 4'd5;
    endcase
end

axi4_master_wr_cg #(
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg_dynamic (
    .cfg_cg_enable       (power_mgmt_enable),
```

```
    .cfg_cg_idle_count   (idle_threshold),
    .cg_gating           (wr_gated),
    .cg_idle             (wr_idle)
    // ... base module ports ...
);
```

22.6.3 Pattern 3: Gating Bypassed

Use Case: Functional verification, DFT scan, latency-critical operation.

```
axi4_master_rd_cg #(
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg_bypassed (
    .cfg_cg_enable       (1'b0),      // clock always runs
    .cfg_cg_idle_count   (4'd0),      // ignored while cfg_cg_enable =
0
    // ... base module ports ...
);
```

With `cfg_cg_enable = 0` the counter is continuously reloaded, `gating` stays low, the ICG enable stays high, and the ready-forcing terms resolve to the base module's own ready signals. The wrapper is then behaviorally equivalent to the base module.

This is the only bypass mechanism. There is no separate scan or test-mode port.

22.6.4 Measuring Gated Cycles

The wrappers do not count gated cycles. Accumulate the status output where the metric is needed:

```
logic [31:0] total_cycles, gated_cycles;

always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1'b1;
        if (cg_gating) gated_cycles <= gated_cycles + 1'b1;
    end
end

// Gated fraction = (gated_cycles / total_cycles) x 100%
```

Note that this counter must itself be clocked from an ungated clock.

22.7 Design Notes

22.7.1 Synthesis and Portability

`clock_gate_ctrl` instantiates a bare `icg` primitive by name:

```
icg u_icg (
    .clk (clk_in),
    .en  (~w_gate_enable),
    .gclk(clk_out)
);
```

`rtl/common/icg.sv` provides a behavioral model (a low-phase `always_latch` on the enable ANDed with the clock) that is correct for simulation and for a standard-cell flow where the name maps to a library integrated clock gate.

ASIC: map `icg` to the foundry integrated clock-gating cell. Verify the enable setup and hold requirements of the chosen cell, and add clock-gating checks to the timing constraints.

FPGA portability note: the `icg` behavioral model is not an FPGA-friendly construct. An inferred latch feeding an AND gate on a clock net will either be rejected or produce a glitch-prone, unconstrained clock. For FPGA targets, either

- replace `icg` with a vendor clock-enable primitive (Xilinx `BUFGCE`, Quartus `ALTCLKCTRL`, and equivalents), or
- hold `cfg_cg_enable` low and use the base (non-`_cg`) module, converting the gating intent into ordinary clock enables that the synthesis tool can infer.

This is a real portability constraint, not a tuning preference. Do not push a `_cg` variant through an FPGA flow without addressing it.

22.7.2 Known Gaps

None. Every `_cg` module in `rtl/amba` – all 34 – instantiates `amba_clock_gate_ctrl`, takes `CG_IDLE_COUNT_WIDTH`, and drives the wrapped module from the gated clock.

This section previously described the eight AXI4 and AXI4-Lite `*_mon_cg` wrappers as a dead, pre-`amba_clock_gate_ctrl` scheme that "saves no dynamic power today", carrying legacy parameters (`ENABLE_CLOCK_GATING`, `CG_IDLE_CYCLES`, `CG_GATE_*`) and per-domain clocks (`aclk_monitor`, `aclk_reporter`, `aclk_timers`) wired to nothing. That was true once; it is the exact opposite of the current RTL, and it is the more dangerous direction of error – an integrator reading it would route around the correct part. Measured on all eight: zero occurrences of any legacy name, two `amba_clock_gate_ctrl` references each, and the wrapped monitor instantiated with `.aclk(gated_aclk)`.

22.8 Related Modules

22.8.1 Available Clock-Gated Variants

22.8.1.1 Transport Modules (22)

Every module below instantiates `amba_clock_gate_ctrl` and takes `CG_IDLE_COUNT_WIDTH`, `cfg_cg_enable`, and `cfg_cg_idle_count`.

Protocol	Location	CG Variants
AXI4	`rtl/amba/axi4/`	`axi4_master_rd_cg`, `axi4_master_wr_cg`, `axi4_slave_rd_cg`, `axi4_slave_wr_cg`
AXI5	`rtl/amba/axi5/`	`axi5_master_rd_cg`, `axi5_master_wr_cg`, `axi5_slave_rd_cg`, `axi5_slave_wr_cg`
AXI4-Lite	`rtl/amba/axil4/`	`axil4_master_rd_cg`, `axil4_master_wr_cg`, `axil4_slave_rd_cg`, `axil4_slave_wr_cg`
APB	`rtl/amba/apb4/`	`apb4_master_cg`, `apb4_slave_cg`, `apb4_slave_cdc_cg` (two gating domains)
APB5	`rtl/amba/apb5/`	`apb5_master_cg`, `apb5_slave_cg`, `apb5_slave_cdc_cg`
AXI-Stream	`rtl/amba/axis4/`	`axis4_master_cg`, `axis4_slave_cg`
AXI5-Stream	`rtl/amba/axis5/`	`axis5_master_cg`, `axis5_slave_cg`

22.8.1.2 Monitor Modules (12)

Protocol	Location	CG Variants	Gating Implemented
AXI5	`rtl/amba/axi5/`	`axi5_master_rd_mon_cg`, `axi5_master_wr_mon_cg`,	Yes

Protocol	Location	CG Variants	Gating Implemented
		axi5_slave_rd_mon_cg, axi5_slave_wr_mon_cg	
AXI4	rtl/amba/axi4/	axi4_master_rd_mon_cg, axi4_master_wr_mon_cg, axi4_slave_rd_mon_cg, axi4_slave_wr_mon_cg	Yes
AXI4-Lite	rtl/amba/axil4/	axil4_master_rd_mon_cg, axil4_master_wr_mon_cg, axil4_slave_rd_mon_cg, axil4_slave_wr_mon_cg	Yes

The _mon_cg wrappers live beside the protocol they wrap, not in rtl/amba/ocelus/ – that directory holds the monitor cores themselves and contains no _cg module at all.

Total: 34 _cg modules, all 34 of which implement clock gating.

22.8.2 Related Documentation

- **Gating Controller:** amba_clock_gate_ctrl.md
- **Per-Protocol Guides:**
  - AXI4 Clock Gating Guide
  - AXI4-Lite Clock Gating Guide
  - AXI-Stream Clock Gating Guide
- **Base Module Documentation:** see the per-protocol directories (axi4/, axi5/, axil4/, apb/, apb5/, axis4/, axis5/)
- **AMBA Overview:** overview.md

22.9 Testing

22.9.1 Functional Verification

Hold `cfg_cg_enable` low for functional regression runs. This yields simpler waveforms, faster simulation, and removes the wake-up backpressure cycle from the transfer timing.

22.9.2 Gating Verification

For gating-specific tests, drive `cfg_cg_enable` high and:

1. Sweep `cfg_cg_idle_count` and confirm `cg_gating` asserts `cfg_cg_idle_count + 1` clocks after the last wakeup.
2. Confirm the wake-up latency — 1 register stage and a first usable gated-clock edge 2 clocks after activity asserts, or 2 stages and 3 clocks on APB, APB5, and AXI5-Stream — and that no transfer is lost across a gate-to-ungate transition.
3. Confirm that `*ready` is low for the whole gated interval on the wrappers listed under Ready Forcing While Gated.
4. Confirm `cfg_cg_enable = 0` reproduces base-module timing exactly.

22.10 References

- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design*, 15(6):630–643, June 1996. DOI: 10.1109/43.503933 — the latch-based glitch-free clock gate inside the wrapped `clock_gate_ctrl` (`icg`).
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3):299–316, June 2000. DOI: 10.1109/92.845896 — fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 3 “Sequential Logic Design”, sec. 3.2.5 (clock gating) and 3.5 (skew from gated clocks) — textbook treatment (secondary source).
- Indexed in `rtl/common/references/README.md` (local PDF: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

22.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index
- Back to Main Documentation Index

23 CXS Pattern Injector and Checker

Modules: `cxs_master_injector.sv`, `cxs_slave_pattern_check.sv` **Location:** `rtl/amba/shared/` **Status:** New (2026-10-16)

23.1 Overview

These are the CXS counterparts of `axis4_master_injector` and `axis4_slave_pattern_check`: the same per-channel LFSR pattern and CRC-32, carried on a credited CXS link (Arm IHI 0079, Issue D) instead of an AXI-Stream. They are compositions; nothing is re-implemented:

```
cxs_master_injector     = axis4_master_injector -> axis_to_cxs ->
cxs_master        -> m_cxs_*
cxs_slave_pattern_check = s_cxs_* -> cxs_slave   -> cxs_to_axis ->
axis4_slave_pattern_check
```

Every `cfg_*` and status port is the wrapped AXIS block's, unchanged, so the expected and actual CRCs stay bit-identical to the AXIS pair, `axi4_slave_rd_injector` and `axi4_slave_wr_crc_check`. A harness that already drives the AXIS pair swaps them module for module.

23.2 The Link

The link profile is the RAPIDS CXS plane's (see AXIS to CXS Adapters, Converters MAS 3.17): Issue D, `CXSMAXPKTPERFLIT` = 2, 16-byte start alignment, `TID` -> `CXSSRCID` (the channel), `TDEST` -> `CXSTGTID`, `CXSUSER` = `{TUSER, TSTRB}` (TUSER dropped above 1016-bit flits), link activation on, no parity.

`cfg_interleave` round-robins channels beat by beat, so packets of different channels are open at the same time on the one link. That is legal CXS because packets are framed per stream (IHI 0079D 2.5.2). Both blocks key on the channel bits of `CXSSRCID` (`STREAM_SRCID_BITS`, default `$clog2(NUM_CHANNELS)`), and both ends must use the same value.

On the checker, `ready_en` still gates the stream: held low, the receive FIFO fills and the link stops granting credits, which is what backpressure is on CXS.

23.3 Parameters

Beyond the wrapped AXIS block's parameters:

Parameter	Block	Default	Description
CXS_MAX_CREDIT	both	32	Credits the link carries (one credit = one flit)
CXS_CREDIT_BEYOND_SPEC	both	0	Allow more than 63 credits; both ends must agree
STREAM_SRCID_BITS	both	`$clog2(NUM_CHANNELS)`	Per-stream framing key on `CXSSRCID`
CXS_IDLE_CYCLES	injector	32	Idle RUN cycles before the link deactivates
RX_FIFO_DEPTH	checker	CXS_MAX_CREDIT	Receive FIFO = the credit pool

`AXIS_DATA_WIDTH` must be 256, 512 or 1024. Full CXS (with `CXSCNTL`) has no 128-bit flit (rule 6), and `TSTRB` fills `CXSUSER` at 1024 bits.

23.4 Status Ports

Port	Block	Meaning
`link_state`	both	`{CXSACTIVEREQ, CXSACTIVEACK}`
`protocol_error`	both	Sticky: a credit, link or framing rule was broken
`unsupported`	checker	Sticky: a flit AXIS framing cannot represent arrived

23.5 Verification

`val/amba/test_cxs_pattern_pair.py` wires the injector's link straight into the checker (`val/amba/tb_cxs_pattern_pair.sv`) and runs every scenario of the AXIS pair test (`bin/TBClasses/cxs/cxs_pattern_pair_tb.py` subclasses the AXIS pair TB): per-channel CRC agreement, beat counts and the exact sequential and interleaved schedules, under backpressure. On top of that, an RDS-DV `CXSMonitor` checks the link on every cycle, both endpoints' `protocol_error` and the checker's `unsupported` must stay low, and the link must reach RUN. The grid covers 4 channels at 512 bits, 2 at 256 and 8 at 1024.

24 SDPRAM Core

Module: `sdpram_core.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

24.1 Overview

`sdpram_core` is the protocol-agnostic Simple Dual-Port RAM backend shared by the four `sdpram_slave_*` protocol wrappers. It owns the inferred dual-port BRAM array, a burst-aware write tracker, a burst-aware read tracker, and a bulk-clear FSM. It exposes a single FUB-shaped slave interface (an AXI superset with `id / addr / len / size / burst` on the address channels and `id / resp / last` on the response channels) so that a protocol-specific wrapper can drop straight on top without any string-switch `generate` plumbing.

24.1.1 Key Features

- Inferred simple dual-port BRAM: one write port (port A) and one read port (port B)
- No reset on the RAM array (BRAM contents are undefined at power-up, cleared only via the bulk-clear FSM)
- FPGA `ram_style = "auto"` attribute for tool-driven BRAM inference
- Burst-aware write and read trackers driven by `axi_gen_addr` (INCR / FIXED any length up to AXI4's 256-beat max)
- Byte-enabled writes via `fub_wstrb`
- Single-cycle read latency on port B
- Bulk-clear FSM that walks the whole array to zero, gated on both sides idle
- FUB-shaped interface that degenerates cleanly to single-beat for AXIL wrappers
- Observation outputs (valid/ready snapshot, BRAM write/read fire pulses)

The `sdpram_slave_*` family needs one memory kernel that behaves identically regardless of which AMBA protocol drives its write and read sides. Rather than duplicate the BRAM, burst logic, and clear FSM in every protocol permutation, that logic lives once here, and the wrappers translate their protocol's FUB into this core's AXI-shaped FUB.

The core speaks exactly one wire format. AXI4 wrappers pass the real `awlen / awsize / awburst / awid` fields straight through; AXIL wrappers, which have no burst or ID fields, feed single-beat defaults (`awlen=0`, `awsize=$clog2(STRB_W)`, `awburst=INCR`, `awid=0`) so the burst tracker collapses to a single-beat path.

Use Cases: - Shared backend for all four `sdpram_slave_{axi4,axil}_{axi4,axil}` wrappers - Memory model / descriptor-RAM / semaphore-RAM inside characterization harnesses - Memory ring backend behind a monitor-bus capture master-write port - Any test bench needing a synthesizable dual-port scratch memory with a fast clear

Key Benefit: One compute kernel with all the burst and clear complexity in a single place, so the protocol wrappers stay thin and no "fake AXI4" fields leak into any external port list.

24.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	FUB transaction-ID width. Passthrough on AXI4 wrappers; tied to a 1-bit zero by AXIL wrappers
ADDR_WIDTH	int	32	Byte-address width on both FUB address channels
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
MEM_DEPTH	int	2048	Number of BRAM words (array depth)
USE_WSTRB	bit	1'b1	Honour WSTRB byte enables on writes. 0 = every write commits the full word.

Derived localparams: - `STRB_W = DATA_WIDTH / 8` — write-strobe / byte-enable width - `ADDR_LSB = $clog2(STRB_W)` — byte-offset bits below the word address - `MEM_AW = $clog2(MEM_DEPTH)` — BRAM word-address width - `WORD_AW = ADDR_WIDTH - ADDR_LSB` — full word-address width

---

24.3 Ports

24.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset (control logic only; never resets the BRAM array)

24.3.2 FUB Write Side (AW + W + B)

Port	Direction	Width	Description
fub_awid	input	AXI_ID_WIDTH	Write transaction ID (echoed back on `fub_bid`)
fub_awaddr	input	ADDR_WIDTH	Write burst base byte-address
fub_awlen	input	8	Burst length minus 1 (0 = single beat)
fub_awsize	input	3	Beat size (log2 bytes) fed to `axi_gen_addr`
fub_awburst	input	2	Burst type: INCR (2'b01) / FIXED (2'b00) / WRAP (2'b10)
fub_awvalid	input	1	Write-address valid
fub_awready	output	1	Write-address ready (accepts when no write active, no B pending, not clearing)
fub_wdata	input	DATA_WIDTH	Write data
fub_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
fub_wvalid	input	1	Write-data valid
fub_wread	output	1	Write-data ready (asserted

Port	Direction	Width	Description
y			while a write burst is active and not clearing)
fub_bid	output	AXI_ID_WIDTH	Write-response ID (captured from `fub_awid`)
fub_bresp	output	2	Write response (OKAY 2'b00; SLVERR 2'b10 reserved for out-of-range)
fub_bvalid	output	1	Write-response valid (asserted while B pending)
fub_bready	input	1	Write-response ready

24.3.3 FUB Read Side (AR + R)

Port	Direction	Width	Description
fub_arid	input	AXI_ID_WIDTH	Read transaction ID (echoed on `fub_rid`)
fub_araddr	input	ADDR_WIDTH	Read burst base byte-address
fub_arlen	input	8	Burst length minus 1 (0 = single beat)
fub_arsize	input	3	Beat size (log2 bytes) fed to `axi_gen_addr`
fub_arburst	input	2	Burst type: INCR / FIXED / WRAP
fub_arvalid	input	1	Read-address valid
fub_arready	output	1	Read-address ready (accepts when no read active, not clearing)
fub_rid	output	AXI_ID_WIDTH	Read-data ID (from the inflight tracker)
fub_rdata	output	DATA_WIDTH	Read data (BRAM port-B output)
fub_rresp	output	2	Read response (OKAY /

Port	Direction	Width	Description
			SLVERR reserved)
fub_rlast	output	1	Last-beat marker of the read burst
fub_rvalid	output	1	Read-data valid (reflects the inflight tracker)
fub_rready	input	1	Read-data ready

24.3.4 Bulk-Clear Control

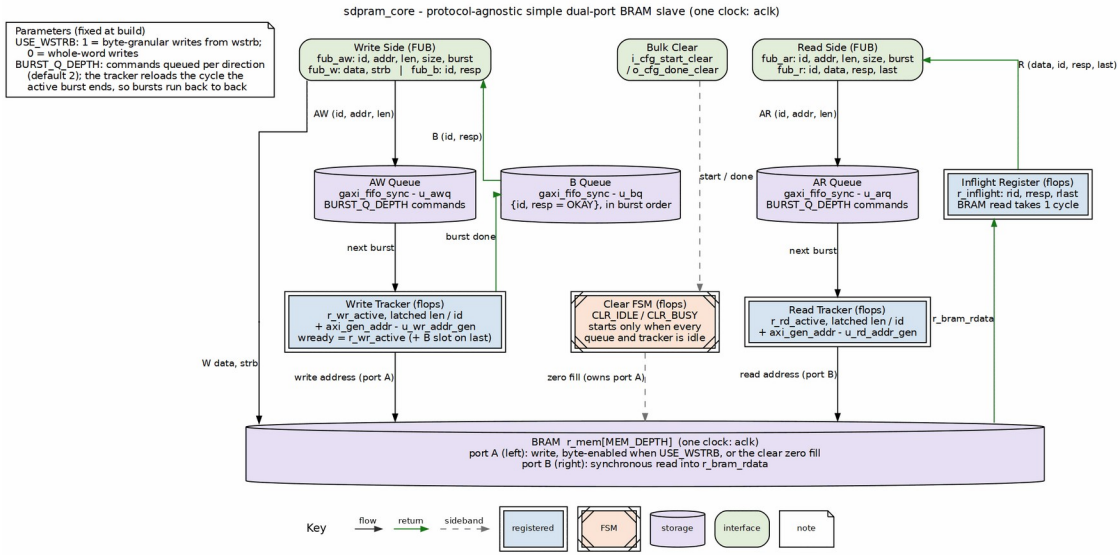
Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request a full-array clear (accepted only when both sides idle)
o_cfg_done_clear	output	1	Asserted when the clear walk has finished

24.3.5 Observation

Port	Direction	Width	Description
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot: `{rready,rvalid, arready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}`
o_dbg_bram_wr	output	1	One-cycle pulse when a BRAM port-A write fires
o_dbg_bram_rd	output	1	One-cycle pulse when a BRAM port-B read issues

24.4 Functional Description

24.4.1 Block diagram



sdpram_core block diagram

Commands go down the middle of each side; data uses the outer lanes. On the write side, AW commands go through the AW queue into the write tracker, which generates each beat's port-A address through `axi_gen_addr`. W data drops down the far-left lane into BRAM port A, and each finished burst returns a B response up through the B queue. On the read side, AR commands go through the AR queue into the read tracker, which drives port B. Read data returns up the far-right lane through the inflight register, which holds the beat's id, resp and last for the one-cycle BRAM read. The clear FSM borrows port A for a zero fill, but only once every queue and tracker is idle.

24.4.2 FUB Interface Contract

The core speaks a single wire format — a FUB-shaped AXI superset — and nothing else. On the address channels it carries `id / addr / len / size / burst`; on the response channels it carries `id / resp / last`. Wrappers translate their protocol into this shape:

- **AXI4 wrappers** pass the real `awlen / awsize / awburst / awid` (and the AR equivalents) straight through.
- **AXIL wrappers** feed defaults: `awlen = 0`, `awsize = $clog2(STRB_W)`, `awburst = 2'b01` (INCR), `awid = 0`. With `len = 0`, the burst tracker degenerates to a single-beat path.

24.4.3 Write Path — Burst Tracker

On `fub_awvalid && fub_awready`, the tracker latches the burst's ID, base address, beat count (`awlen`), size, and burst type, and asserts `r_wr_active`. Each accepted W beat (`fub_wvalid && fub_wready`) writes byte-enabled data into BRAM port A at the current word address, then advances the address through `axi_gen_addr` and decrements the beat counter. When the last beat is accepted (`r_wr_beats_left == 0`), the tracker drops `r_wr_active`, latches a pending B response (`r_b_pending`), and captures the ID and response code. `fub_awready` deasserts while a write is active or a B is pending, so only one write burst is in flight at a time.

24.4.4 Read Path — Burst Tracker

On `fub_arvalid && fub_arready`, the read tracker latches the burst parameters and asserts `r_rd_active`. A read beat issues (`read_issue`) when the burst is active, the core is not clearing, and either the inflight register is empty or the current beat is being consumed (`!r_inflight || fub_rready`). Each issue reads BRAM port B at the current word address (one-cycle latency), advances the address through `axi_gen_addr`, decrements the beat count, and — on the last beat — drops `r_rd_active`. An inflight tracker holds the `(id, rresp, rlast)` for the beat currently presented on `fub_r`, driving `fub_rvalid`; it clears on handshake unless a new issue refills it the same cycle.

24.4.5 Address Generation

Both paths use an `axi_gen_addr` instance (parameterized `AW=ADDR_WIDTH`, `DW=ODW=DATA_WIDTH`, `LEN=8`) to compute the next beat address from the current address, size, and burst type. INCR and FIXED are handled directly by the glue. WRAP addressing is wrap-shaped end to end: the burst type feeds `axi_gen_addr`, whose `len` input is the LATCHED burst length (the pre-2026-08-13 wiring fed the decrementing remainder, which shrank the wrap mask mid-burst and folded addresses early). The AXI4 wrappers still carry a sim-only assertion flagging WRAP until the path is validated by a test.

24.4.6 BRAM Array

The memory is an inferred dual-port array (`(* ram_style = "auto" *) logic [DATA_WIDTH-1:0] r_mem [MEM_DEPTH]`). It is **never reset** — BRAM contents come up undefined and are only zeroed by the bulk-clear FSM. Port A is shared between the clear FSM (when clearing) and byte-enabled writes; port B is a registered read (single-cycle latency). A benign `WIDTHTRUNC` lint-off covers Verilator computing an extra index bit for the never-taken clear wrap path; every index that reaches the array is provably in `[0, MEM_DEPTH-1]` by tracker construction.

24.4.7 Bulk-Clear FSM

The clear FSM has two states (`CLR_IDLE`, `CLR_BUSY`). It accepts `i_cfg_start_clear` only when both sides report idle (`!r_wr_active && !r_b_pending && !r_rd_active && !r_inflight`), which keeps it from glitching `fub_*_ready` mid-transaction. Once busy, it walks `r_clear_addr` from 0 to `MEM_DEPTH-1`, writing zero to BRAM port A each cycle, then returns to

idle and asserts `o_cfg_done_clear`. While `w_clearing` is asserted, `fub_awready`, `fub_wready`, and `fub_arready` are all deasserted so the FSM owns port A cleanly.

24.4.8 Observation Outputs

`o_dbg_fub_vr` packs the ten FUB valid/ready bits for waveform-free probing. `o_dbg_bram_wr` pulses on a real byte-enabled write fire, and `o_dbg_bram_rd` pulses on each read issue.

24.5 Timing Characteristics

This module is **sequential**: it contains 3 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

24.6 Usage Examples

`sdpram_core` is not usually instantiated directly — the `sdpram_slave_*` wrappers do that. The pattern (as used inside every wrapper) is:

```
sdpram_core #(
    .AXI_ID_WIDTH (AXI_ID_WIDTH),
    .ADDR_WIDTH   (ADDR_WIDTH),
    .DATA_WIDTH   (DATA_WIDTH),
    .MEM_DEPTH    (MEM_DEPTH)
) u_core (
    .aclk    (aclk),
    .aresetn (aresetn),

    // Write FUB (from a slave-write leaf, or single-beat defaults for
AXIL)
    .fub_awid    (fub_awid),
    .fub_awaddr  (fub_awaddr),
    .fub_awlen   (fub_awlen),      // 8'h00 for AXIL
    .fub_awsize  (fub_awsize),     // $clog2(STRB_W) for AXIL
    .fub_awburst (fub_awburst),    // 2'b01 (INCR) for AXIL
    .fub_awvalid (fub_awvalid),
    .fub_awready (fub_awready),
    .fub_wdata   (fub_wdata),
    .fub_wstrb   (fub_wstrb),
    .fub_wvalid  (fub_wvalid),
    .fub_wready  (fub_wready),
    .fub_bid     (fub_bid),
    .fub_bresp   (fub_bresp),
    .fub_bvalid  (fub_bvalid),
    .fub_bready  (fub_bready),

    // Read FUB (from a slave-read leaf)
    .fub_arid    (fub_arid),
    .fub_araddr  (fub_araddr),
    .fub_arlen   (fub_arlen),
    .fub_arsize  (fub_arsize),
    .fub_arburst (fub_arburst),
    .fub_arvalid (fub_arvalid),
    .fub_arready (fub_arready),
    .fub_rid     (fub_rid),
    .fub_rdata   (fub_rdata),
    .fub_rresp   (fub_rresp),
    .fub_rlast   (fub_rlast),
    .fub_rvalid  (fub_rvalid),
    .fub_rready  (fub_rready),
```

```
    // Bulk clear + debug
    .i_cfg_start_clear (i_cfg_start_clear),
    .o_cfg_done_clear  (o_cfg_done_clear),
    .o_dbg_fub_vr      (o_dbg_fub_vr),
    .o_dbg_bram_wr     (o_dbg_bram_wr),
    .o_dbg_bram_rd     (o_dbg_bram_rd)
);
```

24.7 Design Notes

24.7.1 No Reset on the RAM Array

The BRAM array is deliberately not reset. Resetting a large inferred RAM prevents BRAM inference on most FPGA flows and costs an enormous number of flip-flops on ASIC. Contents are undefined at power-up; software (or a harness) issues `i_cfg_start_clear` to zero the array when a defined starting state is required.

24.7.2 One Wire Format, Four Wrappers

The core exists so that the burst logic, byte-enable write, single-cycle read, and clear FSM are written exactly once. The four protocol permutations are thin wrappers that only translate their protocol's FUB into this AXI-shaped FUB. AXIL sides supply the burst/ID defaults in exactly one place per wrapper, so no external port list ever exposes a "fake AXI4" field.

24.7.3 WRAP Bursts

WRAP address math is wrap-shaped via `axi_gen_addr` fed with the latched burst length (mask-shrink bug fixed 2026-08-13), but the path remains UNVALIDATED: the AXI4 wrappers assert (sim-only) that `awburst`/`arburst` is not WRAP at the interface boundary until a test drives it. INCR and FIXED are fully supported.

24.7.4 Single Outstanding Burst Per Side

Each side accepts one burst at a time — `fub_awready` deasserts while a write is active or a B is pending, and `fub_arready` deasserts while a read is active. This keeps the tracker state minimal and matches the descriptor-RAM / scratch-memory use cases the family targets.

24.8 Related Modules

24.8.1 Used By

- `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read wrapper
- `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read wrapper
- `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read wrapper
- `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read wrapper
- Characterization harnesses (e.g. `rapids_char_harness` descriptor RAM, `stream_char_harness` memory ring)

24.8.2 Uses

- **axi_gen_addr.sv** — Next-beat address generation for both trackers
- **reset_defs.svh** — `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

24.8.3 See Also

- **sdpram_slave_axi4_axi4.sv** — the AXI4/AXI4 protocol wrapper
- **sdpram_slave_axil_axil.sv** — the AXIL/AXIL protocol wrapper
- **monbus_group_core.sv** — a comparable "one core + protocol wrappers" split for the monitor-bus delivery layer

---

24.9 Testing

No dedicated testbench for this module. It has no `val/**/test_sdpram_core.py`; it is exercised indirectly, through the tests of the modules that instantiate it:

- `sdpram_slave_axi4_axil` – `val/amba/test_sdpram_slave_axi4_axil.py`
- `sdpram_slave_axil_axi4` – `val/amba/test_sdpram_slave_axil_axi4.py`
- `sdpram_slave_axil_axil` – `val/amba/test_sdpram_slave_axil_axil.py`

Indirect coverage exercises this module only in the configurations its parents elaborate. A parameter or mode no parent uses is untested.

24.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

24.10.1 Source Code

- RTL: `rtl/amba/shared/sdpram_core.sv`
- Address gen: `rtl/amba/shared/axi_gen_addr.sv`
- Wrappers: `rtl/amba/shared/sdpram_slave_*.sv`

24.10.2 Documentation

- Architecture: `docs/markdown/rtl-amba/shared/README.md`
- Subsystem guide: `rtl/amba/CLAUDE.md`

Last Updated: 2026-07-15

24.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

25 Simple Dual-Port BRAM Slave Family

Modules: - `sdpram_core.sv` — common backend (BRAM glue + clear FSM); the protocol skids live in the WRAPPERS, not the backend - `sdpram_slave_axi4_axi4.sv` — wrapper: AXI4 write, AXI4 read - `sdpram_slave_axi4_axil.sv` — wrapper: AXI4 write, AXIL read - `sdpram_slave_axil_axi4.sv` — wrapper: AXIL write, AXI4 read - `sdpram_slave_axil_axil.sv` — wrapper: AXIL write, AXIL read

Location: `rtl/amba/shared/` **Category:** Memory / BRAM Slave **Status:** Production Ready

---

25.1 Overview

The sdpram_slave family provides a BRAM-backed simple-dual-port slave with **independent protocol choice on each side** (write, read). It unifies what used to be two separate modules (axi4_sdpram_slave + axil_sdpram_slave) into a single common backend plus four protocol-specific wrappers that expose the exact port shape for their configuration.

The most common deployment in this repo is the **monitor bus memory dump ring** — monbus_axil4_axil4_group's m_axil_* master writes the compressed (or raw) trace into a sdpram_slave_axil_axil, and the host CPU reads it back through the same slave's read port. Wide AXI4 descriptor RAMs (e.g. stream_char_harness::u_desc_ram, 256-bit data, 8-bit AXI ID) instantiate sdpram_slave_axi4_axi4.

25.2 Parameters

All wrappers expose the same scaling knobs as the backend:

Parameter	Default	Notes
ADDR_WIDTH	32	Address width.
DATA_WIDTH	256 (axi4_*) / 64 (axil_axil)	Data-bus width.
MEM_DEPTH	2048 (axi4_*) / 1024 (axil_axil)	BRAM word count.
AXI_ID_WIDTH	8	Present only when at least one side is AXI4.
USER_WIDTH	1	Present only when at least one side is AXI4.
SKID_DEPTH_AW/W/B/AR/R	2/2/2/2/4	Skid buffer depths for each channel.

25.3 Ports

25.3.1 Debug / Observation Outputs

Common to all wrappers (`o_dbg_fub_vr`/`o_dbg_bram_*` come from `sdpram_core`; `o_dbg_vr` and the busy flags come from the wrapper-level skids):

Output	Width	Meaning
`o_dbg_vr`	10	External `{rready,rvalid, aready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}` — R = [9:8], AR = [7:6], B = [5:4], W = [3:2], AW = [1:0]
`o_dbg_fub_vr`	10	Fub-side (post-skid) valid/ready for the same five channels
`o_dbg_bram_wr`	1	One-cycle pulse on BRAM port-A write fire
`o_dbg_bram_rd`	1	One-cycle pulse on BRAM port-B read fire
`o_dbg_busy_wr`	1	Write-side skid busy
`o_dbg_busy_rd`	1	Read-side skid busy

25.4 Functional Description

25.4.1 Why Four Wrappers + a Backend?

SystemVerilog cannot conditionally include or exclude ports from a single module's port list. The port list is a static syntactic construct, fixed at module declaration. So to give each protocol combination an exact port shape — no spurious AXI4-only fields for the caller to tie off — the family is split:

- **One common backend** (`sdpram_core.sv`) with the BRAM port-A/B glue, bulk-clear FSM and burst tracker. Its whole parameter list is `AXI_ID_WIDTH, ADDR_WIDTH, DATA_WIDTH, MEM_DEPTH` – there is no string-switch generate plumbing; its header says the wrappers "drop straight on top".
- **Four wrappers**, each with the right protocol-shaped port list, that instantiate the matching protocol skids (`axi4_slave_wr`, `axil4_slave_rd`, …) AND the core – the protocol skids live in the wrappers, not the backend.

Wrapper	Wr ports	Rd ports
`sdpram_slave_axi4_axi4`	`s_axi_aw*/w*/b*` (full AXI4)	`s_axi_ar*/r*` (full AXI4)
`sdpram_slave_axi4_axil`	`s_axi_aw*/w*/b*` (full AXI4)	`s_axil_ar*/r*` (AXIL only)
`sdpram_slave_axil_axi4`	`s_axil_aw*/w*/b*` (AXIL only)	`s_axi_ar*/r*` (full AXI4)
`sdpram_slave_axil_axil`	`s_axil_aw*/w*/b*` (AXIL only)	`s_axil_ar*/r*` (AXIL only)

Callers pick the wrapper module name matching their fabric. (An older `sdpram_slave` backend with `WR_PROTOCOL`/`RD_PROTOCOL` string parameters no longer exists – this family replaced it.)

25.4.2 Burst Support

- **AXI4 mode** supports `INCR` (`awburst/arburst = 2'b01`) and `FIXED` (`2'b00`) of any length up to AXI4's 256-beat maximum. `WRAP` (`2'b10`) raises a SIMULATION-only `$warning` ("not yet validated") and the burst PROCEEDS – and the burst proceeds with wrap-shaped addressing via `axi_gen_addr` (fed the latched burst length since the 2026-08-13 mask fix). Still UNVALIDATED – no test drives WRAP through the BRAM glue.
- **AXIL mode** is single-beat by construction — the AXIL slaves have no len ports at all, and the WRAPPERS tie the core's fub-side `awlen`/`arlen` to 0 at

the sdpram_core boundary (single-beat defaults), so the burst-aware backend produces exactly one beat per AW/AR. Multi-beat transactions are not expressible in AXIL anyway.

25.4.3 Bulk Clear

All wrappers expose:

- `i_cfg_start_clear` (input) — pulse high to start a memory-wide clear.
- `o_cfg_done_clear` (output) — a STICKY LEVEL: set when the clear FSM finishes and held high until the next clear is accepted. Do not edge-count it as a pulse.

The clear FSM owns BRAM port A while it walks the whole memory writing zeros. It waits for both sides idle before claiming the port, so an in-flight write or read completes cleanly before the clear begins.

25.5 Usage Examples

25.5.1 Migration

If you have an old caller that instantiates the bare `sdpram_slave`:

```
// Old: full AXI4-shape ports, tie off AXI4-only for AXIL side
sdpram_slave #(
    .WR_PROTOCOL ("AXIL"),
    .RD_PROTOCOL ("AXIL"),
    .AXI_ID_WIDTH (1),  // unused
    ...
) u_dump (
    .s_axi_awid    (1'b0),       // ← unsightly tie-off
    .s_axi_awaddr  (axil_awaddr),
    .s_axi_awlen   (8'h00),      // ← unsightly tie-off
    .s_axi_awsize  (3'h0),       // ← unsightly tie-off
    ...
);
```

After migrating to the matching wrapper:

```
// New: AXIL-only ports, no tie-offs needed
sdpram_slave_axil_axil #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (64),
    .MEM_DEPTH  (1024)
) u_dump (
    .s_axil_awaddr  (axil_awaddr),
    .s_axil_awprot  (axil_awprot),
    .s_axil_awvalid (axil_awvalid),
    .s_axil_awready (axil_awready),
    ...
);
```

The backend is unchanged so functional behavior is identical — only the port list shape and signal names differ.

25.6 Related Modules

25.6.1 Use in Ocelus

The `sdpram_slave_axil_axil` wrapper is the canonical SRAM-ring backend for the `monbus_axil4_axil4_group` master writer — both sides AXIL, so no AXI4-only fields anywhere on the harness wiring. For details on the slot stream landing in the ring, see `monbus_compressor.md` (the compressed case) and the raw-record beat layout described in `monbus_group.md`.

25.6.2 Family Relationships

Module	Role
`monbus_axil4_axil4_group`	Most common consumer (memory-dump ring)
`monbus_compressor`	Source of the compressed slot stream landing in the ring
`axi4_slave_wr` / `axi4_slave_rd`	AXI4-side skids instantiated by the WRAPPERS
`axil4_slave_wr` / `axil4_slave_rd`	AXIL-side skids instantiated by the WRAPPERS

25.7 Testing

`val/amba/test_sdpram_slave.py` builds `sdpram_slave_axi4_axi4` (the full-AXI4 wrapper, which contains `sdpram_core`) and drives the four protocol-combination stimulus shapes against it via `@pytest.mark.parametrize`. The other three wrappers are thin port-shape variants over the same core. Sub-tests cover:

1. Single-beat AW/W/B + AR/R round-trip (all 4 combos)
2. AXI4 INCR burst write + read (AXI4-only)
3. AXI4 FIXED burst write (last beat wins) + read (AXI4-only)
4. Random fill + readback (all 4 combos)

```
pytest val/amba/test_sdpram_slave.py -v
```

25.8 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

25.9 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

26 SDPRAM Slave — AXI4 Write / AXI4 Read

Module: `sdpram_slave_axi4_axi4.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

26.1 Overview

`sdpram_slave_axi4_axi4` is one of four protocol-pair wrappers that place a Simple Dual-Port RAM behind two independent AMBA slave interfaces — a write-side slave and a read-side slave. This variant exposes a **full AXI4 slave** on both the write side (AW + W + B) and the read side (AR + R). It directly instantiates the native `axi4_slave_wr` and `axi4_slave_rd` skid leaf modules and pipes their FUB-side outputs into the shared `sdpram_core` backend.

26.1.1 Key Features

- Full AXI4 slave on both write and read sides (burst, ID, size)
- Native `axi4_slave_wr` + `axi4_slave_rd` skid leaves (no protocol emulation)
- Shared `sdpram_core` backend — one BRAM, one clear FSM, one burst tracker set
- Byte-enabled writes, single-cycle-latency reads
- Bulk-clear control input and done flag
- Full complement of debug taps (external and FUB valid/ready, BRAM fire pulses, leaf busy)
- Sim-only assertion flagging unvalidated WRAP bursts

Characterization harnesses need a synthesizable memory that a device-under-test can read and write over AXI4 while a separate host or checker uses the other port. Splitting the write side and read side into two independent AXI4 slaves lets one agent stream data in while another drains it out, with the BRAM as the shared medium.

This is the pure-AXI4 permutation of the family. When both the producer and consumer speak full AXI4 (bursts, IDs), this wrapper avoids any width- or burst-adaptation on either side.

Use Cases: - Memory model behind an AXI4 DUT in a characterization harness - Descriptor RAM with an AXI4 host-write port and an AXI4 engine-read port - Semaphore / scratch RAM shared between two AXI4 masters - General synthesizable dual-port memory with a fast bulk clear

Key Benefit: Full AXI4 on both sides with zero fake fields — the burst/ID information flows straight through the skid leaves into the shared core.

26.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI transaction-ID width on both sides
ADDR_WIDTH	int	32	Byte-address width
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI USER-signal width (carried by leaves, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
`USE_WSTRB`	bit	`1'b1`	Honour WSTRB byte enables on writes. 0 = every write commits the full word.
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

`USE_WSTRB` decides whether this is block RAM or ~23k LUTs. At the default `1'b1` the core's write block holds a full-word clear at one address and a byte-granular write at another inside the same `always_ff`, and that mixed granularity on a muxed address does not map to a BRAM write port: Vivado drops the entire array into distributed RAM. A 64 KB instance measured 28,696 LUT-as-Memory with block RAM sitting at 2.6% used. Set `USE_WSTRB = 0` when the writer never asserts a partial strobe – a job engine writing whole beats, for instance – and both branches become full-word writes that infer block RAM. The default stays `1'b1` so existing instantiations are unaffected.

26.3 Ports

26.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

26.3.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write-address ID
s_axi_awaddr	input	ADDR_WIDTH	Write burst base address
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Beat size (log2 bytes)
s_axi_awburst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlock	input	1	Lock (carried by leaf, ignored by core)
s_axi_awcache	input	4	Cache attributes (ignored by core)
s_axi_awprot	input	3	Protection attributes (ignored by core)
s_axi_awqos	input	4	QoS (ignored by core)
s_axi_awregion	input	4	Region (ignored by core)
s_axi_awuser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_awvalid	input	1	Write-address valid

Port	Direction	Width	Description
s_axi_awready	output	1	Write-address ready
s_axi_wdata	input	DATA_WIDTH	Write data
s_axi_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuser	input	USER_WIDTH	Write-data user sideband
s_axi_wvalid	input	1	Write-data valid
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

26.3.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2

Port	Direction	Width	Description
			bytes)
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)
s_axi_arprot	input	3	Protection attributes (ignored by core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data

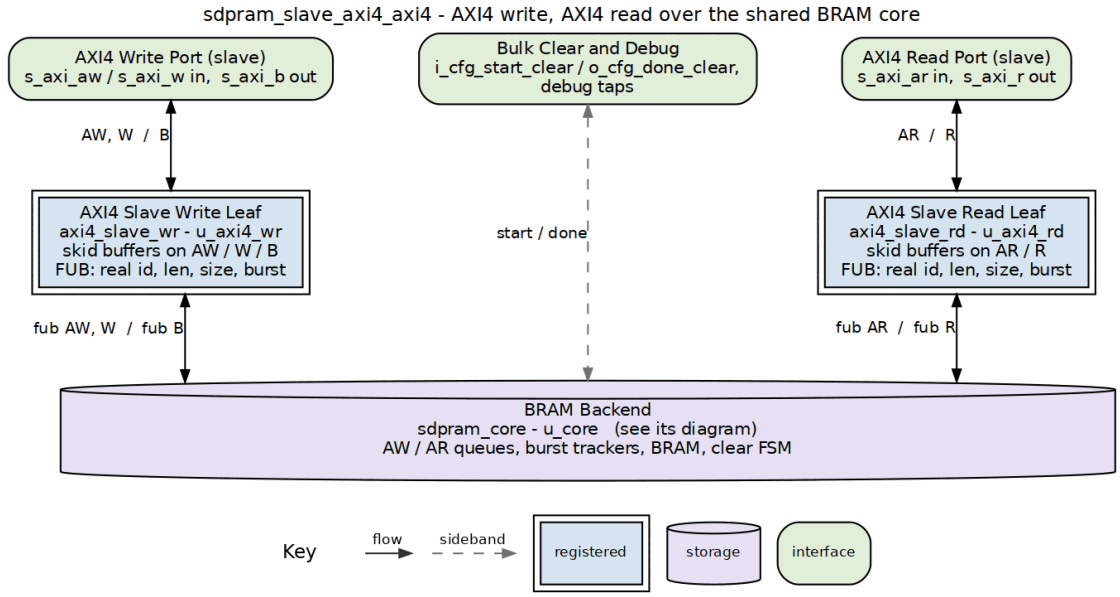
Port	Direction	Width	Description
			valid
s_axi_rready	input	1	Read-data ready

26.3.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot `{rready,rvalid, arready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}`
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_busy_wr	output	1	Write-side skid-leaf busy
o_dbg_busy_rd	output	1	Read-side skid-leaf busy

26.4 Functional Description

26.4.1 Block diagram



sdpram_slave_axi4_axi4 block diagram

The write leaf (AXI4) and read leaf (AXI4) each sit between their bus port and `sdpram_core`, which is drawn as the bottom bar (its queues, burst trackers and BRAM are in its own diagram). Requests go down each side and responses come back up the same edge: AW and W down and B up on the left, AR down and R up on the right. The leaves supply the skid buffers; on the FUB side, both carry real id, len, size and burst. The bulk-clear handshake goes straight to the core.

26.4.2 Structure

The wrapper is three instances and a little glue:

1. `axi4_slave_wr` accepts the external AXI4 write channels and presents a FUB-shaped write interface.
2. `axi4_slave_rd` accepts the external AXI4 read channels and presents a FUB-shaped read interface.
3. `sdpram_core` consumes both FUBs, owns the BRAM, the burst trackers, and the clear FSM.

26.4.3 Field Pass-Through and Tie-Offs

Because both sides are full AXI4, the real `awid / awlen / awsize / awburst` (and the AR equivalents) flow straight through the leaves into the core. The AXI4-only qualifier fields the core does not consume — `lock`, `cache`, `prot`, `qos`, `region`, `user`, and `wlast` — are carried through the

leaf FUB and terminated on `*_unused` nets. The core does not preserve `user` across the BRAM, so `s_axi_buser` and `s_axi_ruser` are hardwired to zero (matching legacy behavior).

26.4.4 Memory Behavior

All memory semantics live in `sdpram_core`: byte-enabled writes on BRAM port A, single-cycle-latency reads on port B, one outstanding burst per side, and the bulk-clear FSM gated on both sides idle. See `sdpram_core` for the full backend description.

26.4.5 Debug Taps

`o_dbg_vr` mirrors the five external channel valid/ready pairs; `o_dbg_fub_vr` mirrors the same for the internal FUB. `o_dbg_bram_wr` / `o_dbg_bram_rd` pulse on real BRAM accesses, and `o_dbg_busy_wr` / `o_dbg_busy_rd` expose each skid leaf's busy flag.

26.4.6 WRAP Assertion

A `translate_off` block asserts on each accepted AW and AR that `awburst`/`arburst` is not WRAP (`2'b10`). WRAP is computed by `axi_gen_addr` but not yet validated through the BRAM glue, so the assertion warns at the sim boundary.

26.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	2 entries
SKID_DEPTH_B	2 entries
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

26.6 Usage Examples

```
sdpram_slave_axi4_axi4 #(
    .AXI_ID_WIDTH (8),
    .ADDR_WIDTH   (32),
    .DATA_WIDTH   (256),
    .MEM_DEPTH    (2048)
) u_mem (
    .aclk    (aclk),
    .aresetn (aresetn),

    // Write-side AXI4 slave (from the producer master)
    .s_axi_awid    (dut_awid),
    .s_axi_awaddr  (dut_awaddr),
    .s_axi_awlen   (dut_awlen),
    .s_axi_awsize  (dut_awsize),
    .s_axi_awburst (dut_awburst),
    .s_axi_awlock  (dut_awlock),
    .s_axi_awcache (dut_awcache),
    .s_axi_awprot  (dut_awprot),
    .s_axi_awqos   (dut_awqos),
    .s_axi_awregion(dut_awregion),
    .s_axi_awuser  (dut_awuser),
    .s_axi_awvalid (dut_awvalid),
    .s_axi_awready (dut_awready),
    // ... W, B, and the AR/R read channels ...

    .i_cfg_start_clear (clear_pulse),
    .o_cfg_done_clear  (clear_done),
    .o_dbg_vr          (dbg_vr),
    .o_dbg_fub_vr      (dbg_fub_vr),
    .o_dbg_bram_wr     (dbg_bram_wr),
    .o_dbg_bram_rd     (dbg_bram_rd),
    .o_dbg_busy_wr     (dbg_busy_wr),
    .o_dbg_busy_rd     (dbg_busy_rd)
);
```

26.7 Design Notes

26.7.1 Native AXI4, No Emulation

Unlike the AXIL-side wrappers, this variant needs no default-field synthesis — both sides are already AXI4, so the burst/ID fields pass through unchanged. The only tie-offs are the qualifier fields the core intentionally ignores, plus the `user` outputs the BRAM does not preserve.

26.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. SystemVerilog cannot conditionally include ports in a single module's port list, so each protocol combination gets its own thin wrapper with the exact port shape; the memory kernel lives once in the core.

26.7.3 WRAP Support Is Sim-Gated

INCR and FIXED bursts of any length are supported. WRAP is flagged by assertion until validated. Callers relying on WRAP semantics should exercise and confirm the path first.

26.8 Related Modules

26.8.1 Used By

- Characterization harnesses (memory model, descriptor RAM, semaphore RAM)
- Any AXI4-to-AXI4 shared-memory test scaffold

26.8.2 Uses

- **axi4_slave_wr.sv** — Write-side AXI4 skid leaf
- **axi4_slave_rd.sv** — Read-side AXI4 skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

26.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend (doc)
- **sdpram_slave_axi4_axil.sv** — AXI4 write + AXIL read variant
- **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
- **sdpram_slave_axil_axil.sv** — AXIL write + AXIL read variant

26.9 Testing

`val/amba/test_sdpram_slave.py` covers this module. It covers this variant alongside its axil sibling.

The test is named for the PAIR rather than the module, which is why an earlier audit looking only for `val/**/test_sdpram_slave_axi4_axi4.py` reported it as having no coverage. It does.

Treat any behaviour described on this page as unverified by simulation.

26.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

26.10.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axi4_axi4.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

26.10.2 Documentation

- Backend spec: `docs/markdown/rtl-amba/shared/sdpram_core.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`

Last Updated: 2026-07-15

26.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

27 SDPRAM Slave — AXI4 Write / AXIL Read

Module: `sdpram_slave_axi4_axil.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

27.1 Overview

`sdpram_slave_axi4_axil` is the family permutation that exposes a **full AXI4 slave on the write side** (AW + W + B) and a **single-beat AXI4-Lite slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. The write side instantiates `axi4_slave_wr` (full AXI4 with id / len / size / burst); the read side instantiates `axil4_slave_rd`, and the wrapper bridges the AXIL read FUB into the core's AXI-shaped read FUB using single-beat defaults.

27.1.1 Key Features

- Full AXI4 write slave (burst, ID, size)
- Single-beat AXIL read slave (no burst / ID)
- AXIL read FUB bridged into the core with `arlen=0`, `arsize=$clog2(STRB_W)`, `arburst=INCR`, `arid=0`
- Shared `sdpram_core` backend
- Byte-enabled writes, single-cycle-latency reads
- Bulk-clear control and debug taps
- Sim-only WRAP-burst assertion on the AXI4 write side

Some harness topologies write bulk data over a high-throughput AXI4 burst port but only need simple, single-word AXIL reads for status polling or spot checks. This wrapper matches that shape: fast AXI4 in, lightweight AXIL out, over one shared BRAM.

The AXIL read side has no burst or ID fields, so the wrapper supplies the missing AXI-shaped fields at the single point where the AXIL read FUB meets the core, letting the core's burst tracker collapse to a single-beat read path.

Use Cases: - AXI4 bulk-write memory model with an AXIL status/read-back port - Descriptor RAM written by an AXI4 engine and polled over AXIL - Any shared memory where the read side is a simple register-style AXIL access

Key Benefit: Full AXI4 write throughput paired with a minimal AXIL read port, with the single-beat adaptation isolated to one place in the wrapper.

27.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 write-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	64	Data-bus / BRAM word width (bits). Shared with the AXI4-Lite side, so 32 or 64 only (IHI 0022H §B1.1.2; elaboration fails otherwise).
USER_WIDTH	int	1	AXI4 write-side USER width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

Derived localparams: `STRB_W = DATA_WIDTH/8`, `FUB_ARSIZE_DEFAULT = $clog2(STRB_W)` (the `arsize` fed to the core for the single-beat read path).

27.3 Ports

27.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

27.3.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write-address ID
s_axi_awaddr	input	ADDR_WIDTH	Write burst base address
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Beat size (log2 bytes)
s_axi_awburst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlock	input	1	Lock (ignored by core)
s_axi_awcache	input	4	Cache attributes (ignored by core)
s_axi_awprot	input	3	Protection attributes (ignored by core)
s_axi_awqos	input	4	QoS (ignored by core)
s_axi_awregion	input	4	Region (ignored by core)
s_axi_awuser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_awvalid	input	1	Write-address valid

Port	Direction	Width	Description
s_axi_awready	output	1	Write-address ready
s_axi_wdata	input	DATA_WIDTH	Write data
s_axi_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuser	input	USER_WIDTH	Write-data user sideband
s_axi_wvalid	input	1	Write-data valid
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

27.3.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes
s_axil_arvalid	input	1	Read-address valid

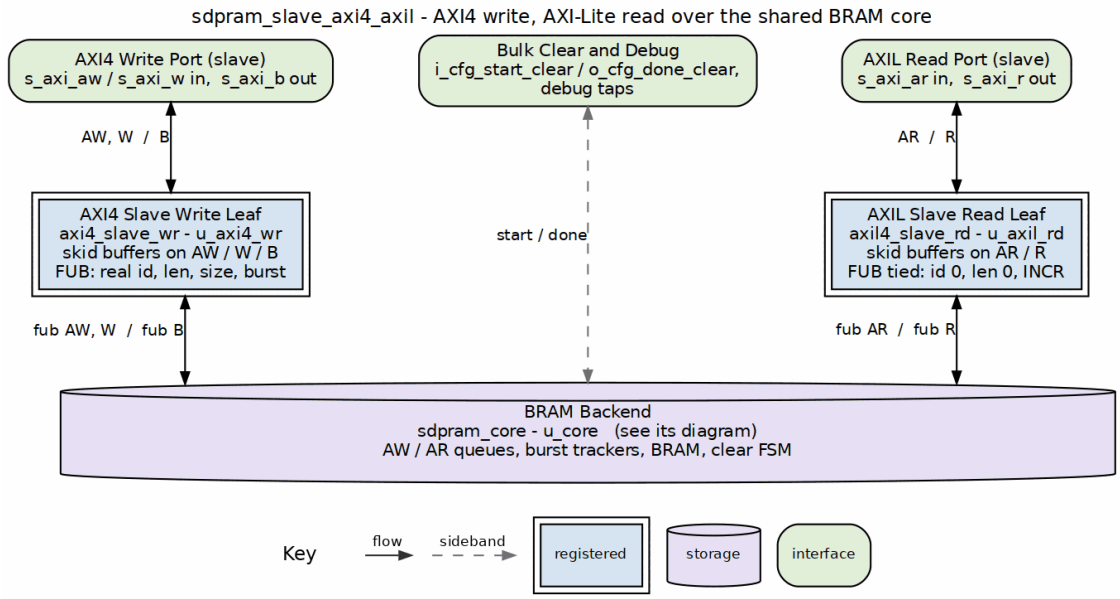
Port	Direction	Width	Description
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

27.3.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot `{rready,rvalid, aready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}` (AXIL R/AR on top, AXI4 B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_busy_wr	output	1	Write-side (AXI4) skid-leaf busy
o_dbg_busy_rd	output	1	Read-side (AXIL) skid-leaf busy

27.4 Functional Description

27.4.1 Block diagram



sdpram_slave_axi4_axil block diagram

The write leaf (AXI4) and read leaf (AXI-Lite) each sit between their bus port and `sdpram_core`, which is drawn as the bottom bar (its queues, burst trackers and BRAM are in its own diagram). Requests go down each side and responses come back up the same edge: AW and W down and B up on the left, AR down and R up on the right. The leaves supply the skid buffers; on the FUB side, the AXI4 write carries real id, len, size and burst; the AXI-Lite read is tied to single beats (id 0, len 0, INCR). The bulk-clear handshake goes straight to the core.

27.4.2 Structure

Three instances: `axi4_slave_wr` (write), `axil4_slave_rd` (read), and `sdpram_core` (backend). The write leaf's FUB carries full AXI4 write fields into the core. The read leaf's AXIL FUB carries only address and data; the wrapper supplies the burst/ID fields the core expects.

27.4.3 AXIL Read Bridge

The AXIL read FUB (`fub_axil_araddr / arvalid / arready` and `rdata / rresp / rvalid / rready`) is wired to the core's read FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- `fub_arid = '0`
- `fub_arlen = 8'h00` (single beat)
- `fub_arsize = 3'(FUB_ARSIZE_DEFAULT)` = `$clog2(STRB_W)` (one full data word per beat)

- `fub_arburst = 2'b01` (INCR)

The core's `fub_rid` and `fub_rlast` outputs go to `*_unused` nets — AXIL has no ID or last field. With `arlen = 0`, each AXIL read is a single-beat access into BRAM.

27.4.4 Write Path and Tie-Offs

The write side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (`lock`, `cache`, `prot`, `qos`, `region`, `user`, `wlast`) terminate on `*_unused` nets. `s_axi_buser` is hardwired to zero since the core does not preserve user across the BRAM.

27.4.5 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

27.4.6 WRAP Assertion

A `translate_off` block asserts on each accepted AW that `awburst` is not WRAP. Only the write side carries this assertion; the AXIL read side is inherently single-beat INCR.

27.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	2 entries
SKID_DEPTH_B	2 entries
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

27.6 Usage Examples

```
sdpram_slave_axi4_axil #(
    .AXI_ID_WIDTH (8),
    .ADDR_WIDTH   (32),
    .DATA_WIDTH   (64),
    .MEM_DEPTH    (2048)
) u_mem (
    .aclk    (aclk),
    .aresetn (aresetn),

    // AXI4 burst write in
    .s_axi_awid    (eng_awid),
    .s_axi_awaddr  (eng_awaddr),
    .s_axi_awlen   (eng_awlen),
    .s_axi_awsize  (eng_awsize),
    .s_axi_awburst (eng_awburst),
    /* ...remaining AW/W/B... */
    .s_axi_bvalid  (eng_bvalid),
    .s_axi_bready  (eng_bready),

    // AXIL single-beat read-back
    .s_axil_araddr  (cpu_araddr),
    .s_axil_arprot  (cpu_arprot),
    .s_axil_arvalid (cpu_arvalid),
    .s_axil_arready (cpu_arready),
    .s_axil_rdata   (cpu_rdata),
    .s_axil_rresp   (cpu_rresp),
    .s_axil_rvalid  (cpu_rvalid),
    .s_axil_rready  (cpu_rready),

    .i_cfg_start_clear (clear_pulse),
    .o_cfg_done_clear  (clear_done),
    .o_dbg_vr          (dbg_vr),
    .o_dbg_fub_vr      (dbg_fub_vr),
    .o_dbg_bram_wr     (dbg_bram_wr),
    .o_dbg_bram_rd     (dbg_bram_rd),
    .o_dbg_busy_wr     (dbg_busy_wr),
    .o_dbg_busy_rd     (dbg_busy_rd)
);
```

27.7 Design Notes

27.7.1 Single-Beat Adaptation Lives in One Place

The only “fake AXI4” fields anywhere in this wrapper are the four read-side defaults (`arid / arlen / arsize / arburst`) fed to the core at the AXIL read FUB boundary. Everything else on the write side is genuine AXI4.

27.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

27.7.3 Read Beat Size

`FUB_ARSIZE_DEFAULT = $clog2(STRB_W)` selects a full-data-word beat for each AXIL read, matching the BRAM word width. Sub-word AXIL reads are not modeled — a read returns the whole BRAM word at the addressed word index.

27.8 Related Modules

27.8.1 Used By

- Characterization harnesses with AXI4 bulk-write and AXIL read-back

27.8.2 Uses

- **axi4_slave_wr.sv** — Write-side AXI4 skid leaf
- **axil4_slave_rd.sv** — Read-side AXIL skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

27.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend (doc)
- **sdpram_slave_axi4_axi4.sv** — AXI4 write + AXI4 read variant
- **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
- **sdpram_slave_axil_axil.sv** — AXIL write + AXIL read variant

---

27.9 Testing

`val/amba/test_sdpram_slave_axi4_axil.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_sdpram_slave_axi4_axil.py -v
```

27.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

27.10.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axi4_axil.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

27.10.2 Documentation

- Backend spec: `docs/markdown/rtl-amba/shared/sdpram_core.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`

Last Updated: 2026-07-15

27.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

28 SDPRAM Slave — AXIL Write / AXI4 Read

Module: `sdpram_slave_axil_axi4.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

28.1 Overview

`sdpram_slave_axil_axi4` is the asymmetric member of the family: a **single-beat AXI4-Lite slave on the write side** (AW + W + B) and a **full AXI4 slave on the read side** (AR + R), both in front of the shared `sdpram_core` backend. The write side instantiates `axil4_slave_wr`; the read side instantiates `axi4_slave_rd` (full AXI4 with id / len / size / burst). AXIL carries no burst or ID fields, so the wrapper bridges the AXIL write FUB into the core's AXI-shaped write FUB using single-beat defaults — the one spot in this design where anything is synthesized rather than genuine.

That shape fits a common characterization pattern: a host CPU populates a memory a word at a time over a simple AXIL write port (a descriptor RAM's host-write port, say), while a high-throughput engine reads bursts back over full AXI4. Supplying the missing AXI-shaped fields at the single point where the AXIL write FUB meets the core collapses the core's write burst tracker to a single-beat path.

28.1.1 Key Features

- Single-beat AXIL write slave (no burst / ID)
- Full AXI4 read slave (burst, ID, size)
- AXIL write FUB bridged into the core with `awlen=0`, `awsize=$clog2(STRB_W)`, `awburst=INCR`, `awid=0`
- Shared `sdpram_core` backend
- Byte-enabled writes, single-cycle-latency reads
- Bulk-clear control and debug taps
- Sim-only WRAP-burst assertion on the AXI4 read side

Use Cases: - Descriptor RAM: host writes descriptors over AXIL, an engine reads them over AXI4 bursts - Semaphore / mailbox RAM: single-word AXIL writes, bursty AXI4 reads - Memory model populated by a control-plane CPU and consumed by a data-plane master

Key Benefit: A simple word-at-a-time AXIL write port with full AXI4 burst read throughput, with the single-beat adaptation isolated to one place in the wrapper.

28.2 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 read-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	64	Data-bus / BRAM word width (bits). Shared with the AXI4-Lite side, so 32 or 64 only (IHI 0022H §B1.1.2; elaboration fails otherwise).
USER_WIDTH	int	1	AXI4 read-side USER width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

Derived localparams: `STRB_W = DATA_WIDTH/8`, `FUB_AWSIZE_DEFAULT = $clog2(STRB_W)` (the `awsize` fed to the core for the single-beat write path).

28.3 Ports

28.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

28.3.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

28.3.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2 bytes)
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)
s_axi_arprot	input	3	Protection attributes (ignored by core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID

Port	Direction	Width	Description
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data valid
s_axi_rready	input	1	Read-data ready

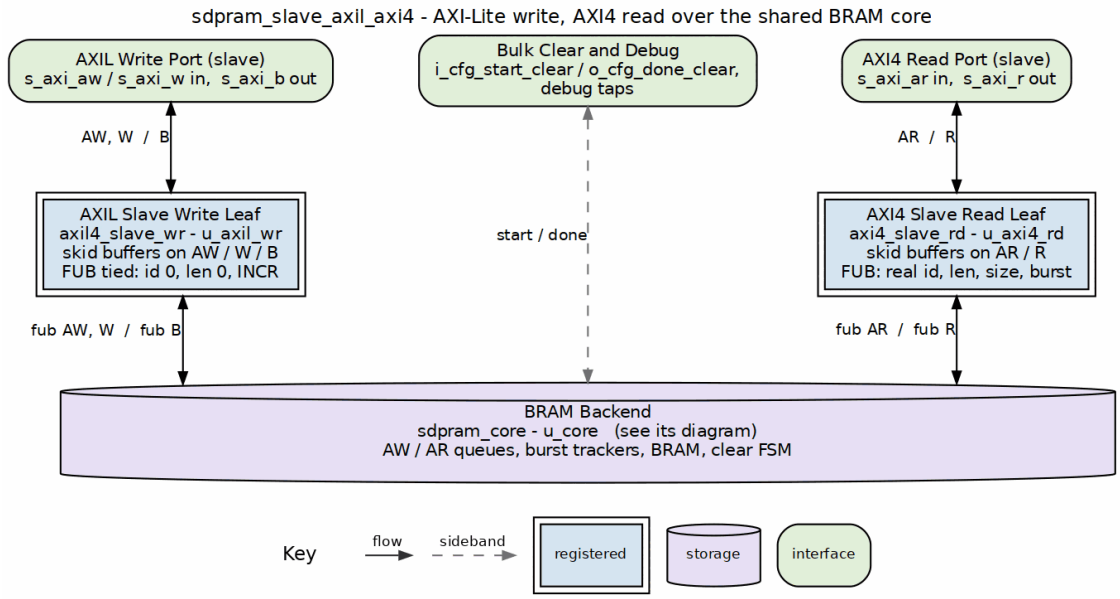
28.3.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot `{rready,rvalid, aready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}` (AXI4 R/AR on top, AXIL B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus	output	1	Write-side (AXIL) skid-leaf

Port	Direction	Width	Description
y_wr			busy
o_dbg_busy_rd	output	1	Read-side (AXI4) skid-leaf busy

28.4 Functional Description

28.4.1 Block diagram



sdpram_slave_axil_axi4 block diagram

The write leaf (AXI-Lite) and read leaf (AXI4) each sit between their bus port and `sdpram_core`, which is drawn as the bottom bar (its queues, burst trackers and BRAM are in its own diagram). Requests go down each side and responses come back up the same edge: AW and W down and B up on the left, AR down and R up on the right. The leaves supply the skid buffers; on the FUB side, the AXI-Lite write is tied to single beats (id 0, len 0, INCR); the AXI4 read carries real id, len, size and burst. The bulk-clear handshake goes straight to the core.

28.4.2 Structure

Three instances: `axil4_slave_wr` (write), `axi4_slave_rd` (read), and `sdpram_core` (backend). The read leaf's FUB carries full AXI4 read fields into the core. The write leaf's AXIL FUB carries only address and data — the wrapper supplies the burst/ID fields the core expects.

28.4.3 AXIL Write Bridge

The AXIL write FUB (`fub_axil_awaddr / awvalid / awready`, `wdata / wstrb / wvalid / wready`, `bresp / bvalid / bready`) is wired to the core's write FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- `fub_awid = '0`
- `fub_awlen = 8'h00` (single beat)
- `fub_awsize = 3'(FUB_AWSIZE_DEFAULT)` = `$clog2(STRB_W)` (one full data word per beat)

- `fub_awburst = 2'b01` (INCR)

The core's `fub_bid` output goes to a `*_unused` net — AXIL has no B-channel ID. With `awlen = 0`, each AXIL write is a single-beat access into BRAM.

28.4.4 Read Path and Tie-Offs

The read side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (`lock`, `cache`, `prot`, `qos`, `region`, `user`) terminate on `*_unused` nets. `s_axi_ruser` is hardwired to zero since the core does not preserve user across the BRAM.

28.4.5 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

28.4.6 WRAP Assertion

A `translate_off` block asserts on each accepted AR that `arburst` is not WRAP. Only the read side carries this assertion; the AXIL write side is inherently single-beat INCR.

28.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	2 entries
SKID_DEPTH_B	2 entries
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

28.6 Usage Examples

```
sdpram_slave_axil_axi4 #(
    .AXI_ID_WIDTH (8),
    .ADDR_WIDTH   (32),
    .DATA_WIDTH   (64),
    .MEM_DEPTH    (2048)
) u_desc_ram (
    .aclk    (aclk),
    .aresetn (aresetn),

    // AXIL host-write port (descriptor population)
    .s_axil_awaddr  (host_awaddr),
    .s_axil_awprot  (host_awprot),
    .s_axil_awvalid (host_awvalid),
    .s_axil_awready (host_awready),
    .s_axil_wdata   (host_wdata),
    .s_axil_wstrb   (host_wstrb),
    .s_axil_wvalid  (host_wvalid),
    .s_axil_wready  (host_wready),
    .s_axil_bresp   (host_bresp),
    .s_axil_bvalid  (host_bvalid),
    .s_axil_bready  (host_bready),

    // AXI4 burst read port (engine fetch)
    .s_axi_arid    (eng_arid),
    .s_axi_araddr  (eng_araddr),
    .s_axi_arlen   (eng_arlen),
    /* ...remaining AR/R... */
    .s_axi_rvalid  (eng_rvalid),
    .s_axi_rready  (eng_rready),

    .i_cfg_start_clear (clear_pulse),
    .o_cfg_done_clear  (clear_done),
    .o_dbg_vr          (dbg_vr),
    .o_dbg_fub_vr      (dbg_fub_vr),
    .o_dbg_bram_wr     (dbg_bram_wr),
    .o_dbg_bram_rd     (dbg_bram_rd),
    .o_dbg_busy_wr     (dbg_busy_wr),
    .o_dbg_busy_rd     (dbg_busy_rd)
);
```

28.7 Design Notes

28.7.1 Descriptor-RAM Fit

This is the natural shape for a descriptor / semaphore RAM: a control-plane CPU pokes single words over AXIL, and a data-plane engine burst-reads them over AXI4. The `rapids_char_harness` descriptor RAM host-write port is exactly this pattern.

28.7.2 Single-Beat Adaptation Lives in One Place

The only "fake AXI4" fields anywhere in this wrapper are the four write-side defaults (`awid / awlen / awsize / awburst`) fed to the core at the AXIL write FUB boundary. Everything else on the read side is genuine AXI4.

28.7.3 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

28.8 Related Modules

28.8.1 Used By

- `rapids_char_harness` descriptor RAM (host-write port)
- Characterization harnesses with AXIL host-write and AXI4 engine-read

28.8.2 Uses

- **axil4_slave_wr.sv** — Write-side AXIL skid leaf
- **axi4_slave_rd.sv** — Read-side AXI4 skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

28.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend (doc)
- **sdpram_slave_axi4_axi4.sv** — AXI4 write + AXI4 read variant
- **sdpram_slave_axi4_axil.sv** — AXI4 write + AXIL read variant
- **sdpram_slave_axil_axil.sv** — AXIL write + AXIL read variant

---

28.9 Testing

`val/amba/test_sdpram_slave_axil_axi4.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_sdpram_slave_axil_axi4.py -v
```

28.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

28.10.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axil_axi4.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

28.10.2 Documentation

- Backend spec: `docs/markdown/rtl-amba/shared/sdpram_core.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`

Last Updated: 2026-07-15

28.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

29 SDPRAM Slave — AXIL Write / AXIL Read

Module: `sdpram_slave_axil_axil.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

29.1 Overview

`sdpram_slave_axil_axil` is the all-AXI4-Lite permutation of the family: a **single-beat AXIL slave on the write side** (AW + W + B) and a **single-beat AXIL slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. It directly instantiates the native `axil4_slave_wr` and `axil4_slave_rd` skid leaves and bridges their AXIL FUB outputs into the core's AXI-shaped FUB by supplying single-beat defaults on both sides. No AXI4-only fields appear anywhere on the wrapper's external boundary — which is exactly what you want from a register-style memory.

Sometimes you just need a lightweight memory shared between two AXIL agents — one writing, one reading — with no burst machinery exposed. This wrapper is exactly that: both sides are single-word AXIL, and the shared backend supplies the burst/ID scaffolding internally. Because AXIL carries no transaction ID, the wrapper carries a 1-bit zero ID through `sdpram_core` purely for type-width bookkeeping. The single-beat defaults are the only "fake" AXI4 fields in the design, and they live in exactly one place per side.

29.1.1 Key Features

- Single-beat AXIL slave on both write and read sides
- Native `axil4_slave_wr` + `axil4_slave_rd` skid leaves
- Both FUBs bridged into the core with `len=0`, `size=$clog2(STRB_W)`, `burst=INCR`, `id=0`
- A 1-bit zero ID carried through the core for typing only (`CORE_ID_WIDTH = 1`)
- Shared `sdpram_core` backend
- Byte-enabled writes, single-cycle-latency reads
- Bulk-clear control and debug taps
- No WRAP assertion needed (both sides are inherently single-beat INCR)

Use Cases: - Lightweight scratch / mailbox RAM shared between two AXIL agents - Small semaphore RAM in a control-plane fabric - Memory-ring backend for the AXIL/AXIL monitor-bus capture master-write port (pairs with `monbus_axil4_axil4_group`) - Any AXIL-only shared-memory test scaffold

Key Benefit: A minimal AXIL-in / AXIL-out shared memory with all burst and clear complexity hidden in the shared core, and no spurious AXI4 fields on the external ports.

29.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	64	Data-bus / BRAM word width (bits). 32 or 64 only: AXI4-Lite data width (IHI 0022H §B1.1.2; elaboration fails otherwise).
MEM_DEPTH	int	1024	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth
USE_WSTRB	bit	1'b1	Honour WSTRB byte enables on writes. 0 = every write commits the full word.

Derived localparams: STRB_W = DATA_WIDTH/8, FUB_AWSIZE_DEFAULT = $clog2(STRB_W) (the awsize/arsize fed to the core), CORE_ID_WIDTH = 1 (a 1-bit zero ID carried through the core for typing only — AXIL has no transaction ID).

> Note: unlike the AXI4-bearing wrappers, this module has **no** AXI_ID_WIDTH or USER_WIDTH parameter, and its DATA_WIDTH / MEM_DEPTH defaults are smaller (64 / 1024), reflecting its lightweight role.

29.3 Ports

29.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

29.3.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

29.3.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

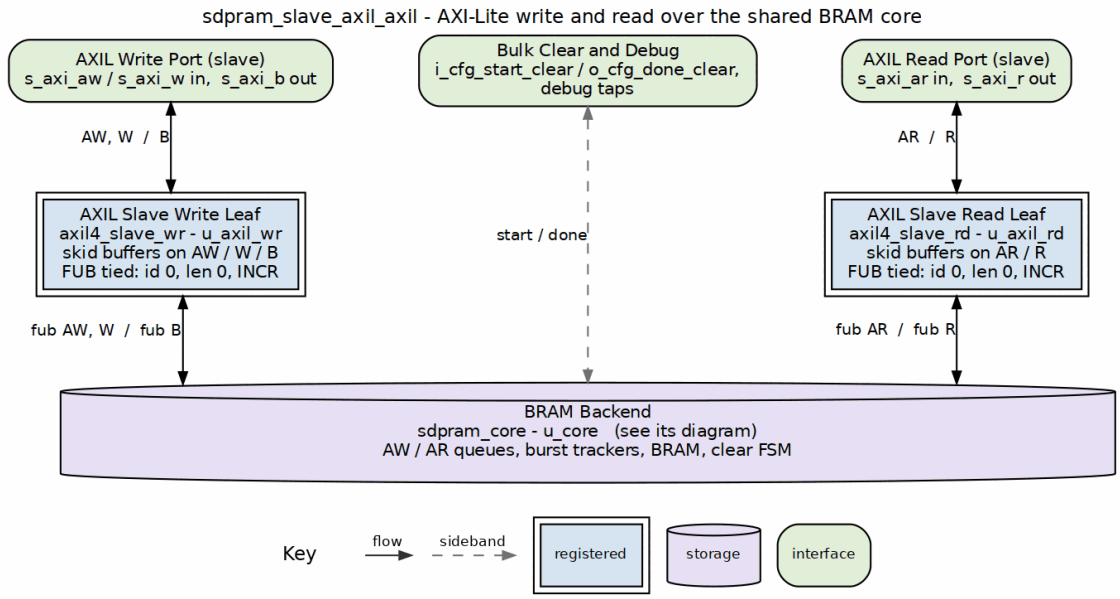
29.3.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot `{rready,rvalid, aready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}`
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse

Port	Direction	Width	Description
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_busy_wr	output	1	Write-side (AXIL) skid-leaf busy
o_dbg_busy_rd	output	1	Read-side (AXIL) skid-leaf busy

29.4 Functional Description

29.4.1 Block diagram



sdpram_slave_axil_axil block diagram

The write leaf (AXI-Lite) and read leaf (AXI-Lite) each sit between their bus port and `sdpram_core`, which is drawn as the bottom bar (its queues, burst trackers and BRAM are in its own diagram). Requests go down each side and responses come back up the same edge: AW and W down and B up on the left, AR down and R up on the right. The leaves supply the skid buffers; on the FUB side, both sides are tied to single beats (id 0, len 0, INCR), so the core's burst trackers degenerate to one beat per command. The bulk-clear handshake goes straight to the core.

29.4.2 Structure

Three instances: `axil4_slave_wr` (write), `axil4_slave_rd` (read), and `sdpram_core` (backend, parameterized with `AXI_ID_WIDTH = CORE_ID_WIDTH = 1`). Both AXIL FUBs carry only address and data; the wrapper supplies the burst/ID fields the core expects.

29.4.3 Single-Beat Defaults on Both Sides

At each FUB-to-core boundary the wrapper ties the AXI-shaped fields the core needs:

Write side: - `fub_awid = 1'b0` - `fub_awlen = 8'h00` - `fub_awsize = 3'(FUB_AWSIZE_DEFAULT)` = `$clog2(STRB_W)` - `fub_awburst = 2'b01` (INCR)

Read side: - `fub_arid = 1'b0` - `fub_arlen = 8'h00` - `fub_arsize = 3'(FUB_AWSIZE_DEFAULT)` - `fub_arburst = 2'b01` (INCR)

The core's `fub_bid`, `fub_rid`, and `fub_rlast` outputs terminate on `*_unused` nets. With `len = 0` on both sides, every access is a single-beat BRAM access.

29.4.4 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding access per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

29.4.5 No WRAP Assertion

Neither side can express a burst, so there is no WRAP-burst assertion in this wrapper (the AXI4-bearing wrappers carry one because their burst-capable side could request WRAP).

29.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	2 entries
SKID_DEPTH_B	2 entries
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

29.6 Usage Examples

```
sdpram_slave_axil_axil #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (64),
    .MEM_DEPTH  (1024)
) u_scratch (
    .aclk    (aclk),
    .aresetn (aresetn),

    // AXIL write port
    .s_axil_awaddr  (wr_awaddr),
    .s_axil_awprot  (wr_awprot),
    .s_axil_awvalid (wr_awvalid),
    .s_axil_awready (wr_awready),
    .s_axil_wdata   (wr_wdata),
    .s_axil_wstrb   (wr_wstrb),
    .s_axil_wvalid  (wr_wvalid),
    .s_axil_wready  (wr_wready),
    .s_axil_bresp   (wr_bresp),
    .s_axil_bvalid  (wr_bvalid),
    .s_axil_bready  (wr_bready),

    // AXIL read port
    .s_axil_araddr  (rd_araddr),
    .s_axil_arprot  (rd_arprot),
    .s_axil_arvalid (rd_arvalid),
    .s_axil_arready (rd_arready),
    .s_axil_rdata   (rd_rdata),
    .s_axil_rresp   (rd_rresp),
    .s_axil_rvalid  (rd_rvalid),
    .s_axil_rready  (rd_rready),

    .i_cfg_start_clear (clear_pulse),
    .o_cfg_done_clear  (clear_done),
    .o_dbg_vr          (dbg_vr),
    .o_dbg_fub_vr      (dbg_fub_vr),
    .o_dbg_bram_wr     (dbg_bram_wr),
    .o_dbg_bram_rd     (dbg_bram_rd),
    .o_dbg_busy_wr     (dbg_busy_wr),
    .o_dbg_busy_rd     (dbg_busy_rd)
);
```

29.7 Design Notes

29.7.1 The Only "Fake" Fields, In One Place Per Side

AXIL has no id / len / burst, so the wrapper feeds `sdpram_core` single-beat INCR defaults. These are the only synthetic AXI4 fields anywhere in the design, and they live in exactly one place per side — the smell of the legacy `generate`-if base is gone.

29.7.2 Monitor-Bus Pairing

This wrapper is the canonical memory-ring backend for the AXIL/AXIL monitor-bus capture path: `monbus_axil4_axil4_group`'s master-write port streams records into an `sdpram_slave_axil_axil` acting as the dump ring.

29.7.3 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

29.8 Related Modules

29.8.1 Used By

- `monbus_axil4_axil4_group` master-write dump ring (memory-ring backend)
- Lightweight AXIL-only shared-memory test scaffolds

29.8.2 Uses

- **axil4_slave_wr.sv** — Write-side AXIL skid leaf
- **axil4_slave_rd.sv** — Read-side AXIL skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

29.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend (doc)
- **sdpram_slave_axi4_axi4.sv** — AXI4 write + AXI4 read variant
- **sdpram_slave_axi4_axil.sv** — AXI4 write + AXIL read variant
- **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
- **monbus_axil4_axil4_group.sv** — Monitor-bus delivery wrapper this RAM backs

---

29.9 Testing

`val/amba/test_sdpram_slave_axil_axil.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_sdpram_slave_axil_axil.py -v
```

29.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h, sec. A3.4.1 “Address structure” (burst addressing via `axi_gen_addr`) and Part B, ch. B1 “AMBA AXI4-Lite” (the single-beat defaults the AXIL wrappers supply). The dual-port RAM itself is a standard technique with no paper.
- Harris, S.L., Harris, D. *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann, 2022, ch. 5 “Digital Building Blocks”, sec. 5.5 “Memory Arrays” (multiported memories) — textbook treatment (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md`

29.10.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axil_axil.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

29.10.2 Documentation

- Backend spec: `docs/markdown/rtl-amba/shared/sdpram_core.md`
- Architecture: `docs/markdown/rtl-amba/shared/README.md`

Last Updated: 2026-07-15

29.11 Navigation

- Back to Shared Infrastructure Index
- Back to rtl-amba Index

30 AMBA Protocol Shims

Location: `projects/components/utility-ip/converters/rtl/` **Test Location:** `val/amba/`
Status: Production (see the module page history for review state)

30.1 Overview

The shims subsystem is where protocol mismatches get solved. These modules bridge between different bus protocols and interface standards — AXI4 to APB, PeakRDL cpuif to cmd/rsp — so components built against different assumptions can coexist in the same AMBA ecosystem without custom glue.

30.1.1 Key Features

- **Protocol Bridging:** Convert between AXI4, APB, and custom interfaces
- **Clock Domain Crossing:** Dual-clock support with proper CDC
- **Width Adaptation:** Automatic data width conversion
- **Burst Decomposition:** Complex transactions → Simple protocol transfers
- **Standards Compliance:** PeakRDL integration, APB/AXI4 conformance
- **Production Ready:** Comprehensive testing and verification

30.1.2 Available Shims

None of these three modules is in `rtl/amba` — all three belong to the converters component, and their documentation now lives with it. The two axi4-to-apb4 shims are specified in that component's MAS chapter 3.4; this book carried a second full copy of each until 2026-08-31. `peakrdl_to_cmdrsp` had no MAS chapter, so its 515-line reference page was MOVED rather than deleted — losing it would have destroyed the only real documentation that module has. This table is kept as a pointer so the shim family stays discoverable from the AMBA book.

Shim	Purpose	Documentation	Status
axi4_to_apb4_shim	AXI4 to APB bridge with dual-clock CDC	converters MAS 3.4	Not in rtl/amba — converters component
axi4_to_apb4_convert	Core AXI4→APB conversion logic	converters MAS 3.4	Not in rtl/amba — converters component
peakrdl_to_cmdrsp	PeakRDL passthrough → cmd/rsp adapter	converters component	Not in rtl/amba — converters component

30.2 Parameters

Most of the art in using these shims is picking buffer depths and width ratios. Here's the guidance I wish someone had handed me the first time.

30.2.1 AXI to APB Bridge

Skid Buffer Depths:

Application	DEPTH_AW/AR	DEPTH_W	DEPTH_R	Notes
Low-latency CPU	2	2-4	2-4	Minimize latency
Moderate DMA	4	8	8	Balance latency/throughput
Burst DMA	8	8	8	Maximize throughput (gaxi_skid_buffer supports DEPTH in 2..8 inclusive (any integer) – anything else now FAILS at elaboration via the DEPTH guard)

Width Conversion:

AXI Width	APB Width	Ratio	Typical Use
32	32	1:1	Simple bridging
64	32	2:1	Modern CPU → legacy periph
128	32	4:1	Wide DMA → narrow periph
64	64	1:1	High-bandwidth peripherals

Side FIFO Depth: - `SIDE_DEPTH` is a THROUGHPUT knob, not a legality constraint: a shallow side FIFO only stalls command issue (entries drain in order as responses retire). The defaults (6 in convert, 4 in the shim) are functionally safe at any burst length - Typical: 4-8 for CPU, 8-16 for DMA

30.2.2 PeakRDL Adapter

Address Width: - Match PeakRDL generation: `--cpuif-addr-width N` - Typical: 12 bits (4 KB register space)

Data Width: - Must match PeakRDL generation (usually 32) - No width conversion supported

---

30.3 Timing

30.3.1 AXI4 to APB Bridge

Latency:

Configuration	Typical Latency	Notes
Same clock, no width convert	5-7 cycles	Minimal overhead
Same clock, 2:1 width convert	8-12 cycles	+1 cycle per APB beat
Dual-clock, no width convert	12-16 cycles	+3-5 cycles per CDC
Dual-clock, 2:1 width convert	15-20 cycles	Combined overhead

Throughput:

Width Ratio	Max Throughput	Notes
1:1	0.5 transfers/cycle	APB overhead
2:1	0.25 transfers/cycle	Width conversion
4:1	0.125 transfers/cycle	More APB beats

30.3.2 PeakRDL Adapter

Latency: - Read: 2 cycles (no stall) - Write: 2 cycles (no stall) - +N cycles if PeakRDL stalls request

Throughput: - Maximum: 1 transaction per 2 cycles - Typical: 1 transaction per 3-5 cycles (with APB)

30.4 Usage Example

30.4.1 Quick Start: AXI4 to APB Bridge

```
// Bridge AXI4 master to APB peripheral bus
axi4_to_apb4_shim #(
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)
) u_bridge (
    .aclk(axi_clk),
    .aresetn(axi_resetn),
    .pclk(apb_clk),
    .presetn(apb_resetn),

    // AXI4 slave interface
    .s_axi_awid(axi_awid),
    .s_axi_awaddr(axi_awaddr),
    // ... AXI signals ...

    // APB master interface
    .m_apb_PSEL(apb_psel),
    .m_apb_PADDR(apb_paddr),
    // ... APB signals ...
);
```

30.4.2 Quick Start: PeakRDL Register Integration

```
// Integrate PeakRDL register block
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH(12),
    .DATA_WIDTH(32)
) u_adapter (
    .aclk(clk),
    .aresetn(rst_n),

    // From APB master stub (cmd/rsp)
    .cmd_valid(cmd_valid),
    .cmd_ready(cmd_ready),
    .cmd_pwrite(cmd_pwrite),
    .cmd_paddr(cmd_paddr),
    .cmd_pwdata(cmd_pwdata),
    .cmd_pstrb(cmd_pstrb),
    .rsp_valid(rsp_valid),
    .rsp_ready(rsp_ready),
    .rsp_prdata(rsp_prdata),
```

```
    .rsp_pslverr(rsp_pslverr),

    // To PeakRDL register block
    .regblk_req(regblk_req),
    .regblk_req_is_wr(regblk_req_is_wr),
    // ... PeakRDL cpuif ...
);
```

30.4.3 Design Patterns

Pattern 1: CPU to Peripherals Bridge

Use the AXI4-to-APB shim for a CPU master accessing peripheral registers:

```
// ARM Cortex-M processor → APB peripherals
axi4_to_apb4_shim #(
    .AXI_ID_WIDTH(12),        // CPU master ID width
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32),
    .DEPTH_AW(2),             // Shallow buffers for low latency
    .DEPTH_AR(2)
) u_cpu_periph_bridge (
    .aclk(cpu_clk),
    .aresetn(cpu_resetn),
    .pclk(periph_clk),
    .presetn(periph_resetn),
    // ... connections ...
);
```

Characteristics: - Low latency (shallow buffers) - Single-beat transactions typical - Dual-clock support

Pattern 2: High-Speed to Low-Speed Bridge

Use width conversion for data rate adaptation:

```
// 64-bit AXI @ 200 MHz → 32-bit APB @ 100 MHz
axi4_to_apb4_shim #(
    .AXI_DATA_WIDTH(64),      // High-speed side
    .APB_DATA_WIDTH(32),      // Low-speed side (2:1)
    .DEPTH_AW(4),             // Deeper for CDC + bursts
    .DEPTH_W(8),              // Extra depth for width conversion
    .DEPTH_R(8),
    .SIDE_DEPTH(8)
) u_width_bridge (
    .aclk(axi_clk_200mhz),
    .pclk(apb_clk_100mhz),
```

```
    // ... connections ...
);
```

Characteristics: - Width adaptation (automatic slicing) - Deeper buffers for burst traffic - Optimized for throughput

Pattern 3: Register Block Integration

Use the PeakRDL adapter for register automation:

```
APB Bus → apb4_slave_stub → peakrdl_to_cmdrsp → PeakRDL Register Block
          (APB→cmd/rsp)     (cmd/rsp→cpuif)     (generated from .rdl)
```

Benefits: - Register automation via PeakRDL - Standard cmd/rsp intermediate interface - Error propagation - Stall handling

30.4.4 Common Use Cases

Use Case 1: SoC Integration

Scenario: ARM Cortex-M processor with AXI4 master accessing APB peripherals.

```
axi4_to_apb4_shim u_cpu_bridge (
    .aclk(cpu_axi_clk),
    .pclk(periph_apb_clk),
    // Connect CPU master → APB peripheral bus
);
```

Benefits: - Enables CPU access to APB peripherals - Handles clock domain crossing - Minimal latency for register access

Use Case 2: DMA to Peripherals

Scenario: DMA engine (AXI4 master) writing to APB peripheral registers.

```
axi4_to_apb4_shim #(
    .DEPTH_AW(8),     // Deep buffers for burst DMA
    .DEPTH_W(8)       // gaxi_skid_buffer legal DEPTHs: 2..8 inclusive
) u_dma_bridge (
    .aclk(dma_clk),
    .pclk(periph_clk),
    // Connect DMA master → APB registers
);
```

Benefits: - Burst decomposition (DMA bursts → APB single-beats) - Deep buffering for DMA traffic - Error reporting back to DMA

Use Case 3: Configuration Registers

Scenario: Control registers for an IP block, auto-generated from SystemRDL.

```
APB → apb4_slave_stub → peakrdl_to_cmdrsp → PeakRDL config_regs
```

Benefits: - Register automation (field reset, RW access, interrupts) - Documentation generation - Software header generation - Type safety

---

30.5 Design Notes

30.5.1 Design Tradeoffs

AXI to APB Bridge — Advantages: - Standard protocols (AXI4 ↔️ APB) - Dual-clock CDC support - Automatic width conversion - Burst decomposition

AXI to APB Bridge — Limitations: - APB serialization (no concurrent transactions) - Performance overhead (APB slower than AXI) - No AXI exclusive access support

Alternatives: - Native APB masters (if no AXI required) - AXI crossbar (if staying in AXI domain)

PeakRDL Adapter — Advantages: - Register automation - Standards-based (SystemRDL) - Documentation generation

PeakRDL Adapter — Limitations: - PeakRDL dependency - Learning curve (SystemRDL) - 32-bit only (typical)

Alternatives: - Custom register implementation - PeakRDL native APB cpuif - Commercial register tools

30.5.2 When to Use Shims

Use the AXI4-to-APB shim when: - Bridging AXI4 master to APB slave - Different clock domains required - Width conversion needed - Burst decomposition required

Use the PeakRDL adapter when: - Automating register block generation - Using SystemRDL descriptions - Need documentation/header generation - Integrating with cmd/rsp infrastructure

Consider alternatives when: - Same protocol both sides (use crossbar) - Need high performance (APB is slow) - Simple registers (custom implementation may be lighter)

30.5.3 Clock Domain Crossing

AXI to APB CDC: - Uses two `gaxi_fifo_async` async FIFOs (cmd aclk→pclk, rsp pclk→aclk) - Gray (or Johnson, via `USE_JOHNSON`) pointer-based synchronization - Safe for arbitrary clock ratios - Proper reset synchronization required

Constraints Required:

```
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_false_path -from [get_clocks pclk] -to [get_clocks aclk]
```

30.5.4 Synthesis Notes

Resource Usage:

Module	LUTs	FFs	BRAM	Notes
axi4_to_apb4_shim (32/32)	~800	~600	0	Minimal config
axi4_to_apb4_shim (64/32)	~1200	~900	0	Width conversion
axi4_to_apb4_convert	~400	~300	0	Core logic only
peakrdl_to_cmdrsp	~50	~100	0	Lightweight

Critical Paths: - AXI packet unpacking (combinatorial) - Width conversion muxing - CDC synchronizers

Recommendations:

```
# Constrain CDC paths in axi4_to_apb4_shim
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
# constrain the two gaxi_fifo_async CDC instances (u_cmd_cdc_fifo /
u_rsp_cdc_fifo);
# there is no cdc_handshake hierarchy in this design
set_max_delay -from */u_cmd_cdc_fifo/* -to */u_cmd_cdc_fifo/* 10.0 -
datapath_only

# Register insertion for wide data paths
set_max_fanout 16 [get_pins */data_shift_reg*]
```

30.5.5 Migration Guide

Upgrading from a legacy AXI-APB bridge is mostly a rename exercise:

Before (legacy):

```
axi_apb_bridge u_bridge (
    .axi_*,  // Legacy AXI naming
    .apb_*   // Legacy APB naming
);
```

After (RTL Design Sherpa):

```
axi4_to_apb4_shim u_bridge (
    .s_axi_*,  // AXI slave interface
    .m_apb_*   // APB master interface
);
```

Key Changes: - Explicit slave/master prefixes - Standardized signal naming - Added CDC support - Parameterized buffering

30.5.6 Troubleshooting

Issue 1: High Latency

Symptoms: slow register access, performance degradation.

Debug: 1. Check buffer depths (DEPTH_*) - deeper = higher latency 2. Verify clock domain crossing overhead 3. Check for width conversion ratio 4. Monitor APB protocol overhead

Solutions: - Reduce buffer depths for low latency - Same-clock operation if possible - Minimize width ratio (prefer 1:1)

Issue 2: PeakRDL Stalls

Symptoms: long delays, timeout errors.

Debug: 1. Check `regblk_req_stall_*` signals 2. Verify PeakRDL internal logic 3. Check for field access conflicts

Solutions: - Review PeakRDL register arbitration - Check for hardware write conflicts - Verify interrupt handling

Issue 3: Data Corruption

Symptoms: incorrect read data, write failures.

Debug: 1. Verify width conversion logic 2. Check byte strobes (WSTRB → bit enables) 3. Monitor error signals (PSLVERR, BRESP, RRESP)

Solutions: - Verify parameter configuration - Check WSTRB alignment - Enable error checking in testbench

---

30.6 Related Modules

- **rtl-amba Overview** - Complete AMBA subsystem
- **AXI4 Modules** - AXI4 infrastructure
- **APB Modules** - APB infrastructure
- **Shared Utilities** - Common modules (CDC, FIFOs)

30.7 Testing

```
# Test AXI to APB bridge
pytest
projects/components/utility-ip/converters/dv/tests/test_axi2apb4_shim.
py -v

# Test PeakRDL adapter
pytest
projects/components/utility-ip/converters/dv/tests/test_peakrdl_to_cmd
rsp.py -v

# Run all shims tests
pytest val/amba/test_*shim*.py -v

# Generate waveforms
pytest
projects/components/utility-ip/converters/dv/tests/test_axi2apb4_shim.
py --vcd=bridge.vcd -v
gtkwave bridge.vcd
```

30.8 References

30.8.1 Protocol Specifications

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part A (AXI4: §A3.2 “Basic read and write transactions”, §A3.4 “Transaction structure”). developer.arm.com/documentation/ihi0022/h – the AXI4 side of the bridge. Citation only.
- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – the APB4 side (ch. 3 “Transfers”, ch. 4 “Operating States”). The earlier listing “AMBA 3 APB v2.0” was wrong: AMBA 3 APB is Version 1.0 (IHI 0024B); Version 2.0 is APB4. Citation only.
- Accellera. *SystemRDL 2.0 Register Description Language*, Jan. 2018 – the register description language PeakRDL compiles. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks in the bridge (`u_cmd_cdc_fifo` / `u_rsp_cdc_fifo`).
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`)

30.8.2 External Resources

- **PeakRDL:** https://github.com/SystemRDL/PeakRDL
- **PeakRDL regblock:** https://github.com/SystemRDL/PeakRDL-regblock
- **PeakRDL Workflow Guide:** `projects/components/utility-ip/converters/rtl/peakrdl_adapter_README.md`

30.8.3 Source Code

- RTL: `projects/components/utility-ip/converters/rtl/`
- Tests: `val/amba/test_peakrdl*.py`
- Framework: `bin/TBClasses/components/`

Last Updated: 2025-10-20

30.9 Navigation

- Back to rtl-amba Index
- Back to Main Documentation Index